



Learn The Mainframe With Gibson

A hands-on guide to z/OS concepts, subsystems, and security on the Gibson simulator

Kev Milne

Edited by Alisdair Gurney

Neuro Training Ltd

Copyright

Copyright © 2026 Kev Milne / Neuro Training Ltd. All rights reserved. Edited by Alisdair Gurney.

Every technique in this book is performed on the Gibson training range, a self-contained, deliberately vulnerable z/OS simulator, and is intended for authorised security education only. Never apply these techniques to systems you do not own or are not explicitly authorised, in writing, to test.

Gibson and this book are independent training tools. IBM, z/OS, RACF, CICS, Db2, IMS, z/VM, and z/TPF are trademarks of IBM Corporation, used here for identification and education only.

Contents

Foreword.....	5
Introduction.....	6
Part I Mainframe Foundations.....	7
Chapter 1 Why the Mainframe Still Runs the World.....	9
Chapter 2 Anatomy of a Mainframe.....	13
Chapter 3 The Vocabulary.....	21
Chapter 4 What Gibson Is.....	26
Part II Install, Launch & Connect.....	30
Chapter 5 Installing Gibson.....	31
Chapter 6 Launching & IPL.....	39
Chapter 7 The Network Map.....	47
Chapter 8 Connecting with c3270 & x3270.....	55
Chapter 9 Your First Logon.....	63
Part III Living on the System: Core z/OS.....	71
Chapter 10 The TSO READY Prompt & Essential Commands.....	72
Chapter 11 ISPF: Panels, Navigation, and the Management Menu.....	79
Chapter 12 Datasets & the Catalog.....	87
Chapter 13 VSAM & IDCAMS.....	95
Chapter 14 The ISPF Editor.....	103
Chapter 15 SDSF: Watching the System Work.....	111
Chapter 16 JES2 & JCL.....	119
Chapter 17 Abends & Debugging.....	127
Chapter 18 OMVS: The z/OS UNIX Side.....	135
Part IV Guarding the System: Security & Identity.....	143
Chapter 19 RACF Fundamentals.....	144
Chapter 20 RACF Resource Protection.....	152
Chapter 21 The RACF Database.....	160
Chapter 22 zSecure: Monitoring the Posture.....	168
Chapter 23 SMF & Forensics.....	176
Part V Operating the System.....	184
Chapter 24 The Master Console.....	185
Chapter 25 PARMLIB, PROCLIB & APF.....	193
Chapter 26 WLM, RMF, GRS & Health Checks.....	201
Chapter 27 Sysprog Tooling.....	209
Part VI The Subsystems.....	217
Chapter 28 CICS: The Transaction Monitor.....	218
Chapter 29 The Banking Stack.....	226

Chapter 30	Db2: The Database.....	234
Chapter 31	IMS: Hierarchical Data.....	242
Chapter 32	z/VM: The Hypervisor.....	250
Chapter 33	z/TPF: Extreme Throughput.....	258
Part VII	Connectivity & Programming.....	266
Chapter 34	FTP & TCP/IP.....	267
Chapter 35	NJE: Network Job Entry.....	275
Chapter 36	IND\$FILE: Terminal File Transfer.....	283
Chapter 37	The Web & API Tier.....	291
Chapter 38	RESTful APIs, z/OS Connect & API Security.....	299
Chapter 39	Programming Gibson.....	309
Part VIII	Reconnaissance: Mapping the Mainframe.....	317
Chapter 40	The Reconnaissance Toolkit.....	318
Chapter 41	nmap on the Mainframe.....	326
Chapter 42	EZRecon: External Footprinting.....	334
Chapter 43	Web-Application Reconnaissance.....	342
Chapter 44	Mainframe Attack Tools.....	350
Chapter 45	OSINT: Open-Source Intelligence.....	358
Chapter 46	LENNOX & Pivoting.....	366
Part IX	The Kill-Chain & Its Defence.....	374
Chapter 47	The Security Model.....	375
Chapter 48	Initial Access & Credential Attacks.....	383
Chapter 49	Privilege Escalation.....	391
Chapter 50	Lateral Movement & Persistence.....	399
Chapter 51	The CTI Platform & HMS Detection.....	407
Chapter 52	The Defender's View.....	414
	The Lab Catalog.....	423
Part XI	Reference.....	434
	Appendix A, Ports & Services.....	434
	Appendix B, RACF Command Reference.....	434
	Appendix C, Console & TSO Quick Reference.....	435
	Appendix D, API Routes & JWT/OAuth Reference.....	435
	Appendix E, CIS z/OS Benchmark Control Map.....	436
	Appendix F, MITRE ATT&CK Technique Mapping.....	436
	Appendix G, Health Checks.....	437
	Appendix H, The Kill-Chain Reference.....	437
	Index.....	438

Foreword

Learning is not linear; it is built on foundations.

Nancy Grimes

[Publisher's note: Nancy Grimes quotes to be inserted here before publication, including the epigraph above and the "RACF is ..." quotation to open the security material. Final wording to be confirmed with the author.]

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum, quis nostrud exercitation.

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo.

Introduction

[Introduction to be written. Placeholder text follows.]

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum, quis nostrud exercitation.

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Part I

Mainframe Foundations

Before you touch a mainframe, you need a feel for what one is and the vocabulary that surrounds it. Part I builds that foundation: why these machines still matter, how they are put together, the words you will keep meeting, and how work actually flows through the system. By the end of it you will be ready to install Gibson and log on for real.

CHAPTER 1

Why the Mainframe Still Runs the World

By the end of this chapter you will be able to:

Explain why mainframes still carry the world's most critical workloads.

Describe what RAS (Reliability, Availability, Serviceability) means and why it matters.

Identify the industries and kinds of work that depend on IBM Z.

Explain why mainframe security has moved from obscurity into the mainstream.

Describe what Gibson is and how it lets you practice safely.

SOMETIME IN THE LAST TWENTY-FOUR hours, you trusted a mainframe with something that mattered to you. You probably did it more than once, and you almost certainly never knew. When you tapped your card at a coffee shop, when your salary landed, when you booked a flight or filed a claim or checked a balance at two in the morning, the request you fired off most likely ended its journey on a machine you have never seen, running software older than the person who served you the coffee. The web app was just the receptionist. The mainframe was the bank.

This is the first thing to understand about mainframes, and the hardest for a lot of people to accept: they did not go away. The press has been writing their obituary since the 1990s, and yet the world's banks, insurers, airlines, retailers, and governments still run their most important workloads on IBM Z. Not as a museum piece they are too scared to switch off (although there is some of that) but because, for a particular kind of work, nothing else does the job as well. If you want to test the security of the systems that actually hold the money, this is where the money is.

The machine that refuses to stop

Ask an engineer what makes a mainframe special and they will eventually say three letters: RAS. Reliability, Availability, and Serviceability. It is an unglamorous phrase for a genuinely remarkable idea, that a computer should be designed, from the silicon up, on the assumption that parts will fail, and that the failure of a part should never become the failure of the system. Processors check each other's arithmetic. Memory corrects its own errors. Components can be replaced while the machine keeps running. It is the computing equivalent of changing a tire at seventy miles an hour without slowing down.

The result is uptime measured not in months but in years. Mainframe shops talk casually about five nines (99.999 percent availability, which works out to roughly five minutes of downtime a year) and many do better than that. There are systems in production today whose operators cannot tell you the last time they were down, because it was before their time. When your business is moving millions of transactions an hour and every one of them is someone's money, that kind of stubbornness is not nostalgia. It is the product.

Where the mainframe actually lives

It is easy to imagine the mainframe as a relic in a basement somewhere. The reality is the opposite: it tends to sit at the very center of the most consequential systems an organization owns. The numbers that get quoted vary, and you should treat any single statistic with care, but the shape of the picture is not in doubt. A large majority of the world's biggest banks run their core processing on the mainframe. The same is true across insurance, where policies and claims live; across airlines, where reservation systems have run on this iron for half a century; across retail and logistics; and across the public sector, where tax, benefits, and records systems quietly carry the weight of entire countries.

What these workloads have in common is that they are unforgiving. A social media feed can drop a post and nobody dies. A payment system that loses a transaction, double-charges a customer, or lets the wrong person move money has failed at the one thing it exists to do. These are the crown jewels, and crown jewels are exactly what a security professional is paid to think about. The mainframe is where the highest-value data and the highest-value transactions converge, which makes it, by definition, the most interesting room in the building.

NOTE

Throughout this book, “the mainframe” means IBM Z hardware running the z/OS operating system, because that is what you will meet in the field and what Gibson simulates. There is a whole family beyond it (z/VM, z/TPF, Linux on Z) and we will get to those. But when a banker or a pen tester says “the mainframe,” they almost always mean z/OS, and so do I unless I say otherwise.

The dinosaur that isn't

The lazy story about mainframes is that they are frozen in 1975. The truth is that they have quietly absorbed almost every major shift in computing while keeping a promise nobody else even attempts: that a program compiled decades ago will still run, unchanged, today. That backward compatibility is not laziness. It is an enormous engineering achievement, and it is the reason a business can build on the platform for forty years without rewriting everything every time the fashion changes.

Underneath the compatibility, the platform has kept moving. Modern IBM Z runs Java and containers and Linux alongside the COBOL and the assembler. It speaks TCP/IP, hosts REST APIs, plugs into hybrid cloud, and runs machine-learning models next to the transactions they score, so a fraud check can happen in the few milliseconds a payment is in flight. The green screen you will spend a lot of this book in front of is one face of the system, and a very useful one for our purposes, but it sits on top of a machine that is far more modern than it looks.

This matters to a security person for a specific reason. A platform that has been accreting capability for fifty years, while never breaking compatibility with the last fifty, is a platform with a deep and layered attack surface. Old defaults linger. New interfaces get

bolted onto old assumptions. The people who truly understand the whole stack are retiring faster than they are being replaced. That combination (immense value, deep complexity, and a shrinking pool of expertise) is precisely the combination that should make your ears prick up.

Why this is your problem now

For most of its history, the mainframe was protected less by its controls than by its obscurity. It lived on private networks, behind terminals nobody outside the building knew how to drive, described in manuals nobody outside the priesthood had read. You could not attack what you could not even log on to. For a long time that was enough.

It is not enough anymore. The mainframe is networked now (it has to be, because the web app and the mobile app and the partner API all need to reach the system of record. The tools to talk to it are open source and a search away. And a generation of researchers) people like Philip Young and Chad Rikansrud, whose work you will see credited throughout this book, have spent the last decade dragging mainframe security into the daylight, building the scanners, the exploits, and the methodology that turn “nobody knows how it works” from a defense into a liability. The obscurity is gone. What is left is whatever security was actually configured, which is often less than anyone hoped.

That is the gap this book is about. Not because breaking into mainframes is clever or transgressive, but because the systems that hold the money deserve to be tested by people who understand them, using a disciplined method you can explain, repeat, and defend. The goal is not to be a tourist who got lucky. It is to be the person in the room who actually knows how the most important computer in the building works, and can say so with evidence.

Enter Gibson

There is one very large obstacle between you and that knowledge: you cannot practice on a real one. You will not be handed a production z/OS LPAR to experiment on, and you absolutely should not go looking for someone else's. Mainframe time is expensive, mainframe mistakes are public, and “I was just learning” is not a defense that ends well. The single biggest reason mainframe security skills are rare is that there has never been a safe, realistic, no-stakes place to build them.

That is what Gibson is for. Gibson is a z/OS mainframe simulator (a self-contained training range that behaves like the real thing across the parts that matter for learning and for security work: the VTAM logon, TSO and ISPF, the job entry system, RACF, the Unix side, CICS and Db2, the console, the network services. It answers a port scan the way a real z/OS system would. It deliberately ships with the kinds of weaknesses real systems acquire over time, and) just as important, it shows you how to find and fix them. It is, in the friendliest possible sense, your crash-test dummy: somewhere you can drive into the wall on purpose, as many times as you like, and learn something every time without anyone getting hurt.

Everything in this book runs on Gibson. Every command you see, you can type. Every screen, you can reach. Every lab ends not with “you’re in” but with “here is what the defender should have done”, because the point of learning to pick the lock is to understand the lock. You do not need access to a real mainframe to follow along, and you do not need to have ever seen a green screen before. You just need to be curious about the most important computer you have never met.

Checkpoint: in your own words, why does a payment system run on a mainframe rather than a farm of web servers? What does “five nines” mean in practice, and why is decades-long backward compatibility both a strength and a security concern? If you can answer those, you have the foundation the rest of Part I builds on.

The mainframe never left. It still carries the core processing of banks, insurers, airlines, retailers, and governments because its design prioritizes reliability, availability, and serviceability above all else (uptime measured in years, not months. That same longevity, decades of unbroken compatibility layered onto a now-networked and thoroughly modern platform, creates a deep and valuable attack surface guarded by a shrinking pool of experts. Mainframe security is no longer protected by obscurity, which is exactly why it deserves disciplined testing) and why Gibson exists, so you can learn the platform safely with every lab paired to the defender’s fix.

What comes next

Before we install anything, we need a shared map. In the next chapter we will open the machine up and look at how it is actually put together, LPARs, the operating system, address spaces, and the crowd of subsystems that live inside a single mainframe like neighborhoods in a city. After that we will sort out the vocabulary, the alphabet soup of VTAM and VSAM and SMF and RACF that can make mainframe documentation read like a foreign language, and we will see how a single unit of work flows through the system from submission to output. Then, with the concepts in hand, we will download Gibson, start it up, point a terminal at it, and log on for the first time.

By the end of Part I you will understand what a mainframe is and why it is built the way it is. By the end of the book you will be able to find your way around one, operate it, and test it like someone who belongs there. Let’s start by taking the lid off.

CHAPTER 2

Anatomy of a Mainframe

By the end of this chapter you will be able to:

Describe the layers of a mainframe: hardware, LPARs, z/OS, and address spaces.

Explain what an LPAR is and how one machine becomes many logical systems.

Identify the major subsystems that share a single z/OS image.

Explain how address spaces isolate running work.

Relate the anatomy to where a security boundary actually sits.

AT THE END OF THE last chapter I promised we would take the lid off. So let's do it. From the outside, a modern mainframe is an anticlimax: a black cabinet about the size of a couple of refrigerators, humming quietly in a data center, indistinguishable from the racks around it. Open it up, though, and the surprise is not what is inside one machine. It is that there is no "one machine" at all. A mainframe is a building full of computers that have agreed to act like a single, very dependable one.

You do not need to memorize the engineering to test a mainframe, any more than a burglar needs a degree in structural design. But you do need a floor plan. This chapter is that floor plan: how one physical box becomes many logical computers, what the operating system does with each of them, how work is boxed up and kept apart, and the crowd of subsystems that live inside a single z/OS image like neighborhoods in a city. Get this map into your head and the rest of the book becomes a guided walk through streets you already recognize.

One box, many machines

Start with the hardware. The physical mainframe is called the central processor complex, or CPC, you will also hear the older term CEC. It is a tank of processors, memory, and I/O built for throughput and for staying up. What turns that single tank into many computers is a hypervisor baked into the hardware called PR/SM, which carves the machine into logical partitions, or LPARs.

An LPAR is a complete, isolated computer. It gets its own slice of processors and memory, its own operating system, and its own identity, and it has no idea the others exist. Think of an apartment building: one structure, but inside it are wholly separate homes, and what happens in one apartment does not leak through the walls into the next. Those walls matter enormously for security, because the LPAR boundary is one of the strongest isolation guarantees in all of computing, hardware-enforced and independently certified to standards most software never gets near. As an attacker you do not break out of an LPAR. You work inside one.

A single mainframe routinely runs many LPARs at once: a production z/OS here, a test z/OS there, perhaps a z/VM partition next door hosting hundreds of Linux guests. They

share the iron and stay strangers. For the rest of this book, when we talk about “the system,” we mean one z/OS running inside one LPAR, which is exactly what Gibson presents you with.

The operating system

Inside our LPAR runs z/OS, the operating system that has carried the mainframe’s most serious workloads for decades. Its family tree runs back through OS/390 and MVS to OS/360 in the 1960s, and that lineage is the point: the compatibility promise from Chapter 1 lives here. z/OS’s job is to run thousands of units of work at the same time (batch jobs grinding through files overnight, online transactions answering in milliseconds, interactive users poking at screens) and to keep them fed, fair, and apart.

The privileged core of z/OS is the nucleus, or supervisor. Everything in the system lives on one side or the other of a hard line the hardware itself draws. On one side is problem state, where ordinary programs run, your code, your jobs, the business applications. On the other is supervisor state, where the operating system runs with the keys to everything. The hardware also tags storage with protection keys, so a program in one key cannot scribble on memory owned by another. That split between the privileged and the unprivileged is the bedrock of mainframe security, and almost every serious attack you will learn in this book is, underneath, a story about crossing from one side of that line to the other.

Address spaces

How does z/OS keep thousands of things running without them treading on each other? It gives each one an address space: its own private map of virtual storage, its own little universe. If you come from the Unix or Windows world, an address space is roughly a process (but on z/OS it is also the fundamental organizing unit of the whole system. Your interactive session is an address space. Each batch job is an address space. And every subsystem we are about to meet) the security manager, the job scheduler, each database, each transaction region, is one or more address spaces of its own.

By default, address spaces are islands: one cannot reach into another’s storage unless the system explicitly permits it. That isolation is good for stability and good for security, and it is also why attackers are pushed toward a small number of privileged doorways rather than simply reading another user’s memory. The most important of those doorways has a name worth tattooing on the inside of your eyelids: the Authorized Program Facility, or APF.

APF is the mechanism by which a program earns the right to run authorized, to enter supervisor state, take key 0, and reach across address spaces. Only programs loaded from libraries on the system’s APF list are allowed to do this, which makes that list one of the most security-critical things on the entire machine. If you can get your code into an APF-authorized library, or trick an already-authorized program into doing your bidding, the

isolation we just admired stops protecting anyone. You have crossed the line. We will spend a whole chapter on exactly this later; for now, just file APF away as the master key.

NOTE

You can see all of this on Gibson rather than just read about it. From the master console, the system display commands report the active address spaces and the IPL and PARMLIB configuration, and the PARMLIB explorer lists the live APF-authorized libraries, including one that has been deliberately left where it should not be, for you to find later. When we reach the console in Part V, that abstract “APF list” becomes something you can actually pull up and read.

A city of subsystems

Here is the idea that makes everything else click. z/OS is not one big program. It is a city of cooperating subsystems, each running in its own address space, each with its own job, its own command language, and its own personality, all living inside the single operating system, connected by well-worn roads, and governed by one set of laws. Once you see the city, the platform stops feeling like a baffling monolith and starts feeling like a place with neighborhoods.

VTAM is the front door. It is the networking layer through which terminals and applications reach the system; when you point a 3270 emulator at a mainframe, you are knocking on VTAM. TSO/E and ISPF are the interactive neighborhood, where users and administrators actually live and work, the command prompt and the menu-driven panels you will spend much of this book inside. JES2, the job entry subsystem, is the batch factory: it takes the jobs people submit, schedules them, runs them, and collects their output.

RACF is the police force and the records office combined (the security manager that decides who may do what, and that every other neighborhood calls before it lets anyone in. CICS is the storefront district, where online business transactions happen at enormous volume; Db2 and IMS are the data districts, the relational and the hierarchical, where the records themselves are kept. z/OS UNIX) you will hear it called USS or OMVS, is the Unix quarter, a complete POSIX environment that is not bolted on but genuinely native, shell and all. And watching over all of it is SMF, the System Management Facility: the city’s tireless recorder, writing down who did what, when, and from where. SMF is the black box, and for a defender it is the single most valuable witness on the system.

Each of these is its own world with its own dialect, which is why a mainframe can feel like a dozen different computers wearing a trench coat. It is also why the documentation is so intimidating: you are not learning one system, you are learning a federation of them. The good news is that they follow patterns, and once you have toured a couple of neighborhoods the others come quickly.

Why the floor plan matters

Hold the city in your mind and a tester’s view of the mainframe falls into place. The big boundaries (between LPARs, between address spaces) are strong, so you rarely attack them head-on. Instead you go for the privileged doorways and the trust between

neighborhoods. Most real mainframe compromise comes down to one of three moves: crossing from problem state to authorized through APF or a careless authorized program; abusing the trust a subsystem has been given, like a transaction region or a started task running with far more power than it needs; or, most often and most embarrassingly, simply using access that RACF was configured to allow but never should have.

That is the whole game, and it is why this chapter came before any command. You now have the map: a box full of isolated computers, an operating system splitting the privileged from the unprivileged, work boxed into address spaces, and a city of subsystems trusting one another under RACF's eye and SMF's pen. Everything from here is detail filling in that frame.

Checkpoint: can you sketch, from memory, the path from physical hardware down through an LPAR to a running address space? Can you name three subsystems that coexist in one z/OS image, and say what an address space protects? Those are the structural facts the rest of the book assumes.

A mainframe is built in layers: physical IBM Z hardware partitioned into LPARs, each running an operating system such as z/OS, which in turn divides its work into isolated address spaces. A single z/OS image is less one program than a city of cooperating subsystems (TSO, JES, VTAM, RACF, CICS, Db2) sharing one set of resources under tight control. Knowing which layer a thing lives in, and where a layer's boundary becomes a security boundary, is the mental model every later chapter builds on.

The acronyms you'll keep meeting

Before we move on, here is something to keep by your side. Every neighborhood we just toured arrives wrapped in its own initials, and the mainframe world hoards acronyms the way a dragon hoards gold. The table that follows collects the important ones (the terms you will keep meeting in this book and in the wild) with what each one actually stands for. Gibson keeps a translation for every one of them; the third column is the one it would rather the auditors did not see. Skim it now, then come back whenever a manual starts speaking in tongues.

Acronym	What it means	Gibson's translation
z/OS	IBM Z operating system.	Zero-day Operating Stress: the place where every simple question becomes a standards meeting.
z/VM	IBM Z virtualization operating system / hypervisor environment.	Zone of Virtual Mayhem: one real machine pretending to be a data centre.
ZVM	Common shorthand/search spelling for IBM z/VM.	Zombie Virtual Machine: it keeps running after everyone swears they shut it down.
TPF	Transaction Processing Facility, now IBM z/Transaction Processing Facility.	Transactions Per Frenzy: airline bookings, cards, and panic at industrial scale.
MVS	Multiple Virtual Storage, the z/OS lineage and programming model.	Mostly Very Serious: because even the jokes need an APAR.
LPAR	Logical Partition.	Little Partition, Astonishing Responsibility: tiny slice, enormous consequences.
IPL	Initial Program Load.	I Pray Loads: the sacred ritual before the system speaks.
DASD	Direct Access Storage Device.	Disks Are Still Disks: but with better suits and worse acronyms.

Acronym	What it means	Gibson's translation
CLPA	Create Link Pack Area.	Clear LPA Anxiety: reboot therapy for confused modules.
TSO	Time Sharing Option.	Type Something Obscure: the READY prompt's entire personality.
ISPF	Interactive System Productivity Facility.	I Still Press F3: the universal survival skill.
SDSF	System Display and Search Facility.	Scroll Down, Still Frustrated: where jobs hide until you learn the line commands.
OMVS	TSO command/interface for the z/OS UNIX shell environment.	Oh My, Very Shell-like: UNIX wearing a 3270 tie.
USS	UNIX System Services, commonly z/OS UNIX.	UNIX, Somehow Still z/OS: same slashes, more RACF.
JCL	Job Control Language.	Just Confusing Lines: three statements, infinite mistakes.
JES	Job Entry Subsystem.	Jobs Eventually Start: usually after you stop staring at SDSF.
NJE	Network Job Entry.	Now Jobs Escape: batch work discovering it has a network.
RACF	Resource Access Control Facility.	Reasonably Accurate Control Facility: until someone has SPECIAL.
SAF	System Authorization Facility.	Security Asks First: the bouncer that calls RACF.
APF	Authorized Program Facility.	Accidentally Powerful Folder: one bad library, one very bad day.
SMF	System Management Facilities.	System Muttering Forever: the mainframe writes everything down.
ICSF	Integrated Cryptographic Service Facility.	I Can Secure Finance: as long as nobody loses the key ceremony.
CKDS	Cryptographic Key Data Set.	Crypto Keys Don't Share: symmetric secrets with a filing cabinet.
PKDS	Public Key Data Set.	Public Keys, Don't Spill: asymmetric keys wearing RACF badges.
TKDS	Token Key Data Set.	Token Keys Demand Supervision: little keys with big opinions.
TKE	Trusted Key Entry.	Touchy Key Experience: the ceremony where everyone suddenly reads the procedure.
RACFDS	RACF data set / RACF database dataset.	RACF's Deep Secrets: where passwords go to become audit findings.
UACC	Universal Access.	Usually Accidentally Catastrophic Control: public access with paperwork.
MFA	Multi-Factor Authentication.	More Friction Added: annoying until it saves you.
SURROGAT	RACF class controlling surrogate job submission.	Submit Under Really Risky Other Guy's Authority Today.
CSFKEYS	RACF class controlling access to ICSF cryptographic keys.	Crypto Stuff: Find Keys? Expect Your Security team.
CSFSERV	RACF class controlling access to ICSF services.	Crypto Services For Seriously Entitled Requesters Verified.
APPL	RACF application class for controlling application logon.	Application Permission Panic Layer.
FACILITY	RACF general resource class for miscellaneous protected functions.	Flexible Access Class, Unfortunately It Loves Important Things.
BPX	z/OS UNIX prefix used on BPX services, parameters, and resources.	Basically POSIX-ish eXcitement.
CICS	Customer Information Control System.	Customers Insist Code Survives: the transaction engine that refuses to retire.
BMS	Basic Mapping Support.	Basically Makes Screens: green-screen origami.
COMMAREA	Communication area passed between CICS programs.	Commonly Mismanaged Memory Area: a tiny parcel of trouble.

Acronym	What it means	Gibson's translation
CEDA	CICS-supplied online resource definition transaction.	Change Everything, Deploy Anyway.
CEMT	CICS master terminal transaction.	Can Everybody Modify This? Hopefully not.
CECI	CICS command interpreter transaction.	Commands Entered, Consequences Immediate.
CESN	CICS sign-on transaction.	Credentials Entered, Security Nervous.
CESF	CICS sign-off transaction.	Customer Escapes Session Finally.
CEDF	CICS Execution Diagnostic Facility transaction.	Can Explain Defects Eventually, Fingers-crossed.
CEBR	CICS temporary storage browse transaction.	Can Examine Buffer Residue.
CMAC	CICS messages and codes transaction.	CICS Muttering About Codes.
CWTO	CICS write-to-operator transaction/command context.	CICS Wakes The Operator.
Db2	IBM Db2 relational database system.	Database Too: because one database acronym was never enough.
SQL	Structured Query Language.	Slightly Questionable Logic: until the WHERE clause behaves.
DDF	Distributed Data Facility.	Database Doorbell Facility: remote callers knocking politely.
DRDA	Distributed Relational Database Architecture.	Databases Remotely Debating Authority.
EBCDIC	Extended Binary Coded Decimal Interchange Code.	Every Byte Confuses Developers Incessantly, Correctly.
ASCII	American Standard Code for Information Interchange.	Almost Sensible Characters, Usually Incomplete.
VTAM	Virtual Telecommunications Access Method.	Very Traditional Access Maze.
TN3270	Telnet 3270 terminal protocol.	Terminal Nostalgia, 3270 Reasons.
AID	Attention Identifier in the 3270 data stream.	Apparently I Did-something: the one-byte clue that tells CICS which key caused the mess.
SNA	Systems Network Architecture.	Still Not Away: enterprise networking archaeology.
TCP/IP	Transmission Control Protocol / Internet Protocol.	The Classic Plumbing: It's Packets.
FTP	File Transfer Protocol.	Files Travel Poorly: especially SYS1.RACFDS.
TLS	Transport Layer Security.	Trust, Laterally Supposed: encrypted, not magically safe.
SSL	Secure Sockets Layer.	Still Saying SSL: even when we mean TLS.
REST	Representational State Transfer.	Requests Eventually Start Trouble.
API	Application Programming Interface.	Attackers Prefer Interfaces.
IMS	Information Management System.	It Might Still-run: hierarchical databases never forget.
VSAM	Virtual Storage Access Method.	Very Stubborn Access Method.
PDS	Partitioned Data Set.	Programs Definitely Somewhere.
PDSE	Partitioned Data Set Extended.	PDS, Eventually Slightly Easier.
PS	Sequential data set; commonly called a physical sequential data set in mainframe usage.	Plainly Sequential: one record after another, like an operator's sighs.
HFS	Hierarchical File System.	Historic File System: UNIX with museum privileges.
ESM	External Security Manager.	Everyone Says 'Mainframe security': usually means RACF/ACF2/TSS.
TTP	Tactics, Techniques, and Procedures.	Things Threat actors Prefer.
OSINT	Open-Source Intelligence.	Oh, So It's Not Theoretical.
IDS	Intrusion Detection System.	I Definitely Saw-that.

Acronym	What it means	Gibson's translation
SIEM	Security Information and Event Management.	Searches In Endless Messages.
SOC	Security Operations Center.	Screens, Operators, Coffee.
JWT	JSON Web Token.	Just Wave Through? Please don't.
T4M	Gibson/offensivesec context: tactics/techniques for mainframes reference.	Tactics For Mayhem: politely documented.
DVCA	Gibson context: Damn Vulnerable CICS Application / 3270 data-event lab.	Deliberately Vulnerable CICS Adventure.
ZSEC	Gibson context: zSecure-like audit command/menu surface.	Zero Sleep, Endless Compliance.

How the layers fit together

It helps to hold the whole stack in mind as a set of layers, each resting on the one below. At the bottom is the hardware (IBM Z) partitioned by PR/SM into logical partitions, the LPARs that let one machine act as many. On an LPAR runs the operating system, z/OS, and beneath the surface of z/OS sit the components you will meet chapter by chapter: JES2 managing batch work, VTAM and TCP/IP carrying the network, RACF deciding who may do what, and the data itself living in cataloged data sets and VSAM. On top of the operating system run the subsystems (CICS for online transactions, Db2 for the database) and the interactive environment, TSO and ISPF, through which you drive all of it. Every acronym in the table above slots into one of these layers, and every later chapter is really a closer look at one of them.

Layer	Example	What it does
Hardware	IBM Z	The physical machine and its processors
Partitioning	PR/SM, LPAR	Divides one machine into many logical systems
Operating system	z/OS	Runs work, manages storage, enforces security
Core components	JES2, RACF, VSAM	Batch, access control, structured data
Subsystems	CICS, Db2	Online transactions and the database
Interactive	TSO, ISPF	How people drive the whole system

The mainframe as layers, each chapter looks closely at one of them.

You do not need to memorize the stack now. The point is to know that when a later chapter talks about RACF or JES2 or VSAM, it is talking about a specific layer of one coherent system, not a loose bag of tools. That mental picture is what turns a pile of initials into an architecture you can reason about.

Why the anatomy matters for security

There is a security reason to learn the anatomy before the tools. Every attack and every defense in the later parts of this book targets a specific layer, and the layers do not protect each other automatically. RACF guards access to data sets, but it does not stop a program that already runs with too much authority. Network controls guard the ports, but they do not decide what a logged-on user may do. The subsystems (CICS, Db2) have their own security on top of the operating system's.

Because the protections are layered, so are the weaknesses. A misconfiguration at one layer (an over-permissive RACF profile, a service exposed too widely, a subsystem trusting the operating system too much) can undermine strength everywhere else. This is why a mainframe assessment is not a single check but a walk through the layers, and why understanding the anatomy is the prerequisite for understanding both the attacks in Part IX and the defenses that answer them.

Keep the layered picture in mind as you read on. Each core chapter that follows takes one layer and shows you how to operate it; each security chapter takes the same layer and shows you how it is attacked and defended. The anatomy you have just learned is the frame that holds all of it together.

What comes next

There is one more piece of groundwork before we install anything, and it is the one that trips people up the most: the vocabulary. Mainframe documentation is written in an alphabet soup (VTAM, VSAM, SMF, RACF, JES, JCL, datasets that are not quite files) and until those words mean something, every manual reads like a foreign language. In the next chapter we will learn just enough of the language to be dangerous, and see how a single piece of work flows through the city from the moment you submit it to the moment its output lands. Then, at last, we install Gibson and log on.

CHAPTER 3

The Vocabulary

By the end of this chapter you will be able to:

Decode the most common mainframe acronyms: VTAM, VSAM, SMF, RACF, JES, TSO, ISPF.

Explain how a unit of work flows from submission to output.

Read a job's lifecycle messages at a high level.

Use the right term for datasets, members, and catalogs.

Navigate mainframe documentation without drowning in jargon.

THERE IS A MOMENT, EARLY in everyone's mainframe career, when they open a manual and discover it is written in a language made entirely of capital letters. VTAM routes a request to TSO, which calls SAF, which calls RACF, while JES2 drains the spool and SMF writes a type 30 record to SYS1.MANA. Every noun is an acronym and every acronym assumes you already know the other five. It reads less like documentation than like a ransom note.

This chapter fixes that. We are going to take the words you will meet most often and pin each one to something concrete, and because this is a book about Gibson, "concrete" means real output from the simulator, not a definition you have to take on faith. You have not logged on yet; that is Part II, and in Part III you will run every command here yourself. For now, treat these as field notes: this is what the words look like once they stop being acronyms and start being things on a screen.

Datasets are not files

Start with the one that trips up every newcomer. On the mainframe, the unit of stored data is not a file. It is a dataset, and the difference is more than a word. A dataset has no folder to live in; instead its name carries its own hierarchy, written as qualifiers separated by dots, with the first qualifier (the high-level qualifier, or HLQ) acting as the owning prefix. SYS1.PARMLIB and SYS1.PROCLIB are not a PARMLIB and a PROCLIB inside a SYS1 folder. They are two complete dataset names that happen to share a first qualifier.

A dataset also has a shape, fixed when it is created: its organization, its record format, and its record length. Ask Gibson to list the system datasets and the shape is right there in the output. Here is the command you will run in Part III, and exactly what it prints:

```
LISTCAT LEVEL(SYS1)
SYS1.PARMLIB          P0 FB LRECL=80
SYS1.PROCLIB          P0 FB LRECL=80
SYS1.LINKLIB          P0 FB LRECL=80
SYS1.LPALIB           P0 FB LRECL=80
SYS1.BROADCAST        PS FB LRECL=80
```

SYS1.RACFDS	PO FB LRECL=80
SYS1.RACFDS.DATABASE	PO VB LRECL=4096
SYS1.VTAMLST	PO FB LRECL=80

A slice of the SYS1 catalog on Gibson. The real listing is longer.

Read the columns. PO means partitioned organization (a partitioned dataset, or PDS, which is the closest thing the mainframe has to a folder: a single dataset that holds named members, like SYS1.PARMLIB holding one member per configuration area. PS means physical sequential) a flat, single-stream dataset, like SYS1.BROADCAST. FB is the record format, fixed-block, and LRECL=80 is the record length, eighty characters, a number you will see everywhere because it is the width of the punched card the platform was born on. None of this is decoration. When you allocate your own datasets in Part III, you will choose exactly these attributes, and getting them wrong is one of the most common ways a job fails before it even runs.

NOTE

Two qualifiers in that listing are worth remembering already: SYS1.PARMLIB, the partitioned dataset whose members configure the running system, and SYS1.RACFDS, where the security database lives. We will come back to both, PARMLIB when we talk about how the system is built, and RACFDS when we talk about how it is broken.

The catalog finds them

If datasets have no folders, how does the system find one by name? Through the catalog, a master index that maps a dataset name to where its data actually lives. You almost never address storage directly; you name a dataset and let the catalog resolve it. The listing you just saw was the catalog answering a question: show me everything whose name begins with SYS1. Lose track of the catalog and you lose track of your data, which is why catalog management is a discipline of its own and why “it is not in the catalog” is a sentence that ruins a mainframer’s afternoon.

JCL describes the work

Most serious work on a mainframe is batch: a unit of work, a job, described ahead of time, submitted, and run without anyone watching. The description is written in Job Control Language, JCL, and for all its fearsome reputation, JCL has only a few moving parts. A JOB statement names the work and says who is paying for it. One or more EXEC statements each run a program. DD statements (data definition) connect the program’s inputs and outputs to real datasets or to the printer-like spool. Here is about the smallest useful job there is, one that copies a couple of lines of text through the system utility IEBGENER:

```
SUBMIT (the job below)
//HELLO    JOB (ACCT), 'FIRST JOB', CLASS=A, MSGCLASS=X
//STEP1    EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
```

```
//SYSUT2 DD SYSOUT=*
//SYSIN DD DUMMY
//SYSUT1 DD *
HELLO, MAINFRAME
//
```

The JOB card names it HELLO and tags it CLASS=A and MSGCLASS=X. The single EXEC step runs IEBGENER. The DD statements wire up its output to the spool with DD SYSOUT=*, its control input to nothing with DD DUMMY, and its data input to a few lines typed right into the JCL with DD *. That is a complete program in the batch sense: not much, but everything a mainframe needs to schedule, secure, run, and account for a piece of work.

JES runs it, and the spool catches the output

When you submit that job, you hand it to the job entry subsystem, JES2, which is the part of the system that turns a piece of JCL into a running, accounted-for unit of work. JES reads the job, queues it, hands it to an initiator to execute, and collects everything it prints into the spool, the shared pool of job output you browse afterward. You do not watch a batch job run. You read what it left behind. Here is what HELLO left behind, captured from Gibson exactly as the spool recorded it:

```
$HASP100 HELLO ON READER
IRR010I USERID IBMUSER IS ASSIGNED TO THIS JOB.
$HASP373 HELLO STARTED - INIT 1 - CLASS A
IEF403I HELLO - STARTED - TIME=11.39.38
IEF236I ALLOC. FOR HELLO STEP1
IEB144I THERE ARE 00000002 RECORDS IN THE OUTPUT DATA SET
IEF142I HELLO STEP1 - STEP WAS EXECUTED - COND CODE 0000
IEF404I HELLO - ENDED - TIME=11.39.38
$HASP395 HELLO ENDED - RC=0000
```

The HELLO job log on Gibson, from reader to RC=0000.

That short block is the entire life of a job, and it is worth reading line by line, because almost everything else in this book is a variation on it.

One job's journey

\$HASP100 HELLO ON READER is JES announcing that it has received the job; the \$HASP prefix marks every message JES2 itself produces, and you will learn to read them at a glance. The very next line is the one a security person should linger on. IRR010I USERID IBMUSER IS ASSIGNED TO THIS JOB is RACF stamping the work with an identity before a single instruction runs. Everything the job then does, it does as IBMUSER, checked against

IBMUSER's permissions. On a mainframe, work has an owner from the moment it is born, and that owner is the first thing an attacker wants to change.

\$HASP373 HELLO STARTED - INIT 1 - CLASS A is JES handing the job to an initiator (a worker address space) in job class A. IEF403I and the messages after it come from the operating system itself as the step runs: IEF236I allocates the datasets the DD statements asked for, IEBGENER does its copy and reports IEB144I THERE ARE 00000002 RECORDS, and IEF142I delivers the verdict every batch operator reads first, STEP WAS EXECUTED - COND CODE 0000. Zero is good. Finally JES closes the books: \$HASP395 HELLO ENDED - RC=0000. The job is done, its output is on the spool, and you can go read it.

Trace those nine lines and you have met, in miniature, half the subsystems from the last chapter doing their actual jobs: JES scheduling and accounting, RACF assigning identity, the operating system allocating and executing, a utility doing the work. And quietly, off to the side, one more subsystem was writing all of it down.

RACF says who, SMF remembers what

RACF (the Resource Access Control Facility) is the subsystem that answers one question, endlessly: is this identity allowed to do this thing? You saw it stamp the job; you can also ask it directly about a user. The command and its output look like this on Gibson:

```
LISTUSER IBMUSER
USER=IBMUSER  NAME=IBMUSER           OWNER=#SYSPROG  CREATED=24.271
DEFAULT-GROUP=SYS1      PASSDATE=25.020  PASS-INTERVAL= 30
ATTRIBUTES=SPECIAL
UACC=NONE  MODEL=NONE  SECLABEL=NONE
REVOKE DATE=NONE      RESUME DATE=NONE
PASSWORD CHANGE REQUIRED=NO
```

Most of that is routine (a default group, a password interval, when the password was last changed. One line is not routine at all: ATTRIBUTES=SPECIAL. SPECIAL is the RACF system-administrator attribute, the closest thing the security database has to a master key, and IBMUSER) the default identity you will start out as on Gibson, has it. Hold that thought; in Part IX it becomes the difference between a tester who is stuck and a tester who owns the system.

Then there is SMF, the System Management Facility, the recorder we met as the city's black box. Every meaningful thing that happened to the HELLO job (that it ran, who ran it, how long it took, what it touched) was written by SMF as records into datasets like the SYS1.MANA you saw in the catalog. A job produces a type 30 record; a security event produces a type 80. For an attacker, SMF is the witness to evade or erase. For a defender, it is the single most valuable source of truth on the system, and a whole later chapter is devoted to reading it. The lesson to carry forward is simple: on a mainframe, almost nothing happens without being written down somewhere.

The front door and the rest of the soup

A few more terms and you have enough to be dangerous. VTAM, the Virtual Telecommunications Access Method, is the front door from the last chapter (the networking layer your terminal talks to, reached over the TN3270 protocol, which carries the green-screen datastream in the mainframe's native EBCDIC character encoding rather than the ASCII the rest of the world uses. VSAM, the Virtual Storage Access Method, is the family of keyed, indexed datasets the platform uses when it needs to look records up by key rather than read them in order) the mainframe's answer to an indexed database file, and you will define one yourself in Part III. TSO is the interactive session you log on to; ISPF is the full-screen panel environment that runs on top of it. You will live in both.

That is the core of the soup. You do not need to have memorized it, the table at the end of the last chapter is there for exactly the moments you forget which three letters mean which thing. What matters is that the words now point at something real: a dataset you can list, a job you can trace from reader to return code, an identity RACF assigns and SMF records. The vocabulary has stopped being a wall and started being a map.

Checkpoint: without looking back, can you say what VTAM, RACF, JES, and SMF each do, and trace a job from submission to output? If a term still feels slippery, the acronym table is yours to revisit, fluency here makes every later command read clearly.

Mainframe documentation reads like a foreign language until the vocabulary clicks. This chapter mapped the alphabet soup (VTAM for the network, TSO and ISPF for interactive work, JES for batch, VSAM for keyed data, RACF for security, SMF for the audit trail) and followed a single unit of work from submission through execution to output. With these words in hand, the commands and screens ahead stop being cryptic and start being readable.

What comes next

That is the end of the groundwork. You know why the mainframe matters, how it is built, and what the words mean. It is time to stop reading about Gibson and start running it. In the next part we will download and install the simulator, take stock of every service and port it exposes, point a real 3270 terminal at it with c3270, and log on for the first time, forced password change, multi-factor prompt, welcome screen, and all. From there on, every command in this book is one you type yourself, and every screen is one you can reach. Let's get you logged on.

CHAPTER 4

What Gibson Is

By the end of this chapter you will be able to:

Describe what Gibson simulates and the boundaries of the simulation.

Identify the estate Gibson presents: hosts, ports, and banners.

Explain how Gibson is used for safe, repeatable security practice.

Recognize the subsystems Gibson exposes.

Set expectations for what you will and won't find versus a real z/OS.

YOU HAVE THE CONCEPTS. YOU know why the mainframe matters, how it is put together, and what its vocabulary means. Now meet the actual machine you are about to run, because Gibson is not quite what the last three chapters may have led you to expect. It has a mainframe at its heart, and everything you have learned applies. But Gibson is bigger than a mainframe. It is a whole estate: a z/OS system surrounded by the web tier, the databases, the second host, the tooling, and the deliberately broken applications that a real mainframe lives among in a real enterprise. This chapter is the map of that estate, so that when you start it up in the next part you know what you are looking at.

A mainframe, and a city around it

In Chapter 2 we toured the mainframe as a city of subsystems. Gibson keeps that city intact (VTAM, TSO and ISPF, JES2, RACF, SDSF, OMVS, SMF, the console, CICS, Db2, IMS) and then builds the rest of the enterprise around it. There is a web and API tier with its own login pages and REST endpoints. There is a vulnerable banking application that spans the green screen, the database, and the website at once. There is a second, separate host on the network for you to discover and pivot to. There is a complete reconnaissance and attack toolkit living inside the Unix side. And there is a threat-intelligence platform watching the whole thing. The point of all of it is the same: to let you learn, and test, a realistic enterprise that happens to have a mainframe at its center, safely, and as many times as you like.

The estate, room by room

The z/OS core is the subject of Parts III through V: the interactive system (TSO, ISPF, the editor), the batch system (JES2, JCL, SDSF), datasets and VSAM, the Unix side, RACF and zSecure, SMF, and the operator console. The online subsystems (CICS, Db2, and IMS) get Part VI, alongside the two other operating environments Gibson runs: z/VM, with its CP and CMS, and z/TPF, the high-volume transaction system that sits behind airline and card networks.

Then comes the part that makes Gibson more than a mainframe. A web and API tier serves a landing site, an operations dashboard, a browser-based 3270 terminal, the CBSA banking REST API, and the FIBS bank's web front end (plus a Tomcat-manager simulator for the classic web-application attacks that so often provide the first foothold. A vulnerable application stack) CBSA, FIBS, and the Damn Vulnerable CICS App (spans CICS, Db2, and the web at once, including a COBOL buffer-overflow path that produces a genuine ASRA abend. A second host, LENNOX, sits on its own port so the network is never just one box. Sysprog tooling) SMP/E, CA SYSVIEW, CA Endevor, and a software package catalog, rounds out the operations side. And inside OMVS lives the toolkit: nmap with mainframe-aware scripts, nikto, EZRecon, OSINT tools, cicspwn, db2connect, PassTicket tooling, racf2john, and more. Watching over all of it is the CTI platform, a threat-intelligence and detection console with an IOC search, a C2 tracker, a MITRE ATT&CK navigator, and an intrusion-detection engine that models a named ransomware crew's kill chain. Every one of these gets its own chapter later; for now, just know the estate is this big.

Every way in

Because Gibson is an estate, it answers on many doors. It exposes sixteen network listeners, and a real part of learning the system is knowing which port does what and how to reach it. Here is the complete map; we will use it constantly, and Chapter 7 returns to it in depth.

Port	Listener	Connect with	What answers
23	TN3270 / telnet	c3270 / x3270	VTAM banner -> TSO logon
2022	USS / OMVS	terminal client	z/OS UNIX shell
3023	z/VM	c3270 host:3023	z/VM logon -> CP/CMS
2380	LENNOX	telnet host:2380	the second training host
21	FTP	ftp host	IBM OS/390 ftpd V2R5; SITE FILETYPE=JES
50000	Db2 DRDA	db2connect	Db2 distributed access
50001	Db2 web services	HTTP	Db2 REST/web endpoint
175	NJE	NJE peer	Network Job Entry (cleartext)
2252	NJE (TLS)	NJE peer + TLS	NJE over TLS
8443	Dashboard	https://host:8443	operations dashboard (admin / gibsonadmin!)
8023	Web terminal	http://host:8023	browser-based 3270 terminal
8080	CBSA REST API	http://host:8080	banking REST API (JSON)
9080	FIBS web	http://host:9080	FIBS bank web front end
80 / 8000	Welcome	http://host	landing sites

Gibson's network listeners. The terminal you will use most is c3270 on port 23.

One detail on that table matters more than the rest and trips up newcomers, so it is worth saying plainly now: the standalone 3270 port has been removed, and c3270 and x3270 connect over port 23, which negotiates the TN3270 datastream. If you point an emulator at port 3270 expecting a green screen, you will get nothing. Point it at 23. Drawn as a picture, the whole estate hangs off that front door like this:

```

c3270 / x3270  ->  port 23  (TN3270 / VTAM front door)
|
+-----+-----+
|          GIBSON  z/OS  (one LPAR)          |
|  VTAM  TSO/ISPF  JES2  RACF  SDSF  OMVS/USS  SMF  Console  WLM/RMF/GRS  |
|  CICS   Db2    IMS   PARMLIB/PROCLIB/APF   REXX/JCL/COBOL/HLASM  |
+-----+-----+
|          |          |          |          |
z/VM :3023  z/TPF      FTP :21      NJE :175 / :2252
Db2 DRDA :50000 / :50001

```

```

Web & API tier   :  Welcome :80/:8000  Dashboard :8443  Web 3270 :8023
                  CBSA REST :8080      FIBS web :9080      Tomcat-manager sim
Vulnerable apps :  CBSA      FIBS (CICS+Db2+web)      DVCA
Second host     :  LENNOX :2380
Inside OMVS     :  nmap  nikto  EZRecon  OSINT  cicspwn  db2connect  ...
CTI platform    :  /cti  (feed, threat actors, IOC search, HMS IDS, PLONK)

```

The Gibson estate: the z/OS LPAR, the other environments, the web tier, the second host, and the toolkit.

Three ways to run it: vulnerable, secure, and no-RACF

Here is the design choice that makes Gibson a teaching tool rather than just a target. The same system can be started in three security postures, and you learn by comparing them. The default is VULNERABLE mode, the training posture, where Gibson deliberately carries the kinds of weaknesses real systems accumulate over the years: weak passwords, over-broad access, a writable APF library, default credentials. It announces itself plainly at startup:

```
GIBSON VULNERABLE TRAINING MODE ACTIVE
```

Flip the system into SECURE mode and the same Gibson hardens itself to a CIS-aligned profile: multi-factor authentication required, the modern KDFAES password algorithm, PROTECTALL(FAIL) so unprofiled datasets are denied by default, the security-relevant RACF classes active and RACLISTed, full audit, and TLS-protected terminal and web ports, with IBMUSER kept only as an audited break-glass account. Its banner tells you the posture has changed:

```

GIBSON SECURE MODE ACTIVE
CIS-ALIGNED SIMULATOR PROFILE LOADED
TSO/TN3270 TLS PORT=1023 HTTPS PORT=8443
IBMUSER BREAK-GLASS ACCOUNT ENABLED AND AUDITED

```

There is a third posture, NORACF mode, which switches RACF authorization checking off entirely, a teaching extreme that shows what a mainframe with no security manager actually behaves like:

GIBSON NORACF MODE ACTIVE - RACF AUTHORIZATION CHECKING DISABLED

NOTE

The three modes are the backbone of how this book teaches security. Almost every attack lab runs against VULNERABLE mode to show how the weakness is exploited, and then points you at SECURE mode to show the same attack failing once the control is in place. That pairing (break it here, watch it hold there) is the disciplined, defensible method this book is built on, and it is why every offensive lab ends on the defender's view.

What you will be able to do

By the time you finish this book you will be able to install Gibson and start it cleanly, connect to every door in that table, log on and move around z/OS like someone who belongs there, allocate and edit datasets, write and submit jobs and read their output, drive CICS and Db2 and the console, run the full toolkit from the Unix side, and carry out a complete, controlled security assessment of the estate, finding the weaknesses in VULNERABLE mode, and confirming the fixes in SECURE mode. You will, in other words, be able to do on Gibson everything a competent mainframe operator and a competent mainframe tester can do on the real thing, with the mainframe concepts learned from the ground up as you go.

Checkpoint: can you describe, in a sentence, what Gibson is and what it is for? Can you name the kinds of services its estate exposes, and explain why practicing on Gibson is safe where practicing on a production system is not? Those answers frame everything you are about to install.

Gibson is a self-contained z/OS training range: it answers a port scan, presents a VTAM logon, runs TSO, ISPF, JES, RACF, the Unix side, CICS, and Db2, and deliberately carries the kinds of weaknesses real systems accumulate, always paired with the defender's fix. It is realistic where realism teaches and bounded where boundaries keep you safe. Knowing what it is, and what it is not, sets the right expectations for the install, launch, and first logon that follow.

What comes next

That is the whole estate in one chapter, and it is the end of the groundwork. Part I has given you the why, the how, the vocabulary, and the map. Part II turns the map into a running system: we will install Gibson, start its services and watch it IPL, walk every listener in that table and confirm each one answers correctly, install and configure c3270 and x3270, and then log on for the first time (through the VTAM banner, the welcome screen, the forced password change, and the multi-factor prompt) and arrive, at last, at the READY prompt. Let's build the system.

Part II

Install, Launch & Connect

Part I gave you the map. Part II turns it into a running system you can log on to. We install Gibson, start its services and watch it IPL, walk every network listener and confirm each one answers correctly, set up the c3270 and x3270 terminal emulators, and then log on for the first time (through the VTAM banner, the welcome screen, the forced password change, and the multi-factor prompt) arriving at the READY prompt with a system that is genuinely yours to drive.

CHAPTER 5

Installing Gibson

By the end of this chapter you will be able to:

Install Gibson and its prerequisites on a Linux host.

Start Gibson and confirm it is listening for connections.

Locate the configuration and data that define the simulated estate.

Recover cleanly from a failed or partial start.

Verify the install before moving on to launch.

GIBSON RUNS ON LINUX. ANYTHING modern will do, but Kali is the natural home: it already carries the recon and offensive tooling you will point at the system later, and it is what Gibson is developed and tested against. You will run the simulator on that Linux host, and connect to it from a 3270 terminal emulator, which can be on the same machine or any other box that can reach it over the network. This chapter gets the simulator installed and running; the next one watches it come up properly, and the two after that connect to it.

What you need

Three things. A Linux host with Python 3 (the installer expects a version that can build a virtual environment, and will offer to install the matching venv package if it is missing. The Gibson package itself, unpacked into a directory of its own. And, on whichever machine you will sit at, a 3270 terminal emulator: c3270 or x3270, which we install in Chapter 8. Root access is optional) you only need it if you want Gibson to listen on the privileged terminal port 23, and we will come back to exactly what that means at the end of the chapter.

Running the installer

Unpack Gibson, change into its directory, and run the installer:

```
cd gibson-mainframe
./install-gibson.sh
```

It identifies itself and the project it is about to set up:

```
Gibson Mainframe Simulator v30.288-freeze installer
Project: /home/kali/gibson-mainframe
```

From there the installer does the unglamorous work so you do not have to. It checks that your Python can create a virtual environment and, on Debian- and Ubuntu-style systems such as Kali, offers to install the matching venv package, trying the version-specific name first, then falling back:

```
sudo apt-get update
```

```
sudo apt-get install -y python3.12-venv # or python3-venv
```

It then creates a self-contained virtual environment under `.venv` so Gibson's dependencies never collide with the rest of your system, and asks one real question: whether to install the optional browser-based 3270 terminal on port 8023, which uses Apache Guacamole and pulls in Docker. Answer no and Gibson is fully usable from c3270; answer yes and you also get a green screen in any web browser. Your choice is remembered in `.gibson-install.conf`, and you can change it later with the controller. If you are scripting the install, the flags do the talking instead of the prompt:

```
./install-gibson.sh --no-web-terminal --non-interactive
```

NOTE

If the installer cannot reach a package manager or lacks `sudo` (common on a locked-down or offline box) it does not fail silently. It prints exactly which package it needs and exits cleanly, and the raw simulator still runs without the browser terminal. You can finish the dependency setup later with `./gibsonctl.sh install-deps` once the access problem is resolved.

What the installer leaves behind

When it finishes, the project directory holds everything Gibson needs to run. The pieces you will actually touch are these:

```
gibson-mainframe/
  gibson/           the Python package - the simulator itself
  install-gibson.sh the installer
  gibsonctl.sh      start / stop / status / preflight controller
  .venv/            Python virtual environment (created by the installer)
  .gibson-pids/     PID files for the running services
  logs/            per-service log files
  .gibson-install.conf your install choices (e.g. the web terminal)
  docs/            the internal technical documentation
  tests/           the test suite and parity harness
```

The Gibson project layout. You drive everything through `gibsonctl.sh`.

The `gibson` directory is the simulator, the package we toured in Chapter 4. The `.venv` holds its Python dependencies. The `.gibson-pids` and `logs` directories appear once you start it: one tracks the running service processes, the other captures what each of them prints, which is the first place to look when something will not start. You rarely open any of these by hand. Almost everything you do to Gibson as an operator goes through one script.

Starting and stopping with gibbonctl

`gibbonctl.sh` is the control panel for the whole estate. It starts and stops the services, reports what is running, runs pre-flight checks, and manages the optional web terminal. Its verbs are few and memorable:

```
./gibbonctl.sh start      # bring the services up
./gibbonctl.sh status    # what is running, on which ports
./gibbonctl.sh stop      # bring everything down
./gibbonctl.sh restart   # stop then start
./gibbonctl.sh preflight  # check the host is ready before starting
./gibbonctl.sh web-status # state of the browser terminal on 8023
```

A plain start brings up the default estate, the terminal and Unix listeners, FTP, the CBSA banking API on 8080, the DVCA and FIBS web applications, and the rest of the services from the port map in Chapter 4. Under the hood the controller is simply running the simulator as a Python module with a set of service flags, which is worth seeing once even though you will never type it yourself:

```
python -m gibbon.cli --serve --with-ftp --with-cbsa-api --with-dvca --with-fibs-web
```

That is the seam between the friendly controller and the simulator proper. `gibbonctl` wraps it, tracks the process IDs, writes the logs, and gives you the simple verbs above. To check the browser terminal's login after enabling it, ask the controller to show the credentials:

```
./gibbonctl.sh web-status --show-credentials
```

A word about ports and privilege

One installation detail will save you a confusing first hour, and it concerns the terminal port. Out of the box, in its default training posture, Gibson is configured to listen for 3270 terminals on port 23 (the classic Telnet port. Port 23 is privileged: binding it requires root, so you will either start Gibson with sufficient privilege or it will use the non-privileged port 1023 instead. SECURE mode, and the controller's own defaults, lean on 1023 precisely so the system can run without root. The practical rule is simple: note which port your host actually bound the terminal listener to) status will tell you, and point your emulator there. If you followed this book's examples to 192.168.0.96 on port 23, you are running privileged; if your green screen is on 1023, that is why. Either way the experience from the emulator is identical.

NOTE

The standalone 3270 port has been removed entirely, so do not aim an emulator at port 3270, nothing listens there. Terminal access is always the Telnet/TN3270 port (23 or 1023), which negotiates the 3270 datastream on connect. We labour this point because it is the single most common reason a first connection attempt hangs.

What the install gives you

Installing Gibson puts a self-contained z/OS training environment on your machine, the emulated system, its subsystems, and the network services a real estate exposes. It runs as a Python application and listens on a set of TCP ports you connect to with a 3270 emulator, exactly as you would reach a real mainframe. Nothing about the workflow in the rest of this book depends on where Gibson runs; once it is installed and listening, you connect the same way every time.

The pieces you get, and where each is covered, are worth a quick map:

Piece	What it is
The emulated system	z/OS-style core: TSO, ISPF, datasets, JES2
Network services	FTP, TN3270, NJE, and a dashboard (Chapter 7)
A 3270 listener	The port your terminal emulator connects to
Sample data	Libraries and users to practice against

What a Gibson install provides, and where this book covers each part.

Lab 5.1

Confirm Gibson is installed and listening

Goal: verify the install by confirming Gibson is running and its terminal port is open.

Background: before you can log on you need to know the system is up and which port to point your emulator at. This lab confirms both.

1. Start Gibson as described in your install notes, and confirm it reports the port it is listening on.
2. From another window, check that the terminal port (23 by default) is open on the host.

Check yourself: Gibson reports that its listener is up, and the terminal port answers. If it does not, the service is not running or a firewall is blocking the port, fix that before moving on.

Why it matters: almost every early problem is really “not running” or “wrong port.”

Confirming both first saves a great deal of confusion at the logon screen.

Blue-Team angle: knowing exactly which ports your system exposes is the first fact a defender establishes. An install that opens more than you expected is worth investigating before anyone connects.

Prerequisites and where things live

Gibson has modest needs: a host that can run Python and the network ports free for its services. The details worth knowing before you start are few, but each one is a common source of first-day friction:

You need	Why
Python runtime	Gibson runs as a Python application
Free TCP ports	23 for terminals, plus 21/175/8443 (Chapter 7)
A 3270 emulator	To connect, x3270 or similar (Chapter 8)

You need	Why
Host reachability	Your emulator must reach the Gibson host

The short list of prerequisites for running and reaching Gibson.

The single most common mistake is a port already in use: if something else on the host is listening on a port Gibson wants, that service will not start, and you will discover it not at install time but when you try to connect. Checking the ports are free before you start turns a puzzling failure into a non-event.

Lab 5.2

Check the ports are free before you start

Goal: confirm the ports Gibson needs are not already in use, before launching it.

Background: a port conflict is the classic silent install failure. Checking first is faster than debugging later.

1. Before starting Gibson, check whether anything is already listening on its ports.

```
ss -tlnp | grep -E ':(21|23|175|8443)'
```

Check yourself: ideally nothing is listening on those ports yet. If something is, that port is taken, stop the conflicting service or change Gibson's configuration before launching.

Why it matters: catching a port conflict before launch saves you from a service that silently fails to start and a connection that mysteriously refuses.

Blue-Team angle: knowing precisely which ports should be listening (and which should not) is the baseline a defender compares against. An unexpected listener is exactly what this check surfaces.

The settings you may need to change

Most installs run with defaults, but a few settings are worth knowing about because they are the ones you change when the defaults do not fit your host:

Setting	When you change it
Terminal port	Something else already uses port 23
Dashboard port	Port 8443 is taken, or you want it elsewhere
Bind address	You want Gibson reachable from other machines
Sample data	You want a clean estate or a seeded one

The handful of settings most installs ever touch.

The pattern to remember is that every service has a port and an address, and every port conflict has the same fix: change the setting or free the port. Once you have connected successfully once, you rarely touch any of this again.

Lab 5.3

Find and read your configuration

Goal: locate Gibson's configuration and identify the ports and address it will use.

Background: knowing where the settings live turns “it will not connect” into a two-minute check rather than a guessing game.

1. Locate the configuration Gibson reads at startup and read the port and address values.

Check yourself: you can state which port your terminal emulator should target and which address Gibson binds to. Those two facts are all you need to connect in Chapter 8.

Why it matters: the connection details you confirm here are exactly what you type into your emulator next chapter. Reading them now prevents the most common first-connection failure.

Blue-Team angle: configuration that binds a service to all addresses rather than localhost widens its exposure. Reading the bind address is a small habit with real security weight.

A clean start, confirmed

When everything is right, a check of the ports before launch shows them free, and the launch then claims them one by one. A healthy “before” looks like silence, nothing listening yet:

```
$ ss -tlnp | grep -E ':(21|23|175|8443)'
```

```
$
```

```
# (no output: the ports are free)
```

```
$ ./launch-gibson
```

```
Gibson starting...
```

```
terminal listener : 0.0.0.0:23
```

```
ftp service       : 0.0.0.0:21
```

```
nje listener      : 0.0.0.0:175
```

```
dashboard         : 0.0.0.0:8443
```

```
Gibson ready. Connect a 3270 emulator to port 23.
```

A clean launch: the ports were free, and Gibson has claimed them.

That final line is the one you are waiting for, it tells you the terminal listener is up and names the port to point your emulator at. From here, Chapter 8 connects to it.

Lab 5.4

Verify every service is reachable

Goal: confirm each service Gibson started is actually listening, not just reported as started.

Background: a service can report STARTED and still be unreachable if it bound to the wrong address. Checking each port from where you will connect closes that gap.

1. From the machine you will connect from, test each of Gibson’s ports in turn.

```
nc -z -v <gibson-host> 21 23 175 8443
```

2. Note which ports answer and which do not.

Check yourself: each port you expect (21, 23, 175, 8443) reports open from where you are testing. A port that Gibson reported as started but that does not answer from your machine points to a bind-address or firewall problem, not a start failure.

Why it matters: “started” and “reachable from where I am” are different claims. Testing from the connecting machine is the check that actually predicts whether you can log on.

Blue-Team angle: the same per-port reachability test is how a defender confirms that a service is exposed only where it should be, and catches one that is reachable from somewhere it should not be.

When a launch fails

It is worth seeing a failed launch too, because the failure is specific and the fix is quick. When a port Gibson wants is already in use, the launch stops and says so plainly:

```
$ ./launch-gibson
Gibson starting...
    terminal listener : 0.0.0.0:23
ERROR: could not bind port 23: address already in use
Gibson did not start.
$ ss -tlnp | grep ':23'
LISTEN 0 128 0.0.0.0:23 users:(("inetd",pid=812))
```

A failed launch names the port and the reason, here, port 23 already in use.

The error names the exact port and the reason, and a quick check shows what already holds it, in this case a stray inetd. The fix is to stop the conflicting service or move Gibson to another port, then launch again. A launch failure is almost never mysterious once you read the line it printed.

Checkpoint: is Gibson installed, starting cleanly, and listening on the expected port? Can you find where its configuration lives and restart it from a clean state? If so, you have a working range to practice on for the rest of the book.

Installing Gibson turns the concepts of Part I into a machine you can actually touch. With the prerequisites in place, the service starts, listens, and presents the simulated estate you will spend the rest of the book exploring. A clean, repeatable install (and knowing how to reset it) is what makes the labs ahead safe to fail in.

What comes next

Gibson is installed and you know how to start, stop, and inspect it. But starting a mainframe is not like starting an app, it boots, and a real mainframe boot has a ritual: the initial program load, the operator replies, the services binding their ports one by one. In the next chapter we run `./gibsonctl.sh start` for real, watch Gibson IPL, answer the startup prompts including the multi-factor PIN, choose between vulnerable and secure mode, and confirm through the dashboard and a status check that the system is genuinely up before we ever point a terminal at it.

CHAPTER 6

Launching & IPL

By the end of this chapter you will be able to:

Start Gibson and follow the IPL sequence to a ready system.

Recognize the key IPL messages and what each milestone means.

Confirm that the core subsystems have come up.

Tell a healthy IPL from one that has stalled.

Know where to look when the system does not come up.

STARTING A MAINFRAME IS NOT like double-clicking an application. A mainframe boots, and the boot has a name and a ritual: the initial program load, or IPL. During an IPL the nucleus comes up, the system reads its parameters, the subsystems start in order, and (this is the part that surprises everyone) the machine stops part way through and asks the human operator questions it will not proceed without answering. Gibson reproduces that ritual faithfully, because learning to read an IPL is learning to read the health of a mainframe. In this chapter we start Gibson for real, watch it IPL, answer the operator prompts, choose its security posture, and confirm it is genuinely up before we point a terminal at it in the chapters that follow.

Bringing it up

From Chapter 5 you already know the command. It is worth running pre-flight first (it checks the host is ready, the ports are free, and nothing is half-running) and then starting:

```
./gibsonctl.sh preflight
./gibsonctl.sh start
```

Behind that single verb, Gibson begins its IPL and the master console comes to life. If you launch the interactive console you watch this happen and drive it yourself; a non-interactive start answers the boot prompts for you with safe training defaults so the system comes up unattended. Either way, the first thing the console shows you is its status panel and the opening moves of the IPL.

The IPL console

Here is what the master console looks like the moment Gibson starts to boot, captured exactly as it renders:

```
+----- GIBSON HERCULES-STYLE CONTROL -----+
| DATE 2026-06-30  TIME 13:09      PROCESSING IDLE      |
| LOADED VOLUMES: WORK01                                     |
| SERVICES STARTED: 12/13  WARNINGS: 0  POWER: ON        |
```

```
| COMMANDS: POWER OFF / Z EOD for simulated shutdown |
+-----+
```

```
IEA794I IBM Z/OS MASTER CONSOLE CONS01 ACTIVE FOR SYSTEM GIB1 AT 2026.181 13:09:41
IEA168I GIBSON IPL INITIATED - NUCLEUS INITIALIZATION PROGRAM ACTIVE
```

```
IEE600I REPLY TO OUTSTANDING IPL REQUESTS WITH R nn,answer
IEE112I 13.09.41 PENDING REQUESTS
R 01 IEA101A SPECIFY SYSTEM PARAMETERS FOR RELEASE z/OS 02.05.00
```

```
IEE600I ENTER ? FOR AVAILABLE COMMANDS
```

The Gibson master console mid-IPL, holding for the operator's reply.

Read it from the top. The status box is Gibson's control panel, the date and time, the loaded disk volumes, how many services have started (twelve of thirteen here), and that the power is on. Below it the IPL narrates itself in the real message numbers a z/OS operator knows by heart. IEA794I announces that the master console is active for the system, here named GIB1. IEA168I says the IPL has begun and the nucleus initialization program is running. And then the boot stops, because of the three lines in the middle.

Write to operator, with reply

IEE600I REPLY TO OUTSTANDING IPL REQUESTS WITH R nn,answer is the system telling you it needs input. The mechanism is the WTOR (write to operator with reply) and it is one of the most important things to understand on a mainframe, because the operator is not a spectator. When the system needs a decision it cannot make alone, it issues a numbered request and waits, possibly forever, for a human to answer. IEE112I PENDING REQUESTS lists what is outstanding, and the request itself is numbered for reply:

```
R 01 IEA101A SPECIFY SYSTEM PARAMETERS FOR RELEASE z/OS 02.05.00
```

That is reply number 01, and the message is IEA101A asking which system parameters to IPL with, which IEASYS member of PARMLIB to use. You answer with R, the reply number, a comma, and your answer. To accept the default member you reply:

```
R 01,IEASYS=00
```

Depending on the posture you are starting, a second request may follow: a prompt for the IPL multi-factor PIN, which you answer the same way (R, its reply number, and the PIN. On the training default that PIN is 1234, and a non-interactive start supplies it automatically. This same reply-numbered mechanic) R nn,answer, is how you will answer every operator request for the rest of the book, so it is worth doing once by hand to feel it.

NOTE

A WTOR that nobody answers will hold the boot indefinitely, that is the point of it. If you ever start Gibson and it seems to hang during IPL, it is almost certainly waiting on a reply. Enter the master

console, look for the PENDING REQUESTS, and answer them with R nn,answer. A mainframe that appears stuck is very often a mainframe waiting politely for a human.

Boot complete

Once the requests are answered, the IPL runs to completion and the console fills with the subsystems reporting in. This is the city of Chapter 2 waking up, one neighborhood at a time:

```
IEA101I SYSTEM PARAMETERS ACCEPTED
IEA347I IEASYS PARMLIB MEMBER SELECTION COMPLETE
IEE254I IPLINFO DISPLAY
SYSTEM IPLED AT 2026.181 13:09:41
RELEASE z/OS 02.05.00 GIBSON TRAINING LPAR
$HASP426 SPECIFY OPTIONS - HASP-II, VERSION SIMULATED
$HASP373 JES2      STARTED
IRR54001I RACF SUBSYSTEM INITIALIZED
BPXM010I OMVS INITIALIZATION COMPLETE
EZZ6035I TCPIP PROC STARTED
IEA602I OPERATOR COMMAND PROCESSING ACTIVE
IEA630I SYSTEM NAME GIBSON
```

IPL complete: parameters accepted and the subsystems started.

Every line is a milestone. IEA101I confirms the parameters you supplied were accepted and IEA347I that the IEASYS member selection finished. IEE254I prints the IPLINFO (when the system was IPLed and that this is the z/OS 02.05.00 Gibson training LPAR. Then the subsystems: \$HASP373 JES2 STARTED brings up the batch system, IRR54001I initializes RACF, BPXM010I completes OMVS) the Unix side, and EZZ6035I starts TCP/IP, which is what makes all those network listeners possible. IEA602I declares that operator command processing is active, so the console will now take your commands, and IEA630I gives the system its name: GIBSON. The machine is up.

Vulnerable, or secure

The IPL is also where Gibson's security posture is decided. Start it plainly and it comes up in VULNERABLE training mode, the default we will use for most of this book. Start it with the secure flag and it applies the hardened, CIS-aligned profile instead:

```
python -m gibbon.cli --serve --secure      # hardened posture
python -m gibbon.cli --serve --vuln       # default training posture
```

You met the three banners in Chapter 4 (VULNERABLE, SECURE, and NORACF) and this is the moment they are chosen. The banner that prints at startup is your confirmation of which world you are now operating in, and therefore which weaknesses are present and

which controls are enforced. For now, start in the default mode; we will deliberately switch to secure mode later to watch attacks fail.

Confirming it is really up

Never reach for a terminal until the system says it is ready. The console's status box already tells you a great deal (services started, warnings, power) and the controller gives you the same picture from the host shell:

```
./gibsonctl.sh status
```

It reports which services are running and on which ports, which is your cross-check against the listener map from Chapter 4. For a friendlier view there is the operations dashboard, served over HTTPS on port 8443 and protected with HTTP basic authentication (the default login is admin with the password gibsonadmin!) where the running services, the connected users, and the security alerts are laid out visually. When the status check is clean and the boot-complete messages have scrolled past, the system is genuinely ready, and only then is it time to connect.

Hands-On Labs

These labs run from the host shell where Gibson is installed.

Lab 6.1

Cold-start Gibson and reach a ready system

Goal: start Gibson and follow the IPL to a ready z/OS.

Background: every session begins by bringing the range up, and reading the IPL is how you know it is healthy before you connect.

1. Start Gibson using the launch command from Chapter 5, and watch the IPL messages scroll.
2. Wait for the system to settle and the TN3270 listener to report active.

```
IST001I TN3270 LISTENER ACTIVE ON PORT 3270
```

Check yourself: the IPL completes and the listener-active message appears. The system is ready once the console settles and the listener is up.

Why it matters: a clean IPL is your assurance the whole estate is available. If the IPL stalls, nothing downstream will work, and you want to know that first.

Blue-Team angle: IPL and console messages are the system's own narration. Defenders baseline a normal start so an abnormal one (a missing subsystem, an unexpected message) stands out.

Lab 6.2

Confirm the listener port

Goal: confirm the TN3270 listener is up and note the port your terminal must use.

Background: a ready system still needs its TN3270 listener running before any terminal can connect.

1. In the launch output, find the listener-active line and read the port number.

Check yourself: the port in the IST001I message is the one you will point c3270/x3270 at in the next chapter. Matching them avoids most connection problems.

Why it matters: most “why won’t it connect” confusion is simply a terminal pointed at the wrong port.

Blue-Team angle: an unexpected listener on an unexpected port is exactly what a defender hunts for, it can be a backdoor or a misconfiguration.

Watching the system come up

When Gibson starts, it brings its services up one at a time and narrates each on the console, exactly as a real system does during and after IPL. The started tasks announce themselves with the same JES2 and console messages you will read everywhere else:

```
$HASP100  FTPD      ON STCINRDR
$HASP373  FTPD      STARTED
IEE352I  GIBDASH STARTED ON PORT 8443
IEE352I  WEBTERM STARTED ON PORT 3271
```

Services announcing themselves on the console as the system comes up.

Each line is a service reporting its state: \$HASP100 and \$HASP373 are JES2 announcing a started task entering the system and beginning to run, and the IEE352I lines are services reporting the port they are now listening on. Reading the startup console is how an operator confirms the system came up cleanly, every service that should be running has said so, on the ports it should be using.

Lab 6.3

Read the startup console

Goal: start the system and read the console to confirm each service came up on its expected port.

Background: the startup console is the first health check. Reading it fluently means you notice at once when something did not start.

1. Start Gibson and watch the console messages as the services come up.
2. Note each STARTED message and the port it reports.

Check yourself: you see a STARTED line for each service, and the ports match what you expect, the terminal listener, the dashboard, and any others. A service that never reports STARTED did not come up.

Why it matters: “did everything start?” is a question you will ask every time you bring a system up. The console answers it, service by service.

Blue-Team angle: the startup console is a baseline. A defender who knows the normal set of started tasks and ports spots an extra, unexpected service the moment it announces itself.

Controlling the started tasks

The services that come up at startup are started tasks, and an operator can pause, stop, and restart them. The console reports each transition with the same message ids you saw at startup:

```
IEE334I GIBDASH STOPPED
IEE341I GIBDASH PAUSED
$HASP395 FTPD      ENDED
```

Services reporting that they have been paused, stopped, or ended.

IEE341I reports a service paused, IEE334I reports one stopped, and \$HASP395 ENDED is JES2 confirming a started task has finished. Being able to read these (and to bring a service back) is the core of operating a system day to day: you do not just start everything once, you manage services through their lifetimes.

Lab 6.4

Stop and restart a service

Goal: stop a non-essential service, confirm it went down on the console, and bring it back.

Background: services are managed, not just launched. Taking one down and back up cleanly is a basic operating skill.

1. Stop a non-essential service (such as the dashboard) and watch for its STOPPED message.
2. Start it again and confirm the STARTED message and its port.

Check yourself: the console shows IEE334I ... STOPPED when it goes down and a STARTED line with its port when it comes back. The service is unreachable in between.

Why it matters: real operating is a cycle of stopping, changing, and restarting services. The console is how you confirm each step actually happened.

Blue-Team angle: unexpected stops and starts of a service can be routine maintenance, or tampering. A defender correlates these console events with who issued them and when.

From power-on to READY: the phases

A real system reaches a usable state in stages, and it is worth knowing the sequence even in a simulator, because the vocabulary appears throughout operations. First the hardware and the LPAR are readied; then the operating system is loaded (the IPL, Initial Program Load) during which core components initialize; then the subsystems and started tasks come up, each announcing itself; and finally the interactive services are listening and you can log on. The console you just read is the tail of that sequence: services reporting that they have started.

Phase	What happens
Hardware / LPAR	The partition is activated and given resources
IPL	The operating system is loaded and initializes
Subsystems	JES2 and others start and announce themselves
Services up	Terminal and network listeners are ready

From power-on to a usable system, in four phases.

You will not drive an IPL in this book, but you will hear the term constantly, and knowing it names “load and start the operating system” keeps later chapters clear.

A full startup, line by line

Put together, a full startup reads as a sequence of services entering the system and reporting ready. This is the console you learn to skim for anything missing:

```
$HASP100 FTPD      ON STCINRDR
$HASP373 FTPD      STARTED
IEE352I FTPD       STARTED ON PORT 21
$HASP100 NJESERV   ON STCINRDR
$HASP373 NJESERV   STARTED
IEE352I NJESERV    STARTED ON PORT 175
IEE352I GIBDASH    STARTED ON PORT 8443
IEE352I WEBTERM    STARTED ON PORT 3271
IEE352I TN3270     STARTED ON PORT 23
*** GIBSON READY ***
```

A full startup console: every service reporting the port it now serves.

Reading top to bottom, each service enters the system and then reports its port. The final ready marker means the terminal service is listening and you can log on. A missing service is a gap in this list, which is exactly why an operator reads it in full rather than glancing at the end.

Checkpoint: can you start Gibson, read the IPL messages, and say which subsystems are up once the system is ready? Can you spot an IPL that has not completed? Reading a boot sequence is a skill you will use every time you bring the range up.

An IPL (Initial Program Load) is the mainframe’s boot, and watching one tells you a great deal about the system. This chapter followed Gibson from a cold start through the milestone messages to a ready z/OS, and showed how to confirm the subsystems you will need are running. Being able to read the launch sequence means you always know whether the range is healthy before you start work.

What comes next

Gibson is up and you can prove it. But “up” means sixteen separate listeners, each its own door, and before you walk through any of them it pays to know what every one of

them is and how to confirm it is answering correctly. The next chapter takes the port map from Chapter 4 and makes it real: we touch each listener in turn, see the response it gives (including the z/OS-accurate fingerprints a scanner sees) and build the mental model of Gibson as a network of services. Then, in Chapter 8, we install the terminal that gets us through the most important door of all.

CHAPTER 7

The Network Map

By the end of this chapter you will be able to:

Scan Gibson to discover its exposed services.

Identify mainframe services from their ports and banners.

Distinguish the TN3270, FTP, and Unix-side services.

Build a picture of the estate before logging on.

Record findings the way a tester documents reconnaissance.

GIBSON IS UP, AND “UP” means it is answering on a lot of doors at once. Chapter 4 gave you the list; this chapter makes it real. We are going to behave exactly the way a tester behaves on the first day of an engagement, not log in, not assume, but knock on every door and listen to what answers. By the end you will know which port carries which service, what each one says when you touch it, and how to confirm it is answering correctly. As a bonus, everything Gibson tells a scanner is deliberately matched to what a real z/OS system says, so the recon you practice here is the recon you would do for real.

Listing what is listening

The single most useful first move against any host is a version scan. From the Unix side of Gibson, or from any machine that can reach it, nmap with the -sV flag asks every open port not just whether it is open but what is running there:

```
nmap -sV 192.168.0.96
```

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-30 13:36 UTC
```

```
Nmap scan report for mainframe (192.168.0.96)
```

```
Host is up (0.0029s latency).
```

PORT	STATE	SERVICE	VERSION
21/tcp	open	ftp	IBM OS/390 ftpd V2R5
2023/tcp	open	tn3270	IBM Telnet TN3270 (Gibson dual-mode)
80/tcp	open	http	Gibson Welcome/CTI
8080/tcp	open	http	Apache Tomcat Manager (Gibson)
9080/tcp	open	http	FIBS Security Academy
50000/tcp	open	drda	IBM DB2 Database Server (QDB2)

```
Nmap done: 1 IP address (1 host up) scanned in 0.04 seconds
```

An nmap -sV scan of Gibson. Every version string is a z/OS-accurate fingerprint.

That one block is the map of the estate as an outsider sees it, and it repays careful reading. Port 21 is FTP, and nmap reports it as IBM OS/390 ftpd V2R5 (a genuine z/OS FTP signature, not a generic one. The terminal listener answers as IBM Telnet TN3270 in Gibson's dual-mode. Three HTTP services identify themselves distinctly: the Gibson Welcome and CTI site, an Apache Tomcat Manager, and the FIBS Security Academy. And port 50000 is DRDA) the distributed database protocol, fingerprinted as an IBM Db2 database server. Six ports, and already you can see the shape of the whole enterprise: a mainframe with FTP and a terminal, a web tier, and a database.

NOTE

Look at the port the terminal answered on: 2023, in this scan. The terminal listener's port depends on how Gibson was launched, 23 when it is started with privilege (the classic Telnet port), 1023 in secure mode and non-root starts, or a raw 2023 in some controller configurations. This is exactly why you scan instead of assume. Confirm the live port with `./gibsonctl.sh status` or a scan, and point your emulator there. If you followed this book to 192.168.0.96 on port 23, that is the privileged default.

FTP, and the door into the spool

Knock on port 21 directly and the banner nmap summarized appears in full:

```
ftp 192.168.0.96
```

```
220-GIBSON IBM FTP CS V2R5 at GIBSON, 13:36:16 on 2026-06-30.
```

```
220 Connection will not be allowed to remain inactive for more than 5 minutes.
```

This is not a generic FTP server, and the FEAT command (which asks a server what extensions it supports) proves it. The reply is unmistakably z/OS:

```
211-Extensions supported:
```

```
EPSV
```

```
PASV
```

```
SITE FILETYPE=FILE
```

```
SITE FILETYPE=JES
```

```
SITE FILETYPE=SQL
```

```
SITE JES STATUS
```

```
SITE JES PURGE
```

```
SIZE
```

```
HELP
```

```
211 End
```

Most of those are ordinary, but three lines should make a security reader sit up: `SITE FILETYPE=JES`, `SITE JES STATUS`, and `SITE JES PURGE`. On z/OS, FTP is not only a file-transfer service, it is a way to submit, monitor, and purge batch jobs. Setting the file type to JES and uploading a piece of JCL submits it to the system. That single capability turns the FTP port into a job-submission channel, and we will return to it as a real technique in Chapter 35. For now, note that the SYST command confirms the personality: 215 UNIX

Type: L8, a Gibson-simulated z/OS FTP service. Everything answers the way the real thing would.

Enumerating the applications behind VTAM

The terminal port hides more than a login screen. Behind VTAM sit named applications (the APPLIDs from Chapter 2) and nmap's mainframe scripts can enumerate them. The vtam-enum script guesses application identifiers and reports which the system accepts:

```
nmap -p 2023 --script vtam-enum 192.168.0.96
PORT      STATE SERVICE
2023/tcp  open  tn3270
| vtam-enum:
|   VTAM Application ID:
|     applid:TSO  - Valid application
|     applid:CICS - Valid application
|     applid:DB2  - Valid application
|_ Statistics: Performed 24 guesses in 1.2 seconds, average tps: 20.0
```

Three valid applications come back: TSO, CICS, and Db2. To a tester that is a target list, the interactive system, the transaction monitor, and the database, each reachable through the terminal front door. The scripts that ship with Gibson's nmap cover the mainframe-specific ground a generic scanner misses: terminal-screen capture, TSO userid enumeration, CICS discovery, and more, all reached through the menu engine we use in Chapter 42.

The database, fingerprinted

Port 50000 deserves its own probe. A Db2-aware script reads the server's identity straight off the DRDA listener:

```
nmap -p 50000 --script db2-das-info 192.168.0.96
PORT      STATE SERVICE VERSION
50000/tcp  open  drda      IBM DB2 Database Server (QDB2)
| db2-das-info:
|   DB2 Version: DSN12015
|   Server Platform: QDB2
|   Instance Name: ZDB2A
|   Location Name: GIBSONDB
|_ Security: RACF external authentication
```

The detail is realistic to the point of being useful: a Db2 version of DSN12015, the location name GIBSONDB, and (the line that matters) security handled by RACF external authentication, telling you the database defers its access decisions to the same security

manager as the rest of the system. Recon like this is not idle; every fact here shapes how you would approach the database in Chapter 30.

The web tier and the other doors

The three HTTP ports are quick to touch with curl, and each has a distinct purpose. Port 80 is the welcome and CTI site; port 8080 presents an Apache Tomcat Manager, the classic web-app foothold; port 9080 is the FIBS bank's web front end. Two more web doors do not always show on a basic scan but matter: the operations dashboard on 8443 over HTTPS, and the browser-based 3270 terminal on 8023. Beyond the web there are the other protocol doors from the Chapter 4 map, z/VM on 3023, the second host LENNOX on 2380, NJE on 175 and its TLS partner on 2252, and the dedicated USS listener on 2022. Each gets its chapter later; the point now is simply that you can confirm every one of them is alive:

```
curl -s http://192.168.0.96:8080/ | head
nc -v 192.168.0.96 175      # NJE: silent until the OPEN record
./gibsonctl.sh status      # the authoritative list of what is up
```

Confirming each answers correctly

That is the whole discipline of this chapter in three moves: scan to see what is open, touch each port to read what it says, and cross-check against the controller's status so you know the map matches reality. Do this once after every start and you will never be surprised by a service that did not come up or a port that bound somewhere unexpected. And because Gibson's responses are matched to genuine z/OS signatures (the OS/390 FTP banner, the TN3270 identity, the Db2 fingerprint, the VTAM applications) the muscle memory you build here transfers directly to a real assessment, where reading these same fingerprints is how you learn what a mainframe is before you ever touch it.

Hands-On Labs

These labs run from the host shell, scanning the Gibson address (use yours).

Lab 7.1

Scan the estate

Goal: use nmap to discover the services Gibson exposes.

Background: reconnaissance precedes everything. You map the estate before you log on.

1. From the host, run a service-version scan against Gibson.

```
nmap -sV 192.168.0.96
```

Check yourself: nmap reports port 23 as a tn3270 service (the VTAM logon) and port 21 as ftp, among others. The tn3270 service on 23 is your way in.

Why it matters: identifying the TN3270 port from a scan is the first concrete step of a mainframe assessment.

Blue-Team angle: defenders watch for service-version scans against the TN3270 and FTP ports, and restrict who on the network can reach them at all.

Lab 7.2

Inventory the services

Goal: turn open ports into an inventory of what each service is and why it matters.

Background: a port number alone is not enough, you need to know what speaks on it.

1. From the scan, record each open port with its service: 21 (FTP), 23 (TN3270 / VTAM logon), 175 (NJE).

Check yourself: for each open port you can say what protocol it speaks and why it matters, the TN3270 port is the interactive front door, FTP a file path, NJE a job-network path.

Why it matters: this inventory is the map every later chapter navigates.

Blue-Team angle: the smaller the exposed surface, the better. A defender justifies every open port and closes the ones no business need requires.

The services on the wire

Seen from the network, the Gibson estate is a set of listening services, each on its own port with its own protocol, exactly what a scan of a real mainframe reveals. This is the map an attacker draws first and a defender knows by heart:

Port	Service	What it is
21/tcp	ftp	Gibson FTPD, file transfer, and a data path
23/tcp	tn3270	Gibson VTAM TN3270, the 3270 terminal service
23/tcp	tn3270	VTAM-compatible line-mode listener
175/tcp	nje	IBM Network Job Entry (JES2)
2252/tcp	ssl/nje	Network Job Entry over TLS
8443/tcp	http	Gibson Dashboard

The Gibson estate as the network sees it: ports, services, and protocols.

Each of these is a door. TN3270 on 23 is how you log on; FTP on 21 moves files and is a classic data path; NJE on 175 (and its TLS form on 2252) carries jobs between systems; the dashboard on 8443 is a web view. Knowing which services exist, on which ports, is the whole point of a network map, you cannot secure or attack what you have not first enumerated.

Lab 7.3

Map the Gibson host

Goal: scan the Gibson host and identify each listening service and the software behind it.

Background: enumeration is the first phase of both an assessment and an attack. A service scan turns an IP address into a map of doors.

1. Run a service and version scan against the Gibson host.

```
nmap -sV <gibson-host>
```

Check yourself: the scan lists the ports above with their services (ftp on 21, tn3270 on 23, nje on 175 and 2252, http on 8443) and version banners such as “Gibson VTAM TN3270.” That set is your network map.

Why it matters: every later recon and kill-chain chapter starts from this map. Knowing the estate’s ports and services is the foundation everything else builds on.

Blue-Team angle: the defender runs the same scan, to know exactly what is exposed, to catch a service that should not be open, and to compare today’s map against the known baseline. Enumeration is a control, not just an attack step.

What a service tells you before you log in

Services announce themselves to anyone who connects, before any authentication. Connect to the FTP port and it greets you with a banner:

```
220-GIBSON FTP service at MVSC, 14:02:59 on 2026-07-01.
```

```
220 Connection will close if idle for more than 5 minutes.
```

An FTP banner, information a service gives away before you authenticate.

A banner like this is a gift to whoever is mapping the estate: it confirms the service is really FTP, hints at the host name, and sometimes reveals software and version. Banner grabbing (collecting these greetings across every open port) is how a map of ports becomes a map of exactly what is running, and it is a standard early move in any assessment.

Lab 7.4

Grab a service banner

Goal: connect to an open port and read the banner the service presents before authentication.

Background: banners turn “a port is open” into “this exact service is running here,” which is what recon is really after.

1. Connect to the FTP port and read the greeting.

```
ftp <gibson-host>
```

Check yourself: the service returns a 220 banner naming itself as the Gibson FTP service, before it ever asks for a username. That banner is intelligence you collected without authenticating.

Why it matters: banners are how enumeration gets specific. The more a service volunteers, the more an attacker knows before trying anything.

Blue-Team angle: verbose banners leak information for free. A defender minimizes what services reveal and watches for the connection pattern (many ports touched briefly) that betrays banner grabbing.

Which doors matter most

Not every open port carries the same weight. Ranking them by what they expose is how a defender prioritizes and how an attacker chooses a first move:

Service	Why it matters
TN3270 (23)	The interactive front door, where credentials are used
FTP (21)	A data path in and out, files, and sometimes JCL
NJE (175/2252)	Job flow between systems, a trust relationship
Dashboard (8443)	A web surface, a different class of exposure

The estate's services ranked by what each one exposes.

TN3270 is where identity is proven, so it is the front line for credential attacks and monitoring alike. FTP and NJE are data and job paths, quieter, but powerful if abused. The dashboard is a web surface with web-style risks. A map is only the start; understanding what each door leads to is what makes it useful.

The scan, in full

Run against the host, a service scan lays the whole estate out at once, the map every later recon chapter starts from:

```
Starting Nmap against gibson-host
PORT      STATE SERVICE  VERSION
21/tcp    open  ftp      Gibson FTPD
23/tcp    open  tn3270   Gibson VTAM TN3270
175/tcp   open  nje      IBM Network Job Entry (JES2)
2252/tcp  open  ssl/nje  IBM Network Job Entry (JES2 over TLS)
8443/tcp  open  http     Gibson Dashboard
Nmap done: 1 host up, 5 services identified
```

A full service scan of the Gibson estate, ports, services, and versions in one view.

Each row is a confirmed door with the software behind it named. This single view (five services, their ports, their versions) is the network map in its most useful form, and it is worth keeping beside you through Parts VIII and IX.

Checkpoint: from a scan alone, can you list the services Gibson exposes and say what each is for? Can you tell the TN3270 logon port from the others? Reconnaissance is the first move in any assessment, and the map you build now guides everything that follows.

Before you log on, you map. This chapter scanned Gibson the way a tester scans an unknown estate, turning open ports and banners into an inventory of services (the TN3270 front door, FTP, the Unix-side daemons) and a sense of what the system is. A careful network map is both good tradecraft and the orientation you need before the first logon.

What comes next

You have knocked on every door and you know which one matters most: the terminal port, where a 3270 session and the path to TSO are waiting. To walk through it you need the right tool, a 3270 emulator that speaks the EBCDIC datastream. The next chapter installs and configures c3270 and x3270, explains the model and keyboard map, shows the macros that make them comfortable, and gets you a green screen pointed at Gibson. After that, we log on.

CHAPTER 8

Connecting with c3270 & x3270

By the end of this chapter you will be able to:

- Install the c3270 and x3270 terminal-emulator family.
- Connect to Gibson's TN3270 port and reach the VTAM logon.
- Use the PF, PA, and Reset keys and clear the X-SYSTEM lock.
- Choose between the graphical and character emulators.
- Fall back to an ASCII path when 3270 is unavailable.

OF ALL SIXTEEN DOORS, ONE matters more than the rest, and you cannot walk through it with an ordinary terminal. The mainframe speaks 3270, a block-mode, EBCDIC-encoded datastream that looks nothing like the line-at-a-time terminals the rest of the computing world uses. To reach a green screen you need an emulator that speaks it. The two you will use are c3270, a text-mode emulator that runs in any terminal window, and x3270, its graphical sibling. They come from the same suite, behave identically once connected, and this chapter gets one of them talking to Gibson.

Installing the emulators

On Kali and other Debian-based systems the emulators are a single package install. The suite brings c3270 and x3270 along with the scripting tools you will meet later:

```
sudo apt-get install -y c3270 x3270
```

That gives you c3270 (the text emulator), x3270 (the graphical one), s3270 (a scriptable engine with no display, for automation), and pr3287 (a printer emulator). For learning, c3270 in a terminal is the quickest to start with; x3270 is nicer when you want real PF-key buttons and a resizable window.

Your first connection

Point c3270 at Gibson's terminal port (port 23 in this book's setup) and connect:

```
c3270 192.168.0.96:23
```

The emulator negotiates the 3270 datastream and the screen fills with Gibson's VTAM logon panel. This is the front door, and it looks like this:

```
*****
*      GIBSON PRODUCTION LPAR      *
*****
```

THIS SYSTEM CONTAINS RESTRICTED INFORMATION FOR AUTHORISED USERS ONLY

```

THE SYSTEM MAY BE MONITORED, RECORDED, AND IS SUBJECT TO AUDIT
UNAUTHORIZED USE OF THE SYSTEM IS PROHIBITED UNDER SCOTTISH LAW.
USE OF THE SYSTEM INDICATES CONSENT TO MONITORING AND RECORDING.
-----

```

G I B S O N T R A I N I N G

```

LU NAME:  LU320      CLIENT IP: 192.168.0.50  CLIENT PORT: 51234
SENSE CODE: AC/00          SERVICE PORT: 23
-----

```

```

GTS0v30.227          GDb30.227          GICS v0.30

```

```
LOGON using L TSO, L CICS, L DB2, L IMS, L ZVM or L TPF.
```

Logon Type:

The Gibson VTAM logon screen, the result of connecting on port 23.

Read it before you touch anything. The banner names the LPAR (GIBSON PRODUCTION LPAR) and the legal notice underneath is the authorized-use warning every real mainframe greets you with, here with a Gibson flourish. The middle block reports your session: the logical-unit name VTAM assigned you, your client address, and the service port you came in on. And the bottom three lines are the ones that matter: the available applications and their versions, and the instruction LOGON using L TSO, L CICS, L DB2, L IMS, L ZVM or L TPF. That is how you choose where to go. Typing L TSO at the Logon Type prompt takes you toward the interactive system; the other names route you to CICS, Db2, IMS, z/VM, or z/TPF. We will actually log on in the next chapter; for now, the point is that the connection works and the front door is open.

If you prefer the graphical emulator, the invocation is the same and you get a window instead of a terminal pane:

```
x3270 192.168.0.96:23
```

Model and screen size

A 3270 terminal has a model number that fixes its screen size. The default, model 3279-2, is the classic twenty-four rows by eighty columns in color, and it is what most panels are designed for. Larger models give you more room (a 3279-3 is 32 by 80, a 3279-5 is 27 by 132) which helps with wide listings in SDSF or the editor. You select one at launch:

```
c3270 -model 3279-2 192.168.0.96:23
```

Start with 3279-2. It matches the screens in this book, and you can always reconnect with a larger model once you know your way around.

The keyboard: PF keys, PA keys, and the dreaded X

The single biggest shock for a newcomer is the keyboard, because 3270 does not work like a PC. You type into fields on a screen, then send the whole screen to the host at once with an action key. Enter is one such key; the others are the program-function and program-attention keys. Here is the set you need from day one:

Enter	submit the screen to the host (an 'AID' key)
PF1 - PF12	function keys; in c3270/x3270 these are F1 - F12
PF13 - PF24	Shift + F1 - F12
PF3	almost always 'exit / go back' in ISPF
PF7 / PF8	scroll backward / forward
PA1 / PA2	attention keys (PA1 often 'break', PA2 'reshow')
Clear	clear the screen buffer
Reset	unlock the keyboard after an input-inhibit (the 'X')

Two things trip up everyone. First, the keyboard locks the instant you press an action key: an indicator appears (often shown as X SYSTEM or just an X) meaning the host is thinking and your typing is inhibited. It unlocks itself when the host replies. If it stays locked because you pressed something the host did not expect, the Reset key clears the inhibit and lets you type again. Second, PF3 is the universal "back out of here" key in ISPF, and PF7 and PF8 scroll. Commit those three to memory and you can already navigate. A complete function-key reference is in Appendix B.

NOTE

If your keyboard seems frozen and nothing you type appears, you are almost certainly looking at an input-inhibit. Press Reset. This one reflex will save you more confusion in your first week than any other single thing in this book.

Macros and scripting

The emulators are scriptable, which is what makes them useful for repeatable testing. Both understand an action language (Connect, String, Enter, PF, Wait, and more) and s3270 runs those actions with no display at all, so you can drive a session from a shell script. A tiny macro that connects and types a logon choice looks like this:

```
Connect("192.168.0.96:23")
Wait(InputField)
String("L TS0")
Enter()
```

You will not need scripting to learn z/OS, but it is worth knowing it is there: when you reach the recon and testing chapters, the same action language lets you automate the repetitive parts of an assessment, and the IND\$FILE file-transfer mechanism in Chapter 36 is driven through the emulators' Transfer action.

The ASCII fallback

Gibson is generous about how you reach it. When a client connects, the listener negotiates 3270 if it can, and falls back to a plain ASCII, line-at-a-time rendering if it cannot. That means you can reach the very same logon screen with nothing more than telnet or netcat, which is invaluable on a stripped-down box with no emulator installed:

```
telnet 192.168.0.96 23
# or
nc 192.168.0.96 23
```

You lose the block-mode niceties (the screen scrolls instead of refreshing in place) but the banner, the application list, and the logon prompt are all there, and Gibson suppresses the echo of your password just as a real terminal would. For everyday use, connect with c3270; for a quick check that the door is answering, or for an environment where you cannot install anything, the ASCII path is always available.

Troubleshooting a dead session

Three problems account for nearly every failed first connection. If the emulator reports the connection was refused, you are aiming at the wrong port: confirm with `./gibsonctl.sh` status whether the terminal bound to 23, 1023, or 2023, and try again. If you connect but the keyboard is locked and nothing happens, that is the input-inhibit, press Reset. And if the screen is there but the characters are garbled, it is almost always a model or character-set mismatch; reconnect with `-model 3279-2` and let the emulator handle the EBCDIC translation, which it does automatically. Get past those three and the green screen behaves.

Hands-On Labs

These labs run from the host shell and the terminal emulator.

Lab 8.1

Install the emulator and connect

Goal: install the c3270/x3270 suite and reach Gibson's VTAM logon.

Background: you need a 3270 terminal to reach the logon screen and everything beyond it.

1. Install the emulator suite.

```
sudo apt-get install c3270 x3270
```

2. Connect to Gibson's TN3270 port.

```
c3270 192.168.0.96:23
```

Check yourself: the VTAM logon screen appears with the GIBSON production banner. You are now at the front door.

Why it matters: c3270 is the character emulator and x3270 the graphical one; both speak the same protocol, so pick whichever suits the moment.

Blue-Team angle: anyone who can reach port 23 can reach the logon. Network segmentation and access control to the TN3270 port are the first line of defense.

Lab 8.2

Recover a locked keyboard

Goal: clear an input-inhibited (X-SYSTEM) keyboard state.

Background: 3270 keyboards lock while the host is busy; recovering is a basic reflex.

1. When the status line shows X-SYSTEM, wait for the host, then press the Reset key.

Check yourself: the keyboard unlocks and you can type again. X-SYSTEM is a normal busy state, not an error.

Why it matters: knowing Reset turns a moment of panic (“the keyboard is dead”) into a non-event.

Blue-Team angle: understanding the 3270 protocol’s states helps a defender read terminal-level logs and tell a stuck session from a malicious one.

The 3270 connection

You reach the system with a 3270 terminal emulator (x3270 on Linux, or another TN3270 client) pointed at the host and the terminal port. The connection is a single command: the emulator opens a TN3270 session to port 23 and presents the full-screen terminal the mainframe expects.

```
x3270 <gibson-host>:23
```

Opening a 3270 session to the Gibson host on the terminal port.

A 3270 session is not a scrolling character terminal like SSH; it is a formatted screen of fields that you fill and submit a whole panel at a time with Enter or a PF key. That is why the connection matters: it establishes the full-screen, field-oriented channel that TSO, ISPF, and every panel in this book rely on. Once the session is open you are looking at the system’s first screen, ready to log on.

Lab 8.3

Open a 3270 session

Goal: connect a 3270 emulator to the Gibson host and reach the first full-screen panel.

Background: every interactive chapter begins from an open 3270 session. Making the connection cleanly is the gateway skill.

1. Start your 3270 emulator pointed at the host and terminal port.

```
x3270 <gibson-host>:23
```

Check yourself: the emulator opens a full-screen 3270 panel (not a scrolling line) and you can see the system's opening screen. If it refuses, the host or port is wrong, or the service is not up (Chapter 6).

Why it matters: the 3270 session is the channel for everything interactive. Recognizing a real full-screen connection (fields, not a scroll) confirms you are truly on the system.

Blue-Team angle: 3270 connections to port 23 are exactly what a defender watches at the perimeter, who is connecting a terminal, from where, and when. The connection you just made is a logged event on a real system.

Talking to a 3270: keys and AIDs

A 3270 does not send every keystroke as you type. You fill fields locally and then send the whole screen with an Attention Identifier key (an AID) which tells the host what you pressed. Knowing the important ones makes the terminal feel natural:

Key	What it does
Enter	Send the screen, the usual submit
PF1-PF24	Function keys, actions defined by each panel
PA1 / PA2	Program-attention keys, attention / reshow
Clear	Clear the screen and send
Tab	Move to the next input field (local, no send)

The 3270 keys that matter, and which ones actually send the screen.

The mental shift from a scrolling terminal is this: typing changes only your local screen; nothing reaches the host until you press an AID key such as Enter or a PF key. That is why panels tell you which PF key does what, those keys are the verbs of every full-screen application in the book.

Lab 8.4

Send a screen with an AID key

Goal: fill a field and send the screen with Enter, then use a PF key, observing that only AID keys reach the host.

Background: understanding that typing is local and AID keys send is the single most important 3270 concept. A short deliberate try makes it concrete.

1. On any panel, type into a field (notice nothing happens on the host) then press Enter.
2. Now press a PF key the panel defines (such as PF3) and watch it act.

Check yourself: the host reacts only when you press Enter or a PF key, not while you type. PF3 performs whatever the panel assigns it, commonly "go back."

Why it matters: every full-screen tool in this book is driven by AID keys. Internalizing "type locally, send with an AID" is what makes them all feel predictable.

Blue-Team angle: because a 3270 sends whole screens, its network traffic looks different from a character terminal, something defenders account for when monitoring terminal sessions.

When the connection will not open

Most connection failures have a handful of causes, and recognizing them by symptom saves time:

Symptom	Likely cause
Connection refused	Service not running, or wrong port
Connection times out	Host unreachable, or a firewall is blocking
Connects then drops	Wrong terminal type, or a protocol mismatch
Blank / garbled screen	Emulator not in 3270 mode

Reading a failed connection by its symptom.

Lab 8.5

Diagnose a refused connection

Goal: tell a “refused” failure from a “timed out” one and reason from the symptom to the cause.

Background: the first connection is where most people get stuck, and the failure message itself usually tells you which of two very different problems you have.

1. If a connection fails, note whether it was refused immediately or timed out after a wait.

Check yourself: “refused” means something answered and said no (the service is down or the port is wrong; “timed out” means nothing answered) the host is unreachable or a firewall is dropping the traffic. The two point at different fixes.

Why it matters: reading the failure correctly sends you straight to the right fix instead of trying everything at random.

Blue-Team angle: from the other side, a firewall that produces timeouts rather than refusals is a deliberate control, it reveals less to a scanner. The symptom you read as a user is a defender’s design choice.

A connection, step by step

Watched closely, opening a 3270 session is a short handshake: the emulator connects, negotiates the terminal type, and the host paints its first screen. A successful open looks like this:

```
$ x3270 gibson-host:23
Connecting to gibson-host, port 23 ...
Connected.
Negotiating TN3270E ... terminal type IBM-3278-2
Bound to session.
(the host paints the TS0/E LOGON panel)
```

A clean 3270 connection: connect, negotiate, bind, and the first panel appears.

The terminal-type negotiation is the moment a 3270 session becomes a 3270 session rather than a raw stream, it is why a plain telnet client shows garble where an emulator shows a formatted screen. Once bound, you are looking at the LOGON panel of Chapter 9.

Checkpoint: can you connect to Gibson with c3270 or x3270, reach the VTAM logon screen, and recover from an input-inhibited (X-SYSTEM) state? Those mechanics are the keyboard you will use for every interactive chapter from here on.

The 3270 terminal is the mainframe's classic face, and this chapter got you in front of it: installing the emulator suite, connecting to Gibson's TN3270 port, reaching the VTAM logon, and learning the PF/PA/Reset keys that make a 3270 keyboard behave. With a working terminal and the ASCII fallback in reserve, you are ready to log on for real.

What comes next

You have a terminal, it is pointed at Gibson, and the VTAM logon screen is waiting. Everything from here happens on that green screen. In the next chapter we finally log on, we choose L TSO, enter a userid and password, survive the forced password change that meets every new user, answer the multi-factor prompt, and arrive at the READY prompt with a session that is genuinely ours. Part II ends the moment you see READY; Part III begins by teaching you what to type next.

CHAPTER 9

Your First Logon

By the end of this chapter you will be able to:

Log on to TSO through the VTAM logon and TSO/E logon panel.

Complete a forced password change and respond to an MFA prompt.

Reach the READY prompt and confirm a successful logon.

Read the last-access message and what it tells you.

Log off cleanly and reconnect.

THE VTAM SCREEN IS ON your terminal and everything is in place. This is the moment the mainframe stops being something you read about and becomes something you operate. In this chapter we go all the way in, choose the interactive system, fill in the logon panel, survive the password change every new user meets, answer the multi-factor prompt if it is active, and arrive at the one prompt every TSO session begins from: READY. It is a short journey, but each step teaches something you will use constantly.

Choosing TSO

Back at the VTAM logon screen from the last chapter, the bottom line invited you to pick an application. You want the interactive system, so at the Logon Type prompt you type the logon command for TSO and press Enter:

```
Logon Type:  L TSO
```

VTAM routes you to TSO, and TSO answers with its logon panel. The other choices (L CICS, L DB2, L IMS, L ZVM, L TPF) would take you straight to those subsystems instead, and we will use them in later parts. For now it is TSO, because TSO is where you live.

The TSO/E logon panel

This is the panel that greets you, exactly as it renders on the green screen:

```
----- TSO/E LOGON -----

Enter LOGON parameters below:          RACF LOGON parameters:

Userid      ==> IBMUSER

Password    ==> _____

New Password ==> _____

Procedure   ==> IKJACCNT                MFA Token ==> _____
```

```

Acct Nmbr ====>
Size          ====> 4096
Perform       ====>
Command       ====>

```

Enter an 'S' before each option desired below:

```

_ Nomail      _ Nonotice    _ Reconnect    _ OIDcard

```

```

PF1/PF13 ==> Help   PF3/PF15 ==> Logoff   PA1 ==> Attention  PA2 ==> Reshow

```

The TSO/E LOGON panel. Userid and Password are the only fields you need at first.

It looks busy, but you only need two fields to start. Userid is who you are (IBMUUSER, the default identity we met in Chapter 3, is already filled in here. Password is the hidden field below it; what you type there does not display. The rest have sensible defaults: Procedure names the logon procedure that builds your session, Size sets your region in kilobytes, and Command lets you run something the instant you log on. The two fields on the right) New Password and MFA Token, are the ones this chapter is really about, because a first logon almost always needs them. Fill in the userid and password, leave the rest, and press Enter. The PF-key line at the foot is worth noting: PF3 logs you off from here, and PF1 is help.

The forced password change

Here is what happens to every new user, and it is not a malfunction. You enter your userid and the password you were given, press Enter, and instead of letting you in, the system tells you the password has expired and refuses to proceed until you set a new one. On z/OS this is policy: an initial password is meant to be changed the first time it is used, so that not even the administrator who created the account knows your working password. The signal is a message to the effect that the password is expired:

```

ICH01034I PASSWORD EXPIRED

```

The fix is on the panel in front of you. You leave your current password in the Password field and type the new one you want in the New Password field, then press Enter. The system records the change and continues the logon with your new credential. From now on, that is your password. If your site enforces history or complexity rules, a weak or reused choice is rejected and you simply try another, which is itself a small lesson in how RACF password policy works, a subject we return to in Part IV.

NOTE

A forced change on first use is one of the controls that separates a hardened system from a soft one. In Gibson's vulnerable training mode you will find accounts that never had this enforced, a weakness you will learn to find and exploit. Watching the secure mode insist on the change, where the vulnerable mode did not, is one of the clearest illustrations in the book of what "hardening" actually means in practice.

Multi-factor, if it is on

The second field on the right is MFA Token. Whether you need it depends on the posture Gibson was started in. In the default vulnerable mode, many accounts log on with a password alone, another deliberate weakness. In secure mode, multi-factor authentication is required, and the logon will not complete until you supply a valid token in that field alongside your password. The mechanism is the same one you answered during the IPL: a second factor the system demands before it trusts you. When MFA is active, type your token into the MFA Token field and press Enter; when it is not, leave the field empty. The presence or absence of that requirement is, again, exactly the kind of thing a tester notes and a defender enforces.

READY

With the password set and any token supplied, the logon completes. RACF stamps the event, the system tells you when you last logged on, and the screen settles onto a single, almost anticlimactic word:

```
ICH70001I IBMUSER LAST ACCESS AT 13:09 ON 30 JUN 2026 RESULT=SUCCESS METHOD=PASSWORD
READY
```

That is it. READY is the TSO command prompt (the mainframe equivalent of a shell's dollar sign) and the line above it, ICH70001I, is RACF reporting your last successful access and the method you used, which is the audit trail beginning to write your session down. You are logged on. To prove the session is genuinely yours, you can ask TSO to describe your environment with the PROFILE command:

```
PROFILE
CHAR(0) LINE(0) PROMPT INTERCOM NOPAUSE NOMSGID NOMODE NOWTP
VER PREFIX(IBMUSER)
DEFAULT LINE/CHARACTER DELETE CHARACTERS IN EFFECT FOR THIS TERMINAL
READY
```

Those are your session settings (prompting behavior, your dataset prefix, your terminal's editing characters) and the fact that the command answered at all confirms you are at a live READY prompt. When you are finished, the LOGOFF command ends the session cleanly and returns you to the VTAM screen, ready for the next person or the next logon.

NOTE

Everything in this chapter works identically over the ASCII path from Chapter 8. Whether you came in through c3270, x3270, or a raw telnet session, the logon panel, the forced password change, the MFA prompt, and the READY prompt behave the same way, Gibson keeps the two paths at parity precisely so a session is a session, however you reached it.

Hands-On Labs

These labs assume you are connected and looking at the VTAM logon screen.

Lab 9.1

Log on to TSO

Goal: take the logon from the VTAM screen all the way to the READY prompt.

Background: logging on is the gateway to every interactive task in the rest of the book.

1. At the VTAM logon, request the TSO application.

L TSO

2. On the TSO/E logon panel, enter the userid and password, and complete any forced password change.

Check yourself: you reach the READY prompt, and the last-access message (ICH70001I ... LAST ACCESS ...) confirms a successful logon.

Why it matters: the last-access line is a small but real security signal, a surprising time or terminal in it is worth a second look.

Blue-Team angle: the last-access message is a user-facing detection aid, and defenders mine the same logon events from SMF to spot misuse.

Lab 9.2

Log off cleanly

Goal: end a TSO session properly and return to the VTAM screen.

Background: leaving a session authenticated and unattended is a real risk; clean logoff matters.

1. From the READY prompt, log off.

LOGOFF

Check yourself: the session ends and you return to the VTAM logon screen, ready for the next user.

Why it matters: an abandoned, still-authenticated session is a classic foothold for an attacker who finds the terminal.

Blue-Team angle: idle-session timeouts and logoff discipline are basic but high-value controls on any interactive system.

The LOGON panel, field by field

The first full-screen panel the system shows is the TSO/E LOGON panel. It asks for the parameters that identify you and start your session:

```
----- TSO/E LOGON -----
Enter LOGON parameters below:          RACF LOGON parameters:
  Userid    ==> IBMUSER
  Password  ==>
  Procedure ==> ISPFPROC                New Password ==>
  Acct Nbr  ==> ACCT#
  Size      ==> 4096
```

The TSO/E LOGON panel: userid, password, and the session parameters.

Read the fields. Userid identifies you to RACF and decides everything you are allowed to do. Password authenticates you, and New Password lets you change it at logon if it has expired. Procedure names the logon procedure that builds your session (ISPFPROC brings you into ISPF). Acct Nmbr and Size are accounting and region parameters a site sets. You will type your userid and password here every time; the rest is usually pre-filled. Pressing Enter submits the whole panel, and RACF decides whether to let you in.

Lab 9.3

Log on and read the panel

Goal: complete the LOGON panel, enter your session, and understand what each field did.

Background: logging on is the first thing you do every session, and the panel is where identity, authentication, and session setup all meet.

1. On the LOGON panel, enter your userid and password, confirm the Procedure, and press Enter.

Check yourself: RACF accepts the credentials and the logon procedure builds your session, leaving you at the READY prompt or the ISPF primary menu. A wrong password is refused by RACF, not by the panel.

Why it matters: this panel is where your identity for the whole session is established. Everything you can and cannot do afterward flows from the userid you entered here.

Blue-Team angle: the LOGON panel is the front door, and every attempt (success or failure) is auditable. Failed logons, logons at odd hours, or a userid logging on from an unexpected place are exactly what a defender watches for.

After you log on

A successful logon leaves you at one of two places, depending on your logon procedure. If the procedure starts ISPF you land on the primary option menu; otherwise you arrive at the bare READY prompt, TSO waiting for a command:

```
IKJ56455I IBMUSER LOGON IN PROGRESS AT 14:03:10 ON JULY 1, 2026
READY
```

A completed logon: TSO confirms it and leaves you at the READY prompt.

IKJ56455I confirms the logon and stamps the time, and READY is TSO telling you it is listening. From here you either work in line mode (Chapter 10) or type ISPF to enter the full-screen environment (Chapter 11). Logging off is just as deliberate: LOGOFF ends the session cleanly, and simply dropping the connection leaves the session to time out.

Lab 9.4

Log off cleanly and log back on

Goal: end your session with LOGOFF and start a fresh one, seeing the full session lifecycle.

Background: a clean logoff frees your session and leaves a tidy audit trail, which matters more on shared systems than beginners expect.

1. From READY, end the session.

LOGOFF

2. Reconnect and log on again with the LOGON panel.

Check yourself: LOGOFF ends the session and returns you toward the LOGON panel; logging on again starts a fresh session with a new IKJ56455I logon message and timestamp.

Why it matters: starting and ending sessions cleanly is basic discipline, and the logon and logoff records are part of the account's history a reviewer reads.

Blue-Team angle: logon and logoff times build a picture of normal behaviour. Sessions that never log off, or logons at unusual hours, are the anomalies a defender looks for in the audit trail.

When logon is refused

RACF, not the panel, decides whether you get in, and it refuses for specific, readable reasons:

Message / symptom	What it means
Password not valid	Wrong password, or the account is revoked
Userid not defined	No such user in RACF
Password expired	Valid, but you must set a new one now
Not authorized	The userid may not use this application

Why RACF refuses a logon, each reason points to a different fix.

Lab 9.5

Read a refused logon

Goal: recognize the difference between a wrong password, an expired password, and a revoked or undefined account.

Background: logon is where identity is tested, and the reason for a refusal tells you whether the problem is the password, the account, or the authorization.

1. Attempt a logon with a deliberately wrong password and read the message.

Check yourself: RACF reports the credentials as not valid rather than letting you in. Note that repeated failures can revoke an account, the same message can hide a different underlying state.

Why it matters: distinguishing “wrong password” from “revoked account” or “not authorized” is the difference between retying and calling for an administrator.

Blue-Team angle: failed logons and revocations are prime signals. A burst of failures against one userid is a guessing attack; the revoke that follows is the control that stops it, both are events a defender watches closely.

From panel to READY, step by step

Submitting the LOGON panel starts a short, visible sequence as the system authenticates you and builds your session:

```
IKJ56455I IBMUSER LOGON IN PROGRESS AT 14:03:10 ON JULY 1, 2026
ICH70001I IBMUSER LAST ACCESS AT 09:22:41 ON JUNE 30, 2026
*** logon proc ISPFPROC building session ***
READY
```

Logon in progress: RACF authenticates, notes your last access, and TSO reaches READY.

Two lines are worth noticing. IKJ56455I confirms the logon and its time; ICH70001I is RACF telling you when the account was last used, a small but real security feature, because a last-access time you do not recognize is a sign someone else has been using your account. Then READY appears, and the session is yours.

Lab 9.6

Read your last-access time

Goal: find and interpret the ICH70001I last-access message that RACF shows at logon.

Background: RACF tells you when your account was last used every time you log on. It is a small line with real security value, and most users never read it.

1. Log on and read the messages that appear as the session starts.
2. Find the ICH70001I line and note the date and time of the last access.

Check yourself: ICH70001I reports a last-access date and time. It should match the last time you personally logged on. A time you do not recognize means someone else may have used the account, exactly the signal the message exists to give.

Why it matters: the last-access time is a free, per-logon integrity check on your own account. Reading it turns a line most people ignore into a habit that can catch a compromise.

Blue-Team angle: last-access data across all accounts is a rich source, dormant accounts that suddenly log on, or access times that cluster at odd hours, are anomalies a defender hunts for in exactly this field.

Checkpoint: can you take the logon all the way from the VTAM screen to the READY prompt, including a forced password change, and then log off cleanly? Can you read the last-access message? That round trip is the gateway to every interactive task ahead.

Logging on is the threshold between looking at a mainframe and using one. This chapter walked the full path (VTAM logon, the TSO/E logon panel, a forced password change, the MFA prompt, and the READY prompt) and showed how the last-access message quietly reports on your session. With a logon you can repeat at will, the core of z/OS is now open to you.

What comes next

You are at READY, and with that, Part II is complete, you can install Gibson, start it, walk its network, connect a terminal, and log on. But READY is a prompt with nothing typed at it yet, and a blinking cursor is only useful if you know what to say. Part III is where the real work begins. It teaches you to live on the system: the essential TSO commands, the ISPF panels and editor you will spend most of your time in, datasets and the catalog, the batch system and its job output, the Unix side, and everything else a mainframe operator does all day. Type your first command, and let's begin.

Part III

Living on the System: Core z/OS

This is the heart of the book. You can install Gibson, start it, and log on; now you learn to work. Over the next nine chapters you will become fluent in the things a mainframe operator does every day (the TSO command line, the ISPF panels and editor where most work happens, datasets and the catalog, VSAM, the batch system and its job output in SDSF and JES, what to do when a job abends, and the Unix side that lives inside z/OS. Each chapter is hands-on: you type the commands, you read the real output, and you finish with labs that turn knowledge into fluency. We begin where every session begins) at READY.

CHAPTER 10

The TSO READY Prompt & Essential Commands

By the end of this chapter you will be able to:

Work at the TSO READY prompt with confidence.

Use HELP, PROFILE, LISTDS, LISTGRP, SEND, and SUBMIT.

List a data set and its members from the command line.

Submit a job and capture its job number.

Tell a supported command from one the system rejects.

READY IS A HUMBLE PROMPT for something so central. It is the command line of TSO (the Time Sharing Option) the interactive monitor you log on to, and it works the way command lines have always worked: you type a command, press Enter, read the response, and READY returns to ask for the next one. Most of your day will be spent one level up, in the full-screen ISPF panels of the next chapter, but ISPF runs on top of TSO, and a handful of commands typed bare at READY will serve you constantly. This chapter teaches that handful.

Getting help

The first command to know is the one that tells you about the others. HELP opens the TSO help facility and lists the topics it knows:

```
HELP
```

```
TSO HELP FACILITY
```

```
HELP TOPICS:
```

```
  CICS  COMMANDS  DATASET  DB2  ISPF  OMVS  PASSTICKET  RACF  SDSF  SECURITY
```

```
Enter HELP topic for details.
```

Those topics map neatly onto the rest of this book, datasets, RACF, SDSF, OMVS, CICS, Db2, PassTicket, security. Type HELP followed by a topic and you get detail on it. It is the right reflex when you are unsure: ask the system before you guess.

Knowing your environment

Before you do anything, it helps to know who and where you are. PROFILE reports your session settings:

```
PROFILE
```

```
CHAR(0)  LINE(0)  PROMPT  INTERCOM  NOPAUSE  NOMSGID  NOMODE  NOWTP
```

```
VER  PREFIX(IBMUSER)
```


DEFAULT LINE/CHARACTER DELETE CHARACTERS IN EFFECT FOR THIS TERMINAL

The line that matters most is PREFIX(IBMUSER). TSO prepends your prefix (normally your userid) to any dataset name you type without quotes. Ask for TEST.DATA and TSO looks for IBMUSER.TEST.DATA; put the name in quotes and TSO takes it exactly as written. That one rule explains a great deal of mainframe behavior, and forgetting it is why a beginner's dataset sometimes ends up with their userid stuck on the front of it.

Looking at datasets

You met the catalog in Chapter 3 with LISTCAT. Its companion is LISTDS, which describes a single dataset, its shape, where it lives, and how it is protected:

```
LISTDS 'SYS1.PARMLIB'
SYS1.PARMLIB
--RECFM-LRECL-BLKSIZE-DSORG-CREATED---EXPIRES---SECURITY--DDNAME---DISP
FB      80      6160      PO      2026-06-30 * *      RACF      SYS00014 KEEP
--VOLUMES--
SBSYS1
```

Read across the columns: the record format and length, the block size, the organization (PO (a partitioned dataset), when it was created, that it is RACF-protected, and the volume it sits on. For a partitioned dataset you can go further and list its members) the named pieces inside it:

```
LISTDS 'SYS1.PARMLIB' MEMBERS
SYS1.PARMLIB MEMBERS
--MEMBER--
BPXPRM00
IEASYS00
IKJTS000
JES2PARM
LNKLST00
LPALST00
PROG00
SCHED00
SMFPRM00
```

Those are the configuration members that build the running system, IEASYS00, the one you selected during the IPL; IKJTS000, which configures TSO itself; PROG00 and the APF list; SMFPRM00 for the audit recorder. You are looking at the system's own settings, and in Part V you will learn to read and change them. For now, simply note that LISTDS with MEMBERS is how you see inside any partitioned dataset.

Asking RACF who is who

Two commands query the security database directly from READY. LISTUSER, which you saw in Chapter 3, describes a user; LISTGRP describes a group and (usefully) who belongs to it:

```
LISTGRP SYS1
GROUP=SYS1 OWNER=IBMUSER SUPGROUP=SYS1
CONNECTED USERS:
  IBMUSER AUTHORITY=JOIN
  RUARIV AUTHORITY=USE
```

The group SYS1 is owned by IBMUSER, and its connected users are listed with the authority each holds within the group. IBMUSER joins with JOIN authority (the highest) while another user, RUARIV, is connected with USE. To a security reader this is reconnaissance: group membership and connect authority are exactly the relationships that decide who can do what, and being able to enumerate them from a plain logon is the first step of understanding a system's access model. We build on this throughout Part IV.

Sending a message, and submitting work

Two more everyday commands. SEND puts a short message in front of another user, immediately if they are logged on, or queued for their next logon if they are not:

```
SEND 'SYSTEM GOING DOWN AT 1700' USER(IBMUSER)
User IBMUSER not active. Message queued for next logon.
```

And SUBMIT hands a piece of JCL to the batch system. You will spend a whole chapter on JCL and JES later, but the command itself is simple, and its reply tells you the job was accepted and gives you the job name and number to track it by:

```
SUBMIT MY.JCL(HELLO)
IKJ56250I JOB IBMUSERJ(JOB00001) SUBMITTED
```

That job number (JOB00001) is how you find the job again in SDSF, which is where its output will appear. SUBMIT is the bridge between the interactive world you are in now and the batch world of Chapter 16.

Leaving, and launching

Two commands end this chapter the way they end most sessions. ISPF takes you up into the full-screen panel environment where the rest of Part III happens, it is the single command you will type most often at READY, because almost everything is easier there. And LOGOFF ends the session cleanly and returns you to the VTAM screen:

```
ISPF
LOGOFF
```

NOTE

Gibson's TSO is a focused subset of the real thing, and it is honest about it. Type a command it does not implement (TIME, LISTALC, STATUS) and you get the genuine z/OS reply, IKJ56500I COMMAND name NOT FOUND, rather than a fake success. That is the real "command not found" message a mainframe gives, so treat it as information, not a fault: it means the command name was not recognized, usually because it is spelled differently here or belongs in another environment such as SDSF or ISPF.

Reading a command's response

TSO tells you exactly what happened, and learning to read its replies is half the skill. A successful command either does its work silently or prints a result; a message beginning with a message id (IKJ for TSO, ICH for RACF, IEF for the system) is the system speaking to you in a documented, searchable language. IKJ56250I after a SUBMIT confirms your job was accepted and gives its number. IKJ56500I COMMAND name NOT FOUND, by contrast, means the command was not recognized, not that something broke, but that the name was wrong or belongs to another environment such as ISPF or SDSF. Reading the message id, looking up its meaning, and acting on it is the loop you will run thousands of times; the ids are not noise, they are the system being precise.

The commands worth memorizing

A small set of TSO commands covers most interactive work. Keep these within reach until they are automatic:

Command	What it does
HELP / ?	List help topics, or help for a command
PROFILE	Show or set your session profile and PREFIX
LISTDS 'dsn'	Show a data set's attributes
LISTDS 'dsn' MEMBERS	List the members of a library
LISTGRP group	Show a RACF group and its connected users
LISTCAT LEVEL(hlq)	List cataloged data sets under a qualifier
SEND 'text' USER(id)	Send a message to another user
SUBMIT 'dsn'	Submit a job for batch execution
ISPF	Enter the full-screen ISPF environment
LOGOFF	End the TSO session

The everyday TSO commands, gathered for reference.

Hands-On Labs

These labs assume you are logged on as IBMUSER at the READY prompt.

Lab 10.1

Inspect a library and its members

Goal: use LISTDS to see a data set's attributes and list the members of a library.

Background: before you edit or submit from a library, you check what it is and what it holds.

1. At READY, list the attributes of a system library.

```
LISTDS 'SYS1.PARMLIB'
```

```
SYS1.PARMLIB
```

```
--RECFM-LRECL-BLKSIZE-DSORG-CREATED---EXPIRES---SECURITY--DDNAME---DISP
  FB      80      6160      PO      2026-06-30 *          RACF      SYS00014 KEEP
--VOLUMES--
  SBSYS1
```

2. Now list the members of that library.

```
LISTDS 'SYS1.PARMLIB' MEMBERS
```

```
SYS1.PARMLIB MEMBERS
```

```
--MEMBER--
```

```
ALLOC00
```

```
BPXPRM00
```

```
CLOCK00
```

Check yourself: the attributes show DSORG PO on volume SBSYS1, and you get a list of members. PO with members confirms a partitioned library (a PDS).

Why it matters: LISTDS is the fastest way to understand a library before you touch it, its format, its size, and what is inside.

Blue-Team angle: enumerating configuration libraries such as SYS1.PARMLIB is also a reconnaissance step. Defenders restrict and monitor read access to sensitive system libraries.

Lab 10.2

Submit a job and capture its number

Goal: submit a job and record the job id you will use to track it in SDSF.

Background: every batch task is identified by a job id; capturing it is how you follow your work to its result.

1. Submit the sample job and read the response.

```
SUBMIT MY.JCL(HELLO)
```

```
IKJ56250I JOB IBMUSERJ(JOB00001) SUBMITTED
```

Check yourself: the system returns IKJ56250I with a job name (IBMUSERJ) and a job number (JOB00001). Write the number down.

Why it matters: that job number is the handle you use in SDSF (Chapter 15) to find the job and read whether it succeeded.

Blue-Team angle: unexpected SUBMITs (especially a job that runs under a surrogate USER= identity) are exactly what a defender reviews in the job log.

Lab 10.3

Check your session profile

Goal: read your TSO profile and confirm the PREFIX that is prepended to unquoted data set names.

Background: the PREFIX decides how an unquoted name like MYDATA is qualified. Knowing yours prevents “why can’t it find my data set” confusion.

1. At READY, display your profile.

PROFILE

```
CHAR(0)  LINE(0)  PROMPT  INTERCOM  NOPAUSE NOMSGID NOMODE  NOWTP
VER PREFIX(IBMUSER)
```

Check yourself: the PREFIX line shows your userid. An unquoted name such as MYDATA is therefore treated as IBMUSER.MYDATA; a fully-quoted name in apostrophes is used as-is.

Why it matters: almost every “data set not found” in early practice is a prefix surprise. Reading your profile once makes the rule concrete.

Blue-Team angle: the PREFIX also shapes what a mistyped command touches. Understanding name qualification is part of understanding what a user can reach by accident or on purpose.

Quoted and unquoted names

One rule prevents most early confusion. A data set name in apostrophes is fully qualified and used exactly as typed, 'SYS1.PARMLIB' means that data set and no other. A name without apostrophes is prefixed with your PREFIX, so at READY the name MYDATA is treated as IBMUSER.MYDATA. The same rule runs through TSO, ISPF, and JCL, and knowing it turns “why can’t it find my data set” into a habit of asking whether you quoted the name. When in doubt, quote fully and you always know what you are naming.

Reading a RACF group

Users are gathered into RACF groups, and one command shows a group and everyone in it. LISTGRP is worth meeting now because groups are where much of the security story lives:

```
GROUP=SYS1  OWNER=IBMUSER SUPGROUP=SYS1
CONNECTED USERS:
  IBMUSER  AUTHORITY=JOIN
  RUARIV   AUTHORITY=USE
```

LISTGRP shows a group, its owner, and the users connected to it.

The output names the group, its owner and superior group, and every connected user with the authority they hold in it, JOIN lets a user add others, USE is ordinary membership. Reading a group this way is the first move in understanding who has access to what, and it is a skill Part IV builds on heavily.

Lab 10.4

Explore a RACF group

Goal: list a RACF group and read the authority each connected user holds.

Background: groups are how access is granted at scale, so reading one is a core skill for both operating and defending.

1. At READY, list the SYS1 group.

`LISTGRP SYS1`

Check yourself: the output shows `GROUP=SYS1`, its owner, and connected users with `AUTHORITY=JOIN` or `USE`. `IBMUSER` holds `JOIN`; note who else is connected and with what authority.

Why it matters: group membership and authority decide a great deal of what users can do. Reading a group is where access reviews begin.

Blue-Team angle: unexpected `JOIN` authority, or an unfamiliar user connected to a powerful group, is exactly what an access review looks for. `LISTGRP` is a defender's everyday tool.

Checkpoint: can you, from `READY`, list a `PDS` and its members, check your profile, send yourself a message, and submit a job, reading each response correctly? Those few commands are the spine of interactive TSO work.

TSO's `READY` prompt is the mainframe's command line, and a handful of commands carry most of the day-to-day work. This chapter put `HELP`, `PROFILE`, `LISTDS`, `LISTGRP`, `SEND`, and `SUBMIT` to work against real Gibson output, including learning which commands the system rejects, so you build an accurate picture rather than a hopeful one. From here, `ISPF` gives these same capabilities a full-screen face.

What comes next

You can now find your way around from a bare prompt: get help, inspect your environment, look at datasets and their members, query the security database, send a message, and submit a job. It is enough to be useful and not yet enough to be comfortable, because almost everyone does this work through panels rather than typed commands. The next chapter enters `ISPF` (the full-screen environment that sits on top of `TSO`) and from there the system becomes a place you navigate rather than a prompt you address.

CHAPTER 11

ISPF: Panels, Navigation, and the Management Menu

By the end of this chapter you will be able to:

- Navigate the ISPF Primary Option Menu and its panels.
- Use the equals-jump and the option/sub-option grammar.
- Read the ISPF status panel and adjust settings.
- Reach the Gibson Management menu and its tools.
- Move fluently between TSO line mode and ISPF.

If TSO IS THE COMMAND line, ISPF is the desktop. The Interactive System Productivity Facility is a full-screen menu and panel environment that runs on top of TSO, and it is where a mainframe professional spends almost the entire working day. You edit datasets in it, browse job output through it, run utilities from it, and administer security with it. You reached it at the end of the last chapter by typing one word at the READY prompt:

```
ISPF
```

The screen clears and the Primary Option Menu takes over. From here you navigate by choosing options rather than remembering commands, which is exactly why ISPF has outlived almost every interface designed since. This chapter teaches you to move around it confidently, and introduces Gibson's own Management menu, a launchpad for the security and office tools you will use throughout the book.

The Primary Option Menu

This is the panel that greets you:

```
ISPF Primary Option Menu
```

```
Menu  Utilities  Compilers  Options  Status  Help
```

```
Option ==>
```

0	Settings	Terminal and user params	User ID . : IBMUSER
1	View	Display source data/listings	Time. . . : 15:51
2	Edit	Create or change source	Terminal. : 3278
3	Utilities	Perform utility functions	Screen. . : 1
4	Foreground	Interactive lang proc	Language. : ENGLISH
5	Batch	Submit job for processing	Appl ID . : ISR
6	Command	Enter TSO/Workstation cmds	TSO logon : ISPFPROC

7	Dialog Test	Perform dialog testing	TSO prefix: IBMUSER
8	Outlist	Display/print held output	System ID : S0W1
S	SDSF	System Display and Search	MVS acct. : ACCT#
12	DB2	DB2I primary menu / SPUFI	Release . : ISPF 7.5
R	RACF	Security admin panels	
M	Management	Management menu	
I	IMS	IMS Connect / OTMA	
X	Exit	Terminate ISPF	

F1=Help F2=Split F3=Exit F7=Up F8=Down F9=Swap F10=Actions F12=Cancel

The ISPF Primary Option Menu. The panel on the right reports your session.

The left column is the menu; the right column is a status panel describing your session (your user ID, terminal model, screen number, the application ID, your TSO prefix, the system name, and the ISPF release. Glance at it whenever you want to confirm who and where you are. The options you will use most are 1 View and 2 Edit, for looking at and changing datasets; 3 Utilities, for the dataset operations of the next chapter; 6 Command, to run a TSO command without leaving ISPF; and the single-letter jumps to the bigger subsystems) S for SDSF, R for RACF, M for the Management menu, and X to leave ISPF altogether. We meet the editor in Chapter 14 and the utilities in Chapter 12; here the goal is simply to move.

Navigating: options, jumps, and the keyboard

Navigation has three moving parts. The first is the Option line: type an option number or letter there and press Enter to descend into that function. The second is the jump command, which is the single most useful habit in ISPF. From anywhere, however deep you are, type an equals sign followed by a path and you go straight there, no backing out screen by screen:

=M.8

That jumps directly to the Management menu and selects its eighth option in one keystroke sequence. The third part is the keyboard you met in Chapter 8, and in ISPF the function keys have settled meanings: PF3 backs out of the current panel, PF7 and PF8 scroll up and down, PF9 swaps between split screens, and PF1 is help. The action bar across the top (Menu, Utilities, Compilers, Options, Status, Help) offers the same functions for those who prefer to point. Master PF3 and the equals-jump and you can already reach any corner of the system quickly.

NOTE

The equals-jump composes. =3.4 opens the Data Set List utility; =M.7 opens the Lynx browser; =X leaves ISPF. Because it works from inside any panel, you never have to remember the way back, you only have to remember where you are going.

Settings

Option 0 opens the settings panel, where a few preferences shape how every other panel behaves:

ISPF Settings

Options

Enter "/" to select option

- _ Command line at bottom
- / Panel display CUA mode
- / Long message in pop-up
- _ Tab to action bar choices

Terminal Characteristics

Screen format . . (1=Data, 2=Std, 3=Max, 4=Part)

Terminal type . . 3278

F1=Help F2=Split F3=Exit F9=Swap F12=Cancel

ISPF Settings. The terminal type and command-line position live here.

You select an option by typing a slash beside it. The two worth knowing early are Command line at bottom, which moves the command line from the top of each panel to the foot (a matter of taste many people feel strongly about) and the Terminal type, which should read 3278 to match the emulator you connected with in Chapter 8. The defaults are sensible; you can leave this panel untouched and come back when something annoys you enough to change it.

The Management menu

Option M is where Gibson extends ISPF beyond the standard. It is a launchpad, a single menu that gathers the security, systems, and office applications you will use across the rest of the book:

Menu Options Info Commands Setup
Management Services

Option ==>

Select a management application:

- 1 zSecure IBM Security zSecure Admin and Audit for RACF
- 2 SMP/E SMP/E software maintenance dialogs

3	SYSVIEW	CA SYSVIEW Performance and Operations Monitor
4	Endevor	CA Endevor SCM software change management
5	EZRecon	EZRecon reconnaissance and assessment toolkit
6	RSS	CTI RSS reader - security news feeds (PF7/PF8 scroll)
7	Lynx	Lynx text web browser rendered in ISPF (PF7/PF8 scroll)
8	Mail	Gibson Office Mail - SMTP/POP3 electronic mail (SYS1.EMAIL)

F1=Help F3=Exit

The Management menu: Gibson's launchpad for security and office tools.

The first five entries are administrative and security tooling, zSecure and SMP/E, the SYSVIEW performance monitor, Endevor change management, and the EZRecon reconnaissance toolkit you will live in during Part VIII. The last three are the ones to try right now, because they turn the green screen into something surprisingly modern.

Option 6, RSS, is a reader for security news feeds rendered straight into ISPF, scrollable with PF7 and PF8. Option 7, Lynx, is a text web browser that runs inside the panel (type a URL, or click a link to follow it. The links render as numbered, highlighted markers; place the cursor on one and press Enter, or type its number, and the browser fetches that page, so you can genuinely browse from a 3270 terminal. And option 8, Mail, is Gibson Office Mail) a full electronic-mail facility with an inbox, sent items, important, and spam folders; you read and write notes, send them over SMTP and download them over POP3, and configure the server on its own panel, with the settings saved to the SYS1.EMAIL dataset. Its colours are deliberately its own, so you always know which application you are in.

NOTE

The mail facility stores your folders in your own datasets (your inbox in IBMUSER.MAIL.IN and your sent notes in IBMUSER.MAIL.OUT) which means you can also browse them as ordinary datasets with the editor of Chapter 14. That is a small but telling illustration of a mainframe truth: on z/OS, almost everything is a dataset underneath, even your email.

The primary option menu

Every ISPF session begins at the primary option menu, and its numbered options are the map of everything the environment does. You will not use them all, but knowing what each is for means you are never lost:

Option	What it does
0 Settings	Terminal and session parameters
1 View	Display source data and listings, read-only
2 Edit	Create or change source with the editor
3 Utilities	Data set, library, and move/copy functions
4 Foreground	Run language processors interactively
5 Batch	Build and submit jobs for processing
6 Command	Enter TSO commands from a full-screen line
8 Outlist	Display and print held job output
M Management	Gibson's RACF and security services menu

The ISPF primary options, the top level of the whole environment.

The one to notice is 3, Utilities, because option 3.4 (the Data Set List) is where you will spend more time than anywhere else: it lists, browses, edits, allocates, and deletes data sets from a single panel. The M option is Gibson's own addition, a Management menu that gathers the RACF and security services this book relies on.

Getting around: options, jumps, and PF keys

ISPF navigation has three moves worth making automatic. You reach a panel by typing its option number and pressing Enter, and you can chain numbers with dots (typing 3.4 from the top goes straight to the Data Set List without stopping at the Utilities menu. From anywhere deeper in the hierarchy you can jump directly by prefixing the target with an equals sign: =3.4 leaps to the Data Set List no matter where you are, and =X exits. And PF3 is the universal step back) it ends the current panel and returns you one level up, saving as it goes in the editor. Those three (option numbers, the equals-jump, and PF3) are the whole grammar of moving through ISPF, and once they are reflexive the environment stops feeling like a maze and starts feeling like a building you know.

Hands-On Labs

These labs assume you are at the ISPF Primary Option Menu.

Lab 11.1

Navigate by option and equals-jump

Goal: move around ISPF by option number and the equals-jump shortcut.

Background: fast navigation is what makes ISPF pleasant, and the equals-jump is the habit that gets you there.

1. From anywhere in ISPF, jump straight to the data set list.

=3.4

2. Now jump straight to the Management menu.

=M

Check yourself: each jump takes you straight to its panel without stepping through the intermediate menus. PF3 backs you out one level at a time.

Why it matters: the equals-jump is the single most time-saving ISPF habit; experienced users navigate almost entirely with it.

Blue-Team angle: knowing the navigation also helps a defender audit exactly which tools and panels are reachable from a given menu.

Lab 11.2

Reach the Management menu tools

Goal: open the Gibson Management menu and see the tools it gathers.

Background: the Management menu is the launch point for the security tools used in later chapters.

1. From the primary menu, select the Management option.

M

Check yourself: the Management menu lists its tools, zSecure, SMP/E, SYSVIEW, Endeavor, EZRecon, RSS, Lynx, and Mail. These are the entry points for later work.

Why it matters: knowing where the tools live now means later chapters can get straight to the technique instead of the navigation.

Blue-Team angle: the set of tools reachable from a menu is itself worth inventorying, each is a capability that can be used or abused, and each deserves access control.

Lab 11.3

Jump straight to the Data Set List

Goal: use the dotted-option and equals-jump forms to reach 3.4 quickly, then return with PF3.

Background: 3.4 is where you will work most, so reaching it without clicking through menus is a habit worth forming on day one.

1. From the primary menu, type 3.4 and press Enter.
2. Press PF3 to return, then from anywhere type =3.4 to jump straight back.

Check yourself: both 3.4 and =3.4 land you on the Data Set List panel, and PF3 returns you one level up each time. The equals form works from any depth; the plain form works from the primary menu.

Why it matters: fast, confident navigation is the difference between fighting ISPF and flowing through it, and every later chapter assumes you can get to 3.4 without thinking.

Blue-Team angle: knowing how someone moves through ISPF also tells you how quickly they can reach sensitive utilities. Fluency is neutral, defenders and attackers alike rely on it, which is why observation, not obstruction, is the control.

Why full-screen changed everything

It is worth pausing on why ISPF matters. Before full-screen panels, every interaction was a typed line-mode command and a printed reply. ISPF put a structured screen in front of the same system: fields to fill, options to pick, lists to act on, all navigable with the cursor and the PF keys. That shift (from typing commands blind to acting on what you can see) is what made the mainframe usable at scale, and it is why almost every tool in this book, from the editor to SDSF to the RACF panels, is an ISPF application. Learning ISPF once is learning the shape of all of them.

The menu in front of you

It helps to see the primary menu exactly as it appears, with the option list on the left and your session details on the right:

0	Settings	Terminal and user params	User ID . . : IBMUSER
1	View	Display source data/listings	Time. . . : 00:21
2	Edit	Create or change source	Terminal. : 3278
3	Utilities	Perform utility functions	Screen. . : 1
4	Foreground	Interactive lang proc	Appl ID . : ISR
6	Command	Enter TSO/Workstation cmds	TSO prefix: IBMUSER
M	Management	RACF and security services	System ID : S0W1

The ISPF primary menu: options on the left, session facts on the right.

The right-hand panel is worth a glance every logon, it confirms who you are, your prefix, and which system you are on, all of which matter the moment you start naming data sets or submitting work.

Lab 11.4

Open the Settings panel

Goal: reach option 0 and see the session parameters you can control.

Background: Settings is where session behaviour is tuned. Visiting it once demystifies options you will otherwise wonder about.

1. From the primary menu, type 0 and press Enter; press PF3 to return.

Check yourself: option 0 opens the Settings panel showing terminal and session parameters, and PF3 returns you to the primary menu. Nothing needs changing, the goal is to know where it lives.

Why it matters: knowing that session behaviour is adjustable, and where, saves confusion later when a default is not what you expected.

Blue-Team angle: session and terminal settings shape what a user sees and can do. Standardized settings are part of a controlled environment; unexpected changes are worth noticing.

Checkpoint: can you move around ISPF by option number and equals-jump, reach the Management menu, and return to where you started without getting lost? Smooth navigation is what makes every full-screen tool that follows feel easy.

ISPF is the full-screen workbench that sits on top of TSO, and most interactive work happens inside it. This chapter mapped the Primary Option Menu, the equals-jump shortcut, the status panel and settings, and the Gibson Management menu that gathers the security tools you will use later. With ISPF navigation in hand, the dataset, editor, and SDSF chapters all become point-and-go.

What comes next

You can now move around ISPF, jump anywhere with an equals sign, set your preferences, and reach Gibson's management tools. What you have been navigating around (the things every panel ultimately reads and writes) are datasets, and they are the subject of the next chapter. We will create them, list them, look inside them, and learn the catalog that keeps track of them all, because nothing else on the system makes sense until datasets do.

CHAPTER 12

Datasets & the Catalog

By the end of this chapter you will be able to:

Explain the dataset model: sequential, partitioned, and members.

Read a DCB: record format, record length, and block size.

Use the ISPF 3.4 dataset list and its line commands.

Allocate a new dataset with the right organization.

Find datasets through the catalog with LISTCAT.

ON MOST SYSTEMS A FILE is a bag of bytes and the programs that read it decide what those bytes mean. The mainframe took the opposite view decades ago, and never changed its mind: here, the container itself knows its shape. A dataset (the mainframe word for a file) carries its record format, its record length, and its organization as properties declared when it is created. Almost everything on z/OS is a dataset: your code, the system's configuration, job output, even, as you saw last chapter, your email. So before anything else makes sense, datasets must. This chapter creates them, lists them, looks inside them, and explains the catalog that keeps track of them all.

What a dataset is

There are two kinds you will meet constantly. A sequential dataset (organization PS, for physical sequential) is a single stream of records, read start to finish, like a plain file. A partitioned dataset (organization PO, universally called a PDS) is more interesting: it is a small library with a directory and named members inside it, so SYS1.PARMLIB holds members like IEASYS00 and PROG00, each an independent piece you can edit on its own. A PDS is how the mainframe stores anything that comes in named pieces: source code, JCL, configuration, macros.

Every dataset also carries its DCB attributes (the data control block) which describe its records. The record format, RECFM, is usually FB, fixed-length blocked; the record length, LRECL, is how many bytes each record holds, classically 80, an inheritance from the punched card; and the block size, BLKSIZE, is how records are grouped on disk. You will see these three everywhere, and you set them when you create a dataset rather than leaving a program to guess.

Naming: qualifiers and the high-level qualifier

Dataset names are not free-form. A name is a series of qualifiers separated by dots, each up to eight characters, up to forty-four characters in total, IBMUSER.TEST.CNTL has three qualifiers. The first, the high-level qualifier or HLQ, matters most: it is the root under which a name is cataloged and, very often, the handle that security uses to decide who

may touch it. Recall the prefix rule from Chapter 10: type a name without quotes and TSO prepends your prefix as the HLQ, so TEST.CNTL becomes IBMUSER.TEST.CNTL. Quote the name and you are giving the fully qualified truth. Getting comfortable with qualifiers is half of getting comfortable with the system.

Listing datasets with 3.4

The fastest way to see what exists is the ISPF Data Set List utility, option 3.4. You give it a dataset-name level (a high-level qualifier to match) and it lists everything beneath it:

```

DSLST - Data Sets Matching SYS1                      Row 1 of 37
Command ==>                                           Scroll ==> PAGE
Enter line command at left:  B Browse  V View  E Edit  M Members  I Info
  Cmd  Name                                           Org Volume
    SYS1.BROADCAST                                PS  SBSYS1
    SYS1.CICS.PROCLIB                             P0  SBSYS1
    SYS1.CLIST                                    P0  SBSYS1
    SYS1.HELP                                    P0  SBSYS1
    SYS1.LINKLIB                                 P0  SBSYS1
    SYS1.LPALIB                                 P0  SBSYS1
    SYS1.MACLIB                                 P0  SBSYS1
    SYS1.MANA                                    PS  SBSMF1
    SYS1.PARMLIB                                P0  SBSYS1
    SYS1.PROCLIB                                 P0  SBSYS1

```

ISPF 3.4 listing the SYS1 datasets, with line commands down the left.

Each row shows a dataset, its organization, and the disk volume it lives on. The power is in the line commands listed at the top: type a letter to the left of any name and press Enter to act on it (B to browse it read-only, V to view, E to edit, M to list a PDS's members, I for full information. This single panel is where most people start almost every task, because it turns a wall of names into something you can point at and open. Note the organizations: the PROCLIBs and LINKLIB are PO) partitioned, while BROADCAST and the SMF dataset MANA are PS, plain sequential streams.

Looking inside, and the catalog

From READY you can ask the same questions by command. LISTDS, from Chapter 10, describes one dataset and, with MEMBERS, lists a PDS's contents. LISTCAT goes wider, listing everything cataloged beneath a level along with its shape:

```

LISTCAT LEVEL(SYS1)
SYS1.BROADCAST                                PS FB LRECL=80
SYS1.CICS.PROCLIB                             P0 FB LRECL=80

```



```

SYS1.LINKLIB          PO FB LRECL=80
SYS1.PARMLIB          PO FB LRECL=80
SYS1.PROCLIB          PO FB LRECL=80

```

That word (cataloged) is the quiet hero of the system. The catalog is the index that maps a dataset name to the volume it lives on, so that you, and every program, can refer to a dataset by name alone and never need to know which disk it sits on. When you create a dataset it is normally cataloged automatically; the entry is what LISTCAT reads. A dataset can in principle be uncataloged (present on a volume but missing from the index) which is exactly the kind of loose end a careful administrator hunts down, and the 3.2 utility offers explicit Catalog and Uncatalog actions for managing it.

Creating a dataset: the 3.2 Allocate panel

To make a new dataset, option 3.2 gives you the Allocate panel, where you declare the shape up front:

```
Allocate New Data Set
```

```
Command ==>
```

```
Data Set Name . . . : IBMUSER.TEST.CNTL
```

```

Space units . . . . . TRACKS    (BLKS TRKS CYLS KB MB BYTES RECORDS)
Primary quantity . . 1          (In above units)
Secondary quantity . 1          (In above units)
Directory blocks . . 10         (Zero for sequential data set)
Record format . . . . FB
Record length . . . . 80
Block size . . . . . 0
Data set name type . PDS        (LIBRARY PDS LARGE BASIC or blank)

```

ISPF 3.2 Allocate. Directory blocks above zero make it a PDS, not sequential.

Read it as a form for the DCB. Space units and the primary and secondary quantities say how much disk to claim and how much more to grab if it fills. Record format, record length, and block size are the FB / 80 / system-chosen trio from earlier. And the single most important field is Directory blocks: leave it zero and you get a sequential dataset; set it above zero (here, ten) and you reserve a directory, which makes it a PDS that can hold members. That one number is the difference between a plain file and a library. Press Enter and the dataset is allocated and cataloged, ready to use.

Everything on this panel can also be done from the command line with ALLOCATE, which is what batch jobs use; the panel simply spares you remembering the keywords while you learn what they mean. Either way, the result is identical: a named, shaped, cataloged dataset.

NOTE

When you allocated the mail facility's folders last chapter, this is what happened underneath, IBMUSER.MAIL.IN was allocated and cataloged exactly like any other dataset. Try option 3.4 with a level of your own userid and you will see your mail datasets listed alongside anything else you own. It is a good way to convince yourself that the abstractions really are datasets all the way down.

Reading a data set's attributes

Every data set carries a set of attributes that describe how its records are shaped and stored, and LISTDS shows them. Here is a real example, the system parameter library:

```
SYS1.PARMLIB
--RECFM-LRECL-BLKSIZE-DSORG-CREATED---EXPIRES---SECURITY--DDNAME---DISP
FB      80      6160      PO      2026-07-01 * *      RACF      SYS00014 KEEP
--VOLUMES--
SBSYS1
```

LISTDS output: the attributes that define a data set.

Read across the line. RECFM is the record format, FB here, fixed-length blocked, the most common. LRECL is the logical record length, 80, the width of one record. BLKSIZE is how records are grouped for storage. DSORG is the data-set organization: PO means partitioned (a library of members), while PS would mean physical-sequential (a single flat file). The SECURITY field even shows that this data set is RACF-protected, and VOLUMES names the disk it lives on. These few fields tell you almost everything about how a data set behaves before you open it.

Attribute	Meaning
RECFM	Record format, FB, VB, F, U
LRECL	Logical record length (record width)
BLKSIZE	Block size, how records are grouped on disk
DSORG	Organization, PS (sequential) or PO (partitioned)
VOLUME	The disk volume the data set resides on
SECURITY	Whether RACF protects the data set

The data-set attributes you will read again and again.

Sequential and partitioned: PS versus PO

The single most important distinction is DSORG. A PS (physical-sequential) data set is one flat stream of records, read start to finish, like a plain file. A PO (partitioned) data set is a library: it holds many named members, each itself a small sequential file, indexed by a directory. When you saw SYS1.PARMLIB's members ALLOC00, BPXPRM00, CLOCK00 and the rest, you were reading the directory of a PO data set. This is why a JCL library, a source library, and the system parameter library are all partitioned: they group related members under one name. The number of directory blocks you request when you allocate a PDS sets how many members it can hold, which is the one allocation decision beginners

most often get wrong, too few directory blocks and the library fills its index long before it fills its space.

Hands-On Labs

These labs assume you are logged on as IBMUSER, at READY or in ISPF.

Lab 12.1

Find your data sets through the catalog

Goal: use LISTCAT to enumerate the data sets cataloged under a high-level qualifier.

Background: the catalog is the index of every data set; LISTCAT is how you query it when you do not already know the names.

1. List everything cataloged under your own userid.

```
LISTCAT LEVEL(IBMUSER)
```

IBMUSER.4CHAR.PIN	PS FB LRECL=80
IBMUSER.COBOL.LAB	P0 FB LRECL=80
IBMUSER.JCL.LAB	P0 FB LRECL=80
IBMUSER.PDS.CODE	P0 FB LRECL=80

Check yourself: each entry shows its organization (PS for a flat sequential file, PO for a partitioned library) and its record format. You now have the real names to work with.

Why it matters: LISTCAT is how you discover what exists. It turns “I don’t know the data set name” into a list you can act on.

Blue-Team angle: LISTCAT against high-value qualifiers is a classic reconnaissance move. Defenders restrict catalog enumeration and watch for sweeps across sensitive qualifiers.

Lab 12.2

Allocate a new data set

Goal: create a new data set with the ISPF 3.2 Allocate panel and choose its organization.

Background: you constantly need a fresh library or file to hold work. Allocation on z/OS is done from the 3.2 panel, not a TSO command line.

1. From the ISPF Primary Option Menu, jump to the Data Set Utility.

```
=3.2
```

2. Enter a new data set name, then use the Allocate (A) line action. Set RECFM to FB and LRECL to 80.

3. Set Directory blocks: 0 for a sequential (PS) data set, or a non-zero number for a partitioned (PDS) library.

Check yourself: the new data set appears in a 3.4 list with the organization you chose, directory blocks above zero make it a PDS, zero makes it PS.

Why it matters: the directory-block count is the single choice that decides PS versus PDS, and it is the most common point of confusion when allocating.

Blue-Team angle: unexpected data set creation in sensitive high-level qualifiers can indicate staging by an attacker. Allocation events belong in the audit trail.

Lab 12.3

Read a data set's attributes with LISTDS

Goal: use LISTDS to read a data set's RECFM, LRECL, and DSORG, and tell a PS from a PO.

Background: before you edit or process a data set, its attributes tell you what it is. LISTDS is the quickest way to find out.

1. At READY or ISPF option 6, list a known data set's attributes.

```
LISTDS 'SYS1.PARMLIB'
```

2. Now list its members to confirm it is partitioned.

```
LISTDS 'SYS1.PARMLIB' MEMBERS
```

Check yourself: the first command shows RECFM FB, LRECL 80, DSORG PO; the second lists members such as ALLOC00 and BPXPRM00. DSORG PO plus a member list confirms a partitioned data set.

Why it matters: reading attributes first prevents the classic mistakes, trying to browse a PDS as if it were flat, or editing a member without knowing the record width the file expects.

Blue-Team angle: LISTDS also reveals the SECURITY field and the volume, so it is a quick way to check whether a sensitive data set is RACF-protected and where it physically lives, exactly the reconnaissance a defender does when auditing storage.

Listing everything you own

One command inventories all the data sets cataloged under a high-level qualifier. LISTCAT LEVEL is how you see, at a glance, everything you own:

IBMUSER.4CHAR.PIN	PS FB LRECL=80
IBMUSER.COBOL.LAB	PO FB LRECL=80
IBMUSER.JCL.LAB	PO FB LRECL=80
IBMUSER.PDS.CODE	PO FB LRECL=80
IBMUSER.REXX.LAB	PO FB LRECL=80

LISTCAT LEVEL(IBMUSER): every data set under one qualifier, with its organization.

Each line shows a data set and its organization, PS for the single flat file, PO for the partitioned libraries. Reading this list is how you take stock of your own storage, and how a reviewer takes stock of someone else's.

Lab 12.4

Inventory your data sets with LISTCAT

Goal: use LISTCAT LEVEL to list every data set under your high-level qualifier and read each one's organization.

Background: before you can manage storage you have to see it. LISTCAT LEVEL is the quickest inventory there is.

1. At READY, list everything under your qualifier.

LISTCAT LEVEL (IBMUSER)

Check yourself: the output lists your data sets, each tagged PS or PO. Count how many are partitioned libraries versus flat files, that ratio tells you how your work is organized.

Why it matters: an inventory is the first step in any cleanup, migration, or audit. LISTCAT LEVEL gives it in one line.

Blue-Team angle: LISTCAT LEVEL against a sensitive qualifier maps what data exists and how it is organized, reconnaissance for an attacker, and an inventory control for a defender.

Browsing a member

A partitioned data set is a library, so you reach its contents a member at a time. From the 3.4 Data Set List you type B beside a library to see its members, then B beside a member to browse it read-only, or E to edit. Browsing first is the safe habit, you read a member without any risk of changing it, which matters when the library is something like the system parameter library:

```
SYS1.PARMLIB MEMBERS
--MEMBER--
ALLOC00
BPXPRM00
CLOCK00
COMMND00
CONSOL00
```

The member list of a PDS, each name is a small file inside the library.

Each of these members is an independent file you can browse or edit on its own, which is exactly why partitioned data sets are used for source, JCL, and configuration: many related pieces, one name, one place to manage them.

Lab 12.5

Browse a member of a library

Goal: open a partitioned data set in 3.4, list its members, and browse one read-only.

Background: browsing is the safe way to read a member (no chance of accidental change) and it is how you inspect libraries you do not own.

1. In 3.4, type B beside a library such as IBMUSER.JCL.LAB to list its members.
2. Type B beside one member to browse it, then PF3 to return.

Check yourself: the member list appears, and browsing a member shows its contents read-only. PF3 steps you back out to the member list and then the data set list.

Why it matters: most of what you read on a mainframe (JCL, source, configuration) lives in library members. Browsing them confidently is a daily skill.

Blue-Team angle: browse access to a sensitive library reveals its contents without leaving an edit trail. Who can browse which libraries is a real access-control question, not just who can change them.

Checkpoint: can you allocate a PDS, list it in 3.4, browse a member, and explain the difference between a PS and a PO dataset and what the directory-block count decides? Datasets are where everything on z/OS is stored, this is bedrock.

Almost everything on z/OS lives in a dataset, and this chapter laid out the model: sequential versus partitioned, members inside a PDS, the DCB attributes that describe a file, and the catalog that finds it. Working the ISPF 3.4 list and the allocate panel against real Gibson output turned the theory into muscle memory. With datasets understood, VSAM and the editor build directly on top.

What comes next

You can now create datasets, find them through the catalog, list them with 3.4, and look inside them. Sequential and partitioned datasets cover most of what you will handle, but there is a third, more powerful organization built for keyed, indexed, record-level access, the one that sits under CICS, Db2, and most of the system's own data. That is VSAM, and the utility that builds and manages it, IDCAMS, is the subject of the next chapter.

CHAPTER 13

VSAM & IDCAMS

By the end of this chapter you will be able to:

Distinguish the VSAM dataset types: KSDS, ESDS, RRDS, and LDS.

Define a VSAM cluster with IDCAMS DEFINE CLUSTER.

Read a cluster's anatomy from LISTCAT output.

Delete a cluster cleanly with DELETE.

Tell interactive IDCAMS functions from batch-only ones.

SEQUENTIAL AND PARTITIONED DATASETS, FROM the last chapter, are read from the front. That is fine for source code and configuration, but it is hopeless for the thing a bank actually does: look up account number 4471-9920 and read that one record, now, out of millions. For that you need keyed, indexed, record-level access, and on z/OS that means VSAM, the Virtual Storage Access Method. Almost every online system you will meet later sits on it: CICS files, much of Db2, and the RACF security database itself are VSAM underneath. This chapter explains VSAM and the utility that builds and manages it, IDCAMS.

What VSAM is

VSAM comes in four flavors, but one dominates. A key-sequenced dataset (a KSDS) stores records in order of a key field and lets you retrieve any record directly by its key, which is exactly the account-lookup problem. The others are the entry-sequenced dataset (ESDS, records in arrival order, used by some logging and Db2 internals), the relative-record dataset (RRDS, addressed by record number), and the linear dataset (LDS, a byte stream used by Db2 and the system). For everyday work, KSDS is the one that matters, so it is the one we build.

A KSDS is not a single object but a cluster: two components that travel together. The DATA component holds the records themselves; the INDEX component holds the keys and points to where each record lives, so a lookup walks the index to find the data. You will see both appear the moment you create a cluster, because IDCAMS builds them for you.

IDCAMS, the access method services utility

IDCAMS (Access Method Services) is the tool for everything VSAM. It runs as a batch step, PGM=IDCAMS, reading its commands from SYSIN, and Gibson also lets you issue its core commands interactively from READY. The commands you need first are DEFINE to create a cluster, LISTCAT to examine one, and DELETE to remove it; in batch you add REPRO to load and copy records and PRINT to dump them. We will take them in that order.

Defining a cluster

DEFINE CLUSTER creates a KSDS. You name it, declare it INDEXED, give the key its length and offset with KEYS, set the record size, and ask for some space:

```
DEFINE CLUSTER (NAME(IBMUSER.CUST.KSDS) INDEXED KEYS(8 0) -
    RECORDSIZE(80 80) TRACKS(2 1))
IDCAMS  SYSTEM SERVICES
IDC0508I DATA ALLOCATION STATUS FOR VOLUME SBVSM1 IS 0
IDC0512I NAME GENERATED- (D) IBMUSER.CUST.KSDS.DATA
IDC0509I INDEX ALLOCATION STATUS FOR VOLUME SBVSM1 IS 0
IDC0512I NAME GENERATED- (I) IBMUSER.CUST.KSDS.INDEX
IDC0181I STORAGECLASS USED IS SCSTD
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
```

Read the response: IDCAMS allocated the data and index, and (this is the part worth noticing) it generated the two component names for you, IBMUSER.CUST.KSDS.DATA and IBMUSER.CUST.KSDS.INDEX, then reported a condition code of zero. KEYS(8 0) means the key is eight bytes starting at offset zero in each record; RECORDSIZE(80 80) sets the average and maximum record length. In a real shop this same command lives inside JCL, where a trailing hyphen continues each line, exactly as it appears in Gibson's own vulnerable banking application:

```
//DEFCLUS  EXEC PGM=IDCAMS
//SYSPRINT DD SYSOUT=*
//SYSIN    DD *
DEFINE CLUSTER (                                -
    NAME( IBMUSER.CUST.KSDS )                  -
    VOLUME( SBVSM1 )                          -
    INDEXED                                   -
    KEYS( 8,0 )                               -
    RECORDSIZE ( 80,80 )                      -
    RECORDS( 500 )                            -
)                                              -
DATA ( NAME(IBMUSER.CUST.KSDS.DATA))         -
INDEX ( NAME(IBMUSER.CUST.KSDS.INDEX))
/*
```

The same DEFINE in batch form: an IDCAMS step reading control statements from SYSIN.

Examining a cluster: LISTCAT ALL

LISTCAT with the ALL keyword is how you read a cluster's anatomy in full, its associations, its attributes, and its allocation:


```

LISTCAT ENT(IBMUSER.CUST.KSDS) ALL
IDCAMS  SYSTEM SERVICES
CLUSTER ----- IBMUSER.CUST.KSDS
      IN-CAT --- USERCAT.VSAM
      HISTORY
      OWNER-IDENT-----IBMUSER      CREATION-----2026.181
      ASSOCIATIONS
      DATA-----IBMUSER.CUST.KSDS.DATA
      INDEX-----IBMUSER.CUST.KSDS.INDEX
      SMSDATA
      STORAGECLASS ---SCSTD      DATACLASS -----DCVSAM
DATA ----- IBMUSER.CUST.KSDS.DATA
      ATTRIBUTES
      KEYLEN----- 8      RKP----- 0
      AVGLRECL----- 80      MAXLRECL----- 80
      CISIZE----- 4096      SHROPTNS(2 3) INDEXED
      ALLOCATION
      SPACE-TYPE-----TRACK      SPACE-PRI--- 2
      VOLUME-----SBVSM1      SPACE-SEC--- 1
INDEX ----- IBMUSER.CUST.KSDS.INDEX
      ATTRIBUTES
      KEYLEN----- 8      LEVELS----- 1
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

```

LISTCAT ALL: the cluster, its DATA and INDEX components, and their attributes.

Everything that defines the cluster is here. The ASSOCIATIONS section confirms the two components belong to the cluster. Under DATA, the ATTRIBUTES tell you the story: KEYLEN 8 and RKP 0 (the relative key position) say the key is the first eight bytes, matching the KEYS(8 0) you specified; AVGLRECL and MAXLRECL are the record sizes; CISIZE 4096 is the control interval, VSAM's unit of transfer. The INDEX component reports its own KEYLEN and how many LEVELS deep its tree runs. Learning to read this output is learning to read VSAM, and it is the first thing you do when a cluster misbehaves.

Deleting a cluster

Because a cluster is really three catalog entries (the cluster and its two components) deleting it cleans up all three at once. DELETE with CLUSTER and PURGE does exactly that:

```

DELETE IBMUSER.CUST.KSDS CLUSTER PURGE
IDC0550I ENTRY (C) IBMUSER.CUST.KSDS DELETED

```

```
IDC0550I ENTRY (D) IBMUSER.CUST.KSDS.DATA DELETED
IDC0550I ENTRY (I) IBMUSER.CUST.KSDS.INDEX DELETED
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
```

Three IDC0550I lines, one for the cluster (C), one for the data (D), one for the index (I), and a clean condition code. PURGE overrides any retention period; without it, IDCAMS would refuse to delete a dataset whose expiration date had not passed.

Why VSAM matters for security

This is not an academic detour. The RACF database (the file that decides who can do anything on the system) is itself a VSAM dataset, SYS1.RACFDS. CICS application files are VSAM. Db2 table spaces sit on VSAM linear datasets. That makes VSAM the substrate of the entire online and security world, and therefore a target: an attacker who can read or copy the RACF database with REPRO, or who finds a CICS file with weak protection, is operating on VSAM. When you reach the RACF database chapter in Part IV and the kill-chain in Part IX, you will be applying exactly the IDCAMS skills from this chapter to the most sensitive datasets on the system, and learning, from the defender's side, how to make sure no one else can.

The four kinds of VSAM data set

VSAM is not one thing but a family, and the differences decide what a data set is good for. Four types cover the ground:

Type	Full name	How records are found
KSDS	Key-Sequenced Data Set	By a key field, the common choice
ESDS	Entry-Sequenced Data Set	By physical order of arrival
RRDS	Relative Record Data Set	By a relative record number
LDS	Linear Data Set	As a byte stream, used by Db2 and others

The VSAM family: pick the type that matches how you retrieve records.

A KSDS is what most people mean by “a VSAM file”: records are found by a key, so it behaves like an indexed table, and it is the type you defined earlier. An ESDS keeps records in the order they arrived and is read sequentially. An RRDS addresses records by number, like slots in an array. An LDS is a flat span of bytes with no record structure at all, the form Db2 table spaces and other system components build on. Choosing the type is the first design decision when you create VSAM data, and it follows directly from how the application needs to reach its records.

Reading a cluster's anatomy

When you defined the KSDS, IDCAMS generated two names (one ending in .DATA and one in .INDEX. That is the anatomy of a KSDS: the DATA component holds the records, and the INDEX component holds the key index that finds them quickly. LISTCAT with the ALL option shows this structure in full) the key length and its position (KEYLEN and RKP), the control-interval size, the space allocation, and the record counts, which is how you inspect

a cluster you did not create. You do not need to memorize every field; you need to recognize that a cluster is a container for a DATA and an INDEX component, and that LISTCAT is where you read their details.

Why VSAM matters for security

This is not an academic detour. The RACF database itself (the file that decides who can do anything on the system) is a VSAM data set. CICS application files are VSAM. Db2 table spaces sit on VSAM linear data sets. That makes VSAM the substrate of the entire online and security world, and therefore a target: a reader who can copy the RACF database, or reach a CICS file with weak protection, is operating on VSAM. When you reach the RACF chapter in Part IV and the kill-chain in Part IX, you will apply exactly these IDCAMS skills to the most sensitive data sets on the system, and learn, from the defender's side, how to make sure no one else can.

Hands-On Labs

These labs assume you are logged on as IBMUSER at the READY prompt.

Lab 13.1

Define a VSAM cluster

Goal: create a key-sequenced (KSDS) cluster with IDCAMS DEFINE CLUSTER.

Background: VSAM clusters underpin most application data on z/OS, and defining one is where that work starts.

1. Define an indexed cluster.

```
DEFINE CLUSTER (NAME(IBMUSER.TEST.KSDS) TRACKS(1 1) RECORDSIZE(80 80) KEYS(8 0)
INDEXED)
```

```
IDCAMS  SYSTEM SERVICES
```

```
IDC0508I DATA ALLOCATION STATUS FOR VOLUME SBVSM1 IS 0
```

```
IDC0512I NAME GENERATED- (D) IBMUSER.TEST.KSDS.DATA
```

```
IDC0512I NAME GENERATED- (I) IBMUSER.TEST.KSDS.INDEX
```

```
IDC0181I STORAGECLASS USED IS SCSTD
```

Check yourself: IDCAMS reports a DATA component and an INDEX component generated, on volume SBVSM1, with a condition code of 0. A KSDS always has both.

Why it matters: the two generated names (.DATA and .INDEX) are the anatomy of a KSDS, and recognizing them is half of reading VSAM.

Blue-Team angle: VSAM datasets hold high-value application data. Who can DEFINE and DELETE them, and how they are RACF-protected, is a real control worth checking.

Lab 13.2

Delete a cluster cleanly

Goal: remove a VSAM cluster and all of its components with DELETE.

Background: cleaning up test clusters (and understanding how deletion cascades) is routine VSAM housekeeping.

1. Delete the cluster, purging it.

```
DELETE IBMUSER.TEST.KSDS CLUSTER PURGE
IDC0550I ENTRY (C) IBMUSER.TEST.KSDS DELETED
IDC0550I ENTRY (D) IBMUSER.TEST.KSDS.DATA DELETED
IDC0550I ENTRY (I) IBMUSER.TEST.KSDS.INDEX DELETED
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0
```

Check yourself: all three entries (cluster (C), data (D), and index (I)) are deleted, and the condition code is 0. Deleting the cluster removes its components with it.

Why it matters: DELETE CLUSTER cascades to the DATA and INDEX components, so you rarely delete them individually.

Blue-Team angle: deleting a VSAM cluster is destructive. DELETE authority should be tightly held, backups should exist, and the action belongs in the audit trail.

Lab 13.3

Read a cluster's anatomy

Goal: use LISTCAT to see the DATA and INDEX components of a KSDS.

Background: every KSDS is really two components working together. LISTCAT is how you confirm that structure on a cluster you did not create.

1. Define a cluster (or reuse the one from Lab 13.1), then list it with the ALL option.

```
LISTCAT ENTRIES(IBMUSER.TEST.KSDS) ALL
```

Check yourself: the output shows a CLUSTER entry with a DATA component and an INDEX component beneath it, each with its own attributes, key length and position, control-interval size, and allocation. Two components under one cluster name confirm a KSDS.

Why it matters: reading a cluster's anatomy is how you understand data you inherit. The DATA/INDEX split is the single most important thing to recognize.

Blue-Team angle: LISTCAT ALL against a sensitive cluster reveals its size, location, and structure, useful to a defender auditing where high-value data lives, and to an attacker mapping it. Access to catalog detail is itself worth controlling.

IDCAMS is the one tool to know

Almost everything you do with VSAM runs through one utility: IDCAMS, the Access Method Services program. DEFINE CLUSTER creates a data set, DELETE removes it, LISTCAT inspects it, REPRO copies records in and out, and PRINT shows their contents. You run IDCAMS as a batch step or from a TSO session, feeding it commands the same way either place. Here is a DEFINE creating a KSDS, and the messages it returns:

```
IDCAMS  SYSTEM SERVICES
IDC0508I DATA ALLOCATION STATUS FOR VOLUME SBVSM1 IS 0
```

```

IDC0512I NAME GENERATED- (D) IBMUSER.TEST.KSDS.DATA
IDC0509I INDEX ALLOCATION STATUS FOR VOLUME SBVSM1 IS 0
IDC0512I NAME GENERATED- (I) IBMUSER.TEST.KSDS.INDEX
IDC0181I STORAGECLASS USED IS SCSTD
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

```

IDCAMS DEFINE creating a KSDS: note the generated DATA and INDEX components.

The two IDC0512I NAME GENERATED lines are the DATA and INDEX components being created automatically, and IDC0001I with condition code 0 confirms success. This is a relief in practice: rather than a dozen tools, VSAM has essentially one, and learning its handful of commands covers creating, deleting, examining, and copying every VSAM data set you will meet.

Deleting a cluster cleanly

Deleting VSAM is as important as creating it, and IDCAMS reports exactly what it removed. DELETE with the CLUSTER and PURGE options takes out the cluster and both of its components in one action:

```

IDC0550I ENTRY (C) IBMUSER.TEST.KSDS DELETED
IDC0550I ENTRY (D) IBMUSER.TEST.KSDS.DATA DELETED
IDC0550I ENTRY (I) IBMUSER.TEST.KSDS.INDEX DELETED
IDC0001I FUNCTION COMPLETED, HIGHEST CONDITION CODE WAS 0

```

DELETE removes the cluster (C) and both the DATA (D) and INDEX (I) components.

The three IDC0550I lines confirm the cluster and its DATA and INDEX components were all removed, and IDC0001I with condition code 0 confirms success. Seeing all three is how you know a KSDS was deleted completely rather than leaving an orphaned component behind.

Lab 13.4

Delete a cluster and confirm all three parts

Goal: delete a KSDS and read the IDC0550I messages that confirm the cluster and both components are gone.

Background: a half-deleted VSAM data set leaves an orphaned component that blocks re-creation. Reading the delete messages is how you avoid that.

1. Delete the cluster you created earlier, with CLUSTER and PURGE.

```
DELETE IBMUSER.TEST.KSDS CLUSTER PURGE
```

Check yourself: three IDC0550I lines report ENTRY (C), (D), and (I) DELETED, and IDC0001I ends with condition code 0. All three parts gone means a clean delete.

Why it matters: clean deletion prevents the orphaned-component errors that confuse beginners when they try to redefine a cluster that was only partly removed.

Blue-Team angle: deletion of VSAM data (especially security-relevant clusters) is an auditable event. Knowing what a clean delete looks like helps a defender spot an incomplete or suspicious one.

Checkpoint: can you define a KSDS, read its DATA and INDEX components in LISTCAT, and delete it cleanly, and say which IDCAMS functions only run in batch? VSAM underpins CICS, Db2, and most application data, so these mechanics pay off everywhere.

VSAM is the access method behind most serious application data on z/OS, and IDCAMS is the tool that manages it. This chapter walked the cluster types, defined and deleted a KSDS against real Gibson output, and read a cluster's DATA and INDEX anatomy from LISTCAT, while being honest about which functions are batch-only. That foundation reappears every time an application's data is on the table.

What comes next

You can now create, inspect, and delete the keyed datasets that the online world runs on. So far, though, every dataset we have made has been empty, we have built containers without putting anything in them. It is time to learn the tool you will use more than any other to create and change the contents of a dataset: the ISPF editor. That is the next chapter, and it is one you will keep coming back to.

CHAPTER 14

The ISPF Editor

By the end of this chapter you will be able to:

- Open data in Edit, View, and Browse and know the difference.
- Use the line commands: insert, delete, repeat, copy, and move.
- Find and change text, including the ALL scope keyword.
- Save, cancel, and exit safely.
- Edit JCL and config the way every later chapter will need.

EVERYTHING UNTIL NOW HAS EITHER looked at datasets or created empty ones. The editor is where you put something inside them, and you will reach for it more than any other tool on the system. You write JCL in it, fix configuration members with it, build REXX and COBOL in it, and inspect data through it. It is a full-screen editor that predates almost every convention you know, and once its handful of commands are in your fingers it is genuinely fast. This chapter makes it familiar.

Edit, View, and Browse

There are three ways to open a dataset, and choosing the right one is a small safety habit. Edit, option 2 or the E line command in the 3.4 list, opens it for change and lets you save. View opens it for change but will not let you save over the original, useful when you only mean to look but might want to copy something out. Browse, the B line command, is strictly read-only: you can scroll and search but not alter a byte, which is the correct choice for production data you must not risk. When in doubt, browse.

The editor screen

Open a member for edit and this is what you see:

```
EDIT  IBMUSER.TEST.CNTL(HELLO)                Columns 00001 00072
Command ===>                                Scroll ===> PAGE
***** ***** Top of Data *****
000100  //HELLO      JOB (ACCT),'IBMUSER',CLASS=A,MSGCLASS=X
000200  //STEP1     EXEC PGM=IEFBR14
000300  //SYSPRINT  DD SYSOUT=*
000400  //SYSIN     DD *
000500      HELLO, MAINFRAME WORLD
000600  /*
***** ***** Bottom of Data *****
```

F1=Help F2=Split F3=Exit F5=Rfind F6=Rchange F7=Up F8=Down F9=Swap

The ISPF editor. The numbers down the left are the prefix area, not data.

Take it apart. The top line names what you are editing and the column range in view. Below it are the Command line, where primary commands go, and the Scroll field, which sets how far PF7 and PF8 move. The body is bracketed by Top of Data and Bottom of Data markers so you always know where the content begins and ends. And the six-digit numbers down the left (000100, 000200) are the prefix area: they are line numbers, not part of your data, and they are also where you type line commands. That dual role of the prefix area is the one idea that makes the editor click.

Line commands: editing by the prefix

Line commands are typed over the numbers in the prefix area and take effect when you press Enter. They are terse and fast:

```
I          insert one line below this one
In         insert n lines (e.g. I3)
D          delete this line
Dn         delete n lines
DD .. DD   delete a block (mark first and last lines)
R          repeat this line
C / CC     copy a line / a block ...
M / MM     move a line / a block ...
A / B     ... to After or Before this line (the destination)
```

So typing I over a line's number and pressing Enter opens a fresh line beneath it; I3 opens three; D deletes; R repeats. Block forms use a doubled letter to mark a range: type DD on the first line and DD on the last, press Enter, and everything between them goes. Copy and move work the same way but need a destination, so you pair C or CC with an A (after) or B (before) on the target line. None of this uses the mouse or a menu; it is all letters typed over numbers, and it is why an experienced user restructures a file faster than they could describe it.

Primary commands: FIND, CHANGE, and saving

Primary commands go on the Command line and act on the whole file. FIND, abbreviated F, locates text and positions you on it:

```
F WORLD
==MSG> FOUND 'WORLD' on line 2
```

CHANGE, abbreviated C, replaces one string with another. Give it the old text, the new text, and (if you want every occurrence) the keyword ALL at the end:

```
C WORLD MAINFRAME ALL
==MSG> 3 CHANGE(S) MADE
```


The editor reports what it did (found on a line, or a count of changes made) so you are never guessing. Two function keys make these repeatable: PF5 (Rfind) finds the next occurrence, and PF6 (Rchange) repeats the last change, which together let you walk through a file confirming each edit. Note the order Gibson expects: the scope keyword ALL comes last, after both strings.

When you are done, three commands end the session. SAVE writes your changes and keeps you in the editor. END, or PF3, saves and leaves. CANCEL discards every change since the last save and leaves, the escape hatch when an edit has gone wrong. Get those three straight and you can edit fearlessly, because you always know how to keep your work and how to throw it away.

NOTE

Because the mail folders from Chapter 11 are ordinary datasets, you can open IBMUSER.MAIL.IN in the editor and read your inbox as raw text, =MSG= blocks and all. You would not normally edit it by hand, but being able to is a vivid reminder of the lesson that keeps recurring: on z/OS, it is datasets all the way down, and the editor opens any of them.

The line commands

Before the commands, get your bearings on the screen itself. An edit session looks like this:

```
EDIT  IBMUSER.TEST.DATA                      Columns 00001 00072
Command ===>                                Scroll ===> CSR
***** ***** Top of Data *****
000100  HELLO WORLD
000200  THIS IS LINE TWO
000300  THIRD LINE HERE
***** ***** Bottom of Data *****
```

An edit session: the Command line at the top, prefix numbers on the left, Top and Bottom of Data markers.

Three regions matter. The Command line at the top takes primary commands that act on the whole member. The prefix area (the six-digit numbers down the left) takes line commands that act on one line. And the Top of Data and Bottom of Data markers bound the editable text. With that map in mind, the line commands make immediate sense.

The editor's real power is in the prefix area, the six-digit numbers down the left of each line. Type a short command over those digits and press Enter, and the editor acts on that line. These are the line commands you will use constantly:

Type in the prefix	Effect
I / In	Insert one line / n lines below
D / Dn	Delete one line / n lines
DD ... DD	Delete a block, marked at both ends
R / Rn	Repeat a line once / n times
C / CC ... CC	Copy a line or a block (with A or B)

Type in the prefix	Effect
M / MM ... MM	Move a line or a block (with A or B)
A / B	Destination: after / before this line
X / XX ... XX	Exclude lines from the display

The prefix-area line commands, the heart of full-screen editing.

Blocks are the idea worth practicing. To copy a run of lines, type C at the top and C at the bottom (or CC on both) then A or B on the destination line, and the whole block moves in one action. Move works the same way with M. Once this becomes muscle memory, restructuring a member is a matter of a few keystrokes rather than retyping.

The primary commands

Alongside the line commands, the Command line at the top of the screen takes primary commands that act on the whole member. FIND and CHANGE you have met; these are the rest of the everyday set:

Command line	Effect
FIND string / F string	Find the next occurrence (PF5 repeats)
CHANGE a b ALL / C a b ALL	Change every a to b (ALL goes last)
SAVE	Write the member, stay in edit
END / PF3	Save and leave
CANCEL	Leave without saving any changes
RESET	Clear pending line commands and messages
COLS	Show a column ruler line
TOP / BOTTOM / M / L	Jump to the top or bottom of the data

The primary commands you type on the editor Command line.

Scrolling, columns, and getting around

A member is usually larger than the screen, so moving around is a skill in itself. PF7 and PF8 scroll up and down; PF10 and PF11 scroll left and right when a record is wider than the screen. The SCROLL field in the top corner controls how far each keystroke moves (PAGE for a whole screen, HALF for half, CSR to move relative to the cursor) and setting it deliberately makes navigation feel effortless. The columns matter too: JCL must live within columns 1 to 71, and the COLS command draws a ruler so you can see exactly where you are. For source that has line numbers in columns 73 to 80, the NUMBER and UNNUM commands turn that numbering on and off. None of this is difficult, but knowing it is the difference between fighting the editor and flowing through it.

Hands-On Labs

These labs assume you have a member open for edit (ISPF option 2, or E from a 3.4 list).

Lab 14.1

Find text in the editor

Goal: locate a string in a member with the FIND command and jump between matches.

Background: finding the right line quickly is the editor's most-used skill, especially in long members.

1. On the editor Command line, find a word.

`F WORLD`

2. Press PF5 (Rfind) to jump to the next occurrence.

Check yourself: the editor positions the cursor on the first line containing the word and reports the find on the message line; PF5 moves to the next match.

Why it matters: FIND plus Rfind is how you navigate a large member without scrolling line by line.

Blue-Team angle: searching source and configuration for secrets (hard-coded passwords, keys, surrogate definitions) is a technique both attackers and defenders use. The editor's FIND is where it starts.

Lab 14.2

Change every occurrence

Goal: replace every occurrence of a word using CHANGE with the ALL scope keyword.

Background: bulk edits (renaming a variable, updating a data set name across a member) are done with CHANGE, not by hand.

1. On the Command line, change one word to another across the whole member. The scope keyword ALL goes last.

`C WORLD MAINFRAME ALL`

Check yourself: the editor reports how many changes it made, and every WORLD is now MAINFRAME. If you put ALL anywhere but last, the change will not span the whole member.

Why it matters: CHANGE ... ALL is powerful and not easily undone (only CANCEL discards it) so save deliberately once you have checked the result.

Blue-Team angle: bulk edits to JCL or configuration can be routine or malicious. Change control and an audit trail are what tell the two apart.

Lab 14.3

Copy a block of lines

Goal: copy a run of lines to another place in the member using block line commands.

Background: block copy and move are the editor's most powerful everyday actions. Doing one deliberately makes the pattern stick.

1. In a member with several lines, type CC in the prefix area of the first and last lines of the block you want to copy.
2. Type A in the prefix area of the line the block should appear after, then press Enter.

Check yourself: the marked block is duplicated immediately after the A line. Using B instead of A would place it before. MM ... MM with A or B moves rather than copies.

Why it matters: restructuring a member (reordering steps in JCL, grouping code) becomes a few keystrokes instead of retyping, once block commands are reflexive.

Blue-Team angle: fluent editing is a working skill for both sides, a defender reshaping configuration under time pressure needs the same reflexes as anyone else.

Lab 14.4

Find and change text

Goal: use the FIND and CHANGE primary commands to locate and edit text across a member.

Background: FIND and CHANGE are how you edit at scale. Doing one deliberately fixes the syntax in your memory.

1. On the Command line, find a word in the member.

```
FIND WORLD
```

2. Now change every occurrence of one word to another.

```
CHANGE WORLD MAINFRAME ALL
```

Check yourself: FIND moves the cursor to the next occurrence and reports it on the status line; CHANGE ... ALL replaces every occurrence and reports how many it made. The ALL keyword must come last.

Why it matters: editing a member of any size by hand is slow and error-prone. FIND and CHANGE turn a sweeping edit into one command.

Blue-Team angle: bulk edits are powerful and easy to over-apply. A careless CHANGE ... ALL on a configuration member can change more than intended, review before you save, and keep a way back.

Lab 14.5

Exclude lines to focus on what matters

Goal: use the X (exclude) line command to hide lines and the display commands to bring them back.

Background: in a large member you often want to see only part of it. Excluding lines collapses the rest out of the way without deleting anything.

1. In a member of several lines, type XX in the prefix area of the first and last lines of a block to exclude, and press Enter.
2. To bring excluded lines back, type RESET on the Command line.

Check yourself: the excluded block collapses to a single dashed marker line showing how many lines are hidden; RESET restores the full display. Nothing is deleted, exclusion only affects what is shown.

Why it matters: excluding lets you work on one region of a large file without distraction, and it pairs with FIND to jump straight to what you need.

Blue-Team angle: reviewing a long configuration or log member is faster when you can hide the noise and focus on the lines that matter, a small skill that pays off under time pressure.

Bounds, tabs, and the ruler

Two editor features repay a little attention. The COLS command draws a ruler across the screen so you can see exactly which column you are in, essential for JCL, which must stay within columns 1 to 71, and for fixed-format data where a field starts at a known position. The ruler looks like this:

```
=COLS> ----+----1----+----2----+----3----+----4----+----5----+----6----+----7--
```

The COLS ruler: a column guide you toggle on when position matters.

The BOUNDS command sets the left and right edges the editor treats as active, so FIND, CHANGE, and shifting operate only within a window of columns, useful when line numbers or sequence fields occupy the last eight columns. You will not need bounds every day, but knowing the ruler and the edges exist turns column-sensitive editing from guesswork into precision.

Lab 14.6

Insert and repeat lines

Goal: use the I and R line commands to add and duplicate lines quickly.

Background: building up a member (a repeating JCL pattern, a table of similar lines) is fastest with insert and repeat rather than typing each line.

1. Type I3 in a line's prefix area to open three new lines below it, and press Enter.
2. Type R on a line to duplicate it, or Rn to duplicate it n times.

Check yourself: I3 opens three blank input lines; R copies the line once and Rn copies it n times. Both act relative to the prefix line you typed them on.

Why it matters: much editing is repetition, similar JCL steps, similar records. Insert and repeat turn that into a couple of keystrokes.

Blue-Team angle: fluent construction of test cases and configurations is a working skill for defenders too, the faster you can build a clean example, the faster you can reproduce and fix a problem.

Checkpoint: can you open a member for edit, insert and delete lines, find a string, change all occurrences of a word, and save, or cancel without saving? The editor is where you will spend real time, so these reflexes matter.

The ISPF editor is where you actually change things on z/OS, and this chapter made it familiar: Edit versus View versus Browse, the prefix-area line commands, FIND and CHANGE with the ALL scope, and the discipline of SAVE, END, and CANCEL, all against real Gibson editor output. Fluency here underpins every later task that edits JCL, configuration, or code.

What comes next

You can now create datasets, shape them, and fill them, the full cycle of making something on the mainframe. Very often the something you make is a job: a piece of JCL

that tells the system to run a program. Once you submit it, it runs out of sight, on the spool, and produces output you need to read. The tool for watching that work and reading that output is SDSF, the System Display and Search Facility, and it is where we go next.

CHAPTER 15

SDSF: Watching the System Work

By the end of this chapter you will be able to:

Enter SDSF and move between its panels from the command line.

Read the ST panel and understand the INPUT, ACTIVE, and OUTPUT job queues.

Find a job by name or owner and interpret its return code.

Open a job's spool data sets and read JESMSGLG, JESJCL, and JESYSMSG.

Use the LOG and DA panels to see live system activity.

Filter the panels with PREFIX and OWNER to find your own work quickly.

WHEN YOU SUBMITTED A JOB in Chapter 10, it vanished. The READY prompt told you it was accepted, gave you a job number, and then nothing (the work ran somewhere you could not see. SDSF, the System Display and Search Facility, is where you see it. It is the operator's window onto the whole system: every job running now, everything queued to run, everything finished and waiting to be read, and the system log itself. If the editor is where you spend your creative time, SDSF is where you spend your watching time, and a security analyst lives in it as much as an operator does) because the spool is where a job's real behavior, intended or not, is written down in full.

Entering SDSF

From the ISPF Primary Option Menu, the letter S takes you to SDSF, and its main menu lists the panels you can switch to:

```

Display  Filter  View  Print  Options  Search  Help
SDSF MENU V2R5M0          GIBPLEX  MVSC 17:44:28          LINE 1-16 (81)
COMMAND INPUT ===>          SCROLL ===> CSR
PREFIX=*          DEST=(ALL)  OWNER=*          SYSNAME=
  NP  NAME        Description          Group
  1   DA          Display active users    Jobs
  2   I           Input queue             Jobs
  3   O           Output queue            Jobs
  4   H           Held output queue        Jobs
  5   ST          Status of jobs           Jobs
  6   LOG         System log              Log

```

The SDSF main menu. Each name is a panel you reach by typing it on the command line.

You rarely navigate this menu by number; instead you type a panel name straight onto the COMMAND INPUT line from anywhere in SDSF. The names worth knowing are DA, the display of active address spaces (what is running right now; I, the input queue of jobs

waiting to start; O and H, the output and held-output queues of finished work; ST, the status of all jobs; and LOG, the live system log. The action bar across the very top) Display, Filter, View, Print, Options, Search, Help (offers the same functions by pointer for those who prefer it. And the four filter fields just below the command line) PREFIX, DEST, OWNER, SYSNAME, narrow what every panel shows, which becomes essential the moment the system is busy with more than your own work.

ST: the status of jobs

Type ST and you see every job the system knows about, including the two we just submitted:

```
SDSF STATUS OF JOBS      GIBPLEX  MVSC 17:44:28      LINE 1-2 (2)
COMMAND INPUT ===>      SCROLL ===> CSR
PREFIX=*      DEST=(ALL)  OWNER=*      SYSNAME=
ACTION=S-Select  ?-Show  P-Purge  C-Cancel  A-Release  /-Values
  NP  JOBNAME  JobID   Owner   Queue    C  ASys  Status  RC    Lines
  1   IBMUSERJ JOB00001 IBMUSER  OUTPUT  A  MVSC  OUTPUT  RC=000 12
  2   IBMUSERJ JOB00002 IBMUSER  OUTPUT  A  MVSC  OUTPUT  RC=000 12
PF1=Help PF3=End PF7=Up PF8=Down
```

SDSF ST: each job with its ID, owner, queue, return code, and line count.

Each row is a job. JOBNAME and JobID identify it (IBMUSERJ, JOB00001) Owner is who submitted it, and Queue is where it sits in its life. A job moves through three queues: it waits on the INPUT queue, runs on the ACTIVE queue, and lands on the OUTPUT queue when it finishes and its output is waiting to be read. If output is held (routed to a class that is not printed automatically) it sits on the held queue, H, instead, which is where most of your jobs will be so that you can read them at leisure.

Two columns earn your attention first. RC is the return code, the single number that says whether the work succeeded; RC=000 here means both jobs ended cleanly. Status echoes the queue state. The line above the column header lists the action characters you type beside a job in the NP column: S selects and browses it, the question mark shows its individual data sets, P purges it from the spool, C cancels one that is still running, and A releases a held job. Those single letters, typed in the prefix column and entered, are how you act on the system's work, the same point-and-act grammar you met in the dataset list and the editor.

A common early mistake is to read RC=000 as “everything is fine” and stop there. The return code reflects the highest step completion code, but a job can end RC=000 and still have done the wrong thing, written to the wrong dataset, skipped a step on a condition, or produced empty output. Always glance at JESYSMSG as well as the return code; the zero is necessary, not sufficient.

DA: what is running right now

Where ST shows every job in any state, DA (Display Active) shows only what is executing at this instant: the live address spaces, including started tasks and TSO users as well as batch jobs:

```
SDSF DISPLAY ACTIVE USERS GIBPLEX MVSC 17:58:50      LINE 1-2 (2)
COMMAND INPUT ==>                                SCROLL ==> CSR
PREFIX=*      DEST=(ALL)    OWNER=*      SYSNAME=
ACTION=S-Select  ?-Show  P-Purge  C-Cancel  A-Release
  NP  JOBNAME  JobID   Owner   Prty Queue   C  ASys   Status
  1   IBMUSERJ JOB00001 IBMUSER  1   OUTPUT  A  MVSC   OUTPUT
  2   IBMUSERJ JOB00002 IBMUSER  1   OUTPUT  A  MVSC   OUTPUT
```

SDSF DA: the address spaces active on the system right now.

DA is the panel an operator watches during the working day. It answers “what is the system doing this second,” and it is where you spot a job consuming too much CPU, a task that should have ended but has not, or simply confirm that a long-running region (CICS, Db2) is up. For a security reader it is reconnaissance in real time: the set of active address spaces tells you what subsystems are running and therefore what is available to interact with.

Reading a job's output

Put a question mark beside a job and SDSF shows you the spool data sets it produced:

```
SDSF JOB DATA SETS JOB00001 GIBPLEX MVSC 17:44:57      LINE 1-3 (3)
ACTION=S-Browse
  NP  DDNAME   StepName ProcStep ByteCount Lines
  1   JESMSG LG JES2           164        4
  2   JESJCL   JES2           141        4
  3   JESYSMSG JES2           169        4
```

Use S n to browse a data set, PF3 to return

A job's spool data sets: the JES message log, the JCL, and the system messages.

Every job produces these three. JESMSG LG is the JES message log (the high-level story of the job entering and leaving the system. JESJCL is the JCL exactly as the system interpreted it, which is invaluable when a symbolic was substituted or a statement overridden. JESYSMSG holds the system messages) allocations, step results, and, when things go wrong, the abend. Type S beside one, or S followed by its number, to browse it. Browsing the whole job stitches them together into the lifecycle you first met in Chapter 3:

```
SDSF OUTPUT DISPLAY IBMUSERJ JOB00001
--- JESMSG LG -----
```

```

000001 $HASP100 IBMUSERJ ON READER
000002 IRR010I USERID IBMUSER IS ASSIGNED TO THIS JOB.
000003 $HASP373 IBMUSERJ STARTED - INIT 1 - CLASS A
000004 IEF403I IBMUSERJ - STARTED - TIME=17.44.57
--- JESJCL -----
000005 //IBUSERJ JOB (ACCT),'TEST',CLASS=A,MSGCLASS=A,USER=IBUSER
000006 //STEP1 EXEC PGM=IEFBR14
000007 //SYSPRINT DD SYSOUT=*
--- JESYSMSG -----
000010 IEF142I IBMUSERJ STEP1 - STEP WAS EXECUTED - COND CODE 0000
000012 $HASP395 IBMUSERJ ENDED - RC=0000

```

The complete output of a successful job, from \$HASP100 to RC=0000.

The job enters (\$HASP100 on the reader) RACF assigns it your identity with IRR010I, JES starts it with \$HASP373, the step runs, and the two lines that decide everything appear in JESYSMSG: IEF142I, STEP WAS EXECUTED, COND CODE 0000, and \$HASP395, ENDED, RC=0000. Those zeros are the whole point. Reading this output fluently (knowing which message tells you what, and which line to jump to when a job fails) is one of the most valuable habits you can build, and it is exactly what the abend chapter trains.

The system log

Finally, the LOG panel is the running narration of the entire system, the same WTO messages that scroll on the operator console, captured and searchable:

```

SDSF SYSTEM LOG          GIBPLEX  MVSC 17:58:50          LINE 1-4 (4)
COMMAND INPUT ==>                               SCROLL ==> CSR
ACTION=S-Browse  FIND string
  NP  SYSNAME  TYPE      VALUE
  1    MVSC    WT0        17.58.50 MVSC $HASP373 IBMUSERJ STARTED
  2    MVSC    WT0        17.58.50 MVSC IEE042I SYSTEM LOG DATA SET
  3    MVSC    WT0        17.58.50 MVSC $HASP395 IBMUSERJ ENDED

```

SDSF LOG: the live system log, every console message searchable with FIND.

Here you see the same \$HASP373 and \$HASP395 messages, but in the context of everything else the system is saying, IPL messages, RACF events, subsystem chatter. The FIND command, on the command line, searches it, which is how you locate a specific event in thousands of lines. For an analyst the log is a primary source: it is where a privilege escalation, a failed logon storm, or an unexpected started task announces itself, and learning to search it is a core detection skill we return to in Part IX.

The output and held queues

ST shows every job, but two panels focus on finished work. The O panel is the output queue (jobs whose output is ready and will be printed or processed by a class. The H panel is the held output queue, and it is where most of your own work lands, because test job classes are usually held: the output waits on the spool for you to read rather than rushing off to a printer. The practical rhythm is simple) submit a job, switch to H or ST, select your job, read it, and when you are done purge it with the P action so the spool does not fill with old output. Housekeeping the held queue is a real operational habit; a spool that fills because no one purged old output can stop a system from accepting new work.

The commands and actions worth memorizing

SDSF rewards a small amount of memorization. These are the panel names you type on the command line and the single-letter actions you type beside a job, the whole working grammar of the tool:

Type this	To do this
DA	Display active address spaces (running now)
ST	Status of all jobs, any queue
I / O / H	Input / output / held-output queues
LOG	The live, searchable system log
OWNER id / PREFIX pat	Filter the panel to your work
S (beside a job)	Select and browse the job or data set
? (beside a job)	Show the job's individual spool data sets
P / C / A	Purge / cancel / release a job
FIND string	Search the current display (PF5 = repeat)

The SDSF grammar: panel names on the command line, actions beside a job.

Hands-On Labs

These labs assume Gibson is running and you are logged on as IBMUSER at the READY prompt. Each takes only a few minutes and builds a habit you will use constantly.

Lab 15.1

Submit a job and read its result

Goal: submit a simple job, find it in SDSF, and confirm it succeeded by reading its return code and system messages.

Background: every batch task on z/OS leaves a trail on the spool. Being able to submit work and then follow that trail to a verdict is the foundation of operating and of investigating. Here you do the full loop once, slowly.

1. From READY, submit the sample job and note the job number it returns.

```
SUBMIT MY.JCL(HELLO)
```

```
IKJ56250I JOB IBMUSERJ(JOB00001) SUBMITTED
```

2. Enter ISPF, then SDSF (option S), and type ST on the command line to list jobs. Find IBMUSERJ with the job number from step 1.

ST

3. Type a question mark beside your job and press Enter to list its spool data sets, then type S beside JESYSMSG to browse the system messages.

Check yourself: in the ST panel your job shows RC=000, and in JESYSMSG you can see the line IEF142I ... STEP WAS EXECUTED - COND CODE 0000 followed by \$HASP395 ... ENDED - RC=0000. If you see both, the job ran correctly.

Why it matters: this submit-watch-verify loop is the heartbeat of mainframe operations. Thousands of production jobs run this way every night, and someone reads exactly these messages to confirm each one. You have just done what a night operator does.

Blue-Team angle: the same spool that confirms success is where a malicious job's behavior is recorded. A defender who reviews JESYSMSG and the LOG for unexpected job names, surrogate USER= values, or steps invoking sensitive programs catches abuse that a return code alone would hide.

Lab 15.2

Filter the panels to your own work

Goal: use the OWNER and PREFIX filters to reduce a busy panel to just the jobs you care about.

Background: on a real system the ST panel can hold thousands of jobs. Filters are how you find yours in seconds rather than scrolling for minutes.

1. On the ST panel, set the owner filter to your userid by typing the command and pressing Enter.

OWNER IBMUSER

2. Observe that only jobs owned by IBMUSER now appear. Then narrow further by job-name pattern with PREFIX.

PREFIX IBMUSER*

Check yourself: the PREFIX= and OWNER= fields near the top of the panel now show your filter values, and every row listed belongs to you. Clear them with OWNER * and PREFIX * to see everything again.

Why it matters: filtering is the difference between SDSF as a firehose and SDSF as an instrument. It is the first thing an experienced user sets on a busy system.

Blue-Team angle: the same filters let a defender hunt. Set OWNER to a service account that should never submit interactive work, or PREFIX to a naming pattern used by an attacker's tooling, and SDSF becomes a focused detection lens.

Lab 15.3

Read the system log with FIND

Goal: open the system log and locate a specific event in it using the FIND command.

Background: the LOG is the system's running narration. Finding one event in it quickly is a core operations and detection skill.

1. From any SDSF panel, type LOG on the command line to open the system log.

LOG

2. Search for the end-of-job message for your job using FIND.

FIND \$HASP395

Check yourself: the cursor lands on a line containing \$HASP395 and your job name, confirming the job ended. Press PF5 (Rfind) to jump to the next occurrence.

Why it matters: real incidents are found by searching logs for the one line that matters among thousands. The mechanics you just used are the same ones a responder uses under pressure.

Blue-Team angle: searching the LOG for ICH408I access-violation messages, repeated failed logons, or unexpected started tasks is front-line detection on z/OS. You will build whole investigations on this single command in Part IX.

Lab 15.4

Purge finished output

Goal: clean up the held-output queue by purging a job you have finished reading.

Background: finished output sits on the spool until someone removes it. Purging your own old jobs is routine housekeeping that keeps the spool healthy.

1. On the ST or H panel, find a job you have already read, type P beside it, and press Enter.

P

Check yourself: the job disappears from the panel, its output has been purged from the spool. A refreshed panel no longer lists it.

Why it matters: on a busy system the spool is a shared, finite resource. A spool that fills because no one purges old output can stop the system from accepting new work.

Blue-Team angle: purge authority is a double edge, it also lets someone erase the evidence of what a job did. Who may purge whose output, and whether purges are logged, is a real audit question.

Checkpoint: can you enter SDSF, list jobs with ST, filter to your own with OWNER, open a job's JESYSMSG, and find an event in the LOG? If any of those is not yet automatic, repeat the matching lab, these five actions are the ones you will perform most in the rest of the book.

SDSF is the window onto the system's work. The ST panel lists every job and its return code; DA shows what is active right now; the H and O queues hold finished output; and LOG is the live, searchable system narration. You act on jobs with single-letter commands in the NP column, read their results in the JESMSG LG, JESJCL, and JESYSMSG spool data sets, and cut a busy system down to size with the PREFIX and OWNER filters. These are the habits of both the operator and the analyst, and you will use them in every chapter that follows.

What comes next

You can now watch the system work and read the full output of anything that runs on it. What you have been reading (those //STEP1 EXEC and //SYSPRINT DD lines in the JESJCL) is JCL, the Job Control Language, and the JES2 subsystem is what schedules and runs it. Now that you can read a job's output, it is time to write the job: the next chapter teaches JCL and the JES2 system that brings it to life.

CHAPTER 16

JES2 & JCL

By the end of this chapter you will be able to:

Read the three core JCL statements: JOB, EXEC, and DD.

Write and submit a minimal job and confirm it ran.

Explain what JES2 does at each stage of a job's life.

Use a procedure, a symbolic parameter, and in-stream data.

Run a TSO command in batch with IKJEFT01.

Recognize the security weight of the USER= parameter on the JOB card.

IN CHAPTER 15 YOU LEARNED to read a job's output (to watch it enter the system, run, and end with a return code. This chapter teaches you to write the job. The language is JCL, Job Control Language, and the subsystem that reads it, schedules it, runs it, and captures its output is JES2, the Job Entry Subsystem. Almost everything that runs in batch on z/OS) the nightly processing of a bank, a report, a database load, a security utility, is a JCL deck handed to JES2. Learning to read and write JCL is learning to make the system do work, and learning where a job's identity comes from is one of the most security-relevant things in this book.

The shape of a job

Here is about the smallest useful job there is, the SURRJOB member from your JCL library:

```
//SURRJOB JOB (ACCT), 'SURROGAT', CLASS=A,MSGCLASS=A,USER=IBMUSER
//STEP1 EXEC PGM=IEFBR14
```

A minimal two-statement job: a JOB card and one EXEC step.

Every JCL statement begins with two slashes in the first two columns, followed by a name, a statement type, and operands. Three statement types do most of the work. The JOB statement (the first line, the "JOB card") names the job (SURRJOB) and carries job-wide information: accounting data in parentheses, the programmer description in quotes, CLASS=A which tells JES2 which input queue to use, MSGCLASS=A which says where the messages go, and (the operand to pay attention to) USER=IBMUSER, which sets the identity the job runs under. The EXEC statement names a step (STEP1) and the program it runs (PGM=IEFBR14, a program that does nothing successfully and is the classic way to test JCL and allocate data sets). A real job adds DD statements (Data Definition) that connect the program to its data sets and SYSOUT.

What JES2 does

When you submit that deck, JES2 takes over, and the messages you read in SDSF narrate exactly what it does:

```

000001 $HASP100 SURRJOB ON READER
000002 IRR010I USERID IBMUSER IS ASSIGNED TO THIS JOB.
000003 $HASP373 SURRJOB STARTED - INIT 1 - CLASS A
000004 IEF403I SURRJOB - STARTED - TIME=23.32.34
--- JESYSMSG -----
000007 IEF236I ALLOC. FOR SURRJOB STEP1
000008 IEF142I SURRJOB STEP1 - STEP WAS EXECUTED - COND CODE 0000
000009 IEF404I SURRJOB - ENDED - TIME=23.32.34
000010 $HASP395 SURRJOB ENDED - RC=0000

```

JES2 narrating a job's life, from the reader to RC=0000.

JES2 reads the deck onto the spool (\$HASP100 SURRJOB ON READER. RACF assigns the job its identity) IRR010I USERID IBMUSER IS ASSIGNED. JES2 then schedules the job from its CLASS A input queue onto an initiator and starts it (\$HASP373 SURRJOB STARTED, INIT 1, CLASS A. The step runs, the system allocates its data sets (IEF236I) and reports the result (IEF142I, COND CODE 0000), and JES2 ends the job and files its output on the output queue) \$HASP395 SURRJOB ENDED, RC=0000. That arc (reader, conversion, execution, output, purge) is the life of every batch job, and JES2 manages all of it. The job classes and MSGCLASS you put on the JOB card are simply how you tell JES2 which queues to use.

A richer deck: procedures and symbolics

Real JCL is rarely two lines. The PROCDEMO member shows the next layer, a procedure, a symbolic parameter, and in-stream data:

```

//PROCJOB JOB (ACCT), 'PROC DEMO', CLASS=A, MSGCLASS=A
//MYPROC PROC WHO=IBMUSER
//STEPA EXEC PGM=IKJEFT01
//SYSTSIN DD *
LISTUSER &WHO
/*
// PEND
//RUN1 EXEC MYPROC, WHO=IBMUSER

```

A procedure (MYPROC) with a symbolic (WHO) and in-stream SYSTSIN data.

A PROC is a reusable block of JCL: everything between the PROC and PEND statements is a named template (here MYPROC) that you invoke with EXEC, just like a program. The WHO=IBMUSER on the PROC line is a symbolic parameter with a default; when RUN1 does EXEC MYPROC, WHO=IBMUSER it passes a value that is substituted wherever &WHO appears. The step runs PGM=IKJEFT01, the TSO terminal monitor in batch, and the DD * statement introduces in-stream data (the lines up to the /* delimiter are fed to the program as SYSTSIN. So this job runs the TSO command LISTUSER IBMUSER in batch. That

pattern) IKJEFT01 with SYSTSIN, is how you run any TSO or RACF command as a job, and it is one you will use and one you will learn to watch for.

The DD statement: connecting a program to its data

The JOB and EXEC statements say who and what; the DD statement (Data Definition) says with which data. A program refers to its files by a symbolic ddname, and the DD statement connects that ddname to a real data set or destination at run time. This indirection is one of JCL's quiet strengths: the same program runs against test data one day and production data the next, changed only by its JCL. Here is a two-step job that creates a data set in the first step and reads it in the second:

```
//MULTI    JOB (ACCT), 'MULTI STEP', CLASS=A, MSGCLASS=A
//STEP1    EXEC PGM=IEFBR14
//NEWDS    DD DSN=IBMUSER.WORK.DATA, DISP=(NEW,CATLG),
//          SPACE=(TRK, (1,1)), DCB=(RECFM=FB, LRECL=80)
//STEP2    EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1   DD DSN=IBMUSER.WORK.DATA, DISP=SHR
//SYSUT2   DD SYSOUT=*
//SYSIN    DD DUMMY
```

A two-step job: DD statements create a data set, then read it.

Read the DD operands. DSN names the data set. DISP is the disposition ((NEW,CATLG) means create it and catalog it, while DISP=SHR means it already exists and may be shared. SPACE requests storage, and DCB carries the record attributes you met in Chapter 12) RECFM and LRECL. Two special forms appear constantly: SYSOUT=* sends output to the spool where SDSF reads it, and DUMMY connects a ddname to nothing at all, which satisfies a program that expects a file it does not actually need. When this job runs, the system reports each allocation and cataloging in the messages:

```
000014 IEF236I ALLOC. FOR MULTI STEP1
000015 IEF285I IBMUSER.WORK.DATA CATALOGED
000016 IEF142I MULTI STEP1 - STEP WAS EXECUTED - COND CODE 0000
000017 IEF236I ALLOC. FOR MULTI STEP2
000020 IEF142I MULTI STEP2 - STEP WAS EXECUTED - COND CODE 0000
000022 $HASP395 MULTI ENDED - RC=0000
```

The system confirms each allocation, the catalog entry, and each step's result.

IEF285I IBMUSER.WORK.DATA CATALOGED confirms the new data set was created and recorded in the catalog, and each step reports its own COND CODE. A multi-step job like this (create, then process) is the backbone of real batch work, and every step's result is visible on the spool.

The statements and operands you will use most

Most JCL is built from a small vocabulary. This table gathers the statements and operands you will reach for again and again:

Statement / operand	Purpose
//name JOB	Names the job; carries account, CLASS, MSGCLASS, USER=
//name EXEC PGM=	Runs a program as a step
//name EXEC procname	Invokes a procedure
//name DD	Connects a ddname to a data set or destination
DSN=	The data set name a DD refers to
DISP=	Disposition: (NEW,CATLG), SHR, OLD, (MOD,KEEP)
SPACE=	How much storage to allocate for a new data set
SYSOUT=*	Route output to the spool for SDSF to read
DD DUMMY / DD *	No data set / in-stream data ending at /*
PROC ... PEND	Define a reusable procedure

A working vocabulary of JCL statements and operands.

Classes, message classes, and the queues

Two small operands on the JOB card decide how JES2 handles the work. CLASS assigns the job to an input class (a queue that an initiator services) which is how a site separates quick jobs from long ones, or interactive test work from overnight production. MSGCLASS assigns the job's messages, and by default its SYSOUT, to an output class, which decides whether the output is printed automatically or held on the queue for you to read in SDSF. A held output class is why your test jobs wait patiently on SDSF's H panel instead of vanishing to a printer. Choosing classes well is how an operations team keeps a busy system flowing; choosing them badly is how one long job blocks a queue that quick jobs needed. You rarely invent classes (a site defines them) but you always pick from them, and knowing what they do turns two mysterious letters on the JOB card into deliberate choices.

Cataloged procedures

The procedure in PROCDEMO was written in-stream, between PROC and PEND, so you could see it. In practice most procedures are cataloged (stored as members of a procedure library, a PROCLIB) and invoked by name alone. When you write EXEC MYPROC, the system searches the PROCLIB concatenation for a member called MYPROC and runs it, exactly as if you had typed it in. This is how sites share standard job skeletons: a compile-and-link procedure, a database-load procedure, a backup procedure, each written once and invoked by hundreds of jobs. You override a procedure's defaults on the EXEC statement (EXEC MYPROC,WHO=EVE) or add and override its DD statements from the calling job, which is how one procedure serves many callers. Cataloged procedures are also a security consideration: a job runs whatever the named procedure contains, so who may change a PROCLIB member matters as much as who may submit the job.

The messages JES2 leaves behind

You have now met most of the messages a job produces. Because you will read them constantly in SDSF, it is worth collecting the important ones and what each announces:

Message	What it announces
\$HASP100	The job has been read onto the spool (on the reader)
IRR010I	RACF has assigned the job its security identity
\$HASP373	The job has started on an initiator, with its class
IEF236I	The system is allocating a step's data sets
IEF285I	A data set was cataloged, kept, or deleted
IEF142I	A step finished, with its COND CODE or ABEND
\$HASP395	The job has ended, with its overall return code

The job-lifecycle messages, gathered for quick reference.

The most common JCL mistake is a column or syntax slip: the // must be in columns 1 and 2, operands must not stray past column 71, and a missing comma or unbalanced quote produces a JCL ERROR, which is different from a bad return code. A JCL error means the job never ran at all; JES2 rejected the deck during conversion. When SDSF shows a job with no step messages and a JCL ERROR in JESYSMSG, look for the punctuation, not the program.

Hands-On Labs

These labs assume you are logged on as IBMUSER with a JCL library to edit (IBMUSER.JCL.LAB).

Lab 16.1

Write and submit a minimal job

Goal: write a two-statement job, submit it, and confirm it ended RC=0000.

Background: IEFBR14 is the “hello world” of JCL, it runs nothing and ends cleanly, so it is the safest way to prove your JCL is correct.

1. Edit a new member and enter a minimal job.

```
//MYTEST JOB (ACCT), 'FIRST JOB', CLASS=A, MSGCLASS=A
//STEP1 EXEC PGM=IEFBR14
```

2. Submit it from the editor or from READY, and note the job number.

```
SUBMIT
```

```
IKJ56250I JOB MYTEST(JOB00001) SUBMITTED
```

3. In SDSF (Chapter 15), find the job and read its JESYSMSG.

Check yourself: the job ends with IEF142I ... COND CODE 0000 and \$HASP395 ... RC=0000. If instead you see a JCL ERROR, check that // is in columns 1-2 and the operands are spelled correctly.

Why it matters: every batch job you ever write starts from this skeleton. Proving the skeleton runs before you add real steps saves hours of debugging.

Blue-Team angle: even a do-nothing job is recorded (job name, owner, time) on the spool and in SMF. Defenders baseline who submits work and when, so an unexpected job from an account that never runs batch stands out.

Lab 16.2

Run a TSO command in batch

Goal: use IKJEFT01 and in-stream SYSTSIN to run a TSO command as a job.

Background: anything you can type at READY can be run in batch this way (reporting, RACF queries, bulk work) which is powerful and worth understanding from both sides.

1. Edit a member with a job that runs LISTUSER in batch.

```
//TSOBATCH JOB (ACCT), 'TSO IN BATCH', CLASS=A, MSGCLASS=A
//STEP1 EXEC PGM=IKJEFT01
//SYSTSPRT DD SYSOUT=*
//SYSTSIN DD *
LISTUSER IBMUSER
/*
```

2. Submit it and, in SDSF, browse the SYSTSPRT output.

Check yourself: the SYSTSPRT data set contains the LISTUSER output (the same text you would see at READY) produced by the batch job. The job ends RC=0000.

Why it matters: IKJEFT01 with SYSTSIN is the bridge between interactive and batch. A huge amount of real automation is exactly this: TSO and RACF commands driven from a job.

Blue-Team angle: because IKJEFT01 can run RACF commands, a job is a way to attempt privilege changes in bulk and away from a watched terminal. Defenders audit batch SYSTSIN for sensitive commands, not just interactive sessions.

Lab 16.3

See where a job's identity comes from

Goal: observe the USER= parameter on the JOB card and the IRR010I message that assigns a job its identity.

Background: a job runs under a security identity, and that identity decides what it is allowed to do. Knowing where it is set is essential to both operating and defending.

1. Submit the SURRJOB member, whose JOB card carries USER=IBMUSER.

```
SUBMIT 'IBMUSER.JCL.LAB(SURRJOB)'
```

2. In SDSF, browse JESMSGGLG and find the identity-assignment line.

```
000002 IRR010I USERID IBMUSER IS ASSIGNED TO THIS JOB.
```

Check yourself: IRR010I names the userid the job runs under. On a job with USER= on the card, that is the requested identity, granted only if the submitter holds surrogate authority for it.

Why it matters: the identity a job runs under, not the person who typed SUBMIT, is what RACF checks for every action the job takes. That distinction is the heart of batch security.

Blue-Team angle: USER= combined with surrogate authority is a legitimate feature, and a classic escalation path when surrogate profiles are too generous. Reviewing who may submit work as whom is a high-value control, and it is exactly what Part IV examines in depth.

Lab 16.4

Write a multi-step job that creates a data set

Goal: write a two-step job that allocates and catalogs a new data set in one step and processes it in the next, and confirm both steps and the catalog entry.

Background: real batch work is rarely a single step, it creates or reads data sets and passes work from one step to the next. This is the smallest realistic example.

1. Edit a member with a two-step job that creates a data set, then reads it.

```
//MULTI    JOB (ACCT) , 'MULTI STEP' , CLASS=A, MSGCLASS=A
//STEP1    EXEC PGM=IEFBR14
//NEWDS    DD DSN=IBMUSER.WORK.DATA, DISP=(NEW,CATLG) ,
//          SPACE=(TRK, (1,1)) , DCB=(RECFM=FB, LRECL=80)
//STEP2    EXEC PGM=IEBGENER
//SYSPRINT DD SYSOUT=*
//SYSUT1   DD DSN=IBMUSER.WORK.DATA, DISP=SHR
//SYSUT2   DD SYSOUT=*
//SYSIN    DD DUMMY
```

2. Submit it and, in SDSF, read JESYSMSG for the catalog and step messages.

Check yourself: JESYSMSG shows IEF285I IBMUSER.WORK.DATA CATALOGED and two IEF142I ... COND CODE 0000 lines, one per step, and the job ends RC=0000. The new data set now appears in a 3.4 list.

Why it matters: create-then-process is the shape of most production batch. Seeing the catalog message and both step results proves the whole job did what its DD statements described.

Blue-Team angle: DISP=(NEW,CATLG) and IEF285I mark data-set creation. A defender watching for new data sets in sensitive high-level qualifiers is reading exactly these messages, batch jobs are a common way to stage data quietly.

Checkpoint: can you write a JOB card and an EXEC step from memory, submit the job, and confirm RC=0000 in SDSF? Can you explain what JES2 does between the reader and the output queue, and what the USER= parameter controls? Those are the skills the rest of the operating and security chapters assume.

JCL is how you tell z/OS to do work, and JES2 is what carries it out. Three statements (JOB, EXEC, and DD) describe a job; JES2 reads the deck, assigns its identity, schedules it by class, runs it, and files its output on the spool you learned to read in Chapter 15. Procedures, symbolics, and in-stream data make real decks reusable, and IKJEFT01 lets a job run any TSO or RACF command. Above all, the JOB card's USER= parameter sets the identity a job runs under, the single most security-relevant field in JCL, and the thread Part IV picks up.

What comes next

Every job in this chapter ended COND CODE 0000 (success. But jobs fail, and when they do the system produces a different and richer kind of output: abend codes, dumps, and the messages that explain what went wrong. The next chapter teaches you to read a failure) to turn a System or User completion code into an understanding of what happened and why.

CHAPTER 17

Abends & Debugging

By the end of this chapter you will be able to:

Tell an abend from a non-zero return code, and know why it matters.

Read an IEA995I symptom dump and find the cause and the failing module.

Distinguish system completion codes from user completion codes.

Recognize the most common abends, S0C7, S806, S913, S322, B37.

Read a security abend (S913) and see it as access control, not a bug.

Know what to do next when a step abends.

EVERY JOB IN THE LAST chapter ended COND CODE 0000. Real jobs fail. When a program does something the system cannot allow (does arithmetic on data that is not a number, calls a module that is not there, touches a data set it is not permitted to) the system stops it immediately. That is an abend, short for ABnormal END, and it is not the same as a bad return code. A return code means the program ran and reported a problem; an abend means the system pulled the plug mid-instruction. Learning to read an abend (to turn a four-character code and a page of hexadecimal into an understanding of what happened) is one of the most useful skills on the platform, and, as you will see, one of the most security-relevant.

Anatomy of an abend

Here is a job that did decimal arithmetic on invalid data (the classic S0C7) as it appears in the job's output in SDSF:

```
000008 IEF450I PAYJOB STEP1 - ABEND=S0C7 U0000 REASON=00000007
000010 IEA995I SYMPTOM DUMP OUTPUT
000011 SYSTEM COMPLETION CODE=0C7 REASON CODE=00000007
000012 DATA EXCEPTION (INVALID DECIMAL DATA)
000014 PSW AT TIME OF ERROR 078D1000 8A1C40C6 ILC 6 INTC 0007
000015 ACTIVE LOAD MODULE ADDRESS=8A1C4000 NAME=PAYCALC
000021 END OF SYMPTOM DUMP JOB PAYJOB STEP STEP1
000022 IEA995I SYSUDUMP TAKEN TO SYS00001
000023 IEF142I PAYJOB STEP1 - STEP WAS NOT EXECUTED - ABEND S0C7
000024 IEF472I PAYJOB STEP1 - COMPLETION CODE - SYSTEM=0C7
000026 $HASP395 PAYJOB ENDED - RC=0012
```

An S0C7 data-exception abend with its IEA995I symptom dump.

Read it from the top. IEF450I announces the abend and the completion code (S0C7) and the reason code. The IEA995I SYMPTOM DUMP then gives you everything you need: the

SYSTEM COMPLETION CODE again, and in plain English the cause (DATA EXCEPTION (INVALID DECIMAL DATA)). The line that usually matters most is the ACTIVE LOAD MODULE: NAME=PAYCALC tells you which program was running when it failed, so you know where to look. The PSW and the general-purpose registers below are for deep diagnosis; you rarely need them for everyday debugging, but they are the raw evidence a dump specialist reads. Finally the system records that a SYSUDUMP was taken, notes IEF142I STEP WAS NOT EXECUTED) a essential phrase, because the step did not complete; it was abnormally terminated, and reports the job's overall result as RC=0012, the conventional code for an abended job.

Each line of the symptom dump answers a specific question, and it helps to know which line to look at for what:

Dump line	What it tells you
SYSTEM COMPLETION CODE	The abend code, what category of failure occurred
REASON CODE	A further code that refines the cause within that category
DATA EXCEPTION (...)	The cause in plain English, read this first
ACTIVE LOAD MODULE NAME	Which program was running when it failed, where to look
PSW AT TIME OF ERROR	The instruction address at the moment of failure
GPR 0-15	Register contents, for deep diagnosis with the full dump
IEA995I SYSUDUMP TAKEN	Where the fuller storage dump was written, if one was requested

How to read a symptom dump: which line answers which question.

System and user completion codes

Completion codes come in two families. A system completion code is set by z/OS itself and is written in hexadecimal with an S prefix (S0C7, S806, S913. A user completion code is set by the application and is written in decimal with a U prefix) U0100, U4039. In the dump above you can see both fields: ABEND=S0C7 U0000 means a system abend with no user code. The rule of thumb is simple: an S-code points you at the system or the environment (bad data, a missing module, a denied resource) while a U-code points you at the application's own logic, and you look it up in that application's documentation.

The abends you will meet most

A handful of system abends account for most of what you will see. S0C7 is a data exception (decimal arithmetic on data that is not valid packed decimal, very often an uninitialized or mis-mapped field. S0C4 is a protection or addressing exception) a program reaching outside the storage it owns. S806 means a module could not be found, a load library is missing from the search order, or a program name is misspelled. S913 is a RACF data-set access denial, which we look at next. S322 means a step ran past its time limit, and B37 or its relatives mean a data set ran out of space. Recognizing these five or six on sight turns most failures from a mystery into a known category with a known first move.

It is worth keeping a short table of them within reach until they become second nature:

Code	Meaning	Typical cause	First move
S0C7	Data exception	Invalid packed decimal in a numeric	Read the dump; find the failing

Code	Meaning	Typical cause	First move
		field	field
S0C4	Protection / addressing	Program referenced storage it does not own	Check the program logic and tables
S806	Module not found	Load library missing or program misspelled	Check JCL, STEPLIB, and the PGM= name
S913	RACF access denied	No authority to a data set or resource	Read the ICH408I; check RACF access
S322	Time limit exceeded	Step ran longer than its TIME= allowed	Raise TIME= or fix a loop
B37	Out of space	Data set filled its space allocation	Increase SPACE= or clean up the volume

The handful of abends that account for most batch failures.

Where the dump goes

When a step abends, the system writes a dump (but only if the JCL asked for one. The symptom dump you just read is always written to the job log, but the fuller storage dump is routed to a DD statement you supply. Three are common, and the one you choose decides how much detail you get. SYSUDUMP gives a formatted, problem-state dump) the everyday choice, readable and enough for most application abends. SYSABEND adds system areas to that, a larger dump for harder problems. SYSMDUMP is an unformatted machine-readable dump for use with the interactive debugger, IPCS. In practice you add a single line to a step so a failure leaves evidence:

```
//SYSUDUMP DD SYSOUT=*
```

That one DD statement is the difference between an abend you can diagnose and one you can only guess at. A job that abends with no dump DD still tells you the completion code in the symptom dump, but you lose the storage detail, so production jobs almost always carry a SYSUDUMP. When you see IEA995I SYSUDUMP TAKEN TO a spool data set, that is the system telling you the dump is waiting for you in SDSF.

You can see the effect in the job's data-set list. Compare an abended job with a clean one and the message data set that carries the dump, JESYSMSG, has grown from a handful of lines to twenty:

```
SDSF JOB DATA SETS JOB00001 GIBPLEX MVSC 23:54:34 LINE 1-3 (3)
```

```
ACTION=S-Browse
```

NP	DDNAME	StepName	ProcStep	ByteCount	Lines
1	JESMSG LG	JES2		158	4
2	JESJCL	JES2		92	3
3	JESYSMSG	JES2		904	20

Use S n to browse a data set, PF3 to return

An abended job's data sets: JESYSMSG is large because it holds the symptom dump.

Return codes, condition codes, and abends

Three terms describe how a step can end, and keeping them straight saves real confusion. A return code (also called a condition code) is a number a program sets to report its own result: 0 for success, 4 for a warning, 8 for an error, 12 for a serious error. The program ran to completion and chose that number; you see it as COND CODE nnnn in IEF142I. An abend is different in kind: the program did not get to choose, because the system stopped it. That is why the message reads STEP WAS NOT EXECUTED rather than COND CODE (there is no return code to report. The job's overall RC, which JES2 prints on the \$HASP395 line, is the highest step condition code, or the conventional 0012 when a step abended. So COND CODE 0008 means "it ran and reported an error," while ABEND S0C7 means "it was killed mid-instruction") two very different situations that call for two very different responses.

A security abend: S913

Not every abend is a bug. Some are the system doing exactly its job, stopping a program from doing something it is not allowed to do. Here is a job that tried to update the RACF database without authority:

```
000008 ICH408I USER(IBMUSER) GROUP(SYS1) NAME(IBMUSER)
000009   SYS1.RACFDS CL(DATASET)
000010   INSUFFICIENT ACCESS AUTHORITY
000011   ACCESS INTENT(UPDATE)   ACCESS ALLOWED(NONE)
000012 IEF450I SECJOB STEP1 - ABEND=S913 U0000 REASON=00000038
000015   SYSTEM COMPLETION CODE=913   REASON CODE=00000038
000016   RACF DATA SET ACCESS DENIED (READ/UPDATE)
000023 IEF142I SECJOB STEP1 - STEP WAS NOT EXECUTED - ABEND S913
```

An S913 abend: the program was denied access, and RACF says so plainly.

This is where debugging and security meet. The abend itself is S913, but the lines above it are the real story: ICH408I, the RACF access-violation message, naming the user, the protected resource (SYS1.RACFDS, the RACF database itself), the access that was attempted (UPDATE), and the access that was allowed (NONE). The program did not malfunction; it was refused, and the refusal terminated the step. To an operator this is a job to investigate. To a defender it is gold: an S913 with an ICH408I is the system telling you, in its own logs, that someone or something attempted to reach a resource it had no right to. Whether that is a misconfigured batch job or a deliberate probe, it is exactly the event you want to see.

The most common debugging mistake is to treat an abend like a return code and simply resubmit the job. An abend means the step was NOT executed (nothing it was supposed to do got done) and rerunning it unchanged will almost always abend again. Read the completion code and the symptom dump first: they tell you the category of failure and, in the ACTIVE LOAD MODULE line, where it happened. Fix the cause, then resubmit.

Putting it together: diagnosing an abend

With the pieces in hand, a real diagnosis follows a short, repeatable path. Start at the bottom of the job and work up: the \$HASP395 line gives the overall result, and an RC=0012 with no useful condition code is your signal that a step abended rather than merely erred. Find the IEF142I line for the failing step (it names the step and the abend code in one place: STEP WAS NOT EXECUTED, ABEND S0C7. Take that code to the IEA995I symptom dump just above it and read the plain-English cause line; for our example, DATA EXCEPTION (INVALID DECIMAL DATA). Then read the ACTIVE LOAD MODULE line to learn which program was running) PAYCALC (so you know whose code or input to examine. At that point you have the three facts that matter: what failed (S0C7), why (invalid decimal data), and where (PAYCALC). Only now do you open the SYSUDUMP for the offset and the registers, and only if the cause is not already obvious. Most abends are solved at the symptom-dump level in under a minute once this path becomes habit; the full dump is for the minority that resist it. The discipline) bottom to top, code then cause then location, is what separates a confident reader of failures from someone who stares at hexadecimal hoping for inspiration.

Hands-On Labs

These labs assume you are logged on as IBMUSER and can submit jobs and read them in SDSF. Each uses a deliberately-named demonstration program that forces a specific, repeatable abend.

Lab 17.1

Read an S0C7 data-exception dump

Goal: provoke an S0C7, then read its symptom dump to find the cause and the failing module.

Background: S0C7 is the abend you will see most. Reading one fluently (cause and location in seconds) is a skill worth drilling.

1. Edit and submit a job whose step forces a data exception.

```
//PAYJOB   JOB (ACCT) , 'PAYROLL' , CLASS=A, MSGCLASS=A
//STEP1    EXEC PGM=BADDEC
```

2. In SDSF, browse the job and find the IEA995I symptom dump.

Check yourself: the dump shows SYSTEM COMPLETION CODE=0C7, the cause DATA EXCEPTION (INVALID DECIMAL DATA), and ACTIVE LOAD MODULE NAME=PAYCALC. The step shows STEP WAS NOT EXECUTED and the job ends RC=0012.

Why it matters: in real shops an S0C7 in a financial program usually means a numeric field held spaces or binary where packed decimal was expected. The dump names the module so you know where to look.

Blue-Team angle: a sudden rash of abends in a normally-clean batch stream can signal corrupted input or tampering. Defenders track abend rates as a health and integrity signal, not just an operations one.

Lab 17.2**Diagnose a module-not-found S806**

Goal: provoke an S806 and recognize it as a load-library or program-name problem rather than a logic error.

Background: S806 is an environment problem, not a data one (the program to run could not be found) so the fix is different from a dump you read line by line.

1. Submit a job that calls a module that is not there.

```
//MODJOB   JOB (ACCT), 'MISSING MOD', CLASS=A, MSGCLASS=A
//STEP1    EXEC PGM=MISSMOD
```

2. In SDSF, read the completion code.

```
000008 IEF450I MODJOB STEP1 - ABEND=S806 U0000 REASON=000000004
000011 SYSTEM COMPLETION CODE=806 REASON CODE=000000004
000023 IEF142I MODJOB STEP1 - STEP WAS NOT EXECUTED - ABEND S806
000024 IEF472I MODJOB STEP1 - COMPLETION CODE - SYSTEM=806
```

Check yourself: the job abends S806. Because the module was never found, there is no application logic to blame, the cause is the search order or a misspelled program name, and that is where you look.

Why it matters: S806 sends you to the JCL and the load-library concatenation, not to the program's code. Knowing the category saves you from debugging the wrong thing.

Blue-Team angle: an S806 for a module that should exist can mean a library was changed or a STEPLIB was redirected. Unexpected load-library changes are worth a defender's attention.

Lab 17.3**Recognize a security abend (S913)**

Goal: provoke an S913 and read it as RACF enforcement, identifying the denied resource and the attempted access.

Background: an S913 is not a malfunction, it is access control working. Reading it correctly is the bridge from operations into security.

1. Submit a job whose step attempts an unauthorized access.

```
//SECJOB   JOB (ACCT), 'RACF DENY', CLASS=A, MSGCLASS=A
//STEP1    EXEC PGM=RACFDENY
```

2. In SDSF, read the ICH408I lines just above the abend.

```
ICH408I USER(IBMUSER) GROUP(SYS1) NAME(IBMUSER)
      SYS1.RACFDS CL(DATASET)
      INSUFFICIENT ACCESS AUTHORITY
      ACCESS INTENT(UPDATE) ACCESS ALLOWED(NONE)
```

Check yourself: the ICH408I names the user, the protected resource (SYS1.RACFDS), the attempted access (UPDATE) and the allowed access (NONE), and the step ends ABEND S913. The job failed because RACF refused it, exactly as designed.

Why it matters: this single message connects two worlds. To an operator it explains a failed job; to a security analyst it is a recorded access violation against one of the most sensitive data sets on the system.

Blue-Team angle: ICH408I with S913 is front-line detection. Searching the log and SMF for these (especially against SYS1.RACFDS, APF libraries, or PARMLIB) surfaces both broken jobs and deliberate probes. You will build investigations on this message in Part IX.

Lab 17.4

Read a time-limit abend (S322)

Goal: provoke an S322 and recognize it as a resource-limit abend rather than a data or logic error.

Background: a step that runs past its allotted CPU time is stopped by the system with an S322. It is one of the few abends that often means “the program was fine but took too long,” which points you at limits and loops, not at the data.

1. Submit a job whose step exceeds its time limit.

```
//TJOB      JOB (ACCT), 'TIME LIMIT', CLASS=A, MSGCLASS=A
//STEP1     EXEC PGM=ABEND322
```

2. In SDSF, read the completion code.

```
000008 IEF450I TJOB STEP1 - ABEND=S322 U0000 REASON=00000000
000011 SYSTEM COMPLETION CODE=322 REASON CODE=00000000
000023 IEF142I TJOB STEP1 - STEP WAS NOT EXECUTED - ABEND S322
```

Check yourself: the step ends ABEND S322 with completion code 322. Nothing is wrong with the data, the step simply ran past its TIME= limit, and the fix is either a higher limit or a loop that never ends.

Why it matters: S322 teaches that not every abend is a defect in the program. Some are the system enforcing a limit, and the right response is to question the limit or the runtime, not the logic.

Blue-Team angle: a job that suddenly starts hitting S322 when it never used to can mean far more data than expected, or a process spinning. Runaway CPU is both an availability concern and, occasionally, a sign of abuse such as crypto-mining on borrowed cycles.

Checkpoint: can you read a symptom dump and name the completion code, the cause, and the failing module? Can you say why an abend is not a return code, and why an S913 is a security event rather than a bug? If so, you can read a failure as fluently as you read a success.

An abend is the system stopping a program mid-instruction, and it is reported with a completion code (a hexadecimal S-code from the system or a decimal U-code from the application. The IEA995I symptom dump turns that code into a cause and, in the ACTIVE LOAD MODULE line, a location; the step shows STEP WAS NOT EXECUTED and the job ends RC=0012. A few codes) S0C7, S0C4, S806, S913, S322, B37, cover most failures, and one of them, S913, is RACF enforcement written into the job log: a debugging skill and a detection skill in the same message.

What comes next

You have now seen z/OS from its classic side (TSO, ISPF, datasets, JCL, the spool. But modern z/OS has a second face: a complete UNIX environment running inside the same operating system, with a shell, a file system, and processes. The next chapter opens OMVS, the z/OS UNIX side, where mainframe and UNIX security models meet) and sometimes collide.

CHAPTER 18

OMVS: The z/OS UNIX Side

By the end of this chapter you will be able to:

Enter the z/OS UNIX shell from TSO and find your bearings.

Navigate the hierarchical file system and read a path.

Read UNIX file permissions and ownership.

List processes and recognize the z/OS UNIX process tree.

Explain where a user's UNIX identity comes from, and what UID 0 means.

See why the UNIX side is both a convenience and an attack surface.

EVERYTHING SO FAR HAS SHOWN you the classic face of z/OS (TSO, ISPF, data sets, JCL, the spool. But z/OS has a second face that surprises newcomers: a complete UNIX environment, running inside the same operating system, with a shell, a hierarchical file system, processes, and UNIX-style permissions. It is called z/OS UNIX System Services, and you reach it through OMVS. This is not an emulator bolted on the side; it is a certified UNIX that shares the same hardware, the same operating system, and) essentially, the same RACF for identity as the MVS side. Learning it matters for two reasons: a great deal of modern z/OS work (TCP/IP, Java, web serving, file transfer) lives here, and the seam between the UNIX and MVS security models is one of the most productive places an attacker looks.

Entering the shell

From the TSO READY prompt, the OMVS command drops you into a UNIX shell, the same `/bin/sh` you would meet on any UNIX. Two commands orient you immediately: `uname` tells you what system you are on, and `id` tells you who you are:

```
OMVS
IBMUSER:/u/ibmuser$ uname -a
OS/390 GIBSON 03.01 1090 POSIX(ON) ZFS DSFS
IBMUSER:/u/ibmuser$ id
uid=10535(IBMUSER) gid=335(IBMUSER) groups=335(IBMUSER)
IBMUSER:/u/ibmuser$ pwd
/u/ibmuser
```

The z/OS UNIX shell: `uname` identifies the system, `id` identifies you, `pwd` shows where you are.

Look closely and the two worlds are already visible in one screen. `uname` reports `POSIX(ON)` (this is a real UNIX) running on a z-series file system (`ZFS`). `id` shows you as an ordinary user with a numeric `uid` and `gid`, exactly as UNIX expects, yet that user is `IBMUSER`, the same identity you log on with on the MVS side. And `pwd` shows your home

directory, /u/ibmuser, a path, not a data set name. You are unmistakably in UNIX, but it is UNIX wearing a 3270 tie.

The hierarchical file system

The UNIX side stores its data in a hierarchical file system (directories and files in a tree rooted at /) not in cataloged data sets. Listing the root shows the familiar UNIX landscape:

```
IBMUSER:/u/ibmuser$ ls -l /
drwxr-xr-x      etc
drwxr-xr-x      tmp
drwxr-xr-x      u
drwxr-xr-x      usr
drwxr-xr-x      var
```

The root of the z/OS UNIX file system, the same directories any UNIX user would expect.

There they all are: /etc for configuration, /tmp for scratch files, /u for user home directories, /usr for programs, /var for variable data. Your home, /u/ibmuser, sits under /u. The mental shift from the MVS side is real but small: where MVS names data by dotted qualifiers and organizes them in a catalog, UNIX names data by slash-separated paths and organizes them in a tree. The two even map onto each other, and it helps to hold the correspondence in mind:

z/OS UNIX (OMVS)	Classic MVS
A file	A member or sequential data set
A directory	A PDS, or a catalog qualifier level
A path: /u/ibmuser/report	A data set: IBMUSER.REPORT
File permission bits (rwx)	A RACF data-set profile
A process (seen with ps)	An address space or job
The /bin/sh shell	The TSO READY prompt
Superuser (UID 0)	Broad power, akin to SPECIAL

The two worlds, side by side: every UNIX idea has an MVS counterpart.

Files, permissions, and ownership

On the UNIX side, access is governed by permission bits carried on each file, not by RACF data-set profiles. Listing a file shows them:

```
IBMUSER:/u/ibmuser$ ls -l
-rwxr-xr-x      README
IBMUSER:/u/ibmuser$ cat README
Gibson USS home directory
```

A file and its permission bits, then its contents.

The string -rwxr-xr-x is the file's permissions. The first character is the type (- for a file, d for a directory). The next nine are three groups of three (read, write, execute) for the owner, the group, and everyone else. So -rwxr-xr-x means the owner may read, write, and

execute, while the group and others may read and execute but not write. You change these with `chmod`, either symbolically or with the octal shorthand where `r=4`, `w=2`, `x=1`, so 755 is `rw-r-xr-x` and 700 is `rw-x-----`, owner-only. This is a genuinely different access model from RACF, and the two coexist on the same system:

Notation	Meaning
<code>r / 4</code>	Read the file (or list a directory)
<code>w / 2</code>	Write the file (or create in a directory)
<code>x / 1</code>	Execute the file (or enter a directory)
First triad	Permissions for the owner
Second triad	Permissions for the group
Third triad	Permissions for everyone else
755	<code>rw-r-xr-x</code> , owner all, others read+execute
700	<code>rw-x-----</code> , owner only

Reading and setting UNIX permissions, symbolic and octal.

Processes

The UNIX side runs processes, and `ps` lists them just as it would anywhere, with a twist that reveals how z/OS UNIX is built:

```
IBMUSER:/u/ibmuser$ ps -ef
  PID  PPID  USER      COMMAND
    1      0 OMVSKERN  BPX0INIT
  100     1  IBMUSER   sh
  101    100  IBMUSER  omvs-shell
```

The process tree: BPX0INIT at PID 1, your shell descending from it.

Every UNIX has a process with PID 1 that is the ancestor of all others; on z/OS UNIX it is BPX0INIT, owned by the OMVS kernel identity, and your shell descends from it through the PPID (parent process id) chain. Reading a process tree (who launched what, under which identity) is a core skill on any UNIX and doubly useful here, because a process on the UNIX side runs under a z/OS UNIX identity that RACF ultimately controls.

The commands are the ones you know

If you know any UNIX, you already know most of z/OS UNIX. The shell offers the standard toolkit, and a built-in help facility lists them by group:

```
IBMUSER:/u/ibmuser$ help
Gibson USS help - grouped commands
Filesystem:
  cat cd chmod chown cp du find grep head ls
  mkdir more pwd rmdir sort tail touch tr uniq
Process / identity:
  id ps whoami kill env
```

Use: `help <command>`, `man <command>`, `whatis <command>`

The z/OS UNIX shell offers the standard command set, grouped by purpose.

These behave as they do on any UNIX, and a handful cover almost everything you will do at the shell:

Command	What it does
<code>ls -l</code>	List files with permissions and ownership
<code>cd / pwd</code>	Change directory / print the current one
<code>cat / more</code>	Show a file, all at once or a page at a time
<code>chmod</code>	Change a file's permission bits
<code>cp / mv / rm</code>	Copy, move, remove files
<code>grep / find</code>	Search text / locate files
<code>id / ps</code>	Show your identity / list processes
<code>man / help</code>	Read the manual for a command

The everyday z/OS UNIX commands, gathered for reference.

The shell environment

Like any UNIX shell, the z/OS UNIX shell keeps an environment, a set of variables that shape how commands behave. The `env` command shows it:

```
IBMUSER:/u/ibmuser$ env
HOME=/u/ibmuser
HOSTNAME=GIBSON
LANG=C
LOGNAME=IBMUSER
PATH=/bin:/usr/sbin:/usr/lpp/IBM/zoau/bin
```

The shell environment: `HOME`, `LOGNAME`, and the `PATH` that finds commands.

`HOME` and `LOGNAME` confirm your identity and starting directory, and `PATH` is the list of directories the shell searches to find a command you type. `PATH` is worth a second look for a security reason: if a directory you can write to appears early in the `PATH`, a program you drop there can be run in place of a real command, a classic UNIX trick that works on z/OS UNIX exactly as it does elsewhere. Reading the environment is how you understand, and check, the context your commands run in.

One identity, two worlds

Here is the idea that ties the chapter together and points straight at Part IV. Your UNIX identity (the `uid` and `gid` that `id` reported) does not come from a UNIX password file. It comes from RACF, specifically from the OMVS segment of your RACF user profile. The same IBMUSER that logs on to TSO carries a UID on the UNIX side, and RACF is the single source of truth for both. That unification is elegant, and it is also where the danger lives. On UNIX, the user with UID 0 is the superuser (root) who bypasses permission checks entirely. On z/OS, a user granted UID(0) in their OMVS segment, or authority to the BPX.SUPERUSER resource, holds that same near-total power. So a RACF change as small

as adding an OMVS segment with UID(0) to an account quietly makes that account root on the UNIX side of a mainframe, a privilege escalation that never touches a single MVS data-set rule.

Bridging to the MVS side

The two file systems are separate (a path is not a data set) but z/OS deliberately provides bridges between them, and those bridges are worth knowing because data crosses them constantly. From the shell, `cp` understands a special form that names an MVS data set, so you can copy a UNIX file into a sequential data set or a PDS member and back. From TSO, the `OGET`, `OPUT`, and `OCOPY` commands move data the same way from the MVS side. A file transferred in over FTP might land in the UNIX file system and then be copied into an MVS data set for a batch job to process; output from a job might be copied out to the UNIX side to be served by a web process. Knowing that these crossings exist (and that they carry data between two different access-control models) is important, because a file that is tightly controlled on one side can become loosely controlled once it crosses to the other. Where data moves between the worlds is exactly where a reviewer checks that its protection moved with it.

The most common beginner error on the UNIX side is forgetting it is case-sensitive and path-based when the MVS side is neither. `README` and `readme` are different files here; `/u/ibmuser/report` is a path, not the data set `IBMUSER.REPORT`; and a command typed in lowercase will not be found if you reach for MVS habits. When something “is not there” on the UNIX side, check the case and the current directory before anything else.

Hands-On Labs

These labs assume you are logged on as `IBMUSER` and can enter OMVS from the `READY` prompt.

Lab 18.1

Enter OMVS and find your bearings

Goal: enter the shell and confirm the system, your identity, and your location with three commands.

Background: `uname`, `id`, and `pwd` are the first three commands to run in any unfamiliar UNIX, they tell you where you are, who you are, and what you are standing on.

1. From `READY`, enter the shell, then run the three orientation commands.

```
OMVS
```

```
uname -a
```

```
id
```

```
pwd
```

Check yourself: `uname` reports `OS/390 GIBSON with POSIX(ON)`; `id` shows your `uid`, `gid`, and `groups`; `pwd` shows `/u/ibmuser`. You are in a real UNIX shell, as the same user you logged on with.

Why it matters: every UNIX task begins from these three facts. Establishing them on the mainframe's UNIX side proves the environment is genuinely POSIX, not a simulation of one.

Blue-Team angle: the `id` output is a security fact worth reading, an ordinary user should have an ordinary uid. A uid of 0 where you did not expect it is the first thing to notice.

Lab 18.2

Explore the file system

Goal: navigate the hierarchical file system, listing the root and your home directory.

Background: the tree rooted at `/` is where the UNIX side keeps everything. Moving around it fluently is the foundation for all the file work that follows.

1. List the root of the file system and note the standard directories.

```
ls -l /
```

2. Change to a directory and list it; then return to your home.

```
cd /etc
```

```
ls -l
```

Check yourself: the root shows `etc`, `tmp`, `u`, `usr`, and `var`; `/etc` holds configuration such as `profile` and `motd`. `cd` with no argument, or `cd /u/ibmuser`, returns you home.

Why it matters: paths, not data set names, address everything on the UNIX side. Reading the tree is how you find configuration, logs, and programs.

Blue-Team angle: `/etc` holds configuration that shapes behaviour and trust. Who can write to files under `/etc` is exactly the kind of question a defender asks, a writable config is a foothold.

Lab 18.3

Read a file's permissions

Goal: list a file, interpret its permission bits, and state who may do what.

Background: on the UNIX side, the permission string is the access rule. Reading it at a glance is a skill you will use constantly.

1. List a file in long form and read its permission string.

```
ls -l README
```

Check yourself: a string like `-rwxr-xr-x` means owner read/write/execute, group and others read/execute only. Convert it to octal (755) and back to prove you can read it both ways.

Why it matters: UNIX permissions decide access on this side of the system independently of RACF data-set rules. Misreading them is misreading who can touch a file.

Blue-Team angle: a world-writable file (`w` in the third triad) is a classic weakness, anyone can change it. Spotting over-permissive bits is a staple of a UNIX security review, and it applies here unchanged.

Lab 18.4

See who is root

Goal: list processes, read the identities they run under, and connect UNIX superuser status back to RACF.

Background: on UNIX, UID 0 is root and bypasses permission checks. On z/OS that power is granted through RACF, which makes “who is root here” a RACF question.

1. List the processes and note the identity each runs under.

```
ps -ef
```

Check yourself: PID 1 is BPXOINIT under the kernel identity, and your shell descends from it under your own userid. No ordinary process should be running as a surprise superuser.

Why it matters: the identity a process runs under decides what it can do. On z/OS UNIX that identity, and whether it is UID 0, is set in RACF, which is where the next part of the book begins.

Blue-Team angle: an OMVS segment with UID(0), or access to BPX.SUPERUSER, makes an account root on the UNIX side without touching MVS data-set rules. Auditing who holds UID 0 is one of the highest-value checks on the whole system, and a thread Part IX pulls hard.

Lab 18.5

Tighten a file's permissions

Goal: change a file's permission bits with chmod and confirm the result by listing it again.

Background: making a file owner-only is one of the most common security actions on any UNIX. Doing it here proves you can both set and read permissions.

1. List a file to see its current permissions.

```
ls -l README
```

2. Restrict it to the owner only, then list it again to confirm.

```
chmod 700 README
```

```
ls -l README
```

Check yourself: after chmod 700 the permission string reads rwx-----, owner may read, write, and execute, and group and others may do nothing. The octal 700 and the symbolic rwx----- describe the same thing.

Why it matters: setting least-privilege permissions is a daily habit on the UNIX side. Being able to make a file private, and to verify it, is basic file hygiene.

Blue-Team angle: over-permissive files are a recurring finding in UNIX reviews, and z/OS UNIX is no exception. chmod is both the tool that creates the problem when misused and the tool that fixes it, a defender checks the bits on sensitive files and tightens the loose ones.

Checkpoint: can you enter OMVS, establish who and where you are, read a permission string, and explain where your UNIX identity comes from and what UID 0 confers? Those are the skills the security chapters build on, because the UNIX side is a full second surface with its own rules and its own risks.

z/OS has a second face: a certified UNIX, reached through OMVS, with a shell, a hierarchical file system, processes, and permission-bit access control. It is genuinely POSIX (uname, id, pwd, ls, ps, chmod all behave as they do anywhere) yet it shares the mainframe's hardware, operating system, and RACF. That shared identity is the crux: a user's UNIX uid and gid come from the RACF OMVS segment, and UID 0 makes an account root. The UNIX side is a convenience for modern workloads and, precisely because it bridges two security models, one of the richest places to look when assessing a mainframe.

What comes next

You have now seen both faces of z/OS, and one word has recurred through every chapter: RACF. It decides who may read a data set, submit a job, hold SPECIAL, or carry UID 0 on the UNIX side. Part IV turns to face it directly. The next chapters open RACF itself (users, groups, profiles, permissions, and the attributes that confer power) the security core that everything you have learned so far has been quietly resting on.

Part IV

Guarding the System: Security & Identity

A running mainframe is only as trustworthy as its controls. Part IV covers RACF and the identity and resource model it enforces, the tools that monitor that posture, and the audit trail it leaves, the machinery that decides who may do what.

CHAPTER 19

RACF Fundamentals

By the end of this chapter you will be able to:

- Read a RACF user profile and its key fields.
- Explain groups, group authority, and the group tree.
- Recognize the powerful attributes: SPECIAL, OPERATIONS, AUDITOR.
- Read system-wide security options with SETROPTS.
- Find who holds power on a system, the core of a RACF audit.
- Relate RACF to ACF2, the other common security manager.

ONE NAME HAS RUN THROUGH every chapter of this book: RACF, the Resource Access Control Facility. It decided that a job could not update SYS1.RACFDS, that IBMUSER holds SPECIAL, that a mailbox is private to its owner, that a UID identifies you on the UNIX side. RACF is the security core of z/OS, and it answers a single question for every action anyone takes: is this identity allowed to do this? This chapter opens RACF itself. It is built from three ideas (users, who act; groups, which gather users; and the attributes that confer power) and the system-wide options that govern how strictly it all runs. The resource profiles that protect data sets come in the next chapter; here we meet the identities and the authority they carry.

The user profile

Every person or started task that RACF knows is a user, described by a user profile. LISTUSER shows one, and it repays careful reading:

```
LISTUSER IBMUSER
USER=IBMUSER  NAME=IBMUSER          OWNER=#SYSPROG  CREATED=24.271
DEFAULT-GROUP=SYS1      PASSDATE=25.020  PASS-INTERVAL= 30
ATTRIBUTES=SPECIAL
UACC=NONE  MODEL=NONE  SECLABEL=NONE
REVOKE DATE=NONE  RESUME DATE=NONE
PASSWORD CHANGE REQUIRED=NO
LOGON ALLOWED  (DAYS)          (TIME)
  ANYDAY              ANYTIME
NO OMVS SEGMENT
```

A RACF user profile, identity, ownership, password policy, and attributes.

Read the important fields. USER and NAME identify the account, and OWNER says who administers it. DEFAULT-GROUP is the group the user is connected to by default (here SYS1. PASS-INTERVAL is how many days a password lasts. ATTRIBUTES is the field to stop

on: SPECIAL means this user is a full RACF administrator, able to change any profile on the system. REVOKE DATE and RESUME DATE control whether the account is currently usable, and LOGON ALLOWED bounds when it may log on. Finally, the profile can carry segments) a TSO segment, an OMVS segment (the source of the UNIX UID from the last chapter), and here there is none. One profile, and you already know who this account is, who runs it, how its password behaves, and that it holds the most powerful attribute RACF has.

Protected and revoked accounts

Not every account is meant to log on. Compare the CICS default user:

```
USER=CICSUSER  NAME=CICS  DEFAULT USER    OWNER=#SYSPROG  CREATED=24.271
DEFAULT-GROUP=SYS1      PASSDATE=25.020  PASS-INTERVAL= 30
ATTRIBUTES=PROTECTED
```

A PROTECTED user, an identity with no password that cannot be logged on to.

ATTRIBUTES=PROTECTED marks an account that has no password and cannot be used to log on or be revoked by bad-password attempts. Protected users exist to be an identity for a started task or a subsystem, something needs a userid to own resources and be checked by RACF, but no person should ever sign in as it. The contrast with IBMUSER is the whole point: one account is a powerful human administrator, the other a faceless service identity, and RACF describes both in the same profile format. Reading the ATTRIBUTES line tells you which kind you are looking at.

Groups and authority

Users are gathered into groups, and access is almost always granted to groups rather than to individuals, it is far easier to manage. LISTGRP shows a group and its members:

```
LISTGRP SYS1
GROUP=SYS1  OWNER=IBMUSER  SUPGROUP=SYS1
CONNECTED USERS:
  IBMUSER  AUTHORITY=JOIN
  RUARIV   AUTHORITY=USE
```

A group, its owner, its place in the group tree, and its connected users.

SUPGROUP names the group's parent (groups form a tree, with SYS1 at the top) and each connected user carries an authority that says what they may do within the group. The levels climb from ordinary membership to full control:

Group authority	What the user may do in the group
USE	Ordinary membership, use the group's access
CREATE	Create data-set profiles for the group
CONNECT	Connect other users to the group
JOIN	Add users and subgroups, full control

Group authority levels, from ordinary member to full control.

So IBMUSER holds JOIN in SYS1 (it can add and remove members) while RUARIV holds only USE. Group authority is distinct from what the group can access; it governs administration of the group itself, which is why a user with JOIN on a powerful group is worth a second look.

The attributes that confer power

A handful of user attributes carry system-wide power, and knowing them is the heart of understanding who can do what. These are the ones that matter:

Attribute	What it confers
SPECIAL	Full RACF administration, change any profile
OPERATIONS	Access to most data sets, for operational work
AUDITOR	Control over auditing and the ability to read it
ROAUDIT	Read-only auditor, see audit settings, not change
PROTECTED	No password; cannot log on (service identity)
(none)	An ordinary user, governed by profiles alone

The RACF attributes that carry system-wide weight.

SPECIAL is total administrative power over RACF. OPERATIONS is nearly as dangerous in a different way (it grants access to most data on the system for operational convenience, which is exactly why it is so often over-assigned and so worth auditing. AUDITOR controls the audit trail itself. The three together) who administers security, who can reach the data, and who watches, are meant to be held by different people, and a single account holding all three is a finding, not a feature.

System-wide options: SETROPTS

Above the individual profiles sit system-wide options, controlled by SETROPTS. Listing them shows how strictly RACF is running:

```
SETROPTS LIST
```

```
RACF OPTIONS
```

```

RACF MODE       : SECURE
GENERIC DATASET  : ACTIVE
GLOBAL ACCESS    : INACTIVE
ERASE            : SIMULATED
AUDIT            : ACTIVE
```

```
CLASSACT
```

```
DATASET FACILITY JESSPOOL CCICSCMD FCICSFCT : ACTIVE
```

SETROPTS: the system-wide switches that govern how strictly RACF enforces.

These switches decide the posture of the whole system. RACF MODE tells you whether RACF is actually enforcing. GENERIC DATASET ACTIVE means generic profiles (like SYS1.***) are in effect. AUDIT ACTIVE means security events are being recorded. The CLASSACT list shows which resource classes are active (DATASET, FACILITY, JESSPOOL

and others) because a class that is not active is not being checked. SETROPTS is the first thing an assessor reads, because a single option turned off here can silently disable protection that every individual profile assumes is in force.

Finding who holds power

Putting it together, the core of a RACF review is a short question: who holds power? SEARCH lists the users, and LISTUSER on each reveals its attributes:

```
SEARCH CLASS(USER)
CICSUSER
IBMUSER
```

Enumerating the users, the first step in finding who is powerful.

From that list you check each account's ATTRIBUTES, and the powerful ones stand out: IBMUSER holds SPECIAL, CICSUSER is a PROTECTED service identity. On a real system with thousands of users this is automated, but the logic is exactly this, enumerate the users, read their attributes, and flag every SPECIAL, OPERATIONS, and AUDITOR to confirm each one is deliberate. That single sweep answers the most important security question a mainframe can be asked.

Administering accounts

Reading profiles is half the job; maintaining them is the other half. Three commands cover the account lifecycle. ADDUSER creates a user, ALTUSER changes one, and CONNECT joins a user to a group. Creating an account looks like this:

```
ADDUSER TESTER PASSWORD(TEMP123) NAME('TEST ACCOUNT')
ICH01024I USER TESTER DEFINED
ICH01025I CONNECT TESTER TO GROUP SYS1 CREATED
ICH01026I PASSWORD CHANGE REQUIRED AT NEXT LOGON
```

ADDUSER creates the account, connects it to a default group, and forces a password change.

Notice that a new account already forces a password change at next logon, a small, sensible default. When an account must be taken out of service, ALTUSER REVOKE disables it without deleting it, and a later LISTUSER shows the change plainly:

```
ALTUSER TESTER REVOKE
USER=TESTER  NAME=TEST ACCOUNT          OWNER=#SYSPROG  CREATED=24.271
ATTRIBUTES=REVOKED
REVOKE DATE=YES  RESUME DATE=NONE
```

A revoked account: ATTRIBUTES=REVOKED and REVOKE DATE=YES, disabled, not deleted.

REVOKE is the everyday tool for suspending access (for a departing employee, a compromised account, or a dormant one) and RESUME reverses it. Because the account is disabled rather than deleted, its history and ownership survive, which matters for both

accountability and investigation. These three commands, and the messages they return, are the working vocabulary of RACF administration.

A note on ACF2

RACF is the most common z/OS security manager, but it is not the only one. Some sites run ACF2 (or Top Secret) instead (external security managers that do the same job through the same interface, SAF, but with different commands and a different default stance. Where RACF grants nothing unless a profile allows it, ACF2 historically defaults the other way and works through rules keyed to logonids (LIDs) rather than user profiles. The concepts translate directly) identities, groups, powerful privileges, resource rules, an audit trail, so everything you learn here carries over; only the syntax changes. When you meet a system whose security commands look unfamiliar, ask which manager it runs before assuming anything is broken.

The most common misreading is to treat group authority and access authority as the same thing. A user's JOIN or USE authority in a group governs administration of that group, not what data the group can reach; access to data comes from resource profiles, which are the next chapter. Confusing the two leads people to think a USE member is harmless when the group itself may hold broad access, or to overlook a JOIN member who can quietly add themselves more power.

Hands-On Labs

These labs assume you are logged on as IBMUSER, which holds SPECIAL and can therefore read every profile on the system.

Lab 19.1

Read a user profile

Goal: display a user profile and identify its owner, default group, password policy, and attributes.

Background: the user profile is the single most informative object in RACF. Reading one fluently is the foundation of every review.

1. Display your own profile.

```
LISTUSER IBMUSER
```

Check yourself: you can point to the OWNER, DEFAULT-GROUP, PASS-INTERVAL, and ATTRIBUTES. Confirm that IBMUSER holds SPECIAL and note whether it has any segments.

Why it matters: every question about what an account can do begins with its profile. Reading it is the first move in operating and in auditing alike.

Blue-Team angle: the ATTRIBUTES line is where power lives. An account that unexpectedly holds SPECIAL or OPERATIONS is exactly what a review is meant to surface.

Lab 19.2

Read a group and its authorities

Goal: list a group and identify each member's authority within it.

Background: access is granted to groups, so knowing who is in a group (and who can administer it) is central to understanding access.

1. List the SYS1 group.

```
LISTGRP SYS1
```

Check yourself: you can name the group's owner and superior group, and state each member's authority, IBMUSER holds JOIN, RUARIV holds USE. JOIN means IBMUSER can add and remove members.

Why it matters: a member with JOIN on a powerful group can extend that power to others. Group authority is a quiet but real avenue of privilege.

Blue-Team angle: reviewing who holds JOIN or CONNECT on sensitive groups catches a path to escalation that attribute checks alone would miss.

Lab 19.3

Find the powerful users

Goal: enumerate the users and identify which hold SPECIAL, OPERATIONS, or AUDITOR.

Background: "who is powerful?" is the single most valuable question in a RACF review, and it is answered by enumeration plus attribute reading.

1. List the users, then read the attributes of each.

```
SEARCH CLASS(USER)
```

```
LISTUSER IBMUSER
```

Check yourself: you can produce a short list of every account holding a powerful attribute (here IBMUSER (SPECIAL)) and confirm each is meant to have it. A protected user like CICSUSER is expected; an unfamiliar SPECIAL user is not.

Why it matters: this sweep is the backbone of a mainframe security assessment. On a real system it is automated, but the logic never changes.

Blue-Team angle: the set of SPECIAL, OPERATIONS, and AUDITOR users should be small, known, and separated across people. Any drift in that set (a new powerful account, one person holding all three) is a high-priority finding.

Lab 19.4

Check the system-wide posture

Goal: read SETROPTS and confirm RACF is enforcing, auditing, and protecting the classes you expect.

Background: individual profiles assume the system-wide switches are set correctly. SETROPTS is where you confirm that assumption.

1. List the system-wide options.

```
SETROPTS LIST
```

Check yourself: RACF MODE is SECURE, AUDIT is ACTIVE, and the classes you rely on (DATASET, FACILITY, JESSPOOL) appear under CLASSACT. A class that is not active is not being checked, whatever its profiles say.

Why it matters: a single option turned off here can silently undo protection everywhere. Reading SETROPTS first tells you whether the rest of the review even means anything.

Blue-Team angle: an inactive class, AUDIT switched off, or WARNING mode left on across profiles are exactly the system-wide weaknesses an attacker hopes for and an assessor checks for first.

Lab 19.5

Create, connect, and revoke an account

Goal: walk an account through its lifecycle (create it, connect it to a group, then revoke it) reading RACF's confirmation at each step.

Background: creating and disabling accounts is daily RACF administration. Doing the full cycle once shows how the commands and their messages fit together.

1. Create a test account with a temporary password.

```
ADDUSER TESTER PASSWORD(TEMP123) NAME('TEST ACCOUNT')
```

2. Confirm and set its group authority, then revoke it.

```
CONNECT TESTER GROUP(SYS1) AUTHORITY(USE)
```

```
ALTUSER TESTER REVOKE
```

Check yourself: ADDUSER reports USER TESTER DEFINED and forces a password change; CONNECT reports the user connected with USE authority; ALTUSER REVOKE leaves LISTUSER showing ATTRIBUTES=REVOKED and REVOKE DATE=YES. The account exists but cannot be used.

Why it matters: this is the exact sequence used to onboard and offboard real users. Knowing that REVOKE disables rather than deletes is the difference between suspending access and destroying an audit trail.

Blue-Team angle: prompt revocation of departing or compromised accounts is a front-line control, and revoked-not-deleted preserves the evidence an investigation needs. Dormant accounts that are never revoked are a standard finding.

Checkpoint: can you read a user profile and name its attributes, explain group authority, list the powerful attributes and what each confers, and read SETROPTS to judge the system's posture? Those skills are the foundation of every security chapter that follows, and of any real assessment.

RACF is the security core of z/OS, answering one question for every action: is this identity allowed? It is built from users, described by profiles that carry ownership, password policy, and attributes; groups, which gather users and grant them authority in a tree; and the powerful attributes (SPECIAL, OPERATIONS, AUDITOR) that confer system-wide weight and are meant to be held separately. SETROPTS governs how strictly the whole system enforces, and the central review question (who holds power?) is answered by enumerating users and reading their attributes. ACF2 does the same job with different syntax. What none of this has covered yet is how a specific resource is protected, which is the next chapter.

What comes next

You now know the identities and the power they can carry. The other half of RACF is the resources: the profiles that protect data sets and other objects, the universal access that sets a floor, the access lists that grant exceptions, and the PERMIT command that maintains them. The next chapter turns to resource protection, how RACF decides, for a specific data set and a specific user, whether the answer is yes or no.

CHAPTER 20

RACF Resource Protection

By the end of this chapter you will be able to:

Create a data-set profile and read it with LISTDSD.

Explain universal access (UACC) and why it is the floor, not the ceiling.

Grant and read exceptions with PERMIT and the access list.

Name the access levels from NONE to ALTER and what each allows.

Distinguish generic from discrete profiles and how RACF matches them.

Protect a general resource such as BPX.SUPERUSER, and trace a decision.

THE LAST CHAPTER COVERED IDENTITIES (who acts. This one covers resources) what they may act on. RACF protects a resource with a profile: a rule that names the resource, sets a default level of access for everyone, and lists the exceptions. Data sets are the classic case, but the same machinery protects a long list of other things (UNIX superuser authority, spool output, operator commands) through general resource classes. Understanding how a profile is built, and exactly how RACF uses it to answer yes or no for a specific user and a specific resource, is the center of mainframe security.

The data-set profile

You protect a data set by defining a profile for it with ADDSD. The profile, not the data set, is what RACF consults:

```
ADDSD 'IBMUSER.SECRET.**' UACC(NONE)
ICH10010I DATASET PROFILE IBMUSER.SECRET.** ADDED
```

Defining a data-set profile with ADDSD, the rule that protects a name pattern.

LISTDSD then shows the profile you created, who owns it, its universal access, and its auditing:

```
LISTDSD DATASET('IBMUSER.SECRET.**') ALL
INFORMATION FOR DATASET IBMUSER.SECRET.**
LEVEL  OWNER    UNIVERSAL ACCESS  WARNING  ERASE
-----
  00    IBMUSER  NONE              NO        NO
AUDITING
-----
FAILURES(READ)
```

A data-set profile: owner, universal access, warning mode, and auditing.

The profile's name is a pattern (IBMUSER.SECRET.***) so it protects every data set whose name matches. UNIVERSAL ACCESS is set to NONE, and AUDITING is set to record

FAILURES(READ), so every denied read is logged. That single profile now governs access to a whole family of data sets, and everything else in this chapter is about the two fields that decide who gets in: the universal access and the access list.

Universal access: the floor

Universal access, UACC, is the level of access granted to anyone who is not specifically listed on the profile. It is the floor beneath everyone. UACC(NONE) means “no access unless I say otherwise”, the safe default. UACC(READ) means “everyone may read this”, which is convenient and, for anything sensitive, dangerous: it grants read access to every user on the system at once, including ones no one is thinking about. The most common single mistake in mainframe security is a sensitive profile left at UACC(READ) or higher, because it silently opens the resource to the entire user population. UACC is a floor, never a ceiling: it can only grant access, and the way to give a specific user more is the access list.

The access list: exceptions

PERMIT adds a user or group to a profile’s access list with a specific level. This is how you grant access above the UACC floor to exactly the people who should have it:

```
PERMIT 'IBMUSER.SECRET.**' CLASS(DATASET) ID(TESTER) ACCESS(READ)
ICH06011I PERMIT SUCCESSFUL FOR 'IBMUSER.SECRET.**' CLASS(DATASET) ID(TESTER)
```

PERMIT grants a specific user a specific access above the UACC floor.

Now TESTER may read the protected data sets, while everyone else is still held at UACC(NONE). You grant to groups the same way, and in practice you almost always permit a group rather than an individual, because it is far easier to manage membership than to maintain long lists of names. Reading a profile’s access list with LISTDSD AUTHUSER shows exactly who has been granted what, the report an auditor reads to answer “who can reach this data?”

The access levels

Both UACC and the access list use the same ladder of access levels, and each rung includes the ones below it:

Access	What it allows
NONE	No access at all
READ	Read the data set
UPDATE	Read and write the data set
CONTROL	Update, plus VSAM control-interval access
ALTER	Full control, including delete and profile change

The access ladder: each level includes everything below it.

The one to watch is ALTER. On a data-set profile, ALTER lets the holder not only change the data but delete it and even modify the profile that protects it, which means a user

with ALTER can grant themselves or others more access. ALTER on a sensitive profile is therefore close to owning it, and it is audited accordingly.

Generic and discrete profiles

Profiles come in two kinds. A discrete profile protects one specific data set by its exact name. A generic profile uses wildcards (like IBMUSER.SECRET.** or the system-wide SYS1.**) to protect a whole family of names with one rule. Generic profiles are how a site protects thousands of data sets without thousands of profiles, and they are the normal case. When more than one profile could match a data set, RACF uses the most specific one: a discrete profile beats a generic, and a more specific generic beats a broader one. This matching rule matters in practice, because a carefully locked-down specific profile can be undermined if a broad generic above it is too permissive, and the reverse (a tight generic with a forgotten open discrete beneath it) happens too.

Beyond data sets: general resources

The same profile machinery protects far more than data sets, through general resource classes. You define a profile in a class with RDEFINE and read it with RLIST. Here is the FACILITY-class profile that guards UNIX superuser authority, the BPX.SUPERUSER from the OMVS chapter:

```
RDEFINE FACILITY BPX.SUPERUSER UACC(NONE)
RLIST FACILITY BPX.SUPERUSER
CLASS      NAME
-----
FACILITY   BPX.SUPERUSER
LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS  WARNING
-----
00     IBMUSER     NONE                      ALTER        NO
```

A general-resource profile in the FACILITY class, protecting UNIX superuser authority.

The structure is identical to a data-set profile (owner, UACC, an access list) but the class decides what is being protected. A user PERMITTED READ to BPX.SUPERUSER can switch to UNIX superuser; JESSPOOL profiles decide who can read whose spool output; OPERCMDS profiles decide who can issue which operator commands. A handful of classes carry most of the weight:

Class	What it protects
DATASET	Data sets, the classic resource
FACILITY	Named authorities like BPX.SUPERUSER
JESSPOOL	Job output on the spool
OPERCMDS	Operator commands
TSOAUTH	TSO authorities such as ACCOUNT
SURROGAT	Whose identity a user may submit work as

Some of the general-resource classes and what each governs.

How RACF decides

With the pieces in hand, the authorization algorithm is short and always the same. When a user asks to access a resource, RACF checks, in order: does the user hold SPECIAL (for the profile) or, for data, OPERATIONS (if so, allow. Otherwise, find the most specific profile protecting the resource. Is the user, or a group they belong to, on its access list with sufficient access? If so, allow. If not, is the UACC high enough? If so, allow. If none of these grant the requested level, deny) and if auditing is set, record it. That denial is the ICH408I message you met in the abend chapter, and the S913 that followed it. Every access decision on the system is this one algorithm, run millions of times a day.

The costliest mistake in resource protection is a permissive UACC on a sensitive profile. UACC(READ) does not mean “readable by its owner”, it means readable by every user on the system, listed or not. Whenever you see a profile protecting anything sensitive, read its UACC first; a floor set too high cannot be fixed by a careful access list above it, because the floor already let everyone in.

Hands-On Labs

These labs assume you are logged on as IBMUSER (SPECIAL) and can define and read profiles.

Lab 20.1

Protect a data set

Goal: create a data-set profile with UACC(NONE) and confirm it with LISTDSD.

Background: protecting data begins with a profile. A profile with UACC(NONE) is the locked-by-default starting point for anything sensitive.

1. Define a profile for a name pattern, closed to everyone.

```
ADDSD 'IBMUSER.SECRET.**' UACC(NONE)
```

2. Read it back.

```
LISTDSD DATASET('IBMUSER.SECRET.**') ALL
```

Check yourself: ICH10010I confirms the profile was added, and LISTDSD shows UNIVERSAL ACCESS NONE with auditing on failures. Nothing but the owner and SPECIAL users can reach the matching data sets yet.

Why it matters: UACC(NONE) is the safe default. Starting closed and granting deliberately is the opposite of the far riskier start-open habit.

Blue-Team angle: FAILURES(READ) auditing means every denied read is recorded, exactly the trail that catches someone probing a resource they cannot reach.

Lab 20.2

Grant an exception with PERMIT

Goal: add a specific user to a profile's access list and confirm the grant.

Background: real access is granted above the UACC floor with PERMIT. Doing one shows how least-privilege access is built, closed by default, opened by exception.

1. Grant a user READ to the protected profile.

```
PERMIT 'IBMUSER.SECRET.**' CLASS(DATASET) ID(TESTER) ACCESS(READ)
```

2. Read the access list.

```
LISTDSD DATASET('IBMUSER.SECRET.**') AUTHUSER
```

Check yourself: ICH06011I confirms the PERMIT, and the access list now shows TESTER with READ while UACC stays NONE. One user has been let in; everyone else is still out.

Why it matters: least privilege is exactly this, a closed floor and a short, deliberate list of exceptions. PERMIT is the tool that builds it.

Blue-Team angle: the access list is the answer to “who can reach this?” Reviewing lists on sensitive profiles (and questioning every name) is core audit work.

Lab 20.3

Find an over-exposed profile

Goal: recognize a profile whose UACC exposes it to the whole system.

Background: the most common real finding is a sensitive profile with too high a UACC. Learning to spot it is a high-value skill.

1. List a profile and read its UNIVERSAL ACCESS and YOUR ACCESS fields.

```
RLIST DATASET SYS1.**
```

Check yourself: a UACC of READ or higher on a sensitive profile means every user has at least that access. Confirm whether the UACC you see is appropriate for what the profile protects, for anything sensitive, NONE is usually correct.

Why it matters: one over-permissive UACC can undo an otherwise careful security design. Finding these is often the single highest-impact result of a review.

Blue-Team angle: a sweep for sensitive profiles with UACC above NONE (SYS1 libraries, security data, production data) is a standard, high-yield assessment step.

Lab 20.4

Protect a general resource

Goal: define and read a profile in a general-resource class, seeing that the machinery is identical to data sets.

Background: the most security-relevant authorities (UNIX superuser, surrogate submission, operator commands) are all general-resource profiles. Protecting one shows the pattern.

1. Define and read the FACILITY profile for UNIX superuser authority.

```
RDEFINE FACILITY BPX.SUPERUSER UACC(NONE)
```

```
RLIST FACILITY BPX.SUPERUSER
```

Check yourself: ICH10002I confirms the profile in class FACILITY, and RLIST shows it with UACC NONE. Only users PERMITTED to it can claim UNIX superuser, the control from the OMVS chapter, now visible as a profile.

Why it matters: the same profile you just read decides who can become root on the UNIX side. Resource classes are where the system's most dangerous authorities live.

Blue-Team angle: the access lists on FACILITY profiles like BPX.SUPERUSER, and on SURROGAT profiles, are among the highest-value things to review, they grant power that never appears in a data-set rule, and Part IX targets exactly these.

Reading your own access to a resource

RLIST answers a question every user and every reviewer asks: for this profile, what access do I actually have? The YOUR ACCESS field gives the answer directly:

```
RLIST DATASET SYS1.**
CLASS      NAME
DATASET    SYS1.**
LEVEL  OWNER      UNIVERSAL ACCESS  YOUR ACCESS
-----
00    IBMUSER     NONE                      ALTER
```

RLIST with the YOUR ACCESS field, the effective access this user has.

Here UACC is NONE, yet YOUR ACCESS is ALTER, because IBMUSER holds SPECIAL and owns the profile. That contrast is the point of reading effective access rather than just the UACC: the answer for a specific user folds in their attributes, group memberships, and the access list. When you need to know what someone can really do, YOUR ACCESS is the field that tells you.

Lab 20.5

Read your effective access

Goal: use RLIST to read the effective access a user has to a profile, and explain why it may differ from the UACC.

Background: the profile's UACC is not the whole story for any one user. Effective access folds in attributes and the access list.

1. List a profile and read its YOUR ACCESS field.

```
RLIST DATASET SYS1.**
```

Check yourself: YOUR ACCESS reflects your real access, ALTER for a SPECIAL owner even when UACC is NONE. For an ordinary user it would show only what the UACC and access list grant.

Why it matters: what a person can actually do is answered by effective access, not by the UACC alone. Confusing the two hides real exposure.

Blue-Team angle: checking effective access for powerful accounts against sensitive profiles reveals reach a UACC review alone would miss.

When access is denied

The other side of a profile is the denial it produces. When a user without sufficient access tries to reach a protected resource, RACF refuses and records it, the message you first met as the cause of an S913 abend:

```

ICH408I  USER(TESTER)  GROUP(SYS1)  NAME(TESTER)
          IBMUSER.SECRET.DATA  CL(DATASET)
          INSUFFICIENT ACCESS  AUTHORITY
          ACCESS  INTENT(UPDATE)  ACCESS  ALLOWED(READ)

```

A denial: the profile did its job, and RACF recorded exactly what was refused.

Read the last two lines together: the user asked for UPDATE, the profile allowed only READ, so the request was refused. This is the profile working exactly as designed, and, logged, it is also the detection signal a defender wants. Every denial names the user, the resource, the access attempted, and the access allowed, which is precisely enough to tell routine misconfiguration from deliberate probing.

Lab 20.6

Read an access denial

Goal: interpret an ICH408I denial, the resource, the access attempted, and the access allowed.

Background: a denial is both the profile enforcing and the system recording. Reading one connects resource protection to detection.

1. Cause or locate a denial and read its ICH408I lines.

Check yourself: the message names the user, the resource and class, ACCESS INTENT (what was attempted), and ACCESS ALLOWED (what the profile permits). Intent above allowed is the refusal.

Why it matters: denials are where resource protection becomes visible. A run of them against one resource is the signature of someone probing it.

Blue-Team angle: ICH408I is front-line detection. Alerting on denials against sensitive profiles (SYS1 libraries, the RACF database) catches probing early.

Checkpoint: can you create a data-set profile, explain UACC as the floor, grant an exception with PERMIT, name the access levels, distinguish generic from discrete, protect a general resource, and recite how RACF reaches a decision? Those are the mechanics behind every access on the system.

RACF protects a resource with a profile that sets a universal access (the floor for everyone) and an access list of exceptions granted with PERMIT. Access runs from NONE through READ, UPDATE, CONTROL, to ALTER, where ALTER is near-ownership. Profiles are discrete or generic, and RACF applies the most specific match. The same machinery, through general-resource classes like FACILITY and SURROGAT, protects authorities well beyond data, including UNIX superuser. Every decision follows one algorithm: SPECIAL or OPERATIONS, then the access list, then UACC, then deny-and-audit. The most common weakness is a UACC set too high, and finding it is often the most valuable thing a review does.

What comes next

Profiles, attributes, and options all live in one place: the RACF database, SYS1.RACFDS. It is the most sensitive data set on the system (the file that decides every answer RACF gives) and it is therefore both the thing a defender guards most closely and the prize an attacker wants most. The next chapter opens the RACF database itself: how it is backed up and verified, how it is unloaded for analysis, and why protecting it is the whole game.

CHAPTER 21

The RACF Database

By the end of this chapter you will be able to:

Explain what SYS1.RACFDS holds and why it is the crown jewel.

Read the database status and its hash inventory.

Back up and verify the database, and treat the backup as sensitive.

Distinguish legacy DES password hashes from modern KDFAES.

Use a hash-exposure review to find accounts that need remediation.

Protect the database itself, the single most important control.

EVERYTHING RACF KNOWS (EVERY USER profile, every group, every resource rule, every password) lives in one data set: SYS1.RACFDS, the RACF database. It is the file that decides every answer RACF gives, which makes it the most sensitive object on the system. A defender guards it above all else, because whoever can read it can, given time, learn its passwords, and whoever can write it can rewrite the rules. This chapter is about that database from the defender's chair: how to read its state, back it up, verify it, inventory the strength of the password hashes it holds, and (the point of the whole chapter) protect it. The tools here are audit tools; the exposures they reveal are the ones you then remediate.

Reading the database status

The status of the database is the place to start. It reports how many users it holds and, essentially, how their passwords are hashed:

```
RACFDB STATUS
PRIMARY  SYS1.RACFDS          USERS=5  LEGACY-DES=3  KDFAES=2
BACKUP   SYS1.RACFDS.BACKUP  USERS=0  FORMAT=TEXT  STALE=NO
POLICY   LEGACY-LAB
LEGACY-DES RECORDS           3
KDFAES/PROTECTED RECORDS    2
JOHN-COMPATIBLE EXPORT      YES
```

The database status, user count and, most importantly, the hash inventory.

Read the hash inventory carefully, because it is a direct measure of exposure. Of five users, three still use LEGACY-DES password hashing and two use modern KDFAES. That single line tells a defender that three accounts are protected by an algorithm that is, by modern standards, weak, and therefore that if the database were ever stolen, those three passwords could most likely be recovered, while the two KDFAES accounts would resist. The whole defensive job flows from that reading: protect the file so it is never stolen, and migrate the weak hashes so that even if it were, less would be lost.

Backing up and verifying

Because the database is irreplaceable, it is backed up and checked for integrity. VERIFY confirms the database is internally consistent:

```
RACFDB VERIFY
```

```
IRRDBV00I SYS1.RACFDS VERIFY COMPLETE - DATABASE CONSISTENT
```

A verify pass confirming the database is internally consistent.

A backup, SYS1.RACFDS.BACKUP, provides recovery if the primary is damaged, and RACF can keep it in step automatically. But a backup of the crown jewel is itself a crown jewel: it holds the same profiles and the same password hashes as the original. A common and serious mistake is to protect SYS1.RACFDS tightly and then leave its backup, or an export of it, in a data set anyone can read. The rule is simple and absolute, every copy of the database is as sensitive as the database, and must be protected identically.

Legacy DES and modern KDFAES

RACF has stored passwords as one-way hashes for its whole history, but the algorithm has changed. The original scheme is a DES-based hash, fast to compute and, on modern hardware, fast to attack by trying candidate passwords until one matches. Its replacement, KDFAES, is deliberately slow and salted, which makes the same guessing attack orders of magnitude more expensive. The difference is not academic: it is the difference between a stolen hash that falls in hours and one that holds for practical purposes. Every account still on legacy DES is a weaker link, which is exactly why the status line broke the population down by algorithm.

Hash	Character
Legacy DES	Fast, unsalted, weak against offline guessing
KDFAES	Slow, salted, strong against offline guessing
PROTECTED user	No password at all, nothing to crack

Why the hashing algorithm decides how much a stolen database costs you.

Finding the exposed accounts

A hash-exposure review turns the inventory into a per-account worklist, exactly what a defender needs to act:

```
ZSEC OFFLINEHASH
```

```
ZSECURE OFFLINE RACF HASH REVIEW
```

USERID	ALG	STATUS	PROVIDER
CICSUSER	KDFAES	PROTECTED	PROTECTED
IBMUSER	KDFAES	PROTECTED	PROTECTED
DUMONT	LEGACY-DES	CRACKABLE-REAL	REAL-DES
FIREID1	LEGACY-DES	CRACKABLE-REAL	REAL-DES
FIREID2	LEGACY-DES	CRACKABLE-REAL	REAL-DES

A hash-exposure review, the accounts flagged CRACKABLE are the remediation list.

This is the defensive heart of the chapter. The review names every account, its algorithm, and a plain verdict: CICSUSER and IBMUSER are on KDFAES and safe; DUMONT, FIREID1, and FIREID2 are on legacy DES and marked crackable. That is a work order. The tooling exists so that a defender can find, before an attacker does, exactly which accounts would fall if the database leaked, and fix them. The reason security teams study how a stolen database would be attacked (unloaded with IRRDBU00, converted with racf2john, and fed to a password cracker) is precisely so they can measure this exposure and close it; the response is never to leave it open.

Protecting the crown jewel

Everything above leads to one control: the database must be unreachable to anyone who does not administer it. SYS1.RACFDS is protected by a profile with UACC(NONE), a very short access list, and auditing on every attempt, so that a program which tries to read or update it is denied and recorded, the ICH408I and S913 you met in the abend chapter were exactly that control firing. The full defensive posture is layered: protect the primary database and every backup and export identically; migrate legacy DES accounts to KDFAES; enforce strong passwords so that even a recovered hash yields little; and audit access so that any attempt to reach the file is seen. Get those right and the crown jewel stays a jewel.

Unloading for legitimate analysis

Defenders do read the database (carefully. IRRDBU00 unloads it to a flat, documented format that analysis tools (and open-source projects such as pyracf and RACFHound) can parse to answer questions no single command does well: every user with a given attribute, every profile with a risky UACC, every access list containing a departed employee. This is powerful and legitimate review work) and the unload output must be handled with the same care as the database, because it contains the same secrets. Where a defender uses the unload to find weaknesses, an attacker who obtained it would use it to find targets; the artifact is identical, and so its protection must be.

The classic error is to lock down SYS1.RACFDS and forget its copies. A tightly protected primary database is undone by a world-readable backup, an IRRDBU00 unload left in a scratch data set, or an export copied to a distribution library. Treat every copy (backup, export, unload) as exactly as sensitive as the original, because it is.

Hands-On Labs

These labs assume you are logged on as IBMUSER (SPECIAL) and are working defensively, measuring exposure in order to close it.

Lab 21.1

Read the database status and hash inventory

Goal: read RACFDB STATUS and interpret the LEGACY-DES versus KDFAES counts as an exposure measure.

Background: the hash inventory is the single most useful exposure number for the database. Reading it is where a password-strength review begins.

1. Display the database status.

RACFDB STATUS

Check yourself: you can state how many users exist and how many are on legacy DES versus KDFAES. The legacy count is the number of accounts that would most likely fall if the database leaked.

Why it matters: this one number quantifies the blast radius of a database compromise. Driving it toward zero is a concrete, trackable security goal.

Blue-Team angle: track the legacy-DES count over time. It should only ever fall; a rise means a new account was created under a weak policy.

Lab 21.2

Verify the database

Goal: run a consistency check and confirm the database is intact.

Background: a corrupt security database is a crisis. Verifying integrity is basic custodianship of the crown jewel.

1. Verify the database.

RACFDB VERIFY

Check yourself: IRRDBV00I reports the database consistent. An inconsistency here would be an operational emergency, handled from the backup.

Why it matters: the database that decides every access decision must itself be sound. Verification is how you know it is.

Blue-Team angle: unexpected inconsistency, or a backup marked stale, can indicate tampering as well as failure, both warrant investigation, not just repair.

Lab 21.3

Build the remediation list

Goal: use a hash-exposure review to produce the list of accounts that must be migrated to strong hashing.

Background: exposure you cannot name you cannot fix. The review turns the inventory into named accounts to act on.

1. Run the offline-hash exposure review.

ZSEC OFFLINEHASH

Check yourself: you can list every account marked CRACKABLE (the ones on legacy DES) as your remediation worklist, and confirm the KDFAES accounts as already safe.

Why it matters: this list is the actionable output of the whole chapter. Migrating each named account to KDFAES and a strong password closes the exposure.

Blue-Team angle: the same review an attacker would love to run against a stolen database is the one a defender runs first, on their own system, to remove the targets before they can be taken.

Lab 21.4

Confirm the database is protected

Goal: confirm SYS1.RACFDS and its copies are protected by profiles with UACC(NONE) and auditing.

Background: the one control that matters most is that the database is unreachable. Confirming it (for the primary and every copy) is the capstone check.

1. List the profile protecting the database and read its UACC and auditing.

```
LISTDSN DATASET('SYS1.RACFDS') ALL
```

Check yourself: the profile shows UACC NONE and auditing on failures, and a short access list of administrators only. Repeat the check for SYS1.RACFDS.BACKUP and any export, each must be protected identically.

Why it matters: hash strength buys time, but keeping the database unreachable is what prevents the attack in the first place. This check is the whole game.

Blue-Team angle: FAILURES auditing on SYS1.RACFDS means every attempt to reach it is recorded. An ICH408I against the RACF database is one of the highest-severity events a monitor can raise.

The database in one view

A single review pulls the whole exposure picture together, the counts, the policy, and whether a backup exists and is current:

```
RACFDB STATUS
PRIMARY  SYS1.RACFDS          USERS=5  LEGACY-DES=3  KDFAES=2
BACKUP   SYS1.RACFDS.BACKUP  USERS=0  FORMAT=TEXT  STALE=NO
POLICY   LEGACY-LAB
LEGACY-DES RECORDS           3
KDFAES/PROTECTED RECORDS     2
LAST SYNC                    2026-07-01T01:46:06+00:00
```

The database at a glance, users, hash mix, backup state, and last sync.

Read as a dashboard, this is a defender's daily health check for the crown jewel: five users, three on weak hashing, a backup present and not stale, a recent sync. Each field maps to an action, drive the legacy count down, confirm the backup is protected and fresh, watch the sync. One screen, and you know the state of the most important data set on the system.

Lab 21.5

Confirm the backup is current and protected

Goal: confirm the database backup exists, is not stale, and is protected as tightly as the primary.

Background: a backup of the RACF database is as sensitive as the database and as useless if stale. Both properties must hold.

1. Read the database status and note the backup line and STALE flag.

`RACFDB STATUS`

2. Confirm the backup data set is protected like the primary.

`LISTDSD DATASET('SYS1.RACFDS.BACKUP') ALL`

Check yourself: the backup shows STALE=NO and a recent sync, and its profile shows UACC(NONE) with auditing. A stale or world-readable backup is a finding in itself.

Why it matters: recovery depends on a current backup, and security depends on that backup being as locked down as the original.

Blue-Team angle: a forgotten, over-exposed backup or export is one of the most common ways a well-protected RACF database leaks.

Unloading the database for review

To answer questions no single command does well, a defender unloads the database with IRRDBU00 into a flat, documented format that analysis tools parse:

```
//UNLOAD EXEC PGM=IRRDBU00,PARM=NOLOCKINPUT
//SYSPRINT DD SYSOUT=*
//INDD DD DISP=SHR,DSN=SYS1.RACFDS
//OUTDD DD DSN=IBMUSER.RACF.UNLOAD,DISP=(NEW,CATLG)
...
IRRDBU00I RACF DATABASE UNLOAD WRITTEN TO IBMUSER.RACF.UNLOAD
```

An IRRDBU00 unload job, the database rendered as a flat file for analysis.

The unload turns the binary database into records that tools such as pyracf and RACFHound can query: every SPECIAL user, every profile with a risky UACC, every access list holding a departed employee. It is legitimate, high-value review work, and the output holds the same secrets as the database, so IBMUSER.RACF.UNLOAD must be protected exactly like SYS1.RACFDS. The artifact a defender uses to find weaknesses is the same one an attacker would use to find targets.

Lab 21.6

Reason about the unload

Goal: explain what IRRDBU00 produces, what questions it answers, and why its output is as sensitive as the database.

Background: the unload is how large-scale RACF review is done, and a classic leak point when its output is left unprotected.

1. Identify where an unload would be written and confirm that data set would be protected.

Check yourself: you can state that the unload is a flat copy of the database contents, that it enables population-wide queries, and that it must carry UACC(NONE) and auditing like the primary.

Why it matters: the unload is powerful for defense and dangerous if leaked. Treating its output as crown-jewel data is the whole discipline.

Blue-Team angle: scan for stray IRRDBU00 output in scratch and distribution libraries, an unprotected unload leaks the entire security database in a form anyone can parse.

The policy that sets the default

One field in the status governs everything downstream: the password policy. It decides which hashing algorithm new accounts get by default, and a lab-grade legacy policy is why weak hashes keep appearing:

```
POLICY    LEGACY-LAB
LEGACY-DES RECORDS      3
KDFAES/PROTECTED RECORDS 2
```

The active password policy, the setting that determines new accounts' hash strength.

Remediation is not only migrating today's weak accounts; it is changing the policy so tomorrow's accounts are strong from creation. A legacy policy quietly re-creates the exposure every time a user is added, which is why fixing the policy comes before, and outlasts, fixing individual accounts.

Lab 21.7

Trace the exposure to the policy

Goal: connect the weak-hash count back to the password policy that produces it.

Background: fixing accounts without fixing the policy means the exposure returns. This lab links symptom to cause.

1. Read the policy field in the database status alongside the hash counts.

```
RACFDB STATUS
```

Check yourself: you can state the active policy and explain that a legacy policy is why new accounts arrive on weak hashing. The durable fix is the policy, not just the accounts.

Why it matters: remediation that ignores the policy is temporary. Fixing the default is what makes the improvement stick.

Blue-Team angle: verify the password policy as part of every review, a strong per-account migration is undone by a weak default that keeps minting legacy hashes.

Checkpoint: can you read the database status and its hash inventory, verify the database, produce a remediation list from a hash-exposure review, and confirm the database and its copies are locked down? Those are the custodial skills for the most sensitive object on the system.

SYS1.RACFDS is the crown jewel (the file that holds every profile, rule, and password hash, and decides every RACF answer. Its status reports a hash inventory that directly measures exposure, because legacy DES hashes are weak against offline guessing while KDFAES is strong. A defender verifies and backs up the database, treats every copy as equally sensitive, uses a hash-exposure review to build a remediation list, migrates weak accounts to KDFAES with strong passwords, and) above all, keeps the database and its copies unreachable with UACC(NONE) and auditing. The same analysis an attacker would run on a stolen database is the one a defender runs first to close the exposure.

What comes next

You have used a review tool to inventory hash exposure without seeing how such tools work at scale. That tooling (zSecure and its relatives) does far more: it reviews events, flags rare and high-risk activity, captures a compliance baseline, and detects drift away from it. The next chapter turns to zSecure as a discipline: how a defender moves from reading one profile at a time to monitoring the security posture of the whole system continuously.

CHAPTER 22

zSecure: Monitoring the Posture

By the end of this chapter you will be able to:

Explain what a security-monitoring product adds beyond raw RACF commands.

Review security events and isolate the rare and high-risk ones.

Read a structured finding with its risk, recommendation, and validation.

Capture a compliance baseline of the system's posture.

Detect drift (changes away from that baseline) continuously.

See how monitoring turns one-time review into ongoing assurance.

READING ONE PROFILE AT A time, as the last chapters did, is how you understand RACF. It is not how you defend a real system, where thousands of users and profiles change every day. For that, sites run a security-monitoring product (zSecure is the common one, with CA Auditor and others alongside it) that reads the same RACF and SMF data but at scale, turning it into reviewed events, prioritized findings, a captured baseline, and continuous drift detection. This chapter is about that shift: from asking "what does this profile say?" to asking "is the whole system still in the state we approved, and what changed?"

The monitoring toolkit

The monitor exposes a set of reviews, each answering a different question a defender asks daily. Its help lists them:

ZSEC HELP

zSecure / Mainframe Security Monitor - HELP (COMMANDS)

ZSEC EVENTS	Recent security events
ZSEC RARE	Rare/high-risk events
ZSEC SUMMARY	Security summary counts
ZSEC BASELINE	Capture compliance baseline (APF/SPECIAL/UID0/UACC)
ZSEC DRIFT	Compare current posture against the baseline
ZSEC RACFDS	RACF database exposure review
ZSEC OFFLINEHASH	Offline hash exposure review

The monitoring toolkit: events, rare-event hunting, baseline, drift, and targeted reviews.

Two ideas organize the set. The event reviews (EVENTS, RARE, SUMMARY) look backward at what has happened. The posture tools (BASELINE, DRIFT) look at what the system currently is, and whether it has moved. The targeted reviews (RACFDS, OFFLINEHASH, and others) drill into a specific high-value area. A defender uses all three modes together, watch the events, know the posture, and inspect the crown jewels.

Reviewing events, and finding the rare ones

The event review lists recent security activity, who did what, and whether it succeeded:

```
ZSEC EVENTS
```

```
ZSECURE SECURITY EVENT REVIEW
```

```
TIME                USER      EVENT                                RESULT
2026-07-01 01:45:37 IBMUSER   SMF TYPE 80 DATASET ACCESS SUCCESS
2026-07-01 01:45:37 IBMUSER   SMF TYPE 80 DATASET ACCESS SUCCESS
```

The event review: a readable stream of who did what, and the result.

On a busy system the event stream is enormous, so the real skill is filtering it down to what matters. The rare-and-high-risk review does exactly that, keeping only the events that touch sensitive resources or represent unusual activity:

```
ZSEC RARE
```

```
ZSECURE RARE / HIGH-RISK EVENT REVIEW
```

```
RARE FILTERS: RACFDS HASH IND$FILE PASSTICKET ICSF SYS1.MAN APF PRIVILEGE
```

The rare-event review keeps only activity touching high-value resources.

The filter list is a map of what a defender worries about: access to the RACF database and its hashes, file transfers (IND\$FILE), PassTicket use, cryptographic services (ICSF), the SMF data itself (SYS1.MAN), APF libraries, and privileged operations. An event that trips one of these filters is worth a human look even on a quiet day, because these are the resources whose abuse signals a real attack rather than routine work.

Findings with risk and recommendation

Beyond listing events, a monitor produces structured findings, each with a severity, the evidence, the risk, a recommendation, and a way to validate the fix:

```
ZSEC SETROPTS
```

```
ZSECURE SETROPTS REVIEW
```

```
ID      SEV  AREA      RESOURCE      EVIDENCE
ZS-INFO INFO  SETROPTS  GIBSON        Simulator state reviewed
RISK: No high-risk state in this control area
RECOMMENDATION: Continue monitoring
VALIDATE: ZSEC SETROPTS
```

A structured finding: severity, evidence, risk, recommendation, and how to validate.

This format is what makes a monitor actionable rather than merely informative. A raw event says what happened; a structured finding says how bad it is, why it matters, what to do about it, and how to confirm you fixed it. On a real review the same shape carries serious findings (an over-permissive UACC, an unexpected SPECIAL user, an inactive class) each with a severity that lets a team triage, and a validate step that closes the

loop. Learning to read a finding, and to act on the recommendation and then run the validation, is the core rhythm of the job.

The compliance baseline

The most powerful idea in monitoring is the baseline: a snapshot of the system's security posture at a moment when it is known to be correct. Capturing one records the facts that must not change silently:

```
ZSEC BASELINE
IBM Security zSecure - Compliance Baseline  V3.1.0
BASELINE CAPTURED: 2026-07-01 01:46:06
SAVED TO: IBMUSER.ZSECURE.BASELINE
  APF LIBRARIES ..... 15
  SPECIAL USERS ..... 2  (FIBSADM, IBMUSER)
  OPERATIONS USERS ..... 0  (none)
```

A compliance baseline, the approved posture, captured so changes can be measured.

The baseline captures exactly the things whose change is dangerous: how many APF libraries exist, who holds SPECIAL, who holds OPERATIONS, which resources sit at what UACC. At capture time these numbers are approved. From then on they are a reference, the shape the system is supposed to keep. Two SPECIAL users, named; zero OPERATIONS users; fifteen APF libraries. Any later deviation is, by definition, a change worth explaining.

Detecting drift

Drift detection compares the live system against the baseline and reports what moved:

```
ZSEC DRIFT
IBM Security zSecure - Compliance Drift  V3.1.0
BASELINE: 2026-07-01 01:47:58    CURRENT: 2026-07-01 01:47:58
NO DRIFT DETECTED - posture matches the captured baseline.
```

Drift detection: the system still matches its approved baseline.

NO DRIFT DETECTED is the report a defender wants, and its opposite is the report that catches an attack. If a new SPECIAL user appears, an APF library is added, or a UACC is loosened, drift detection surfaces exactly that change (not buried in an event stream, but as a clear difference from the approved state. This is how monitoring becomes continuous assurance: capture a good baseline, then let drift detection tell you the moment the system stops matching it. Many of the privilege-escalation moves in Part IX) adding an APF library, granting UID 0, creating a SPECIAL user, are precisely the changes a baseline-and-drift discipline is built to catch.

A monitor is only as good as its baseline and the attention paid to its alerts. The common failures are capturing a baseline from an already-compromised or misconfigured system (so the bad state

becomes the approved state) and letting drift and rare-event alerts pile up unread. Baseline from a known-good posture, and treat a drift report as something to explain, not dismiss.

Hands-On Labs

These labs assume you are logged on as IBMUSER and can run the monitoring reviews.

Lab 22.1

Review events and isolate the risky ones

Goal: read the event review, then narrow to the rare and high-risk events.

Background: the value of a monitor is filtering signal from noise. This lab practices the everyday move from all events to the ones that matter.

1. List recent events, then the rare/high-risk subset.

ZSEC EVENTS

ZSEC RARE

Check yourself: EVENTS shows the full stream; RARE narrows it and shows the filter list (RACFDS, HASH, PASSTICKET, APF, and the rest) that defines high-value activity.

Why it matters: a defender cannot read every event, but must read every risky one. The rare filter is how that becomes possible.

Blue-Team angle: the rare-filter list is a ready-made watchlist. Any hit against RACFDS or APF deserves immediate human attention.

Lab 22.2

Read a structured finding

Goal: run a structured review and read its severity, risk, recommendation, and validation.

Background: findings, not raw events, are what a team acts on. Reading one correctly is how remediation gets prioritized.

1. Run a structured control review.

ZSEC SETROPTS

Check yourself: the finding shows an ID, a severity, the evidence, a RISK statement, a RECOMMENDATION, and a VALIDATE command. You can state what to do and how to confirm it.

Why it matters: the recommendation-and-validate loop is the discipline of remediation, do the fix, then run the validation to prove it took.

Blue-Team angle: severity lets a team triage under pressure. A wall of findings is useless; a wall of findings sorted by severity is a plan.

Lab 22.3

Capture a baseline

Goal: capture a compliance baseline of the current posture and note what it records.

Background: you cannot detect change without a reference. The baseline is that reference, the approved shape of the system.

1. Capture a baseline.

ZSEC BASELINE

Check yourself: the baseline records the APF library count, the SPECIAL users (by name), and the OPERATIONS users, and saves them. Confirm the SPECIAL list matches who you expect.

Why it matters: a baseline captured from a known-good system is the yardstick every later drift check is measured against.

Blue-Team angle: baseline only after you have verified the posture is correct, baselining a compromised system makes the compromise the standard.

Lab 22.4

Detect drift

Goal: compare the live system against the baseline and interpret the result.

Background: drift detection is the payoff of a baseline, it turns “something changed” into a specific, named difference.

1. Run drift detection against your baseline.

ZSEC DRIFT

Check yourself: with nothing changed, drift reports NO DRIFT DETECTED. If you had added a SPECIAL user or an APF library, drift would name that difference.

Why it matters: continuous drift detection is how a monitor catches the exact posture changes that privilege escalation relies on, the moment they happen.

Blue-Team angle: a drift report is a lead, not noise. Every difference from an approved baseline should be explained by a known, authorized change, or investigated as an unauthorized one.

The summary view

Between the full event stream and a single finding sits the summary, the counts that tell you, at a glance, whether today is quiet or not:

ZSECURE SECURITY SUMMARY

EVENTS=6 RARE=5 USERS=1

OTHER 00005

RACFDS 00001

The summary counts, total events, how many were rare, and where they landed.

The value is in the ratio and the categories. Six events, five rare, one touching RACFDS, is a very different day from six hundred events with none rare. The summary is the first thing a defender reads each morning, then drills from the summary into the rare list and into the individual records.

The monitoring rhythm

Together the tools define a daily rhythm. Read the summary for the shape of the day; open the rare list for the events that matter; read the underlying records to establish facts; act on the finding's recommendation; and run its validate step to prove the fix. In parallel, drift detection runs against the baseline so any change to the approved posture surfaces on its own. Events look backward, drift looks at now, the baseline defines correct, together, continuous assurance.

Lab 22.5

Read the summary and drill down

Goal: read the summary counts, then drill from them into the rare events and a specific record.

Background: the monitoring rhythm starts at the summary and drills only where the counts warrant.

1. Read the summary, then follow any rare count into detail.

ZSEC SUMMARY

ZSEC RARE

Check yourself: the summary shows total, rare, and category counts; a non-zero RACFDS or HASH count is your cue to open the rare list and read the records behind it.

Why it matters: drilling only where the counts warrant is what makes monitoring sustainable.

Blue-Team angle: a rising rare count, or a first-ever hit in a category like RACFDS, is a change a summary surfaces immediately and a raw event stream buries.

The watchlist in the rare filter

The rare-event filter is worth reading as a document in its own right, it is the defender's watchlist, encoded:

ZSECURE RARE / HIGH-RISK EVENT REVIEW

RARE FILTERS:

RACFDS	access to the RACF database
HASH	password-hash extraction
IND\$FILE	3270 file transfer
PASSTICKET	one-time credential use
ICSF	cryptographic services
SYS1.MAN	the SMF data itself
APF	authorized-program libraries

The rare-event filter, read as a watchlist of the resources whose abuse signals an attack.

Each entry is a resource whose abuse is a hallmark of a real intrusion rather than routine work: reaching the RACF database, extracting hashes, moving files, minting credentials,

touching crypto, altering the SMF evidence, or changing an APF library. A defender who memorizes this list has memorized the high-value targets of the whole system, and Part IX's attacks are, almost line for line, attempts against exactly these.

Lab 22.6

Turn the filter into alerts

Goal: read the rare-event filter as a watchlist and map each entry to what its abuse would mean.

Background: the filter list is a ready-made set of high-value alerts. Understanding each is understanding what to watch.

1. Run the rare review and read the filter list.

ZSEC RARE

Check yourself: for each filter (RACFDS, HASH, PASSTICKET, APF and the rest) you can say what an event hitting it would suggest an attacker was doing.

Why it matters: alerts are only as good as the list behind them. This filter is a tested list of what matters on a mainframe.

Blue-Team angle: promote every one of these filters to a real-time alert. A single hit on RACFDS or APF outside a change window deserves immediate investigation.

Reading a clean drift report

The everyday drift report is the reassuring one, and reading it correctly matters as much as reading an alarming one:

IBM Security zSecure - Compliance Drift V3.1.0

BASELINE: 2026-07-01 01:47:58 CURRENT: 2026-07-01 01:47:58

NO DRIFT DETECTED - posture matches the captured baseline.

A clean drift report, the posture still matches what was approved.

NO DRIFT DETECTED is not nothing; it is a positive assurance that the APF libraries, SPECIAL users, and UACCs the baseline recorded are all still as approved. Reading it daily builds the habit that makes the alarming report unmissable: when the line instead names a new SPECIAL user or an added APF library, the change stands out precisely because the clean report is so familiar.

Lab 22.7

Prove a change shows up as drift

Goal: reason through what a specific posture change would produce in a drift report.

Background: understanding drift means knowing exactly what a given change looks like when the report runs.

1. Given a baseline, state what drift would report if a SPECIAL user were added, then run a clean check for contrast.

ZSEC DRIFT

Check yourself: with nothing changed, drift reports NO DRIFT DETECTED; you can describe how adding a SPECIAL user or an APF library would instead appear as a named difference.

Why it matters: drift only protects you if you know how a real change surfaces in it.

Reasoning it through once makes the alert legible when it fires.

Blue-Team angle: the posture changes that privilege escalation depends on (new SPECIAL users, new APF libraries, loosened UACCs) are exactly what a drift report names. It is a tripwire for Part IX.

Checkpoint: can you review events and narrow to the high-risk ones, read a structured finding and its validate step, capture a baseline of the posture, and detect drift away from it? Those are the skills that turn point-in-time RACF knowledge into continuous defense.

A security monitor like zSecure reads the same RACF and SMF data as the raw commands, but at scale and with structure. It reviews events and isolates the rare, high-risk ones against a watchlist of sensitive resources; it produces findings carrying severity, risk, recommendation, and validation; and, most powerfully, it captures a compliance baseline of the approved posture (APF libraries, SPECIAL and OPERATIONS users, UACCs) and detects drift away from it. That baseline-and-drift discipline is how a defender catches the privilege-escalation changes of Part IX the moment they occur, turning one-time review into ongoing assurance.

What comes next

Every event these tools review comes from one source: SMF, the System Management Facility, the stream of records z/OS writes for nearly everything that happens. The monitor is a lens; SMF is the light. The next chapter goes to that source, the record types that matter for security, how they are collected, and how a defender reconstructs a forensic timeline from them when an investigation demands the raw truth rather than a summary.

CHAPTER 23

SMF & Forensics

By the end of this chapter you will be able to:

Explain what SMF is and why it is the raw truth behind every review.

Name the record types that matter for security, 30, 80, 110.

Read a security (type 80) record and identify who did what.

Explain where SMF data lives and why it must be protected.

Reconstruct a forensic timeline by correlating records.

Recognize evidence destruction as an attack in its own right.

EVERY REVIEW TOOL IN THE last chapter was a lens; SMF is the light. The System Management Facility is the stream of records z/OS writes for nearly everything that happens on the system (every job that runs, every data set opened, every security decision RACF makes. It is the authoritative record, the raw truth an investigation falls back on when a summary is not enough. For a defender, SMF is two things at once: the source that feeds monitoring, and the evidence that reconstructs an incident after the fact. This chapter goes to that source) the record types that matter, how to read one, and how to turn a pile of records into a timeline of what actually happened.

What SMF records

SMF writes a record whenever something noteworthy occurs, tagged with a record type that says what kind of event it was, and a timestamp, a user, and a job. The records accumulate in a system data set (historically the SYS1.MANx data sets, and on modern systems a log stream) from which they are periodically offloaded for keeping and analysis. The volume is immense: a busy system writes millions of records a day. That completeness is exactly what makes SMF authoritative. A monitor summarizes it, a report samples it, but the records themselves are the ground truth, and an investigation that needs certainty goes to them.

The record types that matter

You do not need all several hundred SMF record types; a handful carry most of the security weight, and knowing them by number lets you go straight to the relevant records:

Type	What it records
30	Job and step accounting, who ran what, using which resources
80	RACF security events, access, violations, logons, changes
81 / 83	RACF initialization and data-set/security changes
110	CICS transactions, online activity
PassTicket	One-time credential generation and use (within 80)

The SMF record types a security investigation reaches for first.

Type 80 is the one to know first: it is RACF speaking, recording every access decision, every violation, every logon and logoff, every profile change. When the last chapter's monitor flagged a rare event, it was reading type 80. Type 30 ties activity to jobs and resource use, type 110 covers the CICS online world, and PassTicket records (carried within type 80) matter because a one-time credential used from an unexpected place is a classic attack signal.

Reading a security record

A structured view of the type-80 records shows the shape of the evidence:

```
ZSEC SMF
ZSECURE SMF REVIEW
SMF STRUCTURED RECORD LIST
RECORD-ID      TYPE SUBTYPE  USERID  JOBNAME  EVENT/RESULT
SMF-33099AFCD 80    RACF      IBMUSER  IBMUSER  JOB SUBMIT SUCCESS
```

A type-80 security record: identity, job, event, and result, with a unique id.

Each record answers the investigator's questions in one line: which record (the id), what kind (type 80, RACF), who (USERID), in what context (JOBNAME), what happened and how it ended (EVENT/RESULT). A single record is a fact. The power comes from having every fact, so that a denied access, a privilege change, and a logon can be placed side by side and read as a story. Reading one record fluently (knowing which field answers which question) is the atom of all forensic work on the platform.

Building a timeline

The deliverable of a forensic review is rarely a single record (it is a timeline. You select the records relevant to an incident (by user, by resource, by time window), order them, and read the sequence. A timeline might show, in order: a logon from an unusual place, a series of denied accesses as an attacker probes, a successful access once they find an opening, a privilege change, and then activity under the new privilege. No single record proves an attack; the sequence does. Correlating across record types) a type-30 job that ran a type-80 privilege change, followed by type-110 activity in a subsystem, is how an investigator turns scattered facts into a narrative that stands up. The monitors of the last chapter automate the common correlations; SMF is what they, and you, correlate.

Protecting the evidence

If SMF is the evidence, then the SMF data is itself a target. An attacker who can clear or alter the SMF stream can erase the record of what they did (and evidence destruction is an attack in its own right, often the last step of a serious intrusion. This is why SYS1.MAN appeared in the last chapter's rare-event filter: access to the SMF data sets is high-risk by definition. The defensive posture mirrors the RACF database exactly) protect the SMF data sets with UACC(NONE) and tight access, audit every attempt to reach them, and offload

records promptly to storage the same people cannot reach. A gap in the SMF timeline is itself a finding: the absence of records where there should be records is evidence of tampering.

The common forensic mistake is to trust a summary when the question demands the source. Monitors and reports are built for speed and can filter, sample, or age out data; an investigation that must be certain goes to the SMF records themselves. The second mistake is leaving the SMF data reachable, evidence that an attacker can delete is evidence you may not have when you need it most.

Hands-On Labs

These labs assume you are logged on as IBMUSER and can run the SMF reviews. Generating a little activity first (submitting a job, listing a profile) gives the reviews something to show.

Lab 23.1

Read a security record

Goal: view the type-80 records and identify the user, job, event, and result in one.

Background: the type-80 record is the atom of RACF forensics. Reading one fluently is the foundation of every investigation.

1. Generate a little activity, then list the security records.

```
SUBMIT 'IBMUSER.JCL.LAB(SURRJOB)'
```

```
ZSEC SMF
```

Check yourself: a type-80 record shows your userid, the job, the event (JOB SUBMIT), and the result (SUCCESS), each in its own field. You can say who did what, when, and how it ended.

Why it matters: every forensic narrative is built from records like this. Reading one is the skill everything else rests on.

Blue-Team angle: a type-80 record with RESULT of a violation (a denied access) is a lead. Filtering type 80 to failures is a fast way to find probing.

Lab 23.2

Identify the record types

Goal: distinguish the security-relevant record types and know which answers which question.

Background: going to the right record type is half of being fast. This lab fixes the map of 30, 80, and 110 in mind.

1. Run reviews for different record types and note what each contains.

```
ZSEC SMF80
```

```
ZSEC SMF30
```

Check yourself: type 80 carries RACF security events; type 30 carries job and step accounting. You can say which type you would open to answer “who accessed this data set?” (80) versus “what did this job consume?” (30).

Why it matters: knowing the record types by their content is what lets you go straight to the evidence instead of searching everything.

Blue-Team angle: correlating a type-30 job record with the type-80 security events it triggered links an action to its actor, the core move in attribution.

Lab 23.3

Sketch a timeline

Goal: order a set of records by time and read the sequence as a narrative.

Background: a timeline, not a single record, is what an investigation delivers. This lab practices turning records into a story.

1. Generate a few distinct events (a logon-equivalent action, a denied access, a submit) then review the records in time order.

ZSEC EVENTS

Check yourself: you can arrange the records into a sequence (what happened first, next, last) and describe the activity as a narrative rather than a list. A denied access followed by a success is a very different story from either alone.

Why it matters: attacks are sequences. Only a timeline reveals the shape (probe, breakthrough, escalation) that no single record shows.

Blue-Team angle: the tell-tale forensic sequence (repeated failures, then a success, then a privilege change) is exactly the pattern a timeline exposes and a single record hides.

Lab 23.4

Protect the evidence

Goal: confirm the SMF data is protected and understand why a gap in the record is itself a finding.

Background: evidence an attacker can delete is evidence you may not have. This lab checks the control that keeps the record intact.

1. Confirm the SMF data sets are protected, as you did for the RACF database.

LISTDSN DATASET('SYS1.MAN**') ALL

Check yourself: the SMF data is protected with UACC(NONE) and auditing, and only a few accounts can reach it. Anyone able to clear it could erase the record of their own activity.

Why it matters: the integrity of the evidence is a precondition for every investigation.

Protecting SMF is protecting your ability to know what happened.

Blue-Team angle: an unexplained gap in the SMF timeline is a high-severity signal, the absence of records where there should be records points at tampering, and evidence destruction is often the final act of a real intrusion.

PassTickets and one-time credentials

One kind of type-80 record deserves its own attention: the PassTicket, a one-time, time-limited credential a system generates so one application can authenticate to another

without a reusable password. Its use is recorded, and reviewing those records is a distinct check:

```
ZSECURE PASSTICKET SMF REVIEW - SMF TYPE 80
RECORD-ID      TYPE SUBTYPE  USERID  JOBNAME  EVENT/RESULT
SMF-33099AFCD 80   RACF     IBMUSER  IBMUSER  PASSTICKET SUCCESS
```

A PassTicket review, one-time credential use, within the type-80 stream.

PassTickets are convenient and, when abused, dangerous: a stolen generation capability lets an attacker mint credentials for other users. That is why their use is audited separately, a PassTicket generated for an unexpected user, or used from an unexpected place, is a strong signal.

Lab 23.5

Review PassTicket activity

Goal: run the PassTicket review and explain why one-time-credential use is audited on its own.

Background: PassTickets bridge applications without shared passwords, which makes their misuse a high-value attack, and their audit a high-value check.

1. Run the PassTicket review.

```
ZSEC PASSTICKET
```

Check yourself: the review lists PassTicket events as type-80 records with the user and result. A PassTicket for a user who should not need one is what this review surfaces.

Why it matters: PassTicket abuse is a quiet way to move between identities and systems. Auditing it separately keeps it visible.

Blue-Team angle: PassTicket generation for privileged users, or a burst of PassTicket activity, is a known lateral-movement pattern worth alerting on.

A record type for every question

In practice you go straight to the record type that answers your question. Reviewing a couple side by side shows how they differ:

```
ZSECURE SMF REVIEW
RECORD-ID      TYPE SUBTYPE  USERID  JOBNAME  EVENT/RESULT
SMF-33099AFCD 80   RACF     IBMUSER  IBMUSER  JOB SUBMIT SUCCESS
```

```
ZSECURE SMF80 SMF REVIEW - SMF TYPE 80
RECORD-ID      TYPE SUBTYPE  USERID  JOBNAME  EVENT/RESULT
SMF-33099AFCD 80   RACF     IBMUSER  IBMUSER  DATASET ACCESS SUCCESS
```

Going straight to the record type that answers the question at hand.

The general review shows the stream; a type-specific review narrows to one kind of event. For “who accessed this data set?” you open type 80; for “what did this job consume?” type 30; for “what happened in CICS?” type 110. Knowing which type answers which question is what turns a search through millions of records into a direct lookup, the difference between minutes and hours in an investigation.

Lab 23.6

Go straight to the right record

Goal: choose the correct SMF record type for a given forensic question and read it directly.

Background: speed in forensics comes from going to the right record type first. This lab drills that choice.

1. For a data-access question, open the type-80 review directly.

ZSEC SMF80

Check yourself: you selected type 80 for a security-access question and read the answer without wading through unrelated records. You can name the type you would choose for a job-accounting or CICS question instead.

Why it matters: the right record type is the difference between a targeted lookup and an endless search. Type fluency is forensic speed.

Blue-Team angle: pre-built queries per record type (failed type-80 accesses, type-80 privilege changes, PassTicket events) turn a mountain of SMF into a handful of ready answers.

Where the records live

Knowing where SMF data physically resides is part of protecting it. Historically the records fill the SYS1.MANx data sets in rotation; modern systems stream them to a log stream, then offload to archive:

SMF DATA SETS

SYS1.MAN1	ACTIVE	used 42%
SYS1.MAN2	ALTERNATE	used 100% (offloaded)
SYS1.MAN3	ALTERNATE	cleared

OFFLOAD TARGET SYS1.SMF.ARCHIVE

Where SMF records live, the MANx data sets in rotation, offloaded to archive.

The rotation matters for forensics: when one data set fills, SMF switches to the next and the full one is offloaded, so the complete record is the live data sets plus the archive together. An investigation that looks only at the live data set may miss what was already offloaded, and an attacker who clears a data set before it is offloaded destroys those records permanently. Protecting all of it, live and archived, is the control.

Lab 23.7

Locate the complete record

Goal: explain where SMF records live across the active data sets and the archive, and why both are needed for a full timeline.

Background: a timeline is only complete if it spans live and offloaded records. Knowing the layout prevents a partial investigation.

1. Identify the active and alternate SMF data sets and the offload target.

Check yourself: you can describe the rotation (active fills, switches, offloads) and state that a complete timeline needs both the live data sets and the archive, all of them protected.

Why it matters: an investigation scoped to only the live data misses offloaded history. Knowing the full layout is knowing where the evidence is.

Blue-Team angle: ensure offload runs reliably and the archive is protected, an attacker who prevents offload, or reaches the archive, can erase history a live-only check would never notice was missing.

A timeline reads as a story

Ordered by time, a handful of type-80 records stop being a list and become a narrative. Read this sequence top to bottom:

TIME	USER	EVENT	RESULT
09:02:11	DUMONT	LOGON	SUCCESS
09:04:37	DUMONT	DATASET ACCESS SYS1.SECRET	INSUFFICIENT
09:04:41	DUMONT	DATASET ACCESS SYS1.SECRET	INSUFFICIENT
09:05:02	DUMONT	DATASET ACCESS SYS1.SECRET	INSUFFICIENT
09:07:55	DUMONT	ALTUSER DUMONT SPECIAL	SUCCESS
09:08:10	DUMONT	DATASET ACCESS SYS1.SECRET	SUCCESS

The same records in time order tell a story: probe, escalate, succeed.

No single line is alarming, but the sequence is unmistakable: a logon, three failed attempts on a protected data set, a self-granted SPECIAL attribute, and then the same access succeeding. That is probe, escalation, and breakthrough, an attack narrative that only the timeline reveals. This is the deliverable of forensics: not a record, but the ordered story the records tell together, and the exact shape a defender learns to recognize.

Lab 23.8

Read the attack in the sequence

Goal: read an ordered set of records and identify the probe-escalate-succeed pattern.

Background: attacks are sequences, and recognizing the shape is the forensic skill that matters most.

1. Order a set of records by time and describe what the sequence shows.

Check yourself: you can point to the failed accesses (the probe), the attribute change (the escalation), and the later success (the breakthrough), and explain why the order is what proves intent.

Why it matters: the same records in any other order prove little; in time order they prove an attack. Sequence is the evidence.

Blue-Team angle: the failure-then-escalation-then-success shape is a high-confidence detection. Correlating a privilege change between failed and successful access to the same resource is a strong, low-false-positive rule.

Checkpoint: can you explain what SMF is, name the record types that matter, read a type-80 record, build a timeline by correlating records, and explain why protecting the SMF data is protecting the evidence itself? Those are the forensic foundations the kill-chain chapters in Part IX build their investigations on.

SMF is the authoritative record of nearly everything z/OS does (the raw truth behind every monitor and the evidence behind every investigation. A few record types carry the security weight: type 80 for RACF events, type 30 for job accounting, type 110 for CICS, PassTicket records within 80. A single record answers who, what, and how it ended; a timeline built by correlating records reveals the sequence) probe, breakthrough, escalation (that an attack actually is. Because the records are the evidence, the SMF data is a target: it must be protected and audited like the RACF database, and a gap in the timeline is itself a finding. This closes Part IV) you now hold the identity, resource, database, monitoring, and forensic foundations of mainframe security.

What comes next

Part IV has been about who may act and how their actions are recorded. Part V turns to running the system itself (operating it from the master console. The next chapter opens the console: the DISPLAY commands that show what is active, the operator commands that start and stop work, and the operations log that records it all) the day-to-day controls of the people who keep a mainframe running, and the same commands an intruder would reach for to understand a system they have just entered.

Part V

Operating the System

Someone has to keep the system running. Part V is the operator and system programmer view: the console, the libraries and parameters that define the system, the workload and serialization that keep it healthy, and the tooling a sysprog relies on.

CHAPTER 24

The Master Console

By the end of this chapter you will be able to:

Use DISPLAY commands to see what is active on the system.

Read D IPLINFO, D TCPIP,PORTS, and D APF output.

Start and stop started tasks and read the console's replies.

Recognize the message-id prefixes and the component behind each.

Explain WTOR replies and the operations log.

See why the same commands aid an operator and an intruder.

PART IV WAS ABOUT WHO may act. Part V is about running the system, and it begins where operators run it: the master console. The console is a command line into the heart of z/OS, and its commands come in two families, DISPLAY commands, which show you the state of the system without changing anything, and action commands, which start, stop, and vary the work the system is doing. An operator lives here. So, notably, does an intruder who has just gained console access: the very first thing anyone does on an unfamiliar system is ask it what it is and what it is running, and the console answers in full. Learning to read it is learning both to operate a mainframe and to understand what an attacker learns the moment they reach it.

Displaying active work

The most-used console command is DISPLAY, abbreviated D. D A,L lists the active work, the started tasks, the batch jobs, and the users logged on:

```
D A,L
IEE114I 00.00.00 2026.107 ACTIVITY
  JOBS      M/S      TS USERS      SYSAS
  FTPD1     STC      IBMUSER      OMVS
  CICS      STC      RUARIV       DB2A
```

D A,L, the active work on the system: started tasks, batch, and users.

The display groups the work: JOBS and started tasks (M/S, for started tasks and mounted systems), the TS USERS logged on interactively, and the system address spaces (SYSAS). Here FTPD1 and CICS are running as started tasks, and the display names the subsystems alongside. For an operator this is the pulse of the system, what is running right now. For anyone assessing the system, it is a map of the services worth attacking, delivered on request.

What is this system?

D IPLINFO answers the most basic orienting question, what system is this, and when did it start:

```
D IPLINFO
IEE254I IPLINFO DISPLAY
SYSTEM IPLED AT 2026.107 08:00:00
RELEASE z/OS 02.05.00 GIBSON TRAINING LPAR
```

D IPLINFO, the release level and when the system was last IPLed.

This one line of intelligence (the z/OS release and the time of the last IPL) tells an operator whether the system is at the expected level and how long it has been up, and tells an assessor which release-specific weaknesses might apply. Uptime since IPL also matters operationally: a system that IPLed unexpectedly recently is worth asking about.

The network, from the console

D TCPIP,PORTS shows the network services the system exposes and their state, the same map the network chapter drew, but from inside:

```
D TCPIP,PORTS
EZZ2500I TCP/IP PORT STATUS
```

SERVICE	STATE	PORT	DESCRIPTION
CICS	STARTED	2323	CICS transaction region
DB2	STARTED	50000	Db2 subsystem
FTPD	STOPPED	21	FTP daemon
GIBDASH	STARTED	8443	Dashboard
JES2	STARTED	--	Job Entry Subsystem
OMVS	STARTED	2022	z/OS UNIX

D TCPIP,PORTS, the network services and their state, seen from the console.

Each row is a service, its state, and its port. From the console an operator confirms that the services that should be up are up, and spots one that should not be. The state column is the point: FTPD here is STOPPED, and an operator who expected it running would investigate. The same view tells an attacker exactly which doors are open and which are closed.

The APF list

D APF lists the authorized program libraries, and this display carries unusual security weight:

```
D APF
CSV450I APF LIST DISPLAY
```

DSNAME	VOLSER	STATUS
SYS1.LINKLIB	MVSRES	APF
SYS1.LPALIB	MVSRES	APF
SYS1.PROCLIB	MVSRES	APF
SYS1.PARMLIB	MVSRES	APF
SYS1.VULNLIB	WORK01	APF
TCPIP.SEZALOAD	MVSRES	APF

D APF, the authorized libraries. Note SYS1.VULNLIB on a work volume.

An APF-authorized library holds programs allowed to run with system privilege, bypassing normal restrictions. That makes the APF list one of the most security-sensitive displays on the system: any program in any of these libraries can, in effect, do anything. Read this list and one entry stands out, SYS1.VULNLIB, on the work volume WORK01, sitting among the trusted system libraries. An unexpected APF library is a five-alarm finding, because write access to it is write access to system privilege. This entry is the thread the next chapter, and the privilege-escalation chapter in Part IX, both pull hard.

Starting and stopping work

Beyond looking, the operator acts. STOP (P) and START (S) control started tasks, and the console confirms each with a message:

```
P  FTPD
IEE334I  FTPD  STOPPED
S  FTPD
IEE352I  FTPD  STARTED
```

Stopping and starting a service from the console, each confirmed by a message.

P stops a task and S starts one; the related VARY (V) command brings devices, paths, and network resources online or offline. These are the levers of day-to-day operation, and, in the wrong hands, of sabotage or cover. Stopping the right service at the right moment can disable a control or hide activity, which is why start and stop of security-relevant tasks are themselves events worth auditing. The console records every one.

WTORs and the operations log

Two more console facts complete the picture. Some situations require an operator to answer: the system issues a write-to-operator-with-reply (a WTOR), numbers it, and waits, and the operator responds with R followed by that number and the reply. Outstanding WTORs are visible with a display and must be answered for work to proceed. Everything that crosses the console (every message, command, and reply) is captured in the operations log (OPERLOG, and the older SYSLOG). That log is both an operator's history and a forensic record: it shows who issued which command and when, which is exactly what an investigation needs to reconstruct operator activity, and exactly what an intruder with console access would want to alter or avoid.

Reading the message ids

Every console message begins with an id, and the letter prefix names the component that issued it. Learning the prefixes lets you read the console at a glance:

Prefix	Component
IEE / IEA	Master scheduler, console, and system (IPL)
IEF	Job scheduling and allocation
CSV	Contents supervision, APF and the link list
EZZ / EZ	TCP/IP (Communications Server)
\$HASP	JES2
ICH	RACF

The message-id prefixes and the component behind each.

A beginner reads console messages as noise; an operator reads them as a language. The prefix tells you the component, the number identifies the exact message, and the text is often terse because the id carries the meaning. Do not skim past a message because it looks cryptic, a CSV450I about the APF list or an ICH408I denial is telling you something a summary never will.

Hands-On Labs

These labs assume you can issue console commands as an operator. DISPLAY commands are safe to run freely; treat START and STOP with the care they deserve.

Lab 24.1

Display the active work

Goal: list the active work and identify the started tasks, users, and subsystems.

Background: knowing what is running right now is the operator's constant question, and the first thing anyone asks of a new system.

1. Display the active work in long form.

D A,L

Check yourself: you can name the started tasks (such as FTPD1 and CICS), the logged-on users, and the subsystems shown alongside. That is the live state of the system.

Why it matters: every operational decision starts from what is currently running. This display is where that knowledge comes from.

Blue-Team angle: an unexpected started task in D A,L (something running that no one authorized) is exactly the kind of anomaly a baseline of normal activity helps you catch.

Lab 24.2

Identify the system

Goal: read D IPLINFO and state the release and last IPL time.

Background: orienting on an unfamiliar system starts with what it is and when it started. One command answers both.

1. Display the IPL information.

D IPLINFO

Check yourself: you can state the z/OS release and the time of the last IPL. Confirm the release matches what you expect for this system.

Why it matters: the release governs which behaviors and which weaknesses apply, and the IPL time tells you how long the current system has been running.

Blue-Team angle: an unexpected recent IPL can be routine maintenance, or the sign of a crash or a forced restart worth explaining.

Lab 24.3**Survey the network and the APF list**

Goal: display the network ports and the APF list, and flag anything unexpected.

Background: two displays carry outsized security weight, the open ports and the authorized libraries. Reading them is a fast posture check.

1. Display the ports, then the APF list.

D TCPIP,PORTS

D APF

Check yourself: you can list the started services and their ports, and (essentially) you spot SYS1.VULNLIB on a work volume in the APF list as an entry that does not belong among the trusted system libraries.

Why it matters: an unexpected open port or an unexpected APF library are two of the highest-value findings on a system, and both are one console command away.

Blue-Team angle: the APF list should match an approved baseline exactly. A library like SYS1.VULNLIB, especially on a writable work volume, is a direct path to system privilege, the escalation Part IX pursues.

Lab 24.4**Stop and start a service**

Goal: stop a non-essential started task and start it again, reading the console's confirmation of each.

Background: controlling started tasks is core operation. Doing it cleanly, and reading the messages, is a basic operator skill.

1. Stop a non-essential service, then start it again.

P FTPD

S FTPD

Check yourself: IEE334I confirms the stop and IEE352I confirms the start. Between them the service is down, as D TCPIP,PORTS would show.

Why it matters: starting and stopping work is the operator's core lever. The confirming messages are how you know the action took.

Blue-Team angle: the stop of a security-relevant task (an audit process, a monitor) can be an attacker disabling a control. Start and stop of such tasks belong on a watchlist.

The operations log as a record

Because every message and command crosses the console, the operations log is a continuous record of the system's life. Read a slice of it and the day is legible:

```
08:00:00 IEE254I SYSTEM IPLED AT 2026.107 08:00:00
08:00:04 $HASP373 JES2      STARTED
08:00:11 IEE352I OMVS      STARTED ON PORT 2022
09:14:22 IEE352I FTPD      STARTED ON PORT 21
10:31:05 P FTPD
10:31:05 IEE334I FTPD STOPPED
10:33:40 ICH408I USER(DUMONT) SYS1.SECRET INSUFFICIENT ACCESS
```

An operations-log slice: IPL, service starts, an operator stop, and a security denial, all timestamped.

Each line carries a time, a message id, and an event, so the log reads as a timeline of everything the system and its operators did. The value cuts both ways: for an operator it is the history to review after an incident, and for a defender it is a forensic record, the ICH408I near the bottom places a denied access at a precise moment, next to the operator actions around it. This is why console access is powerful and why the log itself is protected: whoever can alter it can rewrite the record of what happened.

Displays that answer specific questions

Beyond the everyday displays, the DISPLAY command has a member for almost every question an operator asks, processors, storage, the lock manager, the workload. A few worth knowing:

```
D M=CPU      processor and engine status
D GRS,C      global resource serialization contention
D WLM,SYSTEMS workload manager state
D J,jobname   detail for one job
D U          device and unit status
```

A sampler of targeted DISPLAY commands, each answering one operational question.

The pattern is that D plus a keyword narrows to one area, so an operator diagnosing a problem goes straight to the relevant display rather than reading everything. The same targeting serves an assessor: D M=CPU sizes the machine, D GRS,C reveals contention that might be exploited, and D J drills into a specific job. Knowing which display answers which question is the difference between operating with intent and guessing.

Lab 24.5

Read the operations log as a timeline

Goal: read a slice of the operations log and reconstruct the sequence of system and operator events.

Background: the log is the system's memory. Reading it as a timeline is how you review an incident or a shift.

1. Review the console log for a period and note the IPL, service, operator, and security events in order.

Check yourself: you can arrange the log entries into a timeline (IPL, services starting, an operator action, a security denial) and describe what happened when. Timestamps and message ids make each event unambiguous.

Why it matters: the operations log is the authoritative record of operator activity, and reading it as a timeline is how both operations and forensics use it.

Blue-Team angle: correlate the operations log with SMF for a complete picture, the console log shows operator commands, SMF shows the security decisions, and together they reconstruct an incident. A gap or alteration in the log is itself a finding.

Lab 24.6

Go to the display that answers the question

Goal: choose and run the targeted DISPLAY command for a specific operational question.

Background: fast operation means going straight to the right display. This lab drills matching a question to its command.

1. To size the machine, display the processors; to check contention, display GRS.

D M=CPU

D GRS,C

Check yourself: you selected D M=CPU for a processor question and D GRS,C for a contention question, rather than reading unrelated output. You can name the display you would use for a single job (D J,jobname) or device status (D U).

Why it matters: the targeted display turns diagnosis into a direct lookup. Knowing the map of DISPLAY keywords is what makes an operator fast.

Blue-Team angle: the same targeting helps an assessor move quickly (sizing the system, finding contention, drilling into a suspicious job) which is why unusual DISPLAY activity from an unexpected console is itself worth noticing.

Checkpoint: can you display the active work, identify the system with D IPLINFO, survey the ports and the APF list and flag the odd entry, start and stop a task, and read a message id by its prefix? Those are the operator's core skills, and the first moves of anyone learning a system from its console.

The master console runs z/OS through two command families: DISPLAY, which shows state without changing it, and action commands like START, STOP, and VARY, which change the work the system is doing. D A,L shows active work, D IPLINFO the release and last IPL, D TCPIP,PORTS the network services, and D APF the authorized libraries, where an unexpected entry like SYS1.VULNLIB is a direct route to system privilege. Message-id prefixes name the component behind each line, WTORs demand operator replies, and the operations log records everything. The console is where an operator works and where an intruder orients, which makes fluency in it a skill for both running and defending a mainframe.

What comes next

The console showed the APF list and one entry that did not belong. Where does that list come from, and how does a library get onto it? The answer is the system's configuration, the PARMLIB members read at IPL, the PROCLIB procedures that define started tasks, and the APF list itself. The next chapter opens that configuration: how the live system is assembled from it, how to explore it, and how to hunt for exactly the kind of rogue APF library the console just revealed.

CHAPTER 25

PARMLIB, PROCLIB & APF

By the end of this chapter you will be able to:

Explain how the live system is assembled from PARMLIB at IPL.

Name the PARMLIB members that matter and what each configures.

Explain what PROCLIB holds and how started tasks are defined.

State what APF authorization means and why it equals system power.

Hunt for a rogue APF library by comparing live state to baseline.

Recognize a dynamic APF addition as a persistence and escalation move.

THE CONSOLE IN THE LAST chapter displayed the system's state (its active work, its ports, its authorized libraries. But where does that state come from? A running z/OS system is assembled at IPL from its configuration, and that configuration lives in a few well-known places: PARMLIB, which holds the parameters that shape the system; PROCLIB, which holds the procedures that define started tasks; and, defined within PARMLIB, the APF list of authorized libraries. This chapter opens that configuration. It matters for operation) you cannot run a system you cannot configure, and it matters intensely for security, because the APF list the console revealed, with its out-of-place SYS1.VULNLIB, is defined here, and the hunt for exactly that kind of rogue entry is one of the highest-value reviews on the platform.

PARMLIB: the system's configuration

SYS1.PARMLIB is a library of members, each configuring one aspect of the system, read during IPL to build the running system. An explorer lists them with what each does:

```
PARMLIB LIST
```

```
SYS1.PARMLIB EXPLORER
```

```
MEMBER      DESCRIPTION
```

```
BPXPRM00    ROOT FILESYSTEM('OMVS.GIBSON.ROOT.ZFS')
```

```
COMMND00    COM='START JES2'
```

```
CONSOL00    CONSOLE DEVNUM(01F)
```

```
COUPLE00    COUPLE  SYSPLEX(GIBPLEX)
```

```
GRSRNL00    RNLDEF RNL(INCL) TYPE(GENERIC) QNAME(SYSDSN)
```

```
IEA0PT00    MCCAFTH=(400,600)
```

The PARMLIB explorer, each member configures one aspect of the running system.

Read the descriptions and the system's shape appears: BPXPRM00 configures the UNIX file system, COMMND00 says which commands to issue automatically at startup (here, start JES2), CONSOL00 defines the consoles, COUPLE00 places the system in a sysplex.

The controlling member is IEASYS00, which names which of the other members to use, the master switch for the whole IPL. Change a PARMLIB member and you change what the system becomes at its next IPL, which is why write access to PARMLIB is itself a powerful and audited privilege.

The members that matter

There are dozens of members, but a handful carry the weight for operation and security, and knowing them by name tells you where to look:

Member	What it configures
IEASYS00	The master member, selects the others to use
PROGxx	The APF list, the link list, and system exits
BPXPRMxx	z/OS UNIX, the file system and OMVS limits
SMFPRMxx	SMF, which records are collected and where
CONSOLxx	The consoles and their authority
COMMNDxx	Commands issued automatically at IPL

The PARMLIB members a defender reads first, each shapes posture.

Two of these are security-critical. PROGxx defines the APF list and the system exits, so it decides which libraries hold privileged code, the subject of the rest of this chapter. SMFPRMxx decides what is audited; a record type omitted here is a record type never written, so an attacker who could edit it could blind the system. Reading these members is reading the system's security posture at its source.

PROCLIB: the started tasks

SYS1.PROCLIB holds the JCL procedures that define started tasks, the services the console showed running. Listing it names them:

```
LISTDS 'SYS1.PROCLIB' MEMBERS
SYS1.PROCLIB MEMBERS
--MEMBER--
CICSSTAR  DB2MSTR  DB2START  FTPD
INIT      JES2     LLA        NET
OMVS      RMF
```

PROCLIB members, the procedures behind the started tasks the console displays.

Each member is the procedure that runs when a task is started: JES2 for the job subsystem, OMVS for z/OS UNIX, CICSSTAR and DB2MSTR for the subsystems, FTPD for file transfer. When the console issued S FTPD, it ran the FTPD member of PROCLIB. This makes PROCLIB security-relevant too: a started task runs with the identity and authority its procedure specifies, so a modified PROCLIB member (pointing a trusted task at attacker code, or running it under a more powerful identity) is a route to persistent, privileged execution. Write access to PROCLIB, like PARMLIB, is a serious authority.

APF: authorized means privileged

Now the heart of the chapter. APF (the Authorized Program Facility) is the mechanism that lets certain programs run with system privilege, bypassing the normal restrictions that keep ordinary code in its lane. A program earns that privilege by living in an APF-authorized library. The list of those libraries, defined in PROGxx and visible from the console, is therefore a list of exactly where privileged code may live:

D APF

CSV450I APF LIST DISPLAY

DSNAME	VOLSER	STATUS
SYS1.LINKLIB	MVSRES	APF
SYS1.LPALIB	MVSRES	APF
SYS1.PROCLIB	MVSRES	APF
SYS1.PARMLIB	MVSRES	APF
SYS1.VULNLIB	WORK01	APF

The live APF list, every library here may hold code that runs with system privilege.

The security weight is total: any program in any APF library can, in effect, do anything on the system, turn off controls, read or write any data, alter the security database. It follows that write access to an APF library is equivalent to system privilege, because whoever can put a program into an APF library can run it with full authority. This single fact makes the APF list one of the two or three most important things to get right, and to watch, on any mainframe.

The rogue-APF hunt

Read the APF list again and one entry does not belong: SYS1.VULNLIB, on the work volume WORK01, among the trusted MVSRES system libraries. This is the rogue-APF hunt, and it is a defining mainframe security review. The method is comparison: the live APF list, from D APF, against what PROGxx is supposed to define, and against an approved baseline. Any library that appears in the live list but not the baseline is a finding. A library on a writable, non-system volume is worse, because if ordinary users can write to WORK01, they can drop a privileged program into an APF library, instant system power. The whole exposure is visible in one line of the display, and finding it is the point of the hunt.

Signal in the APF list	Why it matters
Library not in the baseline	Added outside change control, investigate
Library on a work/non-system volume	May be writable by ordinary users
Writable APF library	Equivalent to granting system privilege
Added dynamically, not in PROGxx	Will vanish at IPL, a persistence tell

What to flag when auditing the APF list.

Dynamic APF and persistence

APF libraries can be added two ways: permanently, in PROGxx so they return at every IPL, or dynamically, with a SETPROG APF,ADD command that takes effect immediately but does not survive an IPL. That difference is a detection opportunity. A library that is APF-authorized in the live system but absent from PROGxx was added dynamically, which is exactly what an attacker does to escalate quietly, and a discrepancy a defender can hunt for by comparing the live list to the configured one. The counterpart persistence move is to add the library to PROGxx so it returns after every restart. Either way, the control is the same: know your approved APF list, compare the live and configured versions against it, and treat any difference as an incident until proven a change.

The common mistake is trusting the configured APF list without checking the live one. Because libraries can be added dynamically, PROGxx and the running system can disagree, and the gap is precisely where a quiet escalation hides. Always compare D APF (live) against PROGxx (configured) against your baseline (approved); any two of the three disagreeing is a lead.

Hands-On Labs

These labs assume you can explore PARMLIB and issue the console APF display.

Lab 25.1

Explore PARMLIB

Goal: list the PARMLIB members and identify what several of them configure.

Background: PARMLIB is where the system is defined. Knowing your way around it is the foundation of both configuration and review.

1. List the PARMLIB members with their descriptions.

`PARMLIB LIST`

Check yourself: you can point to BPXPRM00 (UNIX), COMMND00 (auto commands), CONSOL00 (consoles), and name IEASYS00 as the master member that selects the others.

Why it matters: the whole running system is built from these members. Reading them is reading the system's configuration at its source.

Blue-Team angle: SMFPRMxx and PROGxx are the two members a defender reads first, they decide what is audited and where privileged code may live.

Lab 25.2

List the started-task procedures

Goal: list PROCLIB and connect its members to the running started tasks.

Background: every started task the console shows has a procedure in PROCLIB. Linking the two makes the system's startup legible.

1. List the PROCLIB members.

`LISTDS 'SYS1.PROCLIB' MEMBERS`

Check yourself: you can match members (JES2, OMVS, FTPD, CICSSTAR) to the tasks you saw active on the console. Starting FTPD ran the FTPD member here.

Why it matters: a started task is only as trustworthy as its procedure. Knowing where procedures live is knowing where to check that trust.

Blue-Team angle: a modified PROCLIB member can point a trusted task at attacker code or run it under a stronger identity, review write access to PROCLIB as carefully as to PARMLIB.

Lab 25.3

Audit the APF list

Goal: display the APF list and identify any library that does not belong.

Background: the rogue-APF hunt is a defining mainframe review. This lab runs it on the live list.

1. Display the authorized libraries.

D APF

Check yourself: you can identify SYS1.VULNLIB on the work volume WORK01 as out of place among the trusted MVSRES system libraries, an APF library that is a candidate for exploitation.

Why it matters: an unexpected APF library, especially on a writable volume, is a direct route to system privilege. Finding it is one of the highest-impact results a review can produce.

Blue-Team angle: compare the live APF list against an approved baseline on a schedule. Any addition is a finding until it is tied to an authorized change, this is precisely the escalation Part IX attempts.

Lab 25.4

Reason about dynamic APF

Goal: explain how a live-versus-configured APF discrepancy reveals a dynamic addition, and why that is a persistence tell.

Background: dynamic APF additions do not survive IPL, which makes the gap between live and configured a powerful detection.

1. Compare the live APF list against what PROGxx defines and note any difference.

Check yourself: a library APF-authorized in the live system but absent from PROGxx was added dynamically. You can explain that this both grants immediate privilege and would vanish at the next IPL unless also added to PROGxx.

Why it matters: the live-versus-configured comparison is a reliable way to catch a quiet escalation that neither list alone would reveal.

Blue-Team angle: alert on any SETPROG APF,ADD and on any live/PROGxx APF discrepancy. Both are strong signals of an escalation or persistence attempt in progress.

Lab 25.5

Confirm the sensitive members are protected

Goal: confirm PARMLIB and PROCLIB are protected against unauthorized write access.

Background: because these libraries define the system, write access to them is system-level power. Confirming they are locked down is the capstone control.

1. Check the profiles protecting PARMLIB and PROCLIB.

```
LISTDSN DSN('SYS1.PARMLIB') ALL
```

Check yourself: the profiles show UACC(NONE) with a short access list and auditing. Only trusted administrators should be able to write to PARMLIB or PROCLIB.

Why it matters: locking the APF list means nothing if the member that defines it, or the procedures it protects, can be rewritten. The protection must extend to the configuration itself.

Blue-Team angle: write access to PARMLIB, PROCLIB, or any APF library is equivalent to system privilege. Auditing who holds it is as important as auditing who holds SPECIAL.

Inside PROGxx: how the APF list is written

The APF list the console shows is written, statement by statement, in the PROGxx member. Reading it shows how each library gets its authorization, and how a rogue entry would be added:

```
/* SYS1.PARMLIB(PROG00) - program control */
APF ADD DSNAME(SYS1.LINKLIB) VOLUME(MVSRES)
APF ADD DSNAME(SYS1.LPALIB) VOLUME(MVSRES)
APF ADD DSNAME(SYS1.PROCLIB) VOLUME(MVSRES)
APF ADD DSNAME(SYS1.PARMLIB) VOLUME(MVSRES)
APF ADD DSNAME(SYS1.VULNLIB) VOLUME(WORK01)
LNKLST ADD NAME(LNKLST00) DSNAME(SYS1.LINKLIB)
EXIT ADD EXITNAME(SYS.IEFACTRT) MODNAME(ACCTRTN)
```

A PROGxx member, each APF ADD statement authorizes one library; the last one does not belong.

Every APF ADD line names a library and its volume, and each grants that library the power to hold privileged code. Reading top to bottom, the trusted system libraries on MVSRES are followed by one line that stands out: SYS1.VULNLIB on WORK01. In a real review this is exactly how the rogue entry is found, not only in the live display, but in the very member that defines it. PROGxx also adds link-list libraries and system exits, both of which are their own trust and their own attack surface: an EXIT ADD points a system exit at a module, and a module named there runs at the heart of the system.

SETPROG: adding APF authority live

A library can also be authorized without editing PROGxx at all, using the SETPROG command, which takes effect at once:

```
SETPROG APF,ADD,DSNAME=SYS1.VULNLIB,VOLUME=WORK01
CSV410I DATA SET SYS1.VULNLIB VOLUME WORK01 ADDED TO APF LIST
```

SETPROG APF,ADD, immediate APF authorization, and the message that records it.

The CSV410I message is both the confirmation and the detection point. A dynamic add like this grants privilege instantly but leaves PROGxx unchanged, so the authorization disappears at the next IPL, unless the attacker also edits PROGxx to make it persist. That split is the defender's opportunity: watch for the CSV410I message, and compare the live APF list against PROGxx, and a dynamic escalation cannot stay hidden.

Lab 25.6

Read the APF list at its source

Goal: read the PROGxx member and match its APF ADD statements to the live APF list, spotting the rogue entry in the configuration itself.

Background: the rogue-APF hunt is stronger when you read both the live list and the member that defines it. This lab reads the source.

1. View the PROGxx member and read its APF ADD statements.
2. Compare them against the live D APF output.

Check yourself: each APF ADD in PROGxx matches a line in D APF, and SYS1.VULNLIB on WORK01 appears in both as the entry that does not belong. If a library appeared in D APF but not in PROGxx, it was added dynamically.

Why it matters: reading the configuration source, not just the live state, is how you tell a permanent rogue entry from a dynamic one, and how you find both.

Blue-Team angle: monitor PROGxx for changes and the console for CSV410I. An APF ADD you did not authorize, in either place, is a system-privilege escalation and a top-severity event.

Checkpoint: can you explain how the system is built from PARMLIB, name the members that matter, link PROCLIB to the started tasks, state why APF equals system privilege, run the rogue-APF hunt, and explain how a dynamic APF addition is detected? Those are the configuration and escalation-defense skills the rest of Part V and Part IX both depend on.

A running z/OS system is assembled at IPL from its configuration: PARMLIB members that shape the system (IEASYS00 as master, PROGxx for the APF list and exits, SMFPRMxx for auditing), PROCLIB procedures that define the started tasks, and the APF list itself. APF authorization lets a program run with system privilege, so every APF library (and write access to it) is equivalent to system power. The rogue-APF hunt compares the live list against the configured and approved ones to find libraries that do not belong, like SYS1.VULNLIB on a writable volume, and to catch dynamic additions that reveal a quiet escalation. Protecting the configuration (PARMLIB, PROCLIB, and every APF library) is protecting system privilege itself.

What comes next

Configuration decides what the system is; other subsystems decide how well it runs and shares its resources. The next chapter turns to the workload and resource managers, WLM, which decides which work gets served first; RMF, which measures performance; GRS, which serializes access so two jobs do not corrupt the same data; and the Health

Checker, which continuously flags configurations that drift from best practice, including security ones. Together they are how a sysprog keeps a mainframe not just running, but running right.

CHAPTER 26

WLM, RMF, GRS & Health Checks

By the end of this chapter you will be able to:

Explain what WLM does, deciding which work is served first.

Describe RMF's role in measuring system performance.

Explain GRS serialization and read contention.

Run the Health Checker and read its findings by severity.

Recognize that many health checks are security checks.

Act on a high-severity finding and connect it to earlier chapters.

A SYSPROG DOES MORE THAN keep a system running; they keep it running well and safely. Four subsystems make that possible. WLM, the Workload Manager, decides which work gets the machine's resources first. RMF, the Resource Measurement Facility, measures how the system is performing. GRS, Global Resource Serialization, makes sure two jobs never corrupt the same data by fighting over it. And the Health Checker runs continuously in the background, comparing the system against best practice and raising findings when it drifts, many of which are security findings. This chapter covers all four, with the Health Checker as the centerpiece, because it is where operations and security meet most directly.

WLM: deciding what runs first

A mainframe runs many kinds of work at once (online transactions that must respond in milliseconds, batch jobs that can wait, interactive users in between) all competing for the same processors and memory. WLM resolves that competition using goals rather than fixed priorities. Work is sorted into service classes, and each service class is given a goal (a response time or a share of the system) and an importance. WLM then continuously adjusts resources so the most important work meets its goals first:

SERVICE CLASS	GOAL	IMP	ACTUAL
CICSHIGH	RESP 90% < 0.5 SEC	1	MEETING
TSOUSERS	RESP 80% < 1.0 SEC	2	MEETING
BATCHHI	VELOCITY 40	3	MEETING
BATCHLOW	DISCRETIONARY	5	DELAYED

WLM service classes, goals and importance decide who is served first.

The security relevance is easy to miss but real. WLM decides what gets starved when the system is busy, so a rogue job placed in a high-importance class could steal resources from real work, and a critical security task placed in a low class could be starved just when it is needed. Understanding how work is classified is understanding how it can be mis-prioritized, deliberately or by accident.

RMF: measuring the system

You cannot manage what you do not measure, and RMF is how a mainframe is measured. It gathers performance data (processor use, memory, I/O, response times) and presents it both live and historically. A sysprog uses RMF to see whether WLM's goals are being met, to find bottlenecks, and to plan capacity. For a defender, RMF is an unexpectedly useful lens: an attack often has a performance signature. A password-guessing run, a job quietly using the machine for its own computation, an unusual spike in a subsystem, these show up as anomalies in resource use before anyone reads a security log. Baseline performance is, among other things, a security baseline; a sharp, unexplained departure from it is worth investigating on security grounds as much as performance ones.

GRS: serializing access

When two jobs want the same resource (the same data set, the same record) something must keep them from corrupting it. GRS provides that serialization through enqueues: a job requests an ENQ on a resource, holds it while it works, and releases it, and GRS ensures only compatible holders coexist. Usually this is invisible. It becomes visible as contention, when work waits for a resource another job is holding, which D GRS,C displays:

```
IEE600I  REPLY TO DISPLAY GRS,C
        NO ENQ CONTENTION EXISTS
```

A contention display, here, no jobs are waiting on a held resource.

No contention is the healthy state. When contention does exist, the display names the resource, the holder, and the waiters, the information needed to break a logjam. The security angle is denial of service: a job that acquires an ENQ on a critical resource and refuses to release it can stall other work indefinitely, and contention analysis is how a defender spots that a hang is a held lock rather than a broken program. A resource held far longer than it should be is worth a look.

The Health Checker

The Health Checker is the subsystem that most directly serves security. It runs a catalog of checks continuously, each comparing some aspect of the system against a best-practice standard and reporting a status and a severity. Listing the active checks shows what it watches:

```
F HZSPROC,DISPLAY,CHECKS
```

CHECK	NAME	SEV	STATUS
GIBAPF01	Broad APF exposure	MED	ACTIVE
GIBAUD01	Console security audit echo	LOW	ACTIVE
GIBNET01	TN3270/TELNET parameters	LOW	ACTIVE
GIBPTK01	PassTicket profile review	MED	ACTIVE

GIBSUR01 SURROGAT delegation

HIGH ACTIVE

The active health checks, note how many are security checks, each with a severity.

Read the list and a striking fact emerges: nearly every check is a security check. Broad APF exposure is the rogue-APF concern from the last chapter. SURROGAT delegation (rated HIGH) is about who may submit work as another user, a privilege-escalation path Part IX pursues. PassTicket profile review guards the one-time credentials from the forensics chapter. TN3270/TELNET parameters concern how terminals connect. The Health Checker is, in large part, a continuous automated security review running quietly alongside operations, which is exactly why a defender reads its output daily.

Reading the checks by severity

Each check carries a severity that lets a team triage. The severities map directly to how much a finding should worry you:

Severity	What a finding means
HIGH	A serious exposure, address it now (e.g. SURROGAT)
MED	A real weakness, schedule remediation (e.g. APF)
LOW	A hardening opportunity, review when able
EXCEPTION	The check failed its standard, investigate the detail
SUCCESSFUL	The check passed, the standard is met

Health-check severities and how a defender should treat each.

The discipline is the same as any findings-based review: sort by severity, act on HIGH first, and track the rest. A HIGH SURROGAT finding says someone may be able to submit work as another user (investigate before doing anything else. A MED APF finding says the authorized-library posture needs tightening. Because the checks run continuously, a check that was SUCCESSFUL yesterday and is an EXCEPTION today is a change worth explaining) the Health Checker doubles as a drift detector for the specific standards it encodes.

The four together

Taken as a set, these subsystems are how a sysprog keeps a mainframe healthy:

Subsystem	Its job
WLM	Decide which work is served first, by goal and importance
RMF	Measure performance, and reveal anomalies
GRS	Serialize access so shared data is not corrupted
Health Checker	Continuously check the system against best practice

The four health-and-performance subsystems and what each contributes.

Each has an operational purpose and a security dividend: WLM's prioritization can be abused or protected, RMF's measurements expose anomalies, GRS's contention reveals lock-based denial of service, and the Health Checker runs an ongoing security review. A defender who reads all four sees the system's health and much of its security posture in the same glance.

The common mistake is to treat these as purely performance tools and leave their output to the operations team. The Health Checker in particular is a security instrument: ignoring its HIGH and MED findings is ignoring a continuous, automated audit that has already found the problem and rated it for you. Read the checks with security eyes, not only operational ones.

Hands-On Labs

These labs assume you can issue console displays and run the Health Checker.

Lab 26.1

Read the workload priorities

Goal: describe how WLM sorts work into service classes with goals and importance.

Background: understanding what gets served first is understanding what can be starved. This lab reads the workload structure.

1. Display the workload manager's classes and goals.

D WLM,SYSTEMS

Check yourself: you can explain that each service class has a goal and an importance, and that WLM prioritizes higher-importance work when resources are scarce.

Why it matters: prioritization decides what suffers under load. Knowing how it works is knowing how it could be manipulated.

Blue-Team angle: a critical security task starved into a low class, or a rogue job elevated into a high one, are both misuse of WLM worth checking against an approved classification.

Lab 26.2

Check for contention

Goal: display GRS contention and interpret a healthy result.

Background: contention analysis distinguishes a real hang from a held lock. This lab reads the contention display.

1. Display GRS contention.

D GRS,C

Check yourself: no contention is the healthy state. You can explain that, if contention existed, the display would name the resource, the holder, and the waiters.

Why it matters: a resource held too long stalls other work. Reading contention is how you tell a lock-up from a crash.

Blue-Team angle: an ENQ deliberately held on a critical resource is a denial-of-service technique. Contention on a resource that should be transient is worth investigating.

Lab 26.3

Run the Health Checker

Goal: list the active health checks and read their names and severities.

Background: the Health Checker is a continuous automated review. Reading its output is a fast, high-value posture check.

1. Display the active checks.

`F HZSPROC,DISPLAY,CHECKS`

Check yourself: you can list the checks and their severities, and you notice how many are security checks, APF exposure, SURROGAT delegation, PassTickets, terminal parameters.

Why it matters: the Health Checker has already done a security review and rated the findings. Reading it is one of the highest-value few minutes a defender spends.

Blue-Team angle: treat the Health Checker as a scheduled security scan. A new EXCEPTION on a check that used to pass is a change to investigate.

Lab 26.4

Act on the highest finding

Goal: identify the highest-severity health check and connect it to a concrete security exposure.

Background: findings-based work means acting on the worst first. This lab practices triage and connection to earlier chapters.

1. Identify the HIGH-severity check and state what exposure it represents.

Check yourself: GIBSUR01, SURROGAT delegation, is the HIGH finding. You can explain that SURROGAT governs who may submit work as another user (a privilege-escalation path) and that it warrants immediate review.

Why it matters: acting on the HIGH finding first is where remediation effort pays off most. The Health Checker has already told you where to start.

Blue-Team angle: SURROGAT rated HIGH is a direct pointer to the privilege-escalation surface Part IX explores, the Health Checker is naming the same weakness an attacker would seek.

Lab 26.5

Read RMF as a security lens

Goal: explain how a performance anomaly can be a security signal.

Background: attacks have performance signatures. This lab reframes measurement as detection.

1. Consider what an unexplained CPU spike in a subsystem, or a job consuming far more than its peers, might indicate.

Check yourself: you can give at least two examples (a guessing attack, a job using the machine for its own computation) that would appear as a performance anomaly before appearing in a security log.

Why it matters: performance data is an early, hard-to-fake signal. A security-aware reading of RMF catches things logs may miss.

Blue-Team angle: baseline normal resource use so anomalies stand out. A sustained, unexplained departure is worth a security look, not just a tuning one.

An RMF snapshot, read two ways

A live RMF view of resource use reads as a performance report and, with security eyes, as a detection surface. A representative snapshot:

```
RMF MONITOR III - PROCESSOR USAGE
JOBNAME    SERVICE-CLASS  CPU%    STATE
CICS       CICSHIGH      18.4    ACTIVE
DB2MSTR    BATCHHI           6.1    ACTIVE
TSOU001    TSOUSERS             2.0    ACTIVE
BATCH999   BATCHLOW           61.7    ACTIVE  <-- anomalous
SYSTEM TOTAL                88.2
```

An RMF snapshot: one low-class batch job consuming most of the machine is worth a second look.

Operationally, BATCH999 consuming most of the processor while sitting in a low service class is a tuning question. On security grounds it is a lead: a single low-priority job using the machine for heavy computation is the signature of misuse, unauthorized number-crunching, a brute-force run, or code doing something it should not. The value of RMF to a defender is that this stands out in the numbers before it ever reaches a security log, and often before the operator notices a slowdown.

A closer look at a HIGH finding

A health check is most useful when you read its detail, not just its severity. The HIGH SURROGAT check expands into a finding with evidence and a recommendation, in the same shape as the zSecure findings from Part IV:

```
CHECK(GIBSUR01)  SURROGAT delegation                SEV(HIGH)
STATUS:  EXCEPTION
EVIDENCE: SURROGAT profiles permit submit-as for 2 users
RISK:    a permitted user may submit work as another identity
ACTION:  review SURROGAT access lists; remove unneeded permits
VALIDATE: re-run GIBSUR01
```

The HIGH SURROGAT finding in detail, evidence, risk, action, and how to validate the fix.

This is the whole value of the Health Checker in one screen: it has found a specific exposure, rated it HIGH, explained the risk, told you what to do, and given you the command to confirm the fix. SURROGAT delegation (the right to submit work under another user's identity) is a privilege-escalation path, so a HIGH finding here goes to the top of the queue. Reading the detail turns a severity into an action.

Lab 26.6

Read a finding in full and plan the fix

Goal: expand a HIGH health-check finding, read its evidence and recommendation, and state the remediation and validation.

Background: a severity tells you what to do first; the detail tells you what to do. This lab reads a finding end to end.

1. Display the detail of the HIGH SURROGAT check and read its evidence and action.

Check yourself: you can state the exposure (permitted submit-as for other identities), the remediation (review and trim the SURROGAT access lists), and the validation (re-run the check). That is a complete remediation plan from one finding.

Why it matters: findings-based security is only useful if you act on the detail. The recommend-then-validate loop is how a finding becomes a fix that stays fixed.

Blue-Team angle: SURROGAT access lists deserve the same scrutiny as SPECIAL, they grant the power to act as someone else. This check and its detail point straight at the escalation Part IX attempts.

Lab 26.7

Spot the anomalous job in RMF

Goal: read a resource snapshot and identify the job whose usage does not fit its priority class.

Background: the fastest security read of RMF is looking for a low-priority job consuming high resources. This lab drills that pattern.

1. Display processor usage by job and service class.

RMF MONITOR III

Check yourself: you can identify a job whose CPU use is far out of line with its service class (a low-class job dominating the machine) and explain why that mismatch is worth a security look, not just a tuning one.

Why it matters: the class-versus-usage mismatch is a simple, solid anomaly. Attacks that consume resources reveal themselves this way before any log entry.

Blue-Team angle: alert on sustained high CPU from low-importance work. It is a low-false-positive signal for unauthorized computation running on the system.

Checkpoint: can you explain WLM's goal-based prioritization, RMF's role as a measurement and anomaly lens, GRS serialization and contention, and (above all) run the Health Checker and act on its findings by severity? Those skills keep a system healthy and surface much of its security posture automatically.

Four subsystems keep a mainframe running well and safely. WLM prioritizes work by goal and importance, deciding what is served first and what can be starved. RMF measures performance and, read with security eyes, exposes the anomalies attacks produce. GRS serializes access so shared data is not corrupted, and its contention display reveals both logjams and lock-based denial of service. The Health Checker runs a continuous catalog of checks (most of them security checks like APF exposure and SURROGAT delegation) each rated by severity, making it a standing automated security review. A defender who reads all four sees the system's health and much of its security posture in a single pass.

What comes next

These subsystems watch the running system. Building and maintaining it (installing software, managing source, keeping service current) is the work of a different toolset. The next chapter closes Part V with the sysprog's tooling: SMP/E, which installs and maintains system software; CA SYSVIEW, the real-time monitor; and CA Endevor, which manages source and change. These are the tools that keep a mainframe current and controlled, and whose own privileges make them worth protecting.

CHAPTER 27

Sysprog Tooling

By the end of this chapter you will be able to:

- Explain what SMP/E does and the SYSMOD maintenance lifecycle.
- Read SYSMOD status and distinguish PTFs, APARs, and USERMODs.
- Use SYSVIEW to monitor the system in real time.
- Explain what Endeavor manages and why change control is a security control.
- Recognize that these tools carry system-level privilege.
- See why protecting the toolchain protects the system.

THE SUBSYSTEMS IN THE LAST chapter watch a running system. Building and maintaining it is a different craft, with its own tools. A systems programmer (a sysprog) installs software with SMP/E, watches the live system through a real-time monitor like CA SYSVIEW, and controls source and change with a tool like CA Endeavor. These are the instruments that keep a mainframe current, correct, and under control. They also carry enormous authority (the power to install code, to see everything, and to move programs into production) which makes them both essential to operations and, precisely because of that power, worth protecting. This chapter closes Part V with the toolchain a sysprog lives in.

SMP/E: installing and maintaining software

SMP/E, the System Modification Program/Extended, is how z/OS software is installed and kept current. Every product, every fix, every local change goes through it, tracked in a consolidated inventory. Its main menu shows the shape of the work:

```
SMPE
SMP/E MAIN MENU - GIBSON TRAINING SIMULATION
1  CSI          - Consolidated software inventory
2  ZONES        - GLOBAL, TARGET, DLIB zones
3  SYSMODS      - FMIDs, PTFs, APARs, USERMODs
4  RECEIVE      - Simulate receiving maintenance
5  APPLY        - Simulate applying maintenance to target zone
```

The SMP/E main menu, the inventory, the zones, and the maintenance actions.

The key ideas are zones and SYSMODs. SMP/E keeps software in zones: a global zone that tracks everything received, a target zone that is the running system, and a distribution zone that holds the master copies. A SYSMOD (a system modification) is any unit of software or fix, and it moves through the zones in a controlled lifecycle. SMP/E exists so that the state of the system's software is always known and always reversible, which is as much a security property as an operational one: you cannot secure code you cannot account for.

SYSMODs and the maintenance lifecycle

Listing the SYSMODs shows both what is installed and where each stands in its lifecycle:

```
SMPE SYSMODS
SMP/E SYSMOD STATUS

SYSMOD      STATUS
HZSEC10     APPLIED
UGIB261     RECEIVED
UJES200     ACCEPTED
UZSEC80     HELD
```

SYSMOD status, each fix in its stage: received, applied, accepted, or held.

A SYSMOD moves from RECEIVED (in the global zone, not yet installed) to APPLIED (installed in the target zone, testable, still reversible) to ACCEPTED (committed to the distribution zone). HELD means a SYSMOD is blocked pending an action, usually because it is known to cause a problem. The types matter too, and they carry different security weight:

SYSMOD type	What it is
FMID	A base product or function, the foundation
PTF	A vendor Program Temporary Fix
APAR	A fix for a specific reported defect
USERMOD	A local, custom modification, site-written

SYSMOD types, a USERMOD is local code, and deserves extra scrutiny.

The USERMOD is the one to watch. A PTF or APAR comes from the vendor with a known provenance; a USERMOD is code someone at the site wrote and installed into the system through SMP/E. A legitimate USERMOD is normal; an unexpected one is a way to introduce modified or malicious behavior with the system's own blessing. Applying a SYSMOD confirms with a message:

```
SMPE APPLY
GIMSMP003I SYSMOD UGIB261 APPLIED TO TARGET ZONE
```

Applying maintenance, the SYSMOD moves into the running target zone.

SYSVIEW: the real-time monitor

Where RMF measures and reports, CA SYSVIEW gives a live, interactive window into the running system, the console a sysprog watches during operations. Its menu spans the whole system:

```
SYSVIEW
SYSVIEW PERFORMANCE AND OPERATIONS MONITOR

1 System Overview
2 Active Jobs / Address Spaces
3 CICS Region Monitor
```

4 DB2 Subsystem Monitor

5 TCP/IP Summary

The SYSVIEW menu, a live monitor spanning the system and its subsystems.

The System Overview is the at-a-glance health screen, and the Active Address Spaces view lists every job with its resource use, the same anomaly-hunting surface RMF offers, but live:

```
SYSVIEW ACTIVE ADDRESS SPACES
JOBNAME  ASID  CPU% STORAGE STATUS
GIBSON    0031  07.4 128M   ACTIVE
GIBCIICS  0042  01.1 064M   ACTIVE
DB2A      0052  00.8 096M   ACTIVE
RSSCTI    0061  00.0 008M   IDLE
```

SYSVIEW active address spaces, every job, live, with its resource use.

For an operator this is the pulse of the system in real time. For a defender it is a live watch for the unexpected: an address space that should not exist, a job consuming far more than its peers, a subsystem behaving oddly, all visible the moment they appear. A real-time monitor is a real-time detection surface, and knowing how to read one is a security skill as much as an operational one.

Endevor: managing source and change

The last tool governs how code changes reach the system. CA Endevor is a source-control and change-management system: it holds the source for programs, tracks every version, and moves code through defined stages (typically development, test, and production) under controlled actions. Its menu is the entry to that discipline:

```
ENDEVOR
C1 ----- ENDEVOR PRIMARY OPTIONS MENU -----
OPTION ==>
0  DEFAULTS    - Specify Endevor session defaults
1  DISPLAY     - Display environment information
2  FOREGROUND  - Perform element actions in foreground
3  BATCH       - Build element action requests
```

The Endevor menu, the gateway to controlled source and change management.

Endevor's stages enforce a path: code is developed, then promoted to test, then to production, and each promotion is a controlled, recorded action. This is where change control becomes a security control. Who may move code to production, whether the same person can both write and promote it, and whether every change is recorded and reviewable are all security questions, because production code runs with production authority. Segregation of duties (the developer cannot be the sole approver of their own promotion) is enforced here, and its absence is a classic finding: an attacker who can push

code straight to production has bypassed every review that stands between a change and the running system.

The toolchain is privileged

Step back and the common thread is authority. SMP/E can install code into the system. Endeavor can move code into production. SYSVIEW can see everything running. Each tool exists to wield system-level power on a sysprog's behalf, which means each tool is also a target: compromise the toolchain and you compromise the system, cleanly and with its own blessing. Protecting these tools (who can run them, who can approve their most powerful actions, and whether their activity is audited) is therefore protecting the system itself. The defensive posture mirrors everything from Part IV: least privilege on who may use the tools, separation of duties on their powerful actions, and auditing of what they do.

Tool	Purpose and its risk
SMP/E	Installs software, controls what code exists
CA SYSVIEW	Monitors live, sees all activity
CA Endeavor	Moves code to production, gates every change

The sysprog toolchain, each tool's power is also its risk.

The common mistake is to treat the sysprog toolchain as purely operational and exempt it from security review. These tools carry the highest authority on the system, to install code, to promote it to production, to observe everything. A weak control here undoes strong controls everywhere else: an unrestricted USERMOD path or an Endeavor promotion with no separation of duties is a direct route into the running system.

Hands-On Labs

These labs assume you can reach the SMP/E, SYSVIEW, and Endeavor interfaces.

Lab 27.1

Explore SMP/E

Goal: navigate the SMP/E menu and explain zones and SYSMODs.

Background: SMP/E is how software gets onto the system. Understanding its structure is understanding software provenance.

1. Open SMP/E and read its main menu.

SMPE

Check yourself: you can explain the global, target, and distribution zones, and that a SYSMOD is a unit of software or fix tracked in the inventory.

Why it matters: knowing what software is installed, and how, is the foundation of both maintenance and security.

Blue-Team angle: the software inventory is a provenance record. Code on the system that SMP/E cannot account for is a red flag.

Lab 27.2

Read the SYSMOD status

Goal: list the SYSMODs and identify their types and lifecycle states.

Background: the SYSMOD list shows what is installed and where each stands. USERMODs and HELD items deserve special attention.

1. List the SYSMOD status.

SMPE SYSMODS

Check yourself: you can read each SYSMOD's state (RECEIVED, APPLIED, ACCEPTED, HELD) and you know that a USERMOD is local code warranting extra scrutiny.

Why it matters: the state tells you what is live and reversible; the type tells you the provenance. Both matter for trust.

Blue-Team angle: an unexpected USERMOD is a way to inject modified behavior with the system's blessing. Review every USERMOD's origin and purpose.

Lab 27.3

Monitor with SYSVIEW

Goal: use SYSVIEW to read the live system and its active address spaces.

Background: a real-time monitor is a real-time detection surface. This lab practices reading it.

1. Open SYSVIEW and view the system overview and active address spaces.

SYSVIEW

Check yourself: you can read the overview's health figures and list the active address spaces with their resource use, and you could spot one that does not belong or consumes too much.

Why it matters: watching the live system is how anomalies are caught as they happen, not after the fact in a log.

Blue-Team angle: an unexpected active address space, seen live in SYSVIEW, is an early sign of unauthorized activity, the same anomaly RMF shows, but sooner.

Lab 27.4

Explore Endeavor's change path

Goal: explain how Endeavor stages move code to production and why that is a security control.

Background: change control is security control. This lab connects the promotion path to separation of duties.

1. Open Endeavor and read its options and stage concept.

ENDEAVOR

Check yourself: you can describe the development-to-test-to-production path and explain why the same person should not both write and approve a promotion to production.

Why it matters: production code runs with production authority. Who can put it there is a first-order security question.

Blue-Team angle: a promotion path with no separation of duties lets one person push code straight to production, bypassing every review, a classic and serious finding.

Lab 27.5

Reason about tool privilege

Goal: explain why the sysprog toolchain must be protected as tightly as any other high-authority resource.

Background: the tools carry system-level power. Treating them as ordinary utilities is a mistake. This lab makes the risk explicit.

1. For SMP/E, SYSVIEW, and Endeavor, state the authority each holds and the control that should guard it.

Check yourself: you can say that SMP/E installs code, Endeavor promotes it, and SYSVIEW observes all of it, and that each needs least-privilege access, separation of duties on powerful actions, and auditing.

Why it matters: a weak control on a powerful tool undoes strong controls elsewhere. The toolchain deserves the same rigor as SPECIAL and APF.

Blue-Team angle: include the sysprog toolchain in every access review. The power to install and promote code is among the most dangerous authorities on the system.

Zones: the software inventory

SMP/E keeps its software in zones, and listing them shows the structure that makes maintenance safe and reversible:

SMP/E ZONES

ZONE	TYPE	STATUS
GLOBAL	GLOBAL	ACTIVE
MVST100	TARGET	ACTIVE
DLIB100	DLIB	ACTIVE

The SMP/E zones, global tracks everything, target is the running system, DLIB holds masters.

The three zones each play a role: the GLOBAL zone records everything received, the TARGET zone (here MVST100) is the software the system actually runs, and the DLIB (distribution) zone holds the master copies from which the target can be rebuilt. This separation is why SMP/E maintenance is reversible, a change applied to the target can be backed out because the distribution masters remain intact. For a defender it is also an integrity control: the state of every zone is known, so unaccounted-for code has nowhere legitimate to hide.

Watching a subsystem live

SYSVIEW drills into individual subsystems as well as the whole system. The CICS region monitor, for instance, shows each region's live activity:

SYSVIEW CICS REGION MONITOR

REGION	STATUS	SESSIONS	TRANSACTIONS	AVG RESP
GIBICIS	ACTIVE	0000	000001	0.03

The SYSVIEW CICS region monitor, live sessions, transaction counts, and response time.

Region status, active sessions, transaction volume, and response time, all live, are what an operator watches to know a subsystem is healthy. The same numbers are a detection surface: a sudden spike in transactions, a jump in sessions, or response times that climb without an obvious cause can be the first visible sign of an attack against the subsystem (an injection run, a flood, or credential abuse) appearing here before it surfaces anywhere else.

Lab 27.6

Read the zones and a subsystem monitor

Goal: list the SMP/E zones and read a SYSVIEW subsystem monitor, explaining what each tells you.

Background: zones explain software provenance; subsystem monitors show live health. Both are daily sysprog reads with security value.

1. List the SMP/E zones.

SMPE ZONES

2. Open a SYSVIEW subsystem monitor.

SYSVIEW 3

Check yourself: you can name the global, target, and distribution zones and their roles, and read a subsystem monitor's live figures (sessions, transactions, response time) well enough to notice an anomaly.

Why it matters: zones tell you what code is trusted and reversible; live monitors tell you how the running subsystems behave. Together they cover provenance and behavior.

Blue-Team angle: a transaction or session spike in a subsystem monitor is an early attack signal, and an unaccounted-for change to a zone is an integrity concern, both are worth watching.

Checkpoint: can you explain SMP/E and the SYSMOD lifecycle, read SYSMOD status and spot a USERMOD, monitor live with SYSVIEW, describe Endeavor's change path and why it is a security control, and explain why the whole toolchain is privileged? Those are the sysprog skills (and the review points) that keep a mainframe current and controlled.

The sysprog toolchain builds and maintains the system. SMP/E installs and tracks all software through zones and a SYSMOD lifecycle (received, applied, accepted) where USERMODs, being local code, warrant special scrutiny. CA SYSVIEW is a live monitor and therefore a live detection surface, showing active address spaces and their resource use as they happen. CA Endevor governs source and change, moving code through development, test, and production, which makes it the enforcement point for change control and separation of duties. Every one of these tools carries system-level authority (to install code, to see everything, to promote to production) so protecting the toolchain, with least privilege, separation of duties, and auditing, is protecting the system itself. This closes Part V.

What comes next

Parts I through V have built the platform and the disciplines that run and secure it. Part VI turns to what the platform is actually for: the online subsystems and applications that make mainframes the backbone of banking and commerce. It opens with CICS, the transaction monitor that serves millions of online interactions a day, its regions, its transactions, and the sign-on and administration screens that are, in turn, their own security surface.

Part VI

The Subsystems

Most real mainframe work runs inside a subsystem. Part VI tours the major ones, the CICS transaction monitor and the banking stack it drives, Db2 and IMS for data, and z/VM and z/TPF, so you can see where business logic and data actually live.

CHAPTER 28

CICS: The Transaction Monitor

By the end of this chapter you will be able to:

Explain what CICS is, regions and transactions.

Use CEMT, the master terminal, to inquire and control.

Recognize the transactions that matter, and which are dangerous.

Understand CECI and CEDA and why they are powerful.

Name the CICS RACF classes that protect transactions and commands.

See the CICS attack surface a defender must lock down.

PART VI TURNS FROM RUNNING the mainframe to what it runs. For online work (the millions of banking, retail, and booking interactions a large enterprise handles every day) the engine is CICS, the Customer Information Control System. CICS is a transaction monitor: it takes a short request from a terminal or an application, runs a program to handle it, and returns a response, thousands of times a second, reliably, for decades. It is the software behind a great deal of the world's commerce. For a defender it is also a rich attack surface, because CICS ships with a set of powerful built-in transactions (to sign on, to control the region, to run commands, to define new resources) and each of those is something that must be locked down. This chapter opens CICS: what it is, how you drive it, and where its security lives.

Regions and transactions

Two words unlock CICS. A region is a running instance of CICS, an address space that hosts applications and their resources. A transaction is a short unit of work, named by a four-character identifier, that runs a program to do one thing. You inquire into a region with CEMT, and it describes itself:

```
CEMT INQUIRE SYSTEM
```

```
CICS MASTER TERMINAL
```

```
INQUIRE SYSTEM
```

```
-----
Sysid(CICS) Applid(CICS) Mvsimg(GIB1) Jobname(GICS)
Cicsts(05.06.00) Aos(0001) MaxTask(00050) CurTask(00001)
Start(2026/07/01 10:13:40) Uptime(0:00:00)
```

A CICS region describing itself, its identifiers, release, and task limits.

The region has a Sysid and an Applid that name it on the system and the network, a CICS TS release level, and task limits, MaxTask caps how many transactions may run at once, with CurTask showing how many are running now. This is the first thing to read about any

CICS: what it is, what version, and how busy. The Applid in particular matters for security, because it is how other systems and attackers address this region across the network.

The master terminal: CEMT

CEMT, the master terminal transaction, is the operator's window into a region, and its control panel. It can inquire into every kind of resource and, essentially, change them. Its menu shows the verbs:

```
STATUS:  ENTER ONE OF THE FOLLOWING
```

```
Discard   Inquire   Perform   Set
```

The CEMT verbs, Inquire to look, and Set, Perform, and Discard to change.

INQUIRE reads; SET, PERFORM, and DISCARD change. Inquiring into the running tasks and the loaded programs shows what a region is doing:

```
CEMT INQUIRE TASK
```

```
INQUIRE TASK
```

```
-----
```

```
Tas(000001) Tra(CICS) Fac(LU320) RUNNING Use(SYSTEM) Prog(DFHSIP)
```

CEMT INQUIRE TASK, the transactions running right now, and under which identity.

Each task shows its number, the transaction, the terminal, its state, the user it runs under, and the program. That last detail (the identity a task runs under) is a security fact: a task running as SYSTEM has different reach than one running as an ordinary user. The power of CEMT is that the same transaction that reads all this can also change it: SET can disable a transaction, PERFORM can shut the region, DISCARD can remove a resource. CEMT is therefore one of the most privileged transactions in CICS, and who may run it is a first-order security question.

The transactions that matter

CICS ships with a family of built-in transactions, each a four-character CE-something. A handful carry the security weight, and knowing them is knowing the CICS attack surface:

Transaction	What it does
CESN / CESF	Sign on to / sign off from CICS
CEMT	Master terminal, inquire and control the region
CECI	Command interpreter, run CICS commands interactively
CEDA	Resource definition, define programs and transactions
CEBR	Browse temporary storage
CEDF	Execution diagnostic facility, step through a program

The built-in CICS transactions, several are powerful enough to require tight control.

CESN signs a user on, establishing their identity for the session, the CICS front door. CEMT controls the region. CECI runs commands. CEDA defines resources. The last three are powerful enough that, in a secured region, ordinary users have no business running them

at all. Leaving CEMT, CECI, or CEDA open to general users is one of the most common and serious CICS misconfigurations.

CECI: the command interpreter

CECI, the command-level interpreter, lets a user type and execute CICS commands interactively. Its menu is a catalog of the CICS API:

```
CECI
STATUS:  ENTER ONE OF THE FOLLOWING
ABend    DEFine    FREE      READ      RETUrn    WRITE
ADD       DELETE   GETMain   READQ    REWRite   WRITEQ
ADDRess   DEQ      LINK      RECEIVE  R0ute     XCTL
```

CECI, an interactive gateway to the CICS command set, including READ and WRITE.

Read the verbs and the danger is obvious: READ and WRITE reach files, READQ and WRITEQ reach queues, LINK and XCTL transfer to other programs. CECI is a legitimate and valuable tool for a developer testing commands, and, in the wrong hands, an interactive way to read and modify any resource the region can reach, without writing or installing a program. An attacker who reaches CECI in a poorly secured region has an interactive console into the application's data. This is precisely why CECI belongs behind transaction security, available only to those who genuinely need it.

CEDA: defining resources

CEDA defines the resources a region uses, its programs, its transactions, its files. Its menu is the vocabulary of resource definition:

```
CEDA
ENTER ONE OF THE FOLLOWING
Add      ALter     APpend    Check     C0py     DEFine
```

CEDA, define, alter, and install the resources a region runs.

DEFINE creates a resource; ALTER changes one; and installing a definition makes it live. The security weight is that CEDA can define a new transaction pointing at a new program, which is to say, it can install a backdoor. A user with CEDA can add a transaction that runs code of their choosing under the region's authority, and make it persist. Like CECI and CEMT, CEDA is a transaction that ordinary users must never be able to run; it is a definition-time route to persistent, privileged execution inside CICS.

Securing CICS

CICS security rests on RACF, through a set of CICS-specific resource classes that you first saw active in the SETROPTS output back in Part IV. They decide who may run which transaction and issue which command:

Class	What it protects
TCICSTRN / GCICSTRN	Transactions, who may run each one
CCICSCMD	CICS system-programming (SP) commands
FCICSFCT	Files defined to the region
PCICSPSB	Program specification blocks (IMS access)

The CICS RACF classes, transaction and command security for the region.

The core control is transaction security through TCICSTRN: with it active, running a transaction requires RACF authorization, so CEMT, CECI, and CEDA can be restricted to the few who need them while everyone else is denied. CCICSCMD guards the powerful system-programming commands, and FCICSFCT guards the files. A properly secured region grants each user only the transactions their job requires (the same least-privilege principle as everywhere else) and an assessment of a CICS region is, in large part, checking exactly who can run the dangerous built-in transactions.

The classic CICS mistake is leaving the powerful built-in transactions open. CEMT, CECI, and CEDA are development and operations tools, not everyday user transactions; if TCICSTRN is not active, or the access lists are too broad, an ordinary user (or an attacker who signed on as one) can control the region, read and write its data interactively, or install a backdoor. Always confirm transaction security is active and that the dangerous transactions are tightly held.

Hands-On Labs

These labs assume you can reach a CICS region and issue its transactions.

Lab 28.1

Inquire into the region

Goal: use CEMT to read a region's identity, release, and task limits.

Background: the first thing to learn about any CICS region is what it is. CEMT INQUIRE SYSTEM answers it.

1. Inquire into the CICS system.

`CEMT INQUIRE SYSTEM`

Check yourself: you can name the Sysid, Applid, CICS TS release, and the MaxTask limit. The Applid is how this region is addressed across the network.

Why it matters: every later question about a region starts from its identity and limits. This is the orientation step.

Blue-Team angle: the Applid is a network-facing identifier, it is what recon tools enumerate. Knowing your regions' Applids is knowing what an attacker sees.

Lab 28.2

Read the running tasks

Goal: use CEMT to list running tasks and note the identity each runs under.

Background: what is running now, and as whom, is the operational and security pulse of a region.

1. Inquire into the tasks, then the loaded programs.

CEMT INQUIRE TASK

CEMT INQUIRE PROGRAM

Check yourself: you can list the running tasks with their transaction, state, and the user they run under, and see the loaded programs and whether each is enabled.

Why it matters: a task's identity decides its reach. Reading who tasks run as is reading the region's live privilege picture.

Blue-Team angle: an unexpected task, or a task running under a more powerful identity than it should, is exactly the anomaly CEMT surfaces.

Lab 28.3

Explore CECI

Goal: open the command interpreter and recognize why it is powerful.

Background: CECI is a developer's friend and an attacker's dream. Seeing its command set makes the risk concrete.

1. Open CECI and read its command menu.

CECI

Check yourself: you can see that CECI offers READ, WRITE, LINK, and the rest of the CICS API interactively, an interactive way to reach files, queues, and programs without installing any code.

Why it matters: CECI turns the CICS API into an interactive console. In an unsecured region that is direct access to the application's data.

Blue-Team angle: confirm CECI is restricted by transaction security. An ordinary user who can run CECI can read and write the region's resources at will.

Lab 28.4

Explore CEDA

Goal: open resource definition and explain why defining a transaction is a persistence risk.

Background: CEDA defines what a region runs. That power is exactly what a backdoor needs.

1. Open CEDA and read its action menu.

CEDA

Check yourself: you can see that CEDA can DEFINE and install transactions and programs, and you can explain that a defined transaction pointing at attacker code is a persistent backdoor under the region's authority.

Why it matters: definition is persistence. A tool that can add a transaction can add a way back in that survives restarts.

Blue-Team angle: CEDA access is definition-time control of the region. Restrict it tightly and review resource definitions for transactions or programs no one authorized.

Lab 28.5

Audit transaction security

Goal: confirm transaction security is active and that the dangerous transactions are restricted.

Background: a CICS assessment is largely a check of who can run the powerful built-in transactions. This lab runs that check.

1. Confirm the CICS classes are active and review access to the dangerous transactions.

SETR0PTS LIST

Check yourself: the CICS classes (TCICSTRN, CCICSCMD, FCICSFCT) are active, and access to CEMT, CECI, and CEDA is limited to the few who need them. Broad access to any of these is a finding.

Why it matters: transaction security is the control that keeps ordinary users out of the powerful transactions. Confirming it is the heart of a CICS review.

Blue-Team angle: inactive TCICSTRN, or broad access to CEMT/CECI/CEDA, is a high-severity finding, it means anyone who can sign on can control or subvert the region.

The programs behind the transactions

Every transaction runs a program, and CEMT lists the programs a region has loaded, the code actually resident and available:

CEMT INQUIRE PROGRAM

```

-----
Prog(DFHWBADX) Leng(000184) Resc(000000) Use(000000) Enabled
Prog(DFH0XCMN) Leng(004096) Resc(000000) Use(000000) Enabled

```

CEMT INQUIRE PROGRAM, the loaded programs, their size, use count, and status.

Each entry shows the program name, its length, how many times it is in use, and whether it is enabled. The DFH-prefixed names are IBM's own CICS programs; application programs appear here too once loaded. For a defender this list is an inventory of what code the region can run, and an enabled program that no one recognizes (especially one not from a known application) is worth investigating, because a transaction defined by CEDA points at exactly such a program. Reading the program list against an approved inventory is how an installed backdoor is found.

Stepping through a transaction: CEDF

CEDF, the execution diagnostic facility, lets a developer step through a transaction as it runs, pausing at each CICS command to inspect and even change the data in flight:

```

TRANSACTION: BANK   PROGRAM: PAYPROG   TASK: 0000123
STATUS: COMMAND EXECUTION COMPLETE
EXEC CICS READ FILE('ACCTS')

```

RESPONSE: NORMAL EIBRESP=0000

ENTER: CONTINUE PF2: SWITCH HEX PF4: SUPPRESS DISPLAYS

CEDF, stepping through a live transaction, inspecting each command and its result.

CEDF is invaluable for debugging: it shows every EXEC CICS command, its arguments, and its response, and lets the developer alter values before the command runs. That same capability is a security concern, stepping through a live transaction exposes the data it handles, and altering values in flight can change what a transaction does. Like the other diagnostic transactions, CEDF is a tool that must be restricted to those who legitimately develop and support the applications, never left open to ordinary users of a production region.

Lab 28.6

Inventory the region's programs

Goal: list the loaded programs and reason about how an unrecognized one relates to a defined transaction.

Background: the program list is the region's code inventory. An unaccounted-for program is the far end of a CEDA-defined backdoor.

1. Inquire into the loaded programs.

CEMT INQUIRE PROGRAM

Check yourself: you can read each program's name, size, use count, and status, and you can explain that an enabled program matching no known application, reachable by a defined transaction, is a backdoor candidate.

Why it matters: a transaction is only a name; the program behind it is the code that runs. Inventorying programs is how you verify what a region can actually do.

Blue-Team angle: compare the loaded-program list against an approved inventory. A program no one authorized, especially paired with a recently defined transaction, is a strong indicator of an installed backdoor.

Checkpoint: can you explain regions and transactions, use CEMT to inquire and understand its control power, name the dangerous built-in transactions and why CECI and CEDA are risky, and check that transaction security restricts them? Those are the CICS operating and security skills the banking chapter builds on next.

CICS is the transaction monitor behind much of the world's online commerce: a region hosts applications, and transactions (four-character units of work) run programs to serve requests. CEMT, the master terminal, both inquires into and controls a region; CECI runs the CICS API interactively; CEDA defines and installs resources. The last three are powerful enough to read and write any resource or install a backdoor, which is why CICS security rests on RACF transaction classes (TCICSTRN, CCICSCMD, FCICSFCT) that restrict who may run each transaction and command. Securing CICS is, in large part, ensuring transaction security is active and the dangerous built-in transactions are held to the few who need them.

What comes next

CICS is the engine; the next chapter shows what it drives. Gibson's banking stack (CBSA and FIBS, built on CICS, Db2, and a web tier) is a working model of how a real bank's core runs on the mainframe. It is also where several application-level weaknesses live, including the classic COBOL storage-overflow path that ends in an ASRA abend. The next chapter walks that stack end to end: how the pieces fit, and where the application layer can be attacked and defended.

CHAPTER 29

The Banking Stack

By the end of this chapter you will be able to:

Describe the layers of a mainframe banking application.

Read the customer and account data model.

Name the COBOL programs that make up the banking logic.

Trace a transaction (a transfer) from screen to data and back.

Explain the classic COBOL storage-overflow bug and its ASRA abend.

Apply the secure fix: validate length before moving data.

CICS IS THE ENGINE; A banking application is what it drives. Gibson includes a working model of a bank's core (CBSA, the CICS Bank Sample Application, alongside FIBS and DVCA) built the way real cores are built: BMS screens on 3270 terminals, CICS transactions, COBOL programs holding the business logic, Db2 tables holding the data, and a web tier in front. Studying it teaches two things at once. First, how the layers of a real mainframe application fit together, which is how the majority of the world's financial transactions are actually processed. Second, where such applications break, including the single most classic mainframe application bug, a COBOL storage overflow that corrupts adjacent data and ends in an ASRA abend. This chapter walks the stack end to end, then walks that bug and its fix.

The anatomy of a banking stack

A banking application on the mainframe is a stack of layers, each with a clear job, and a request passes down through them and back up:

Layer	What it is
Presentation	BMS 3270 maps, and, in front, a web tier
Transaction	CICS transactions that route each request
Business logic	COBOL programs that do the banking work
Data	Db2 tables holding customers and accounts

The layers of a mainframe banking application, presentation, transaction, logic, data.

A teller keys an account number into a 3270 screen; the screen is handled by a CICS transaction; the transaction calls a COBOL program; the program reads or updates Db2; and the result flows back up to the screen. The same core is increasingly reached through a web and API tier as well, so the very same COBOL logic serves both a green-screen teller and a mobile app. This layering is the model to hold in mind, because a weakness can live in any layer (an unvalidated web input, an overly trusting transaction, an unsafe COBOL move, an over-permissive Db2 grant) and an assessment walks all of them.

The data model: customers and accounts

At the bottom of the stack is the data. CBSA models a bank the way a real one does, customers, and the accounts belonging to them, keyed by a sort code and an account number:

ACCOUNT	CUSTOMER	TYPE	BALANCE
00-00-01 / 00000101	1001	CURRENT	1,500.00
00-00-01 / 00000102	1001	SAVINGS	8,500.00
00-00-01 / 00000103	1002	CURRENT	315.55
00-00-01 / 00000104	1003	CURRENT	2,090.10

The CBSA data model, accounts keyed by sort code and number, belonging to customers.

Each account has a sort code (the branch), an account number, an owning customer, a type, and a balance. One customer can hold several accounts. This is the data the COBOL programs read and change, and it is also the data an attacker wants, to read balances they should not see, or to alter them. The access controls that protect it live in Db2 and RACF, which the next chapter takes up; here the point is simply that the balances are the crown jewels of the application, and everything above exists to serve and protect them.

The COBOL programs

The business logic is a set of COBOL programs, each doing one banking operation. Their names say what they do:

Program	What it does
BNKMENU	The main banking menu
INQCUST / INQACC	Inquire a customer / an account
CRECUST / CREACC	Create a customer / an account
UPDACC	Update an account
DBCRFUN	Debit or credit an account
XFRFUN	Transfer funds between accounts

The CBSA COBOL programs, each a unit of banking logic behind a transaction.

These are ordinary COBOL programs of the kind that run the world's banks, decades old in concept, utterly central in practice. DBCRFUN moves money into or out of one account; XFRFUN coordinates a debit and a credit to move it between two. Each is invoked by a CICS transaction and works against Db2. Understanding that the logic is plain COBOL matters for security, because COBOL's handling of fixed-length fields is the source of the classic bug this chapter builds toward.

A transfer, end to end

Follow a transfer through the stack. The teller selects transfer on the menu (BNKMENU) and keys a from-account, a to-account, and an amount into a BMS screen. The transfer transaction invokes XFRFUN. XFRFUN validates the request, then calls the debit-credit logic to subtract the amount from the source account and add it to the destination (two

Db2 updates that must both succeed or both fail, so they are done as a unit of work that CICS commits together. The result) success, or a reason for failure such as insufficient funds, flows back to the screen. This is a complete mainframe transaction: screen to transaction to COBOL to Db2 and back, atomic and logged. It is also where input handling matters, because the amount and the account numbers are user input flowing into fixed-length COBOL fields.

Where the application breaks: the COBOL overflow

COBOL works with fixed-length fields laid out contiguously in storage. When a program moves input into a field without checking its length, and the input is longer than the field, the excess spills into whatever storage sits next to it, corrupting an adjacent field. This is the COBOL storage overflow, and Gibson demonstrates it defensively. An oversized value keyed into a field that expects far less overruns it, corrupts the field beside it, and ends in a CICS ASRA abend:

```
CBSA BUFFER OVERFLOW SIMULATION
INPUT LENGTH: 96  TARGET FIELD: 32
SIMULATED EFFECT: ADJACENT APPROVED-FLAG CORRUPTED
CICS-LIKE ABEND: ASRA SIMULATED - PROGRAM BNKUPD
DFHAC2206 TRANSACTION ABEND ASRA PROGRAM BNKUPD
SECURE FIX: validate length before MOVE / RECEIVE MAP
```

A COBOL storage overflow, 96 bytes into a 32-byte field corrupts an adjacent flag, ASRA.

Read what happened. Ninety-six bytes of input were moved into a field with room for thirty-two. The extra sixty-four bytes did not vanish (they overwrote the storage next to the field, which here held an approved-flag. Two things followed, and both matter. The visible one is the ASRA abend: ASRA is the CICS abend for a program check) a storage or data exception (and it is the application-layer cousin of the S0C7 and S0C4 abends from Part III. The subtler and more dangerous one is the corruption: an adjacent approval flag was overwritten. In a banking program, corrupting the wrong byte can flip a decision) an unapproved transaction marked approved. A storage overflow is not only a crash; it is a data-integrity failure, and that is why it is the most consequential class of mainframe application bug.

The secure fix

The fix is stated on the last line of the simulation, and it is the one habit that prevents the whole class of bug: validate length before you move. Concretely, a COBOL program must check that input does not exceed the target field before a MOVE, and must handle screen input with the length that RECEIVE MAP reports rather than assuming it fits. Where the input might be longer, the move must be bounded, reference-modified to the field's size, or rejected outright with an error to the user. The principle is exactly the input-validation discipline of any language: never trust the length of what came in, and never write past

the end of what you have. A single length check before the move turns a crash-and-corrupt into a clean, handled rejection.

Practice	Why it prevents the overflow
Check input length before MOVE	Stops the excess from ever being written
Use the length from RECEIVE MAP	Handles the real input size, not an assumed one
Bound the move to the field size	Excess is truncated, not spilled into neighbors
Reject over-length input	Turns corruption into a clean error

Secure-coding practices that prevent the COBOL storage overflow.

The mistake at the root of the overflow is trusting input length. A COBOL program that moves a screen field or an API value into a fixed field without checking the length assumes the input fits, and when it does not, the excess corrupts whatever lies next to the field. Never move unvalidated input into a fixed-length field; the length check is one line, and it prevents both the crash and the silent data corruption.

Hands-On Labs

These labs explore the banking stack and its classic bug. The overflow is studied to understand and fix it, the Blue-Team goal throughout.

Lab 29.1

Map the stack

Goal: identify the layers of the banking application and what each contributes.

Background: understanding the layers is the prerequisite for finding a weakness in any of them.

1. For a banking request, name the layer that handles each stage from screen to data.

Check yourself: you can trace a request through presentation (BMS/web), transaction (CICS), business logic (COBOL), and data (Db2), and name what each layer does.

Why it matters: a weakness can live in any layer. Knowing the layers is knowing where to look.

Blue-Team angle: each layer boundary is a place input crosses and trust is assumed. Those boundaries are where validation belongs and where attacks aim.

Lab 29.2

Read the data model

Goal: read the customer and account model and explain the keys.

Background: the data is what everything else exists to serve and protect. Reading the model orients the whole application.

1. List the accounts and note their sort code, number, owner, and balance.

Check yourself: you can explain that an account is keyed by sort code and number, belongs to a customer, and holds a balance, and that one customer may hold several accounts.

Why it matters: the balances are the application's crown jewels. Knowing the model is knowing what must be protected.

Blue-Team angle: read access to balances a user does not own, and write access to balances at all, are the two exposures that matter most, controlled in Db2 and RACF.

Lab 29.3

Trace a transfer

Goal: follow a transfer through the programs and explain why the debit and credit are one unit of work.

Background: a transfer is the canonical banking transaction. Tracing it shows how the layers cooperate and where atomicity matters.

1. Describe the path of a transfer from BNKMENU through XFRFUN to Db2 and back.

Check yourself: you can explain that XFRFUN debits one account and credits another as a single unit of work that CICS commits together, so both succeed or both fail, never half a transfer.

Why it matters: atomicity is what keeps money from being lost or duplicated. A transfer that could half-complete would be a integrity disaster.

Blue-Team angle: a bug or attack that breaks atomicity (a debit without a credit) is a direct financial-integrity issue. Unit-of-work handling is a security property, not just a correctness one.

Lab 29.4

Understand the overflow

Goal: reproduce the storage overflow in the simulation and explain both its effects, the abend and the corruption.

Background: understanding the bug precisely is the prerequisite for fixing it. This lab studies the overflow defensively.

1. Trigger the buffer-overflow simulation and read the reported effect and abend.

Check yourself: you can explain that oversized input (96 into 32) overwrote an adjacent approved-flag and caused an ASRA abend, and that the corruption (not just the crash) is the dangerous part.

Why it matters: the overflow is both a crash and a data-integrity failure. Seeing both effects is what motivates the fix.

Blue-Team angle: ASRA abends spiking in a program can signal malformed or hostile input probing for exactly this weakness. Correlate ASRA rates with input sources.

Lab 29.5

Apply the secure fix

Goal: state the COBOL changes that prevent the overflow and confirm they close both effects.

Background: the fix is a habit, not a heroic effort. This lab makes it concrete.

1. Describe the length check and bounded move that prevent the overflow, before any MOVE or after RECEIVE MAP.

Check yourself: you can state that validating input length before the move, using the RECEIVE MAP length, and bounding or rejecting over-length input all prevent the excess from ever being written, closing both the abend and the corruption.

Why it matters: one length check per untrusted move eliminates the whole bug class. Secure coding here is cheap and decisive.

Blue-Team angle: length validation before moves is a reviewable coding standard. A code review or scan for unbounded moves of screen and API input finds the exposures before an attacker does.

The overflow in COBOL, and its fix

The bug and the fix are clearest in the code itself. The unsafe version moves screen input straight into a fixed field; the safe version checks the length first:

```
* --- UNSAFE: no length check before the move ---
01 WS-INPUT      PIC X(96).
01 WS-NAME-FIELD PIC X(32).
01 WS-APPROVED   PIC X(01).
    MOVE WS-INPUT TO WS-NAME-FIELD.    *> spills 64 bytes
*                                     *> WS-APPROVED corrupted

* --- SAFE: validate length, then bound the move ---
    IF FUNCTION LENGTH(WS-INPUT) > 32
        MOVE 'E' TO WS-RESULT    *> reject over-length input
    ELSE
        MOVE WS-INPUT(1:32) TO WS-NAME-FIELD
    END-IF.
```

The overflow and its fix in COBOL, the unsafe MOVE versus a length-checked, bounded MOVE.

In the unsafe block the three fields sit next to each other in storage, so moving 96 bytes into the 32-byte name field writes 64 bytes past its end (straight over WS-APPROVED. The safe block does the one thing that prevents it: it checks the input length and either rejects the input or bounds the move to 32 bytes with reference modification, so nothing is ever written past the field. This is the entire lesson in six lines) the difference between a program that corrupts an approval flag and abends, and one that cleanly refuses bad input.

The same core, through the web

The modern face of the stack is a REST tier that exposes the very same COBOL logic to web and mobile clients. A request for an account arrives as HTTP and returns JSON:

```
curl http://gibson:8080/api/v1/cbsa/vuln/account/00000101
200 OK
```

```
{ "account": { "SORT_CODE": "00-00-01", "ACCOUNT_NUMBER": "00000101", "CUSTOMER_ID": "1001",
  "customer": "1001", "type": "CURRENT",
  "ACTUAL_BALANCE": "1500.00", "STATUS": "OPEN"}, "training": "IDOR ownership check
bypassed" }
```

The banking core reached over REST, the same account data, served as JSON to a web client.

The web tier does not replace the COBOL; it wraps it, so a mobile app and a green-screen teller drive the same programs against the same Db2 data. That is powerful and it widens the attack surface: the same input-validation and access-control questions now apply at the web boundary too, and an unvalidated web parameter can reach the same fixed-length fields as a screen input. The web and API tier is important enough that Part VII returns to it in full; here the point is that securing the core means securing every door into it, old and new.

Lab 29.6

Read the bug and write the fix

Goal: read the unsafe COBOL move, explain the corruption, and state the length-checked replacement.

Background: secure coding is best learned at the line level. This lab reads the exact bug and its exact fix.

1. Study the unsafe MOVE and identify which field is corrupted and why.
2. State the length check and bounded move that prevent it.

Check yourself: you can explain that the unsafe MOVE writes past WS-NAME-FIELD into WS-APPROVED, and that checking the length and bounding the move to 32 bytes (or rejecting over-length input) prevents both the corruption and the abend.

Why it matters: seeing the fix at the line level makes it repeatable. This is the pattern to apply to every untrusted move in the codebase.

Blue-Team angle: an automated scan for unbounded MOVES of screen or API input into fixed fields is a high-yield code review, it finds this exact exposure across a whole application before an attacker probes for it.

Checkpoint: can you describe the banking stack's layers, read its data model, name the COBOL programs, trace a transfer as an atomic unit of work, explain the storage overflow's two effects, and state the secure fix? Those skills connect the application layer to everything beneath it, and to the attacks Part IX aims at exactly this layer.

A mainframe banking core is a stack: BMS and web presentation, CICS transactions, COBOL business logic, and Db2 data, with a web tier serving the same COBOL to modern clients. CBSA models it faithfully (customers and accounts keyed by sort code and number, and programs like XFRFUN that move money as atomic units of work. Its most consequential weakness is the classic COBOL storage overflow: unvalidated oversized input moved into a fixed-length field spills into adjacent storage, corrupting a neighboring value (an approval flag) and raising an ASRA abend. The bug is both a crash and a data-integrity failure, and the fix is a single discipline) validate length before moving, use the real input length, and bound or reject the rest, that eliminates the whole class.

What comes next

Beneath the COBOL sits the data, and the data lives in Db2. The next chapter opens the database: SPUFI and SQL, the Db2 catalog, the distributed access that lets applications reach it over the network through DDF and DRDA, and the SQL-injection paths that arrive with any application that builds queries from user input. Db2 is where the balances actually live, which makes how it is queried, and how it is secured, the next essential piece of the banking story.

CHAPTER 30

Db2: The Database

By the end of this chapter you will be able to:

- Query Db2 interactively with SPUFI and SQL.
- Read the Db2 catalog, the database describing itself.
- Explain plans and packages and the EXECUTE privilege.
- Describe DDF and DRDA, reaching Db2 over the network.
- Explain SQL injection and why concatenating input is the flaw.
- Apply parameterized queries as the fix.

BENEATH THE COBOL OF THE banking stack sits the data, and on the mainframe that data usually lives in Db2, IBM's relational database. The account balances the last chapter cared about are Db2 rows. This chapter opens the database: how you query it with SQL through SPUFI, how the catalog lets the database describe itself, how plans and packages turn SQL into something runnable, and how DDF exposes Db2 across the network. It ends with the application-layer attack that arrives with any program that builds SQL from user input (SQL injection) and the single practice that prevents it. Db2 is where the crown-jewel data lives, so how it is queried and how it is secured are the heart of protecting a mainframe application.

Querying Db2: SPUFI and SQL

The interactive way to run SQL against Db2 is SPUFI, SQL Processor Using File Input. You type SQL, it runs, and it shows the result set. A query against the catalog looks like this:

```
SELECT NAME, CREATOR FROM SYSIBM.SYSTABLES
DSNE616I NUMBER OF ROWS DISPLAYED IS 39
NAME          CREATOR  TYPE  DBNAME  TSNAME
-----
SYSTABLES     SYSIBM   T     DSNDB06 SYSTSTAB
SYSUSERAUTH   SYSIBM   V     DSNDB06 SYSUSER
SYSDBAUTH     SYSIBM   T     DSNDB06 SYSDBAUT
EMPLOYEES     GIBSON   T     GIBDB   EMPLOYE
ACCOUNTS      GIBSON   T     GIBDB   ACCOUNTS
```

SPUFI, interactive SQL against Db2, here listing tables from the catalog.

SQL itself is the standard relational language (SELECT to read, INSERT, UPDATE, and DELETE to change) and Db2 speaks it exactly as any relational database does. The DSNE616I line reports how many rows came back, and the result is a table of them. SPUFI is a developer's and administrator's everyday tool, and it is also, for anyone who reaches

it, a direct window into the data, which is why SPUFI access, and the authority behind it, is itself a security consideration.

The catalog: the database describing itself

Db2 keeps a complete description of itself in the catalog, a set of tables under the SYSIBM creator that record every object in the database. The query above was reading one of them. The catalog tables that matter most are these:

Catalog table	What it describes
SYSTABLES	Every table and view in the database
SYSCOLUMNS	Every column of every table
SYSPLAN / SYSPACKAGE	The application plans and packages
SYSUSERAUTH	System-level authorities held by users
SYSDBAUTH	Database-level authorities held by users

The Db2 catalog, the database's own description of its objects and authorities.

The catalog is a gift to an administrator and to an attacker alike. An administrator queries it to understand the database. An attacker who can read it learns the entire structure, every table, every column, and, through SYSUSERAUTH and SYSDBAUTH, exactly who holds which privileges. Reading the authorization tables answers “who can touch the ACCOUNTS table?” in a single query, which is why access to the catalog's authorization views is worth controlling: it is the map an attacker would draw first.

Plans, packages, and EXECUTE

Application SQL is not usually typed live; it is written into a program, then bound into a package and a plan, the prepared, optimized, executable form of that SQL. Listing them shows the applications the database serves:

PLANS	PACKAGES
DSNTEP2	PAYROLL
SPUFI	EMPMaint
CBSAPLAN	
PAYPLAN	

Db2 plans and packages, the bound, runnable form of application SQL.

A plan or package bundles an application's SQL so it runs efficiently and under controlled authority. The key security concept here is the EXECUTE privilege: to run a plan, a user needs EXECUTE authority on it, and that authority (not direct table access) is often how applications reach data. This is a feature, because it lets a bank grant a teller the right to run the banking application without granting direct SELECT on the ACCOUNTS table. But it also means EXECUTE on a powerful plan can be as valuable as table access itself, and belongs in any review of who can do what.

DDF and DRDA: Db2 over the network

Db2 is not reached only from the mainframe. The Distributed Data Facility (DDF) exposes it to the network so that remote applications (a web server, a Java program, a reporting tool) can connect and run SQL using DRDA, the distributed database protocol. Displaying the data-sharing group shows the network identity of the database:

```
-DB2A DISPLAY GROUP
DSN7100I -DB2A DISPLAY GROUP REPORT
GROUP ATTACH NAME: DB2G1
MEMBER NAME       : DB2A
SUBSYSTEM ID      : G0SDB2
LOCATION           : GIBSONDB2
VERSION           : 12.1
```

The Db2 group, its members and its network LOCATION, reachable via DDF/DRDA.

The LOCATION name, GIBSONDB2, is how remote clients address this database over DRDA, on the Db2 port you saw in the console's port display (50000). DDF is enormously useful and, being a network-facing door into the database, a significant attack surface: it accepts connections and authentication from off the mainframe, so weak DDF authentication, or a DRDA-speaking client with stolen credentials, is a path straight to the data that bypasses the green screen entirely. Any Db2 assessment includes what DDF exposes and how it authenticates.

Securing Db2

Db2 access is controlled by privileges, granted with GRANT and removed with REVOKE, and increasingly integrated with RACF so that one security manager governs the whole system. The privileges form a familiar ladder:

Privilege	What it allows
SELECT	Read rows from a table
INSERT / UPDATE / DELETE	Change rows
EXECUTE	Run a plan or package
DBADM	Administer a database
SYSADM	Full Db2 authority, everything

Db2 privileges, from reading a row to full system authority.

SYSADM is Db2's equivalent of RACF SPECIAL (total authority) and, like SPECIAL, should be held by very few and audited closely. The everyday controls are the table privileges and EXECUTE, granted on least-privilege lines: a user or application gets exactly the access its job requires. Where Db2 is integrated with RACF, these decisions flow through the same external security manager as the rest of the system, so the who-can-touch-the-data question is answered in one place. The exposure to hunt for is the same as everywhere: over-broad grants, especially SELECT on sensitive tables or SYSADM held too widely.

SQL injection

The most common application-layer attack against a database arrives whenever a program builds SQL by pasting user input into a query string. Consider an account search that looks up a customer id. With normal input it does what you expect:

```
INPUT: 1001
SQL:   SELECT * FROM CBSA.ACCOUNT WHERE CUSTOMER_ID = '1001'
ROWS:  2
```

A normal search, the input is a customer id, and two matching accounts return.

Now give it input that is not a customer id but a fragment of SQL. Because the program pastes the input directly into the query, the input becomes part of the command:

```
INPUT: 1001' OR '1'='1
SQL:   SELECT * FROM CBSA.ACCOUNT WHERE CUSTOMER_ID = '1001' OR '1'='1'
ROWS:  4  (all accounts returned)
```

SQL injection, crafted input changes the query's logic and returns every account.

The added OR '1'='1' makes the WHERE clause true for every row, so the query returns all accounts, not just the customer's. This is SQL injection, and its consequences run from reading data one should not see to modifying or deleting it, because the same trick works on UPDATE and DELETE. The root cause is precise and worth stating exactly: the program treated user input as part of the SQL command rather than as mere data. That confusion between code and data is the entire vulnerability, and it points straight at the fix.

The fix: parameterized queries

The cure is to never build SQL by concatenating input. Instead, the query is written once with a placeholder (a parameter marker, the ? in Db2, or a host variable in a program) and the input is supplied separately as a value bound to that placeholder. The database then treats the input strictly as data, never as part of the command:

```
* UNSAFE - input concatenated into the SQL text
"SELECT * FROM CBSA.ACCOUNT WHERE CUSTOMER_ID = '" + input + "'"

* SAFE - parameter marker; input bound as a value
SELECT * FROM CBSA.ACCOUNT WHERE CUSTOMER_ID = ?
(then bind the input to the marker as data)
```

The fix, a parameter marker keeps input as data, so it can never become SQL.

With a parameter marker, the input 1001' OR '1'='1 is looked up as a literal customer id that simply does not exist, returning nothing, the injection is inert. This is the same principle as the COBOL length check from the last chapter: never trust input, and keep the boundary between what the program means and what the user supplied absolutely clear.

Parameterized queries are the standard, decisive defense against injection in every language and every database, Db2 included.

The mistake behind every injection is building a command out of untrusted input. If input is concatenated into SQL text, no amount of filtering or escaping is as reliable as simply not doing it: use a parameter marker so the database treats input as data. Blocklists of bad characters are brittle and get bypassed; parameterization removes the vulnerability entirely.

Hands-On Labs

These labs query Db2 and study injection defensively, to understand and prevent it.

Lab 30.1

Query with SPUFI

Goal: run a SELECT against the catalog and read the result set.

Background: SPUFI is the interactive door to Db2. Running a query is the foundation of everything else.

1. Select tables from the catalog.

```
SELECT NAME, CREATOR FROM SYSIBM.SYSTABLES
```

Check yourself: the query returns a list of tables with their creators, and the DSNE616I line reports the row count. You can read a Db2 result set.

Why it matters: SQL is how the data is reached. Running a query is the first skill for both using and assessing a database.

Blue-Team angle: who can run ad-hoc SQL through SPUFI is itself a control, interactive query access to production data is powerful and should be limited.

Lab 30.2

Read the catalog for authorities

Goal: query the authorization catalog tables to see who holds which privileges.

Background: the catalog answers “who can touch this data?” directly. This lab reads it as an auditor would.

1. Query SYSUSERAUTH and SYSDBAUTH for held authorities.

```
SELECT * FROM SYSIBM.SYSDBAUTH
```

Check yourself: you can read which users hold which database and system authorities, and identify any broad grant (SYSADM, or SELECT on a sensitive table) that warrants review.

Why it matters: the authorization catalog is the Db2 access map. Reading it is how a review finds over-privilege.

Blue-Team angle: an attacker reads these same tables to plan. Reading them first, and trimming over-broad grants, closes the map before it can be used.

Lab 30.3

Inspect DDF

Goal: display the Db2 group and identify its network LOCATION and port.

Background: DDF is the network door into Db2. Knowing what it exposes is part of the attack surface.

1. Display the Db2 group.

```
-DB2A DISPLAY GROUP
```

Check yourself: you can name the group, the member, and the LOCATION (GIBSONDB2) that remote DRDA clients use, and relate it to the Db2 port from the console's port display.

Why it matters: DDF accepts connections from off the mainframe. Its authentication is a first-class part of Db2's security posture.

Blue-Team angle: a DRDA client with valid credentials reaches the data without ever touching a 3270. Weak DDF authentication is a direct, network-facing exposure.

Lab 30.4

Understand injection

Goal: reproduce the injection in the simulation and explain why the crafted input changes the query's logic.

Background: understanding injection precisely is the prerequisite to preventing it. This lab studies it defensively.

1. Search with a normal id, then with crafted input, and compare the built SQL and the rows returned.

Check yourself: you can show that normal input returns the customer's accounts, while 1001' OR '1'='1 makes the WHERE clause always true and returns every account, and you can explain that the input became part of the command.

Why it matters: seeing the crafted input become SQL makes the code-versus-data confusion concrete, and motivates the fix.

Blue-Team angle: injection attempts often show as SQL errors or anomalous result sizes.

Monitoring for malformed queries and unexpectedly large result sets is a useful detection.

Lab 30.5

Apply parameterized queries

Goal: state the parameter-marker rewrite and explain why it makes the injection inert.

Background: the fix is a single, universal pattern. This lab makes it concrete.

1. Rewrite the vulnerable query to use a parameter marker and describe how the input is bound.

Check yourself: you can state that the query is written once with a ? and the input is bound as a value, so 1001' OR '1'='1 is looked up as a literal id that does not exist, the injection returns nothing.

Why it matters: parameterization removes the vulnerability rather than filtering around it. It is the decisive, standard defense.

Blue-Team angle: a code review or scan for dynamic SQL built by concatenation finds injection exposures wholesale. Mandating parameter markers is a reviewable, enforceable standard.

Granting and revoking access

Privileges are handed out with GRANT and taken back with REVOKE, and each is recorded in the catalog. Granting a teller the right to run the banking plan, rather than direct table access, looks like this:

```
GRANT EXECUTE ON PLAN CBSAPLAN TO TELLER01
DSNE615I NUMBER OF ROWS AFFECTED IS 1
GRANT SELECT ON GIBSON.ACCOUNTS TO REPORTER
DSNE615I NUMBER OF ROWS AFFECTED IS 1
REVOKE SELECT ON GIBSON.ACCOUNTS FROM REPORTER
DSNE615I NUMBER OF ROWS AFFECTED IS 1
```

GRANT and REVOKE, handing out and reclaiming Db2 privileges, each recorded in the catalog.

Read the pattern: the teller is granted EXECUTE on the banking plan (the right to run the application) rather than SELECT on the ACCOUNTS table, so they can bank without being able to browse balances directly. The reporter is granted, then revoked, direct SELECT. This is least privilege in practice: prefer EXECUTE on a controlled plan over broad table access, and revoke what is no longer needed. Every GRANT is visible in SYSDBAUTH and SYSUSERAUTH, so the catalog is both the record of what was granted and the place a review checks that nothing was granted too widely.

Lab 30.6

Audit the grants

Goal: read who holds EXECUTE and table privileges, and identify any grant that is broader than the job requires.

Background: over-broad grants are the most common Db2 finding. This lab practices spotting them.

1. Query the authorization catalog and review the grants on sensitive objects.

```
SELECT GRANTEE, TCREATOR, TTNAME FROM SYSIBM.SYSTABAUTH
```

Check yourself: you can list who holds SELECT, UPDATE, or EXECUTE on sensitive tables and plans, and flag a direct SELECT on ACCOUNTS where EXECUTE on the plan would suffice, or any SYSADM held too widely.

Why it matters: the difference between EXECUTE on a plan and SELECT on a table is the difference between using an application and browsing its data. Auditing grants enforces that line.

Blue-Team angle: prefer EXECUTE on controlled plans over direct table grants, revoke promptly, and review SYSADM regularly, it is Db2's SPECIAL, and just as dangerous when spread too far.

Checkpoint: can you query Db2 with SPUFI, read the catalog including its authorization tables, explain plans and EXECUTE, describe DDF as a network door, and (above all) explain SQL

injection and fix it with parameter markers? Those skills secure the layer where the application's data actually lives.

Db2 is where mainframe application data lives. SPUFI runs SQL interactively; the catalog (SYSTABLES, SYSCOLUMNS, and the SYSUSERAUTH/SYSDBAUTH authorization tables) lets the database describe itself and its privileges; plans and packages are the bound, runnable form of application SQL, reached through the EXECUTE privilege; and DDF exposes Db2 to the network over DRDA, a powerful feature and a real attack surface. Access is governed by GRANTed privileges from SELECT to SYSADM, increasingly through RACF. The classic application attack, SQL injection, arises when a program builds SQL from concatenated input so that input becomes command; the decisive fix is the parameterized query, which keeps input as data. As with the COBOL overflow, the lesson is to never trust input and never blur code and data.

What comes next

Db2 is the relational database, but it is not the only one on the platform. Some of the oldest and highest-volume workloads run on IMS, a hierarchical database and transaction manager that predates the relational model and still processes enormous volumes in banking and insurance. The next chapter turns to IMS (its hierarchical databases and its transactions) and to what changes, and what stays the same, when the data model is a tree rather than a table.

CHAPTER 31

IMS: Hierarchical Data

By the end of this chapter you will be able to:

Explain the hierarchical data model, segments in a tree.

Read an IMS database definition (DBD).

Navigate data with DL/I calls: GU, GN, ISRT, and the rest.

Read a program view (PSB/PCB) and explain PROCOPT.

Describe IMS as a transaction manager and its security.

Compare IMS access control to Db2's, and secure both.

NOT ALL MAINFRAME DATA IS relational. IMS (the Information Management System) predates the relational model and still runs some of the largest, highest-volume workloads in banking and insurance, precisely because it is fast and it has never stopped working. IMS is two things in one: a hierarchical database (IMS DB), where data is a tree of segments rather than a set of tables, and a transaction manager (IMS TM), a peer of CICS that schedules and runs transactions. This chapter covers both, and its real subject is what changes (and what stays the same) when the data model is a tree. The security ideas carry over from Db2, but they wear an unfamiliar shape: access is granted not by a SQL GRANT but by the program view, the PSB, which is worth understanding on its own terms.

The hierarchical model

In IMS, data is organized as a tree. A database definition (a DBD) describes the segment types and how they nest, parent to child. The classic parts database makes it concrete:

IMS DBD

DBD PARTSDB ACCESS=HDAM (hierarchical parts database)

SEGMENT	PARENT	KEY FIELD	FIELDS
PART	(root)	PARTNO	PARTNO, DESC, TYPE
STOCK	PART	WHSE	WHSE, QTY, LOC
ORDER	PART	ORDNO	ORDNO, QTY, STATUS

Hierarchy: PART -> (STOCK, ORDER)

An IMS DBD, the segment hierarchy: a PART root with STOCK and ORDER children.

Read the tree. PART is the root segment, keyed by PARTNO. Beneath each part hang two kinds of child segment: STOCK, recording where the part is warehoused, and ORDER, recording orders for it. This is the hierarchical model in miniature: data is not spread across tables joined by keys, it is nested, with children physically belonging to their parent. The relationships are built into the structure rather than expressed by matching

values, which is why IMS is so fast for the access patterns it was designed for, and why reaching the data feels so different from SQL.

DL/I: navigating the tree

There is no SQL in IMS. Programs reach data through DL/I (Data Language One) a set of calls that navigate the hierarchy one segment at a time. You get a specific segment, then get the next, walking the tree. A qualified get-unique reads a part directly:

```
IMS DLI GU PART(PARTNO=P200)
DL/I GU STATUS=' ' SUCCESSFUL
I/O AREA: PART PARTNO=P200 DESC=GASKET TYPE=FLAT
POSITION: PART(P200)
```

A DL/I GU (Get Unique), a direct, qualified read of one segment.

The call returns the segment into an I/O area and leaves a position in the database. A get-next then moves to the following segment in hierarchical sequence, here, from the part down to its first stock child:

```
IMS DLI GN
DL/I GN STATUS=' ' SUCCESSFUL
I/O AREA: STOCK WHSE=W01 QTY=0200 LOC=C03
POSITION: PART(P200) -> STOCK(W01)
```

A DL/I GN (Get Next), stepping to the next segment in hierarchical order.

The two-character status field is DL/I's equivalent of an SQLCODE: blanks mean success, and codes like GE (not found) or GB (end of database) report what happened. The full call set is small and navigational:

DL/I call	What it does
GU	Get Unique, direct, qualified read of a segment
GN	Get Next, next segment in hierarchical sequence
GNP	Get Next within Parent, stay under the current parent
ISRT	Insert a new segment
REPL	Replace the current segment
DLET	Delete the current segment

The DL/I call set, navigate and modify the hierarchy one segment at a time.

Where SQL is declarative (you describe the rows you want and Db2 finds them) DL/I is navigational: you tell IMS exactly how to walk the tree. This is more work for the programmer and, for the right patterns, far faster. It also shapes the security model, because access is controlled not by naming rows a user may read, but by defining the view of the tree a program is given.

The program view: PSB and PROCOPT

A program does not see the whole database. It sees the view its PSB (Program Specification Block) grants, and that view is the heart of IMS data security. The PSB lists,

in its PCBs, exactly which segments a program is sensitive to and what it may do with them:

IMS PSB

PSB PARTPSB LANG=COBOL

PCB TYPE=DB DBDNAME=PARTSDB PROCOPT=A KEYLEN=14

SENSEG NAME=PART, PARENT=0

SENSEG NAME=STOCK, PARENT=PART

SENSEG NAME=ORDER, PARENT=PART

PROCOPT A = all (GU/GN/GNP/ISRT/REPL/DLET)

A PSB, the program's view: which segments it can see (SENSEG) and do (PROCOPT).

Two controls live here. SENSEG (sensitive segment) lists which segments the program can see at all; a segment not named is invisible to that program, as if it did not exist. PROCOPT (processing option) says what the program may do: PROCOPT=A allows everything (get, insert, replace, delete), while PROCOPT=G is read-only and rejects any attempt to change data. This is IMS's access model, and it is genuinely different from Db2's. Instead of granting a user privileges on tables, IMS gives a program a tailored view of the tree and a set of allowed operations. A read-only reporting program is given a PROCOPT=G PSB and simply cannot write, no matter what its code attempts, the restriction is in the view, not the code.

IMS as a transaction manager

IMS TM is the other half, a transaction manager that, like CICS, accepts input, schedules a program to handle it, and returns a response. Transactions are named by trancodes, and the operator commands display and control them. Displaying the active regions shows the work running:

/DIS ACTIVE

DFS000I	REGID	JOBNAME	TYPE	TRAN/STEP	STATUS
DFS000I	0001	IMS1DEP	TP	PART	WAIT-INPUT
DFS000I	0002	IMS1BMP	BMP	IVTNO	ACTIVE

89001/140233

IMS active regions, the message and batch regions running transactions.

When a transaction runs, IMS schedules it against a PSB and reports the lifecycle: DFS058I TRANSACTION scheduled, then completed. Transactions arrive from 3270 terminals and, increasingly, from the network through OTMA (the Open Transaction Manager Access interface) which lets off-platform clients submit IMS transactions. OTMA is IMS's equivalent of Db2's DDF: a powerful network door, and therefore a security surface. IMS enforces transaction security, and a rejected attempt reports it plainly: DFS3662W OTMA SECURITY VIOLATION when a transaction or command is not permitted.

Securing IMS

IMS security rests on the same RACF foundation as the rest of the system, applied to IMS resources: who may enter which transaction, issue which command, and (through OTMA) connect from the network. But the data-access control is the PSB, and that is the shift to internalize. The comparison with Db2 lays it out:

Aspect	IMS (hierarchical)	Db2 (relational)
Data shape	Tree of segments	Tables of rows
Relationships	Parent-child nesting	Keys and joins
Access API	DL/I navigation	SQL (declarative)
Data control	PSB: SENSEG + PROCOPT	GRANT on tables

IMS versus Db2, the same jobs, a different model, a different access control.

The security lesson is that in IMS you audit the PSBs as much as the RACF profiles: a program with a PROCOPT=A view of sensitive segments can change them, and a PSB that names segments a program has no business seeing is an over-broad grant in hierarchical clothing. Transaction and command security through RACF, tight OTMA authentication, and least-privilege PSBs together are IMS's answer to the same questions Db2 answers with GRANT and DDF controls.

The mistake is to look for a GRANT statement and, finding none, assume IMS data is unprotected. IMS controls data access through the PSB (the SENSEG list and the PROCOPT) not through per-user table grants. Reviewing IMS security means reviewing which programs have which views and processing options, alongside the RACF transaction and OTMA controls. A permissive PSB is the IMS equivalent of an over-broad SQL grant.

Hands-On Labs

These labs explore the hierarchical database and IMS transaction management.

Lab 31.1

Read the hierarchy

Goal: read a DBD and describe the segment tree.

Background: the hierarchy is the whole model. Reading a DBD is where IMS understanding begins.

1. Display the database definition.

IMS DBD

Check yourself: you can name the root segment (PART) and its children (STOCK, ORDER), and explain that children physically belong to their parent rather than being joined by a key.

Why it matters: the tree shape governs how data is reached and secured. Reading the DBD is reading the model.

Blue-Team angle: the DBD tells you what sensitive data exists and where it sits in the tree, the first input to deciding which programs should be able to see which segments.

Lab 31.2

Navigate with DL/I

Goal: use GU and GN to read a segment and step through the hierarchy.

Background: DL/I is navigational, not declarative. This lab practices walking the tree.

1. Get a part directly, then get the next segment.

```
IMS DLI GU PART(PARTNO=P200)
```

```
IMS DLI GN
```

Check yourself: GU returns the qualified part with a blank status, and GN steps to its first child (a STOCK segment). You can read the status field as DL/I's success indicator.

Why it matters: reaching hierarchical data means navigating it. GU and GN are the core moves you build everything from.

Blue-Team angle: DL/I access leaves its own trail; understanding the call pattern is part of interpreting what a program actually did to the data.

Lab 31.3

Read the program view

Goal: read a PSB and explain SENSEG and PROCOPT as access controls.

Background: the PSB is IMS's data-access control. Reading one is the heart of an IMS security review.

1. Display the PSB and read its PCB, SENSEGs, and PROCOPT.

```
IMS PSB
```

Check yourself: you can state which segments the program is sensitive to and what PROCOPT allows, and explain that a PROCOPT=G program cannot write no matter what its code attempts.

Why it matters: the PSB, not the code, decides what a program may do to the data. Reading it is reading the real access boundary.

Blue-Team angle: audit PSBs for over-broad SENSEG lists and PROCOPT=A on sensitive segments, they are the hierarchical equivalent of over-permissive grants.

Lab 31.4

Display active work

Goal: display the active IMS regions and transactions.

Background: IMS TM runs transactions in regions. Displaying them is the operational pulse, as CEMT is for CICS.

1. Display the active regions.

```
/DIS ACTIVE
```

Check yourself: you can read the active regions, their type (message or batch), and the transaction each is running or waiting for.

Why it matters: knowing what is running, and being able to display and control it, is core IMS operation.

Blue-Team angle: an unexpected transaction scheduled, or a rejected one reported as an OTMA security violation, is exactly the kind of event a defender watches for in IMS.

Lab 31.5

Audit IMS access

Goal: bring the PSB, transaction security, and OTMA together into an IMS access review.

Background: securing IMS means auditing both RACF controls and the PSBs. This lab connects them.

1. For a sensitive database, identify which programs have write access via PROCOPT and confirm transaction and OTMA security are enforced.

Check yourself: you can name the programs with PROCOPT=A on sensitive segments, confirm that transaction entry and OTMA connections require authorization, and flag any PSB granting more than a program needs.

Why it matters: IMS security is the PSBs plus the RACF controls together. Auditing one without the other leaves a gap.

Blue-Team angle: the DFS3662W OTMA security violation is a detection signal; over-broad PSBs are a configuration finding. A full IMS review covers both the runtime events and the static views.

Changing the tree, and the read-only guard

Writing to the hierarchy uses the same navigational style. An insert places a new segment, positioned by its key:

```
IMS DLI ISRT PART(PARTNO=P400,DESC=WIDGET,TYPE=ROUND)
DL/I ISRT STATUS=' ' SUCCESSFUL
I/O AREA: PART PARTNO=P400 DESC=WIDGET TYPE=ROUND
POSITION: PART(P400)
```

A DL/I ISRT, inserting a new PART segment into the hierarchy.

That insert succeeded because the program's PSB has PROCOPT=A, which permits writes. The security value of PROCOPT becomes concrete the moment the view is switched to read-only. With a PROCOPT=G view, the very same insert is refused by IMS itself, before it touches the data:

```
PSB PARTPSB now PROCOPT=G (get-only); ISRT/REPL/DELET will return AM
IMS DLI ISRT PART(PARTNO=P500,DESC=BOLT,TYPE=HEX)
DL/I ISRT STATUS='AM' REJECTED - PROCOPT DOES NOT ALLOW ISRT
```

The read-only guard, a PROCOPT=G view returns status AM and refuses the write.

The AM status is IMS enforcing the program view: the write is rejected not by the application's logic but by the PSB's processing option, so a reporting program given a get-only view cannot alter data even if its code tries. This is the whole point of PROCOPT as a control, the restriction lives in the view, where the program cannot reach it, exactly as a

database privilege lives outside the code that runs under it. Auditing which programs hold PROCOPT=A on sensitive segments is therefore auditing who can change the data.

A transaction, scheduled

On the TM side, entering a trancode schedules a program against a PSB and runs it, with the lifecycle reported on the console:

```
DFS058I 14:02:33 TRANSACTION PART SCHEDULED PSB PARTPSB REGION 0001
DFS058I TRANSACTION PART COMPLETED
```

```
DFS3662W OTMA SECURITY VIOLATION - TRANSACTION PAYX REJECTED
```

IMS transaction scheduling, and, below, a rejected transaction reported as an OTMA violation.

The DFS058I messages trace a transaction from scheduled to completed, tying it to the PSB that gave it its data view. The DFS3662W line is the security counterpart: an attempt to run a transaction the requester is not authorized for, refused and logged. Together they show IMS TM doing both jobs at once (running authorized work and rejecting unauthorized work) and both lines are events a defender reads, the first for operations and the second for security.

Lab 31.6

Prove the read-only view holds

Goal: show that a PROCOPT=G program view rejects writes regardless of the operation attempted.

Background: the PSB, not the code, decides what a program may do. This lab proves the guard by trying to write through a read-only view.

1. With a write-capable view, insert a segment; then switch to a get-only view and attempt the same insert.

```
IMS PSB GET
IMS DLI ISRT PART(PARTNO=P500,DESC=BOLT,TYPE=HEX)
```

Check yourself: the insert succeeds under PROCOPT=A and is rejected with status AM under PROCOPT=G. The refusal comes from the view, not from any check in the program.

Why it matters: a control the program cannot bypass is a strong control. PROCOPT enforces read-only at the database, where application code cannot reach it.

Blue-Team angle: give reporting and query programs get-only PSBs by default. A program that can write when it only needs to read is unnecessary risk, the AM rejection is the guard working as designed.

Checkpoint: can you read a DBD and describe the hierarchy, navigate with DL/I, read a PSB and explain SENSEG and PROCOPT as the data-access control, display active transactions, and audit IMS access across PSBs and RACF? Those skills secure a data model that is everywhere in banking and insurance and looks nothing like a table.

IMS is the mainframe's hierarchical database and transaction manager, older than the relational model and still central to high-volume banking and insurance. Its data is a tree of segments described by a DBD (a root with nested children) reached not by SQL but by navigational DL/I calls (GU, GN, GNP, ISRT, REPL, DLET). Data access is controlled by the program view: the PSB, whose SENSEG list decides which segments a program can see and whose PROCOPT decides what it can do, so a read-only view cannot write regardless of the code. IMS TM schedules transactions by trancode, reachable from terminals and, over OTMA, from the network, with RACF enforcing transaction and command security. Securing IMS means auditing the PSBs and the RACF controls together, the hierarchical answer to the same questions Db2 answers with GRANT and DDF.

What comes next

IMS and Db2 run on top of z/OS. But z/OS is not the only operating system the mainframe hosts. The next chapter turns to z/VM (the hypervisor that can run many operating systems at once, including many copies of Linux, on a single machine. Its CP and CMS environments are a world of their own, with their own security model, and they are increasingly where modern mainframe workloads) and modern attack surfaces, live.

CHAPTER 32

z/VM: The Hypervisor

By the end of this chapter you will be able to:

Explain what z/VM is, a hypervisor running many guests at once.

Distinguish CP (the control program) from CMS (the guest OS).

Read the CP directory, where virtual machines are defined.

Explain the privilege-class security model (A through G).

Run a privilege-class creep audit and apply Just Enough Authority.

See why the hypervisor is the trust boundary for every guest.

EVERYTHING SO FAR HAS RUN on z/OS. But z/OS is only one operating system the mainframe can host, and often it is not running directly on the hardware at all (it is running as a guest of z/VM. z/VM is a hypervisor: it virtualizes the entire machine so that many operating systems run at once, each believing it has a mainframe to itself. Those guests can be copies of z/OS, and increasingly they are dozens or hundreds of Linux systems, all sharing one physical machine. z/VM has two faces) CP, the control program that is the hypervisor itself, and CMS, a lightweight single-user operating system you run inside a virtual machine. Its security model is unlike anything in the z/OS chapters: privilege classes, defined in a user directory. And because it sits beneath every guest, z/VM is the ultimate trust boundary, whoever controls the hypervisor controls everything running on it.

CP and CMS

Two layers make up z/VM. CP (the Control Program) is the hypervisor: it creates and runs virtual machines, manages the real hardware, and handles the privileged commands that control the system. CMS (the Conversational Monitor System) is a small, single-user operating system that runs inside a virtual machine, and it is where an ordinary user actually works, much as TSO is on z/OS. From CMS you manage your virtual machine's own resources, such as its minidisks:

QUERY DISK

LABEL	VDEV	M	STAT	CYL	TYPE	BLKSZ	FILES	BLKS	USED-	(%)
CMSA01	191	A	R/W	010	3390	4096	3	200	17	
CMSS01	190	S	R/O	010	3390	4096	1	200	17	
CMSY01	19E	Y	R/O	010	3390	4096	0	200	17	

Ready; T=0.01/0.01

CMS minidisks, the virtual disks a guest owns, each with a mode and access.

Each line is a minidisk (a virtual disk carved from real storage and given to this virtual machine, with a mode letter (A, S, Y) and a read/write or read-only status. This is the guest's own little world, isolated from other guests by CP. The isolation is the point: CP

gives each guest its own virtual devices and keeps them separate, so one guest cannot see another's minidisks) unless the directory or a command grants sharing, which is exactly the kind of thing a security review checks.

Virtual machines and the directory

Every guest is defined in the CP directory, z/VM's equivalent of the RACF database. The directory entry for a guest names its userid, its virtual storage, its minidisks, and, critically, its privilege classes. Reading the directory shows the shape of the system:

```

USERID    CLASSES  STORAGE  ROLE
MAINT ABCDEFG 128M system maintenance, all classes
SYSADMIN ABCDEFG 64M administrator, all classes
OPERATOR  ABCDE    64M      system operator
DIRMAINT  BG        32M      directory manager
RACFVM    BG        32M      the RACF service machine
GUEST     G         32M      ordinary user

```

The CP directory, each guest's userid, privilege classes, and storage.

Read the CLASSES column, because it is where z/VM security lives. MAINT and SYSADMIN hold ABCDEFG (every privilege class, meaning total control of the hypervisor. OPERATOR holds ABCDE. GUEST holds only G. A guest's classes decide which CP commands it may issue, and therefore how much of the system it can touch, so the directory is the single most important artifact in a z/VM security review) the place where power is granted, one letter at a time.

The privilege classes

CP privilege is divided into classes A through G, each granting a category of command. The division is the whole model, and it is worth knowing:

Class	What it grants
A	Primary system operator, system-wide control (FORCE, SHUTDOWN)
B	Real resource control, attach and vary real devices
C	System programmer, alter host storage (STORE)
D	Spooling control, manage other users' spool files
E	System analyst, examine host storage (DISPLAY, DUMP)
F	Service representative, real device diagnosis
G	General user, control one's own virtual machine

The CP privilege classes A-G, each a category of command a guest may issue.

Class G is ordinary: it lets a guest run its own virtual machine and nothing more. The others are powerful, and two are especially dangerous. Class C lets a guest alter host storage (the real memory of the hypervisor) and class E lets it examine that storage. A guest with C or E can read or change what other guests are doing at the hypervisor level, which is a route to compromising the whole machine. Class A can shut the system down

or force off any guest. This is why the classes a guest holds matter so much: they are not conveniences, they are the reach of that guest into everyone else's world.

Class creep and Just Enough Authority

The most common z/VM finding is class creep, guests that have accumulated more privilege classes than their role needs, usually because classes were added over time and never removed. An audit compares each guest's held classes against a baseline of what its role actually requires:

```
CP PRIVILEGE-CLASS CREEP AUDIT
```

```
-----
GUEST      HELD      BASELINE  EXCESS  RISK
OPERATNS   ABCDE      ABDG      CE      alter host storage, examine host
OPERATOR   ABCDE      ABDG      CE      alter host storage, examine host
-----
```

```
Remediation: reduce each guest to Just Enough Authority; split
the single all-class admin into hypervisor- and storage-admin roles.
```

A privilege-class creep audit, held versus baseline, with the excess and its risk.

Read the finding. The OPERATOR and OPERATNS guests hold classes ABCDE, but their role baseline is only ABDG (so they carry an excess of classes C and E, exactly the two that grant altering and examining host storage. A system operator has no business reaching into hypervisor memory, yet these guests can. The remediation is the principle at the heart of z/VM security: Just Enough Authority) reduce every guest to the classes its role genuinely needs, and split an all-powerful admin into narrower roles so no single guest holds ABCDEFG. This is least privilege in z/VM's own language, and running the creep audit is how you find where it has been violated.

z/VM, Linux, and the trust boundary

Modern mainframes run enormous numbers of Linux guests under z/VM, connected to each other by in-memory networking (HiperSockets) that never touches a physical wire. This is efficient and it concentrates risk in one place: the hypervisor. CP is the trust boundary beneath every guest, so the security consequence is stark (whoever compromises CP compromises every guest at once, z/OS and Linux alike. A Linux guest that could break out to CP, or a guest with class C that can alter hypervisor storage, is not just a compromised guest; it is a compromised machine. This is why the privilege classes matter beyond their own commands: they are the wall between one guest and the hypervisor that all the others depend on. Guarding that wall) keeping class C and E out of guests that do not need them, protecting the directory, monitoring privileged CP commands, is the core of defending a virtualized mainframe.

The z/VM mistake is treating privilege classes as harmless role labels. Class C and class E are not conveniences for operators; they grant the ability to alter and examine the hypervisor's own

memory, which is power over every other guest. A guest holding classes it does not need is the z/VM equivalent of an unnecessary SPECIAL attribute, review the directory, run the creep audit, and reduce every guest to Just Enough Authority.

Hands-On Labs

These labs explore z/VM and audit its privilege model.

Lab 32.1

Work in CMS

Goal: use CMS to query your virtual machine's minidisks.

Background: CMS is where a guest works. Querying its disks orients you in the virtual machine.

1. Query the minidisks.

QUERY DISK

Check yourself: you can read the minidisks with their mode letters and read/write status, and explain that each is a virtual disk private to this guest unless sharing is granted.

Why it matters: the guest's isolated resources are the unit of z/VM. Knowing your minidisks is knowing your virtual machine.

Blue-Team angle: shared minidisks are a data-sharing path between guests. Unexpected sharing in the directory is worth investigating.

Lab 32.2

Read the directory

Goal: read the CP directory and identify each guest's privilege classes.

Background: the directory is where power is granted. Reading it is the start of any z/VM review.

1. Review the directory entries and their CLASSES.

Check yourself: you can identify which guests hold all classes (ABCDEFG, like MAINT and SYSADMIN) and which hold only G, and explain that the classes decide each guest's reach.

Why it matters: the directory is z/VM's access map. Reading it is how you find who can do what.

Blue-Team angle: the set of all-class guests should be tiny and known. An unexpected guest with broad classes is a top finding.

Lab 32.3

Understand the classes

Goal: explain what classes A, C, E, and G grant, and why C and E are dangerous.

Background: the classes are the model. Knowing which grant hypervisor reach is the security core.

1. For each of A, C, E, and G, state what it allows.

Check yourself: you can say that A is system operator, C alters host storage, E examines host storage, and G is an ordinary user, and explain that C and E give a guest reach into the hypervisor and thus into other guests.

Why it matters: the classes are power, not labels. Knowing which reach the hypervisor is knowing where the danger is.

Blue-Team angle: classes C and E are the ones to scrutinize hardest, they cross the wall between a guest and the hypervisor every other guest depends on.

Lab 32.4

Run the class-creep audit

Goal: compare each guest's held classes against its baseline and identify the excess.

Background: class creep is the common z/VM finding. This lab runs the audit that finds it.

1. Run the privilege-class creep audit and read the excess and risk per guest.

Check yourself: you can show that OPERATOR and OPERATNS hold ABCDE against a baseline of ABDG, an excess of C and E, the ability to alter and examine host storage that their role does not require.

Why it matters: the audit turns "too much privilege" into a named, actionable excess per guest. It is the heart of a z/VM review.

Blue-Team angle: class creep is privilege escalation by accumulation. The creep audit is the standing control that catches it before an attacker uses it.

Lab 32.5

Apply Just Enough Authority

Goal: state the remediation, reduce guests to baseline and split the all-class admin.

Background: finding the excess is half the job; removing it is the other half. This lab plans the fix.

1. For each over-granted guest, state the classes to remove; for the all-class admin, state how to split the role.

Check yourself: you can say that OPERATOR and OPERATNS should lose classes C and E to return to their baseline, and that a single ABCDEFG admin should be split into separate hypervisor-admin and storage-admin roles held by different guests.

Why it matters: Just Enough Authority is least privilege in z/VM. Applying it shrinks the blast radius of any single compromised guest.

Blue-Team angle: no single guest should hold every class in a well-run system. Splitting the all-powerful admin is separation of duties for the hypervisor.

Why class E is dangerous, made concrete

It is worth seeing exactly what an over-granted class buys, because the abstraction “reach into the hypervisor” understates it. A guest holding class E can DISPLAY host real storage, and that storage contains the memory of every other guest. Running such a display from an operator guest that should never have had class E surfaces other guests’ secrets:

```
DISPLAY HOST  (CP privilege class E - examine host real storage)
R021000  OWNER=RACFVM      <== NOT YOUR GUEST
        RACF database master key (in RACFVM nucleus buffer)
        RACFKEY=7F3A9C12E0B45D8A
R048000  OWNER=MAINT      <== NOT YOUR GUEST
        MAINT logon material (clear in CP logon work area)
R055000  OWNER=LINUX01    <== NOT YOUR GUEST
```

The class-E exposure, a guest examining host storage reads other guests’ secrets, including keys.

This is the danger stated plainly. The operator guest, carrying class E only through creep, can read the RACF service machine’s master key, the MAINT guest’s logon material, and into a Linux guest’s memory, none of which belong to it. Class C is worse still, because it can alter that storage, not just read it. This single display is the whole argument for the creep audit and Just Enough Authority: a class that lets one guest read every other guest’s memory must never be held by a guest that does not absolutely require it. The remediation is not optional hardening; it closes a path to every credential and key on the machine.

The remediation in practice

Applying Just Enough Authority to the finding is direct: strip the excess classes and re-audit until each guest matches its baseline.

```
APPLY JEA REMEDIATION
OPERATOR  ABCDE -> ABDG   (removed C,E)
OPERATNS  ABCDE -> ABDG   (removed C,E)
admin role ABCDEFG -> split: HVADMIN(ABF) + STGADMIN(CE)
RE-AUDIT: no guests over baseline - NO CLASS CREEP DETECTED
```

The remediation, excess classes removed, the all-class admin split, and a clean re-audit.

After remediation the operator guests can no longer touch host storage, and the single all-powerful admin has become two narrower roles so that examining and altering storage require different guests, separation of duties for the hypervisor. The clean re-audit is the proof the fix took, exactly the recommend-then-validate loop from the monitoring chapter, applied to z/VM. A standing creep audit then keeps the system at Just Enough Authority over time.

Lab 32.6

See the exposure, then close it

Goal: demonstrate what an over-granted class E permits, then apply the remediation and confirm it with a re-audit.

Background: seeing the exposure motivates the fix. This lab connects the risk directly to the remediation and its validation.

1. From a guest that holds class E through creep, examine host storage and note what belongs to other guests.
2. Remove the excess classes, split the all-class admin, and re-run the creep audit.

Check yourself: with class E, the guest reads other guests' storage, including keys and logon material; after removing C and E and splitting the admin, the re-audit reports no class creep and the guest can no longer reach host storage.

Why it matters: the exposure and its closure are two halves of one lesson. Seeing what class E permits is what makes Just Enough Authority feel urgent rather than theoretical.

Blue-Team angle: treat any guest holding class C or E as a top-priority review item, and validate every remediation with a re-audit. A guest that can read host storage can read every secret in every guest, there is no higher-value z/VM finding.

Issuing CP commands

CP commands are how a guest talks to the hypervisor, and which ones a guest may issue is governed entirely by its classes. A general-user (class G) guest can query and control its own machine; a query of who is logged on is class-appropriate and harmless:

CP QUERY USERS

LOGON AT 10:41:03 GUESTS = 006 DIALED = 000

OPERATOR OPERATNS MAINT SYSADMIN TCPIP GUEST

Ready; T=0.01/0.01

A class-appropriate CP QUERY, what a general-user guest may legitimately ask.

A command like QUERY USERS is available broadly because it reveals only who is on, not the contents of anyone's machine. The privileged commands (FORCE to log off another guest (class A), STORE to alter host memory (class C), DISPLAY of host storage (class E)) are refused to a guest that lacks the class, which is the model doing its job. Reading which commands succeed and which are refused for a given guest is a practical way to confirm its real reach matches its intended role.

Lab 32.7

Confirm a guest's reach matches its role

Goal: issue class-appropriate and privileged CP commands from a guest and confirm the class model enforces the boundary.

Background: the surest test of a guest's privilege is what its commands do. This lab probes the boundary safely.

1. From a general-user guest, run a harmless query, then attempt a privileged command it should not be allowed.

CP QUERY USERS

Check yourself: the query succeeds, while a privileged command the guest lacks the class for is refused. The guest's real reach matches the classes in its directory entry.

Why it matters: confirming the boundary by exercising it is stronger than reading the directory alone, it proves the classes are enforced, not just declared.

Blue-Team angle: a guest that can issue a privileged command it should not have the class for is a misconfiguration or a compromise, the command working when it should be refused is the alarm.

Checkpoint: can you distinguish CP from CMS, read the directory and a guest's classes, explain the privilege model and why classes C and E are dangerous, run the class-creep audit, and apply Just Enough Authority? Those skills secure the hypervisor that every guest on a virtualized mainframe depends on.

z/VM is the mainframe's hypervisor, running many guests (z/OS and, increasingly, large numbers of Linux systems) on one machine. CP is the control program that virtualizes the hardware; CMS is the lightweight OS a guest runs inside its virtual machine. Guests are defined in the CP directory, and their power is set by privilege classes A through G: G is an ordinary user, while classes like C and E grant the ability to alter and examine hypervisor storage, reach into every other guest. The common weakness is class creep, guests holding more classes than their role needs, found by a creep audit and fixed by Just Enough Authority: reduce each guest to its baseline and split any all-class admin into narrower roles. Because CP is the trust boundary beneath every guest, guarding the classes and the directory is guarding the whole virtualized machine.

What comes next

z/VM hosts many operating systems; the next chapter covers one more of the mainframe's specialized worlds. z/TPF (Transaction Processing Facility) is the extreme end of the platform, an operating system built for the very highest transaction volumes on earth, running airline reservations and payment networks at scales measured in tens of thousands of transactions a second. It closes Part VI by showing how far the mainframe's specialization goes, and what security means when raw throughput is everything.

CHAPTER 33

z/TPF: Extreme Throughput

By the end of this chapter you will be able to:

Explain what z/TPF is and why it exists.

Read system status and understand I-streams and utilization.

Explain the ECB, z/TPF's ultra-light unit of work.

Use functional messages, the operator interface.

Explain loadsets and live code activation.

Describe z/TPF's security model and attack surface.

PART VI CLOSES AT THE extreme end of the platform. z/TPF (the Transaction Processing Facility) is an operating system built for one thing: the very highest transaction volumes on earth. It runs the world's airline reservation systems and the card and payment networks that authorize purchases, at scales measured in tens of thousands of transactions every second, with response times in milliseconds and uptime measured in decades. To do this it throws away almost every convenience of a general-purpose operating system in favor of raw speed. z/TPF looks and feels unlike anything in the earlier chapters (there is not even an online help facility) and it is worth understanding precisely because it shows how far the mainframe's specialization goes, and what security means when throughput is the entire point.

Built for throughput

z/TPF is stripped to the bone. Where z/OS is a rich, general environment, z/TPF is a lean, assembler-heavy engine tuned so that a single transaction costs as little as physically possible. The philosophy shows even in small things: asked for help, it replies that there isn't any, use the manual. The operator sees the system through a status message:

```
ZSTAT
CPU-B SS-BSS SSU-HPN
CSMP0097I 110449 SYSTEM STATE: NORM   ONLINE: YES
      I-STREAMS ACTIVE: 4   CPU UTIL: 07%
      ECBS DISPATCHED: 0   HIGH ECB: 004000
      LOADSETS: BASE, AVL1, CC01
*1821104*
```

z/TPF system status, state, active I-streams, utilization, ECBs, and loadsets.

Every line is dense with z/TPF vocabulary. SYSTEM STATE NORM means the system is running normally. I-STREAMS ACTIVE: 4 counts the parallel instruction streams (the processing engines working at once. CPU UTIL shows how hard they are working. The ECBS and HIGH ECB lines count the unit of work that defines z/TPF, and LOADSETS names

the code currently active. The line endings) CPU-B at the top, the *1821104* terminator at the bottom, are z/TPF's terse message framing. This is a system that says exactly what it must and nothing more.

The ECB: the unit of work

The heart of z/TPF is the ECB (the Entry Control Block. Every transaction that enters the system becomes an ECB: an extremely lightweight unit of work that is created, dispatched, does its job, and is released, often in well under a millisecond. Where z/OS runs work as address spaces and tasks) relatively heavy, relatively few, z/TPF runs enormous numbers of ECBs, creating and destroying them at a rate that lets one system handle tens of thousands of transactions a second. The status display's ECBS DISPATCHED and HIGH ECB numbers are how an operator gauges that flow. The ECB is why z/TPF is fast: it has stripped the unit of work down to almost nothing, so the overhead per transaction is almost nothing. Understanding z/TPF is largely understanding that everything is an ECB, and that the system's whole design serves making ECBs cheap.

Functional messages

Operators drive z/TPF through functional messages (short commands that all begin with Z, entered from a terminal identified by its LNIATA address, with the main operator console known as the CRAS. ZSTAT was one. There are functional messages to display and control every part of the system, and they are the entire operator interface) there is no rich console session, just these terse, powerful messages. The design mirrors the rest of z/TPF: a minimal, fast interface with nothing extra. For anyone learning the system, the functional messages are the vocabulary to master, because they are how you see and change what a running z/TPF is doing.

z/TPF term	What it is
ECB	Entry Control Block, one transaction's unit of work
I-stream	An instruction stream, a parallel processing engine
Functional message	A Z-command, the operator interface
LNIATA	The address identifying an operator terminal
CRAS	The main operator console
Loadset	A named set of application code, activated live

The core z/TPF vocabulary, the terms that make the system legible.

Loadsets: changing code without stopping

A system that must never stop still has to be updated, and z/TPF does this with loadsets. A loadset is a named package of application code that can be loaded and activated into the running system without an outage. The status display showed several active: BASE, AVL1, CC01. Activating one is a functional message, confirmed tersely:

ZTPLD BASE

CPU-B SS-BSS SSU-HPN

CLDR0050I LOADSET BASE ACCEPTED AND ACTIVATED

1821105

Activating a loadset, new code brought live into a running system without an outage.

Loadsets are how a system that authorizes millions of payments a day gets new code without ever going offline, and they are also a security control point of the first order. Whoever can activate a loadset can change the code that runs in a payment or reservation system, which is to say, whoever controls loadset activation controls the application. Loadset management is therefore among the most sensitive authorities on a z/TPF system, and unexpected loadset activity is exactly the kind of event a defender watches for: it is the z/TPF equivalent of an unauthorized code deployment straight into production.

The data: flights and cards

What z/TPF processes is as high-value as data gets. The records it handles are flight records (the availability and bookings behind an airline reservation system) and card records, the account and authorization data behind a payment network. A z/TPF system is often the engine that decides, in the few milliseconds a card terminal waits, whether a purchase is approved. That places z/TPF at the center of some of the most regulated and most targeted data in existence: payment card information under PCI rules, and the personal and travel data of millions of passengers. The security stakes are not abstract; a z/TPF system is precisely the kind of high-value target that motivates the whole discipline of this book.

Security in z/TPF

z/TPF secures itself differently from z/OS, and the differences follow from its design. There is no general RACF session for end users; instead, authority is attached to terminals and functional messages (the CRAS and privileged LNIATAs can issue powerful messages that an ordinary terminal cannot) and the applications enforce their own access control over the flight and card data. Because z/TPF is a closed, purpose-built system, much of its exposure comes not from interactive users but from its edges: the network gateways that feed it transactions from airlines, travel agencies, and payment processors. Those gateways, and the functional-message authority model, are where a z/TPF assessment concentrates.

Aspect	z/TPF	z/OS
Purpose	Extreme-volume transactions	General-purpose
Unit of work	ECB (ultra-light)	Address space / task
Operator interface	Functional messages (Z...)	Console (D..., etc.)
Security model	Terminal authority + gateways	RACF

z/TPF versus z/OS, the same platform, a radically different design and security model.

The lesson for a defender is to meet z/TPF on its own terms. The RACF-centric thinking of the earlier chapters does not map directly; instead, the questions become which terminals hold which authority, who can activate a loadset, and how the gateways that feed the

system authenticate what they send. Different questions, same principle (least privilege on powerful actions, control of who can change the running code, and tight authentication at every door) applied to the fastest transaction engine on the planet.

The mistake is to approach z/TPF as if it were z/OS with a strange command set. Its security does not live in RACF profiles but in terminal authority, functional-message privilege, loadset control, and gateway authentication. Assessing a z/TPF system means asking its own questions (who can activate code, which terminals are privileged, how transactions arrive) not looking for a SETROPTS that is not there.

Hands-On Labs

These labs explore z/TPF through its functional messages.

Lab 33.1

Read system status

Goal: display z/TPF status and interpret its state, I-streams, and utilization.

Background: ZSTAT is the operator's first look at a z/TPF system. Reading it orients you.

1. Display the system status.

ZSTAT

Check yourself: you can read SYSTEM STATE, the count of active I-streams, the CPU utilization, and the active loadsets, and explain what each tells you about the running system.

Why it matters: the status message is the operator's dashboard for a system with no rich console. Reading it fluently is basic z/TPF literacy.

Blue-Team angle: an unexpected loadset in the status, or a state other than NORM, is a fast signal that something has changed on a system that should be boringly steady.

Lab 33.2

Understand the ECB

Goal: explain the ECB as z/TPF's unit of work and why it enables extreme throughput.

Background: everything in z/TPF is an ECB. Understanding it is understanding the system.

1. From the status display, read the ECB figures and explain what an ECB is.

Check yourself: you can explain that each transaction becomes a lightweight ECB created and released in a fraction of a millisecond, and that this is why one z/TPF system can handle tens of thousands of transactions a second.

Why it matters: the ECB is the design idea behind z/TPF's speed. Everything else follows from making the unit of work almost free.

Blue-Team angle: ECB volume and dispatch rate are the performance baseline; a sharp, unexplained change can signal abnormal load (including an attack) on a system whose whole value is steady throughput.

Lab 33.3

Use functional messages

Goal: issue functional messages and understand the operator interface.

Background: functional messages are the whole operator interface. This lab practices them.

1. Issue a status functional message and read its terse response framing.

ZSTAT

Check yourself: you can read a functional-message response, including its CPU-B header and its terminator line, and explain that all begin with Z and are entered from an authorized terminal.

Why it matters: functional messages are how you see and control z/TPF. There is no other interface to learn.

Blue-Team angle: which terminals can issue which functional messages is the z/TPF authority model, a privileged message from an unexpected terminal is a red flag.

Lab 33.4

Examine loadsets

Goal: read the active loadsets and explain live code activation and its risk.

Background: loadsets are how code changes without an outage, and a first-order security control. This lab examines them.

1. From the status, read the active loadsets and describe what activating one does.

Check yourself: you can name the active loadsets and explain that activating one brings new application code live into a running system, so control over loadset activation is control over the application.

Why it matters: loadset activation is production code deployment on a system that never stops. Who can do it is a first-order security question.

Blue-Team angle: unexpected loadset activation is the z/TPF equivalent of an unauthorized production deployment, monitor for it and tightly control who can issue it.

Lab 33.5

Reason about z/TPF security

Goal: describe how z/TPF security differs from z/OS and where its attack surface lies.

Background: z/TPF must be assessed on its own terms. This lab frames the right questions.

1. List the z/TPF-specific security questions: terminal authority, loadset control, and gateway authentication.

Check yourself: you can explain that z/TPF secures itself through terminal and functional-message authority, loadset control, and gateway authentication rather than a general RACF session, and that its edges (the transaction gateways) are the main surface.

Why it matters: applying z/OS assumptions to z/TPF misses its real controls. Asking its own questions is how you assess it correctly.

Blue-Team angle: the high value of flight and card data makes z/TPF a prime target. Concentrate on the gateways that feed it and on who can activate code, the two places its throughput-first design exposes the most.

How a transaction flows

It helps to picture the path a single transaction takes, because it is the whole system in miniature. An input message arrives, z/TPF creates an ECB for it, dispatches that ECB onto one of the parallel I-streams, the ECB reads or updates a record and builds a response, and then it is released, all typically in well under a millisecond:

```
INPUT MESSAGE  (from gateway / terminal)
|
v   create ECB
[ ECB 004001 ] --dispatch--> I-STREAM 2
|   read FLIGHT/CARD record, apply logic
v   build response
RESPONSE OUT  -> ECB released
(elapsed: << 1 ms; multiply by tens of thousands / sec)
```

The z/TPF transaction path, create an ECB, dispatch to an I-stream, respond, release.

Multiply that path by tens of thousands per second across four active I-streams and you have z/TPF at work. The design is visible in the flow: nothing is heavier than it must be, the ECB exists only for the life of the transaction, and the I-streams run many at once. For a defender, the same flow marks the points that matter, the gateway where the message enters (authentication), the record it touches (data access), and the code that applies the logic (loadset control). Each is a place the throughput-first design concentrates both value and risk.

The loadset lifecycle

Changing code live follows a controlled sequence (load the code, then activate it) each step confirmed by a CLDR message, so a running payment or reservation system is updated without an outage and with a clear audit trail:

```
ZTPLD CC02          load a new loadset
CLDR0048I LOADSET CC02 LOADED - NOT YET ACTIVE
ZTPLD CC02 ACTIVATE
CLDR0050I LOADSET CC02 ACCEPTED AND ACTIVATED
ZSTAT
LOADSETS: BASE, AVL1, CC01, CC02
```

The loadset lifecycle, load, activate, and confirm, live in a running system.

Each CLDR message is a record of a code change to a system that never stops, which makes the loadset log one of the most security-relevant trails on a z/TPF system. A load followed by an activation is a production deployment; an activation no one authorized is unauthorized code running in a payment network. Reviewing loadset activity, and controlling who may issue ZTPLD, is the z/TPF counterpart of the change-control and code-promotion discipline from the sysprog-tooling chapter, applied where the stakes are highest.

Lab 33.6

Trace a transaction and a code change

Goal: describe the ECB path of a transaction and the load-then-activate lifecycle of a loadset, identifying the security-relevant points in each.

Background: the transaction flow and the loadset lifecycle are the two things that most define z/TPF operation and its security. This lab connects both to their control points.

1. Describe how an input message becomes an ECB, is dispatched, responds, and is released.
2. Describe the load-then-activate loadset sequence and confirm the new loadset in ZSTAT.

ZSTAT

Check yourself: you can trace a transaction from gateway to released ECB and name the gateway, record, and loadset as its control points; and you can describe loading and activating a loadset and confirm the active set changed.

Why it matters: the transaction path shows where value and risk concentrate, and the loadset lifecycle shows how the running code is changed. Both are where z/TPF security lives.

Blue-Team angle: watch two things above all, gateway authentication for the messages coming in, and loadset activation for the code going live. They are the entrance and the deployment door of a system whose data is among the most valuable anywhere.

Checkpoint: can you read z/TPF status, explain the ECB and why it enables extreme throughput, use functional messages, describe loadsets and live activation, and frame z/TPF security in its own terms? Those skills let you assess the fastest transaction engine on the platform, and close Part VI's tour of the subsystems the mainframe was built to run.

z/TPF is the mainframe's extreme-throughput operating system, engineered for the highest transaction volumes on earth (airline reservations and payment networks at tens of thousands of transactions a second. It achieves this by stripping the unit of work to the ECB, an ultra-lightweight entity created and released in a fraction of a millisecond, dispatched across parallel I-streams. Operators drive it through terse functional messages (Z-commands) from authorized terminals, and code is changed live through loadsets) a control point as sensitive as production deployment. Its data, flight and card records, is among the most valuable and regulated anywhere. z/TPF secures itself not through RACF but through terminal and functional-message authority, loadset control, and gateway authentication, the same least-privilege principles as the rest of the platform, asked in the system's own language.

What comes next

Part VI has toured what the mainframe runs (CICS, banking applications, Db2, IMS, z/VM, and now z/TPF. Every one of them is reached, ultimately, over the network. Part VII turns to that connectivity: how data and jobs move to and from the mainframe. It opens with FTP and TCP/IP) the file-transfer and networking layer, including the mainframe-specific twists, like FTP that can submit jobs to the internal reader, that make it both powerful and a classic exfiltration and access path.

Part VII

Connectivity & Programming

A mainframe rarely stands alone. Part VII covers the ways data and jobs move on and off the platform, FTP, NJE, terminal transfer, and the modern web and API tier, and the languages that drive it all.

CHAPTER 34

FTP & TCP/IP

By the end of this chapter you will be able to:

Describe TCP/IP on the mainframe and its key ports.

Explain why mainframe FTP is far more than file transfer.

Use FTP's FILE, JES, and SQL modes and transfer types.

Explain how SITE FILETYPE=JES lets FTP submit and retrieve jobs.

Identify FTP's exposures: cleartext, anonymous, and code execution.

Harden FTP: TLS, access control, and monitoring.

EVERYTHING IN PART VI IS reached over the network, and Part VII is about that connectivity (how data and jobs move to and from the mainframe. It begins with the oldest and most revealing example: FTP. On most systems FTP is a simple file-transfer tool, and treating mainframe FTP the same way is a serious mistake. On z/OS, FTP can read and write data sets directly, run Db2 queries, and) most importantly, submit jobs to the internal reader, which means FTP is not merely a way to move files but a way to run code and retrieve its output over the network. This chapter covers TCP/IP on the mainframe and then FTP in depth, with a clear defensive purpose: understand how much power mainframe FTP carries so you can detect its misuse and lock it down.

TCP/IP on the mainframe

The mainframe is a full TCP/IP host. Its Communications Server provides the network stack, and its services listen on ports exactly as any server does. You saw the port map in the console chapter; it is worth recalling, because it is the mainframe's network-facing attack surface:

```
EZZ2500I  ACTIVE PORTS
          PORT  SERVICE          STATUS
          21    FTPD (FTP)        LISTEN
          23    TN3270            LISTEN
          50000  DB2 DDF           LISTEN
          8443   GIBSON DASHBOARD LISTEN
```

The mainframe's listening ports, each an entry point a defender must account for.

Each listening port is a door. FTP on 21, TN3270 on 23, Db2's DDF on 50000, the dashboard on 8443, every one is a service that accepts connections from the network, and every one is a place authentication and access control must hold. A first step in assessing any mainframe is exactly this: what is listening, and what does each service allow? FTP is first on the list, and it is the one whose power is most often underestimated.

FTP: more than file transfer

Connecting to mainframe FTP looks ordinary, a banner, then a login:

```
220-GIBSON FTP service at MVSC
220 Connection will close if idle.
USER ibmuser
331 Send password please.
PASS *****
230 IBMUSER is logged on. Working directory is "IBMUSER.".
```

A mainframe FTP login, ordinary-looking, but the working directory is a data-set prefix.

The first surprise is in the last line: the working directory is a data-set high-level qualifier, not a Unix path. Mainframe FTP browses and transfers data sets. It has transfer types (ASCII for text, which translates between EBCDIC and ASCII; BINARY for exact bytes; and EBCDIC for native transfers) and the choice matters, because getting it wrong corrupts data in the character-set translation. But the deeper surprise is that FTP has modes, selected with SITE FILETYPE=, that change what it talks to entirely:

SITE FILETYPE=	What FTP then accesses
SEQ (default)	Data sets, read and write them as files
JES	The JES job queue, submit jobs and retrieve output
SQL	Db2, run queries and fetch results

FTP's FILETYPE modes, the same protocol reaching data sets, jobs, or the database.

That one FTP connection can, by switching FILETYPE, reach the file system, the batch scheduler, or the database is what makes mainframe FTP so powerful, and so dangerous when it is not controlled. The JES mode is the one that turns FTP from a data channel into an execution channel.

FILETYPE=JES: FTP that submits jobs

With SITE FILETYPE=JES set, a file uploaded by FTP is not stored, it is submitted to JES as a job. Upload a piece of JCL, and the mainframe runs it:

```
SITE FILETYPE=JES
PUT testjob.jcl
250-It is known to JES as JOB00001
250 Transfer completed successfully.
```

FILETYPE=JES, an uploaded file becomes a submitted job, run by the mainframe.

The job now runs with the authority of the FTP user, and its output can be listed and retrieved over the same FTP connection:

```
JOBID      JOBNAME    OWNER      STATUS     RC
JOB00001   TESTJOB    IBMUSER    OUTPUT     0000
```

Listing jobs in JES mode, FTP can retrieve the output of the jobs it submits.

Stop and consider what this means. Anyone with FTP credentials and FILETYPE=JES can submit arbitrary JCL (which is to say, run arbitrary work) on the mainframe, and read the results, entirely over the network, without ever opening a terminal session. FTP becomes remote code execution. This is a legitimate, useful feature (it is how automated systems submit batch work) but it is also one of the most important exposures on a mainframe, because it collapses “has FTP access” into “can run code.” Understanding this is the whole reason a defender treats mainframe FTP with far more caution than FTP anywhere else.

The exposure

Put the pieces together and FTP’s risk profile is substantial. It spans credentials, access, and execution:

Exposure	Why it matters
Cleartext credentials	Plain FTP sends the password unencrypted, sniffable
Anonymous access	If enabled, no credentials needed at all
Data-set read/write	FTP reaches data sets, a path to read or alter them
FILETYPE=JES	Submitting jobs is remote code execution
Bulk transfer	Large outbound transfers are an exfiltration channel

FTP’s exposures, from sniffable passwords to remote code execution and exfiltration.

Each row is a real path. Plain FTP transmits the password in the clear, so anyone watching the network learns it. If anonymous access is enabled, even that is unnecessary. Once in, FTP can read data sets (a direct route to sensitive data) and write them. FILETYPE=JES turns access into execution. And FTP is a natural exfiltration channel: a large outbound transfer is exactly how data leaves. These are not exotic; they are the reasons FTP appears in real mainframe incidents, and every one has a defense.

Securing FTP

The defenses map directly onto the exposures, and together they turn FTP from a liability into a controlled service:

Control	What it addresses
Require TLS (FTPS)	Encrypts credentials and data on the wire
Disable anonymous access	Forces authentication for every session
Restrict FILETYPE=JES	Limits who can submit jobs via FTP
RACF data-set protection	FTP obeys the same profiles as any access
SMF logging and monitoring	Records transfers and submissions for review

Hardening FTP, each control closes one of the exposures above.

The single most important control is encryption: requiring TLS closes the cleartext-credential exposure that makes everything else easier for an attacker. Disabling anonymous access forces authentication. Restricting FILETYPE=JES to the automation accounts that genuinely need it keeps FTP from being a general-purpose execution channel. Essentially, FTP access to data sets is still governed by RACF (the same profiles that protect a data set from a TSO user protect it from an FTP user) so tight data-set

profiles matter here too. And because FTP is logged to SMF, monitoring gives the detection layer: unusual transfers, especially large outbound ones, and unexpected job submissions are exactly the signals a defender watches for. FTP is safe when it is encrypted, authenticated, access-controlled, and watched.

The defining FTP mistake is treating mainframe FTP like FTP anywhere else, a harmless file-mover. It is not: it reaches data sets, runs Db2 queries, and through FILETYPE=JES submits jobs, so FTP access can mean code execution and data access, not just file transfer. Compounded by cleartext credentials, that makes plain FTP one of the highest-value exposures on a mainframe. Require TLS, control the modes, and monitor it.

Hands-On Labs

These labs explore mainframe FTP and its controls. The job-submission power is studied to understand and defend it.

Lab 34.1

Map the listening ports

Goal: display the mainframe's active ports and identify each service.

Background: the listening ports are the network attack surface. Mapping them is the first step of any assessment.

1. Display the active TCP/IP ports.

`D TCPIP,,NETSTAT,PORTLIST`

Check yourself: you can name the services listening (FTP on 21, TN3270 on 23, Db2 DDF on 50000, the dashboard on 8443) and explain that each is a door to secure.

Why it matters: you cannot defend a service you do not know is listening. The port map is the inventory of the network surface.

Blue-Team angle: an unexpected listening port is an unexpected service, and possibly an attacker's. Compare the port list against an approved baseline.

Lab 34.2

Transfer with the right type

Goal: connect with FTP and use ASCII versus BINARY correctly.

Background: mainframe FTP transfers data sets, and the wrong transfer type corrupts data in EBCDIC/ASCII translation. This lab practices choosing correctly.

1. Log in, set the transfer type, and move a text and a binary file.

ASCII

BINARY

Check yourself: you can explain that ASCII translates between EBCDIC and ASCII for text and BINARY moves exact bytes, and that the working directory is a data-set prefix, not a Unix path.

Why it matters: choosing the transfer type is basic mainframe FTP literacy, and getting it wrong silently corrupts data.

Blue-Team angle: FTP reaching data sets means RACF data-set profiles apply to FTP too, confirm sensitive data sets are protected against FTP access, not just TSO.

Lab 34.3

Submit a job through FTP

Goal: use FILETYPE=JES to submit a job and retrieve its output, and understand the power this represents.

Background: FILETYPE=JES turns FTP into remote code execution. This lab studies that capability defensively.

1. Set JES mode, upload a small piece of JCL, and list the resulting job.

```
SITE FILETYPE=JES
```

```
PUT testjob.jcl
```

Check yourself: the upload is submitted as a job (known to JES as a JOBID), runs with your authority, and its output is retrievable over FTP. You can explain that this makes FTP access equivalent to code execution.

Why it matters: “has FTP” becoming “can run code” is one of the most important facts about mainframe security. Seeing it makes the risk concrete.

Blue-Team angle: restrict FILETYPE=JES to the automation accounts that need it, and alert on job submissions via FTP from anywhere else, they are execution events, not transfers.

Lab 34.4

Assess the FTP exposure

Goal: enumerate FTP’s exposures for a given configuration.

Background: assessing FTP means checking encryption, anonymous access, modes, and data-set protection together. This lab runs that assessment.

1. For an FTP service, determine whether TLS is required, whether anonymous access is enabled, who can use FILETYPE=JES, and whether sensitive data sets are protected.

Check yourself: you can list the exposures present (cleartext credentials without TLS, anonymous access if enabled, broad FILETYPE=JES, unprotected data sets) and rate them by severity.

Why it matters: a structured assessment turns “FTP is risky” into a concrete, prioritized findings list. That is the deliverable of a review.

Blue-Team angle: plain FTP without TLS is typically the highest finding, the sniffable password undermines every other control. Fix encryption first.

Lab 34.5

Harden FTP

Goal: state the controls that turn FTP into a safe, monitored service.

Background: each exposure has a remediation. This lab plans the hardening.

1. For each exposure found, state the control that closes it.

Check yourself: you can map cleartext to TLS, anonymous access to disabling it, FILETYPE=JES to restriction, data-set access to RACF profiles, and exfiltration to SMF monitoring, a complete hardening plan.

Why it matters: FTP is safe when encrypted, authenticated, access-controlled, and watched. The plan is the same every time.

Blue-Team angle: after hardening, validate, confirm TLS is enforced, anonymous is off, and monitoring fires on a test transfer. Controls unverified are controls unproven.

The full round trip: submit, run, retrieve

The execution channel is complete only because FTP can also read back what the job produced. Retrieving the submitted job's output over the same connection returns its full spool, messages, JCL, and any data it wrote:

```
GET JOB00001
----- JESMSG LG -----
$HASP100 T ON READER
IRR010I USERID IBMUSER IS ASSIGNED TO THIS JOB.
$HASP373 T STARTED - INIT 1 - CLASS A
IEF403I T - STARTED - TIME=11.32.39
----- JESJCL -----
```

Retrieving job output over FTP, the round trip is complete: submit, run, read results.

The IRR010I line is the telling one: the job ran under IBMUSER's identity, with all of that user's authority, and its output came straight back over FTP. That is the whole remote-execution loop in one exchange (upload code, the mainframe runs it as you, and you read the results) with no terminal, no interactive session, nothing but an FTP client. A defender reading this recognizes it immediately as the shape of a remote-execution channel, and the retrieval half is why exfiltration and code execution are the same exposure here: whatever a submitted job can read, FTP can carry away.

FILETYPE=SQL: FTP queries the database

The third mode points FTP at Db2. With FILETYPE=SQL, an uploaded query is run and its result set returned as a file:

```
SITE FILETYPE=SQL
PUT query.sql

NAME                CREATOR    TYPE
SYSTABLES           SYSIBM     T
SYSUSERAUTH         SYSIBM     V
SYSINDEXES          SYSIBM     T
```


250 Query complete; results in JOBSHOW_SYSTABLES.txt

FILETYPE=SQL, FTP runs a Db2 query and returns the result set as a file.

Now the same FTP connection reaches the database directly, running SQL and pulling back results. Everything from the Db2 chapter applies (the authorization tables an attacker wants to read, the injection risk if the query is built from input) but reached over FTP rather than SPUFI. Three modes, one connection: FTP can touch the file system, the batch scheduler, and the database, which is exactly why controlling and monitoring it matters so much. A single set of FTP credentials, unencrypted on the wire, is a key to all three.

Lab 34.6

Trace the round trip and the SQL mode

Goal: complete the submit-run-retrieve loop and run a query through FTP, and explain why retrieval makes execution and exfiltration one exposure.

Background: FTP's danger is the full loop (code in, results out) across all three modes. This lab exercises it defensively.

1. In JES mode, submit a job and then retrieve its output.

GET JOB00001

2. In SQL mode, run a query and read the returned result file.

SITE FILETYPE=SQL

Check yourself: you can show that a submitted job runs under your identity and its output returns over FTP, and that FILETYPE=SQL runs a query and returns results, so one FTP session reaches files, jobs, and the database.

Why it matters: seeing all three modes in one session makes the scope of FTP access concrete, it is not file transfer, it is broad remote access.

Blue-Team angle: monitor FTP for job submissions and SQL activity, not just transfers, and require TLS so the credentials that unlock all three modes are never exposed on the wire.

Checkpoint: can you map the mainframe's listening ports, use FTP's modes and transfer types, explain how FILETYPE=JES turns FTP into remote code execution, enumerate FTP's exposures, and harden it with TLS, access control, and monitoring? Those skills turn the most underestimated service on the mainframe into a controlled one.

The mainframe is a full TCP/IP host with services listening on well-known ports, and FTP is the one whose power is most underestimated. Mainframe FTP transfers data sets directly, with transfer types (ASCII, BINARY, EBCDIC) that must match the data, and it has modes selected by SITE FILETYPE= that reach the file system, Db2, or (through FILETYPE=JES) the job queue. That last mode lets FTP submit jobs and retrieve their output, making FTP access equivalent to remote code execution. Combined with cleartext credentials, optional anonymous access, data-set reach, and its natural fit as an exfiltration channel, plain FTP is among the highest exposures on a mainframe. The defenses are direct: require TLS, disable anonymous access, restrict FILETYPE=JES, rely on RACF for data-set protection, and monitor via SMF, encrypted, authenticated, controlled, and watched.

What comes next

FTP moves data and jobs between the mainframe and the wider network. The next chapter covers how mainframes move work between each other: NJE, Network Job Entry, which links multiple systems into a network that can pass jobs and output across nodes. Gibson models several NJE nodes, and the trust between them (how one node accepts work from another) is its own security boundary, and its own attack path when that trust is misplaced.

CHAPTER 35

NJE: Network Job Entry

By the end of this chapter you will be able to:

Explain what NJE is and how nodes pass work between each other.

Read the node definitions and their AUTH trust levels.

Explain the difference between clear and TLS NJE transport.

Describe cross-node job execution and why it is a pivot path.

Explain NMR message spoofing between nodes.

Secure NJE with least trust, TLS, and the RACF NODES class.

FTP MOVES WORK BETWEEN THE mainframe and the outside world. NJE (Network Job Entry) moves work between mainframes. It links systems into a network in which nodes pass jobs, output, commands, and messages to one another, so a job submitted on one system can run on another and return its results. This is enormously useful in enterprises that run many mainframes, and it is built on trust: each node decides how much to trust the nodes it connects to. That trust is the whole security story of NJE. Where it is well-scoped, NJE is a controlled way to share work; where it is too broad or unauthenticated, it becomes a lateral-movement path between systems and a channel for spoofed commands. Gibson models a small NJE network (the nodes GIBSON, HAL, and ORAC) and this chapter uses it to make the trust boundaries, and their abuse, concrete.

The NJE network

An NJE network is a set of nodes connected by lines. Each node is a JES system with a name and a number; each line is a connection to another node, today usually over TCP/IP. NJE is store-and-forward: a node accepts work addressed to it, and forwards work addressed elsewhere. Displaying the network shows the nodes and what each is trusted to do:

```
$D NODE
NODE(GIBSON)  NUMBER(001) AUTH=(JOB,NET,DEVICE,SYSTEM) SECURE=OPTIONAL
NODE(HAL)     NUMBER(002) AUTH=(JOB,NET) SECURE=TLS-LABELLED
NODE(ORAC)    NUMBER(003) AUTH=(JOB) SECURE=CLEAR
```

The NJE nodes, each with a trust level (AUTH) and a transport security setting.

Three nodes, three very different trust postures. GIBSON, the home node, is authorized for everything. HAL is trusted for jobs and network commands and connects over labelled TLS. ORAC is trusted only to send jobs, and connects in the clear. Read the lines that carry them, and the topology is complete:

```
$D LINE
LINE1 UNIT=TCPIP STATUS=ACTIVE NODE=HAL
```

```
LINE2 UNIT=TCPIP STATUS=ACTIVE NODE=ORAC
```

The NJE lines, active TCP/IP connections from GIBSON to HAL and ORAC.

NJE listens on well-known ports (175 for clear connections and 2252 for TLS) and, like a cautious server, says nothing until a connecting node sends its OPEN record identifying itself. That identification, and the trust attached to it, is where NJE security lives.

The trust model: AUTH levels

The AUTH setting on each node is the heart of NJE trust. It declares what kind of work this system will accept from that node, and the levels escalate in power:

AUTH level	What the node may send
JOB	Jobs to be run, the basic NJE function
NET	Network (JES) commands, control the job network
DEVICE	Device-control commands
SYSTEM	System operator commands, the most powerful

NJE AUTH levels, from accepting jobs to accepting system operator commands.

The escalation matters enormously. A node trusted only for JOB can submit work (significant already, since a job can do anything its identity allows. A node trusted for SYSTEM can issue operator commands on your system as if from your own console. GIBSON trusting itself for everything is expected; the question in any real network is whether the other nodes are trusted for more than they need. A node trusted for SYSTEM that only ever needs to send jobs is a standing over-trust) the NJE equivalent of an unnecessary privileged grant, and exactly the kind of finding a review looks for.

Cross-node job execution

The core NJE function is running a job submitted elsewhere. A job routed from ORAC to execute at GIBSON arrives, runs, and reports, across the network boundary:

```
JOB41668 $HASP122 X(JOB41668 FROM ORAC) RECEIVED AT GIBSON
JOB41668 IEF403I X - STARTED - TIME=10.57.32
JOB41668 IEF404I X - ENDED - TIME=10.57.34
```

A cross-node job, submitted at ORAC, received and run at GIBSON.

This is NJE doing its job: a job from ORAC ran on GIBSON. It is also, viewed from the other side, lateral movement. If an attacker controls or can submit through ORAC, and GIBSON trusts ORAC to send jobs, then the attacker can run work on GIBSON, a pivot from one system to another that rides entirely on the NJE trust relationship. The identity the job runs under on GIBSON is decided by how GIBSON maps inbound work from ORAC, which is precisely what the RACF NODES class governs. Cross-node execution is the feature and the risk in one: sharing work between systems is exactly the capability an attacker wants for moving between them.

NMR injection: spoofing between nodes

Beyond jobs, NJE carries messages and commands between nodes as nodal message records, NMRs. If a receiving node does not properly authenticate the sender, a crafted NMR can be made to appear as though it came from a trusted node, and a command inside it is accepted on that basis:

```
NMR received: SRC=GIBSON (claimed)  CMD=$D JOBQ
$HASP000 OK
```

An NMR whose source is spoofed as a trusted node, the command is accepted.

The command was accepted because it appeared to come from GIBSON, a trusted source (even though the sender only claimed to be GIBSON. This is the store-and-forward trust problem in sharp form: trust attached to a node name is only as good as the assurance that the sender really is that node. On a clear (unencrypted, unauthenticated) line, that assurance is weak, and an attacker positioned on the network can inject NMRs that impersonate a trusted node to issue commands. The defense is to make node identity strong) which is exactly what TLS transport and the NODES class provide.

Securing NJE

NJE security comes down to trusting less, authenticating better, and watching the boundary. The threats and their remediations line up cleanly:

Threat	Remediation
Over-trusted node (broad AUTH)	Reduce AUTH to what the node needs
Cross-node execution as a pivot	RACF NODES class maps inbound work tightly
NMR command spoofing	Require TLS; authenticate node identity
Cleartext transport (port 175)	Require TLS transport (port 2252)
Unmonitored inbound work	Log and alert on cross-node jobs and commands

NJE threats and remediations, least trust, strong identity, and monitoring.

The central control is the RACF NODES class, which governs how inbound work from each node is authorized and what identity it runs under (the mechanism that stops a job from ORAC running as a powerful user on GIBSON. Alongside it, the AUTH levels should be trimmed to each node's real need, TLS transport (port 2252) should replace clear connections so node identity is cryptographically assured and NMRs cannot be spoofed, and cross-node jobs and commands should be logged and monitored. NJE is safe when each node is trusted for exactly what it needs, authenticated so it cannot be impersonated, and watched at the boundary) the same least-trust principle the whole book returns to, applied between systems.

The NJE mistake is trusting a node name without assuring the node's identity, and trusting it for more than it needs. A node authorized for SYSTEM commands over a clear line is a system whose operator console an attacker on the network can reach by impersonation. Trim AUTH to need, require TLS, and use the NODES class to control inbound work, name-based trust without authentication is not trust, it is an opening.

Hands-On Labs

These labs explore the NJE network and its trust boundaries. The trust-abuse paths are studied to detect and close them.

Lab 35.1

Map the NJE network

Goal: display the nodes and lines and describe the network.

Background: knowing the nodes and how they connect is the start of any NJE review.

1. Display the nodes and the lines.

```
$D NODE
```

```
$D LINE
```

Check yourself: you can name the nodes (GIBSON, HAL, ORAC), their numbers, and the active lines connecting them, and explain that NJE is store-and-forward.

Why it matters: the topology is the map of trust relationships. You cannot assess trust you have not mapped.

Blue-Team angle: an unexpected node or line is an unexpected trust relationship, compare the NJE definitions against an approved topology.

Lab 35.2

Read the trust levels

Goal: read each node's AUTH and identify any over-trust.

Background: AUTH is the trust model. Reading it finds nodes trusted for more than they need.

1. For each node, read its AUTH and its transport security.

Check yourself: you can explain that JOB, NET, DEVICE, and SYSTEM escalate in power, and identify that a node trusted for SYSTEM or connecting in the clear warrants scrutiny.

Why it matters: over-trust between nodes is the NJE equivalent of an over-broad grant. Reading AUTH is how you find it.

Blue-Team angle: a node that only sends jobs should be AUTH=(JOB) only. Anything more is unnecessary trust, and ORAC connecting CLEAR is a transport finding.

Lab 35.3

Observe cross-node execution

Goal: observe a job submitted at one node running at another and explain the pivot risk.

Background: cross-node execution is NJE's core function and a lateral-movement path. This lab studies it defensively.

1. Observe a job routed from ORAC executing at GIBSON and note the receiving messages.

Check yourself: you can explain that a job from ORAC ran on GIBSON because GIBSON trusts ORAC to send jobs, and that this is a pivot path if ORAC is compromised.

Why it matters: the same feature that shares work between systems moves an attacker between them. The NODES class decides what identity that work runs as.

Blue-Team angle: use the RACF NODES class to map inbound work to a constrained identity, never a powerful one, and monitor cross-node job receipts.

Lab 35.4

Understand NMR spoofing

Goal: explain how a spoofed NMR gets a command accepted as if from a trusted node.

Background: name-based trust without authentication can be impersonated. This lab studies the spoof defensively.

1. Examine an NMR whose claimed source is a trusted node and note that the command is accepted.

Check yourself: you can explain that the command was accepted because it appeared to come from a trusted node, and that on a clear line the sender's identity is not assured, so it can be spoofed.

Why it matters: trust attached to a name is only as strong as the assurance of the name. Weak assurance turns trust into an opening.

Blue-Team angle: TLS transport gives cryptographic node identity, closing the spoof. A clear NJE line into a trusting node is a high-priority finding.

Lab 35.5

Secure the trust boundary

Goal: state the controls that make NJE least-trust, authenticated, and monitored.

Background: each NJE threat has a remediation. This lab plans the hardening.

1. For over-trust, spoofing, cleartext, and unmonitored work, state the control that addresses each.

Check yourself: you can map broad AUTH to trimming it, spoofing and cleartext to requiring TLS, inbound work to the NODES class, and blind trust to monitoring, a complete NJE hardening plan.

Why it matters: NJE is safe when nodes are trusted for exactly what they need, cannot be impersonated, and are watched. The plan is consistent every time.

Blue-Team angle: the NODES class plus TLS is the backbone, one controls what inbound work may do, the other assures who sent it. Validate both after changes.

The definition and the sockets

A node's own NJE definition names it to the network and lists the sockets it listens on.

Reading it shows both the identity GIBSON presents and the ports it accepts connections on:

```
NJEDEF  NODENUM=001  OWNNAME=GIBSON  NETSERV=GIBNJE
SOCKET(GIBSOCK) PORT=175  STATUS=ACTIVE
SOCKET(GIBTLS)  PORT=2252 STATUS=ACTIVE TLS=YES
```

The NJE definition and sockets, the clear port 175 and the TLS port 2252 both listening.

Both sockets are active: the clear port 175 and the TLS port 2252. That is itself worth a note, because as long as the clear port accepts connections, a peer (or an attacker) can choose to connect without encryption, and node passwords and NMRs cross the wire in the clear. A hardened configuration disables or firewalls the clear socket and requires the TLS one, so there is no unencrypted path to fall back to. An active clear NJE socket is the transport-layer finding that underlies the spoofing risk.

The OPEN handshake and node authentication

When a node connects, it sends an OPEN record naming itself and offering its node password; the receiver answers ACK or NAK. A correct password is accepted:

```
OPEN OHOST=HAL RHOST=GIBSON  (password HAL123)
OPEN OHOST=HAL RHOST=GIBSON ACK NODE=002
```

The NJE OPEN handshake, a node authenticates with its node password and is acknowledged.

A wrong password is refused with a NAK and a reason code:

```
OPEN OHOST=HAL RHOST=GIBSON  (password WRONGPW)
NAK REASON=0x08 INVALID NODE PASSWORD
```

A rejected handshake, an invalid node password is refused with a NAK.

The node password authenticates the OPEN, which is the basic identity check. But two weaknesses follow on a clear line. First, the password itself crosses the wire unencrypted, so it can be captured and replayed. Second, the very difference between ACK and NAK lets an attacker probe node names and passwords, a valid node draws a different response than an invalid one, which is exactly how a node-brute recon script enumerates the network. TLS transport addresses both: it hides the handshake from capture and binds identity to a certificate rather than a guessable shared password. This is the concrete reason the clear socket is a finding and TLS is the remediation.

Lab 35.6

Inspect the sockets and handshake

Goal: read the NJE sockets and the OPEN handshake, and explain why a clear socket and password-only authentication are weaknesses.

Background: node identity is asserted in the OPEN handshake and carried over a socket that may or may not be encrypted. This lab inspects both.

1. Display the NJE definition and sockets, noting which ports are active.

```
$D SOCKET
```


2. Examine an accepted and a rejected OPEN handshake and note the ACK/NAK difference.

Check yourself: you can explain that the clear socket allows unencrypted connections exposing node passwords, and that the ACK/NAK difference both authenticates a peer and leaks whether a node name and password are valid, usable for enumeration.

Why it matters: the socket and handshake are where node identity is established. Weak transport there undermines every trust decision above it.

Blue-Team angle: disable or firewall the clear socket, require TLS, and treat repeated NAKs from one source as node-name enumeration, an early recon signal against the NJE network.

The NODES class: mapping inbound work

The RACF NODES class is what decides the identity that inbound work runs under, the control that keeps a job from ORAC from executing as a powerful user on GIBSON. A NODES profile maps work arriving from a node to a constrained local identity:

```
RDEFINE NODES ORAC.USERJ.* UACC(NONE)
  APPLDATA('NJEUSER')      <- inbound jobs run as NJEUSER
SETROPTS CLASSACT(NODES) RACLIST(NODES)
ICH408I job from ORAC mapped to NJEUSER (least privilege)
```

The RACF NODES class, inbound work from a node is mapped to a constrained identity.

With this in place, a job that arrives from ORAC does not run as whatever user it claims; it runs as NJEUSER, a deliberately low-privilege identity, so even a job from a compromised peer is boxed into what that identity may do. This is the decisive control on cross-node execution: the AUTH level decides whether a node may send jobs at all, and the NODES class decides how little those jobs are trusted once they arrive. Together they turn NJE from open trust into scoped, least-privilege sharing, the difference between a convenience and a pivot path.

Lab 35.7

Constrain inbound work with NODES

Goal: use the RACF NODES class to map inbound work from a node to a constrained identity, and explain why this closes the pivot.

Background: cross-node execution is only dangerous if inbound work runs with real privilege. The NODES class removes that.

1. Define a NODES profile mapping inbound jobs from a peer node to a low-privilege identity, and activate the class.

```
RDEFINE NODES ORAC.USERJ.* APPLDATA('NJEUSER')
```

Check yourself: you can explain that inbound jobs from ORAC now run as NJEUSER rather than any claimed identity, so a job from a compromised peer is confined to a low-privilege account.

Why it matters: the NODES class is what makes cross-node execution safe. Without it, inbound work can run with far more authority than a peer should be able to invoke.

Blue-Team angle: map every inbound node to the least identity its legitimate work needs, and audit NODES profiles alongside AUTH levels, the two together define how much a peer is really trusted.

Checkpoint: can you map an NJE network, read the AUTH trust levels and spot over-trust, explain cross-node execution as a pivot and NMR spoofing as impersonation, and secure the boundary with least trust, TLS, and the NODES class? Those skills protect the trust relationships that link mainframes to one another.

NJE links mainframes into a store-and-forward network that passes jobs, output, commands, and messages between nodes. Each node carries an AUTH trust level (JOB, NET, DEVICE, SYSTEM, escalating in power) and a transport setting from clear to TLS. The core function, running a job submitted at another node, is also a lateral-movement path: a job from ORAC runs on GIBSON because GIBSON trusts ORAC, with the identity governed by the RACF NODES class. And because trust is attached to node names, a node whose identity is not cryptographically assured can be impersonated (a spoofed NMR gets a command accepted as if from a trusted source. Securing NJE means trusting each node for exactly what it needs, requiring TLS so identity cannot be spoofed, mapping inbound work tightly through the NODES class, and monitoring the boundary) least trust between systems, the same principle applied one level up.

What comes next

FTP and NJE move whole files and jobs. The next chapter covers a different kind of transfer, one that rides inside the 3270 terminal session itself: IND\$FILE, the file-transfer mechanism built into the terminal protocol. It has its own quirks (ASCII versus binary, record formats, the CRLF handling that trips people up) and its own security considerations, closing out the file-movement chapters before Part VII turns to the web and API tier.

CHAPTER 36

IND\$FILE: Terminal File Transfer

By the end of this chapter you will be able to:

Explain how IND\$FILE transfers files inside a 3270 session.

Use the IND\$FILE GET and PUT commands.

Choose ASCII, CRLF, or binary correctly for a transfer.

Recognize why the wrong transfer type corrupts data.

Read the messages of a transfer in progress.

Assess IND\$FILE's exposure and the controls that apply.

FTP AND NJE ARE SEPARATE network services with their own ports. IND\$FILE is different in a way that matters for both use and security: it rides inside the 3270 terminal session itself. When you are logged on at a terminal, IND\$FILE moves files between the host and your workstation through the very same 3270 datastream that carries your screens, using file-transfer support built into the terminal emulator. There is no extra port and no separate login, the transfer travels the terminal channel you are already using. It is the classic way to get a file on or off the mainframe from a PC, and it is notorious for one thing: the ASCII-versus-binary and line-ending choices that quietly corrupt files when they are wrong. This chapter covers how IND\$FILE works, how to drive it, the quirks, and the security angle of a transfer that hides inside an ordinary session.

Transfer inside the terminal session

IND\$FILE is a cooperating pair. On the host, IND\$FILE is a program, invoked from TSO or ISPF. On the workstation, the terminal emulator has built-in file-transfer support. When a transfer starts, the two cooperate over the 3270 datastream: the emulator and the host program exchange the file in a series of buffers carried as terminal data, in what is called CUT mode. The user sees an ordinary terminal session; underneath, file bytes are flowing through it. The consequence worth holding onto is that IND\$FILE uses the authority and the channel of the logged-on session, it is not a new connection with its own login, but an activity within one that already exists. That shapes both how it is controlled and how visible it is.

The IND\$FILE command

A transfer is driven by the IND\$FILE command, usually issued through the emulator's transfer dialog, which sends it to the host. The direction is set by GET or PUT: GET brings a data set down to the workstation, PUT sends a workstation file up to a data set.

```
IND$FILE GET IBMUSER.SOURCE.COBOL ASCII CRLF
TRANS03 File transfer started.
```

TRANS15 Transfer complete: 4,096 bytes in 2 seconds.

IND\$FILE transfer completed successfully.

An IND\$FILE GET, bringing a data set down to the workstation as text.

The command names the host data set, the direction, and the options that control translation. A PUT reverses the direction, sending a local file up into a data set:

IND\$FILE PUT IBMUSER.UPLOAD.DATA BINARY

TRANS03 File transfer started.

TRANS15 Transfer complete: 8,192 bytes in 3 seconds.

IND\$FILE transfer completed successfully.

An IND\$FILE PUT, sending a workstation file up into a data set as exact bytes.

GET and PUT are the whole interface; everything else is options. And the options are where the care is required, because they decide whether the bytes arrive intact or scrambled.

ASCII, binary, and CRLF: the quirks

The mainframe stores text in EBCDIC, in records; a workstation stores text in ASCII, in lines ended by carriage-return/line-feed. IND\$FILE's options bridge that gap, and choosing them correctly is the entire skill:

Option	What it does
ASCII	Translate between EBCDIC and ASCII, for text
CRLF	Convert record boundaries to/from CR/LF line endings
(none)	Binary, exact bytes, no translation at all
APPEND	Add to an existing file rather than replacing it

The IND\$FILE options, translation for text, or exact bytes for binary.

The rule is simple once stated. For text (source code, JCL, reports) use ASCII and CRLF, so the character set is translated and the mainframe's records become the workstation's lines. For anything binary (a program, a load module, a compressed archive, an image) use no translation options at all, so the bytes are moved exactly. The failure modes are the classic ones. Transfer EBCDIC text as binary and you get a file of unreadable bytes on the PC. Transfer a binary file with ASCII translation and you get a corrupted, useless program, because the translation mangled bytes that were never characters. The decision comes down to one question:

Transferring...	Use
Text (source, JCL, reports)	ASCII CRLF
Program, load module, archive, image	Binary, no options
Unsure	Binary, then inspect the result

The transfer-type decision, the single most important choice in an IND\$FILE transfer.

Getting this right is most of what using IND\$FILE well amounts to. When a transferred file looks like garbage, the transfer type is almost always the cause, and re-doing it with the correct options fixes it. This is the same EBCDIC-versus-ASCII issue that appeared with

FTP's transfer types (the mainframe's character set never stops mattering) wearing IND\$FILE's clothes.

Security considerations

IND\$FILE's security profile follows from where it lives. Because it runs inside the logged-on session, it uses that user's authority: a GET can only read data sets the user could read anyway, and a PUT can only write where the user could write. RACF data-set profiles therefore govern IND\$FILE exactly as they govern any other access (the control is already there. But two things make IND\$FILE worth specific attention. First, it is a data-movement path, so it is a route for exfiltration (GET sensitive data down to a workstation) and for bringing files in (PUT a tool or script up to the host). Second, because the transfer travels inside the terminal datastream rather than over a named service port, it can be less visible to network monitoring than an FTP transfer) there is no separate connection to flag. The defensive posture is therefore to lean on the controls that do apply: tight RACF data-set protection, which bounds what any session can move, and host-side and emulator logging of transfers where available, so that IND\$FILE activity is not a blind spot.

Consideration	Control
Exfiltration via GET	RACF data-set read protection bounds it
Bringing tools in via PUT	RACF write protection; review upload targets
Lower network visibility	Host/emulator transfer logging
Same authority as the session	Session authority is the transfer's limit

IND\$FILE security, the session's authority bounds it, and RACF and logging are the controls.

Two mistakes recur with IND\$FILE. The everyday one is the wrong transfer type (text as binary or binary as ASCII) which silently corrupts the file; the fix is to match the type to the content. The security one is assuming that because IND\$FILE has no port of its own, it needs no attention: it is a real data-movement path bounded only by the session's data-set authority, so tight RACF profiles and transfer logging matter here as much as anywhere.

Hands-On Labs

These labs practice IND\$FILE transfers and reason about their controls.

Lab 36.1

Understand the mechanism

Goal: explain how IND\$FILE transfers a file inside a 3270 session.

Background: IND\$FILE is unlike FTP and NJE, it rides the terminal channel. Understanding that is the key to its behavior and its security.

1. Describe the cooperation between the host IND\$FILE program and the emulator over the 3270 datastream.

Check yourself: you can explain that IND\$FILE is a host program and an emulator feature exchanging file buffers over the terminal channel, using the session's authority and channel.

Why it matters: where a transfer lives shapes how it is controlled and how visible it is. IND\$FILE lives inside the session.

Blue-Team angle: because it rides the terminal channel, IND\$FILE has no port to monitor, the controls are data-set authority and transfer logging, not network rules.

Lab 36.2

Transfer a file both ways

Goal: use GET and PUT to move a file down and up.

Background: GET and PUT are the whole interface. This lab exercises both.

1. GET a data set to the workstation, then PUT a file back up.

```
IND$FILE GET IBMUSER.SOURCE.COBOL ASCII CRLF
```

```
IND$FILE PUT IBMUSER.UPLOAD.DATA BINARY
```

Check yourself: GET brings the data set down and PUT sends a file up, each reporting bytes transferred, and you can explain that direction is set by GET versus PUT.

Why it matters: moving files on and off the host from a workstation is a daily task. GET and PUT are how it is done through the terminal.

Blue-Team angle: a PUT is an upload into a data set, confirm the target is one the user should be able to write, and that RACF protects sensitive libraries from casual upload.

Lab 36.3

Choose the right transfer type

Goal: select ASCII/CRLF for text and binary for programs, and explain the failure modes.

Background: the transfer type is the one choice that most often goes wrong. This lab drills it.

1. For a text file and a binary file, state which options to use and what happens if you choose wrong.

Check yourself: you can say text needs ASCII CRLF and binary needs no options, and explain that text-as-binary gives unreadable bytes while binary-as-ASCII gives a corrupted program.

Why it matters: matching the type to the content is most of using IND\$FILE well. The wrong type silently corrupts the file.

Blue-Team angle: a corrupted transfer is usually a type mismatch, not tampering, but confirm, because a mangled load module could also be a sign of the wrong file being moved.

Lab 36.4

Read a transfer in progress

Goal: read the transfer messages and confirm success.

Background: IND\$FILE reports its progress and result. Reading the messages confirms the transfer and its size.

1. Start a transfer and read the started, byte-count, and completion messages.

Check yourself: you can read the TRANS messages (started, bytes transferred, completed) and confirm the transfer succeeded and how much data moved.

Why it matters: the byte count and completion message are the record that a transfer happened and how large it was, useful for both operation and review.

Blue-Team angle: an unexpectedly large GET is the same exfiltration signal as a large FTP transfer, the byte count is worth noticing even inside a session.

Lab 36.5

Assess the IND\$FILE exposure

Goal: enumerate IND\$FILE's security considerations and the controls that apply.

Background: IND\$FILE is a data-movement path bounded by session authority. This lab frames its assessment.

1. List the exfiltration, upload, and visibility considerations, and the control for each.

Check yourself: you can explain that IND\$FILE uses session authority, that RACF data-set profiles bound both GET and PUT, and that its lower network visibility makes host and emulator logging important.

Why it matters: a transfer with no port of its own is easy to overlook. Naming its controls keeps it from being a blind spot.

Blue-Team angle: rely on RACF data-set protection as the primary bound on IND\$FILE, and enable transfer logging so terminal-channel file movement is recorded, not invisible.

What corruption looks like

The failure modes are worth seeing, because recognizing them saves hours. Take a data set holding the text HELLO. Transferred correctly, with ASCII and CRLF, it arrives as readable text. Transferred as binary (no translation) the EBCDIC bytes land untranslated and the workstation shows nonsense:

```
HOST (EBCDIC):  C8 C5 D3 D3 D6          = HELLO
```

```
GET ... ASCII CRLF  ->  H E L L O          (correct)
```

```
GET ... (binary)    ->  È Å Ó Ó Ö          (untranslated garbage)
```

Text transferred as binary, the EBCDIC bytes arrive untranslated and display as garbage.

The reverse mistake is worse because it is quieter. Transfer a binary file (a load module, say) with ASCII translation, and IND\$FILE dutifully runs every byte through an EBCDIC-to-ASCII conversion. Bytes that were never characters are altered, and the result is a load module that is subtly, irreparably wrong:

```
BINARY LOAD MODULE (bytes): 47 F0 F0 05 ...
```

```
PUT ... (binary)    ->  47 F0 F0 05 ...    (intact, runs)
```

```
PUT ... ASCII       ->  47 30 30 05 ...    (translated -> broken)
```

A binary file transferred with ASCII, translation alters bytes that were never text, breaking it.

The lesson in both figures is the same: translation is correct only for text. When a transferred file will not open, will not run, or shows garbage, the transfer type is the first

suspect, and re-transferring with the right options almost always fixes it. This is why matching the option to the content is the whole of using IND\$FILE well.

Record formats and line endings

The CRLF option deserves its own note, because it bridges two different ideas of a line. The mainframe stores text as records, often fixed-length, described by a record format (RECFM) such as FB (fixed blocked) with a logical record length (LRECL). A workstation stores text as a byte stream with CR/LF markers between lines. The CRLF option converts between the two:

```
HOST DATA SET: RECFM=FB LRECL=80  (records, no line endings)
  record 1: 'IDENTIFICATION DIVISION.' + padding to 80
  record 2: 'PROGRAM-ID. HELLO.'      + padding to 80
GET ... ASCII CRLF ->  each record becomes one CR/LF line
                        trailing padding trimmed
```

The CRLF option, fixed-length host records become CR/LF-terminated workstation lines.

Without CRLF, an 80-byte fixed record arrives as 80 bytes including its padding, with no line break (so a source file becomes one long run of text with trailing blanks. With CRLF, each record becomes a proper line and the padding is trimmed. This is why source and JCL transfers specify ASCII CRLF together: ASCII fixes the character set, CRLF fixes the line structure. Understanding RECFM and LRECL) the record shape from the data-set chapters, is what makes the CRLF option make sense.

Lab 36.6

Recognize a corrupted transfer

Goal: given a corrupted result, diagnose the transfer-type mistake and state the correct re-transfer.

Background: recognizing corruption by its signature saves hours. This lab drills the diagnosis.

1. For a file that arrives as untranslated bytes, and for a load module that will not run, state which option was wrong and the correct transfer.

Check yourself: you can identify text-transferred-as-binary (untranslated EBCDIC, fix with ASCII CRLF) and binary-transferred-as-ASCII (altered bytes, fix with binary and no options).

Why it matters: diagnosing by signature turns a mysterious broken file into a one-line fix, re-transfer with the right type.

Blue-Team angle: confirm a corrupt load module is a transfer-type mismatch and not the wrong (or tampered) file; the byte pattern usually tells you which.

Lab 36.7

Handle record formats

Goal: explain how CRLF converts fixed-length records to workstation lines and why source transfers need it.

Background: the CRLF option bridges records and lines. This lab connects it to RECFM and LRECL.

1. For an FB LRECL=80 source data set, describe what a GET produces with and without CRLF.

Check yourself: you can explain that without CRLF each 80-byte record arrives with its padding and no line break, while with CRLF each record becomes a trimmed CR/LF line, which is why source and JCL use ASCII CRLF.

Why it matters: the record-versus-line difference is the mainframe's, and CRLF is how IND\$FILE reconciles it. Knowing RECFM makes the option obvious.

Blue-Team angle: a transferred source file with 80-column runs and trailing blanks is a missing-CRLF symptom, not a security issue, but knowing the normal shape helps you spot the abnormal one.

Appending and partitioned-data-set members

Two more everyday cases round out IND\$FILE. The APPEND option adds to an existing workstation file instead of replacing it, useful for accumulating report output across several GETs; and a PUT can target a specific member of a partitioned data set by naming it in parentheses, the same member syntax from the data-set chapters:

```
IND$FILE GET IBMUSER.REPORT.DAILY ASCII CRLF APPEND
```

```
TRANS15 Transfer complete: 2,048 bytes appended.
```

```
IND$FILE PUT IBMUSER.SOURCE.PDS(NEWPGM) ASCII CRLF
```

```
TRANS15 Transfer complete: member NEWPGM stored.
```

APPEND accumulates into one workstation file; a member name targets one PDS member.

The member-targeting PUT is the security-relevant one to notice: it writes a named member into a partitioned data set, which for a source or a load library means adding or replacing a program. Exactly as with FTP and NJE, the control that matters is who can write that library, a PUT into a load library that the user should not be able to update is either a misconfiguration or an attempt to plant code. RACF protection of the target library is what bounds it, and reviewing write access to program libraries covers the IND\$FILE upload path along with every other.

Lab 36.8

Target a PDS member safely

Goal: PUT a file into a named PDS member and reason about the write-access control that bounds it.

Background: a member-targeting PUT can add a program to a library. The control is RACF write access to that library.

1. PUT a workstation file into a named member of a partitioned data set, then state which RACF profile governs whether it is allowed.

```
IND$FILE PUT IBMUSER.SOURCE.PDS(NEWPGM) ASCII CRLF
```

Check yourself: you can explain that the PUT stores a member into the PDS, that this can add or replace a program, and that RACF write protection on the library is what decides whether the user may do it.

Why it matters: uploading into a program library is code placement. The write-access control on that library is the same one that governs every other path to it.

Blue-Team angle: review who can write to source and load libraries, since a member-targeting PUT into one is a code-placement path, IND\$FILE is just one more way to reach it.

Checkpoint: can you explain how IND\$FILE transfers files inside a 3270 session, use GET and PUT, choose ASCII/CRLF versus binary correctly, read a transfer's messages, and assess its exposure and controls? Those skills cover the terminal-channel file transfer that every mainframe user eventually meets.

IND\$FILE transfers files inside the 3270 terminal session, cooperating between a host program and the emulator over the terminal datastream rather than a separate service port. Direction is set by GET (host to workstation) and PUT (workstation to host), and the options decide translation: ASCII and CRLF for text, which converts EBCDIC to ASCII and records to line endings, or no options for binary, which moves exact bytes. Choosing wrong silently corrupts the file (text as binary becomes unreadable, binary as ASCII becomes a broken program) so matching the type to the content is the core skill. Because it runs inside the session, IND\$FILE is bounded by the user's data-set authority and governed by RACF profiles exactly as any access is, but its lower network visibility, as a transfer with no port of its own, makes RACF protection and transfer logging the controls that keep it from being a blind spot for exfiltration or upload.

What comes next

FTP, NJE, and IND\$FILE are the traditional ways onto and off the mainframe. But the modern mainframe is also a web server and an API host, and that tier is where much of today's access (and much of today's attack surface) lives. The next chapter turns to Gibson's web and API layer: the banking REST APIs, the web front ends, the browser-based 3270 terminal, and the administration dashboard, each a modern door into a very traditional system, and each with the web vulnerabilities that come with it.

CHAPTER 37

The Web & API Tier

By the end of this chapter you will be able to:

Explain how a modern mainframe serves web pages and APIs.

Query the banking REST API and understand its exposure.

Recognize the browser-based 3270 terminal and web front ends.

Identify the default-credential risk on the admin dashboard.

Explain the Tomcat manager as a web-to-code-execution path.

Harden the web tier with web-application security rigor.

THE TRADITIONAL DOORS ONTO THE mainframe (FTP, NJE, IND\$FILE, the green screen) are no longer the only ones. The modern mainframe is also a web server and an API host: the same COBOL, CICS, and Db2 that run the bank are now wrapped in REST APIs and web front ends and reached from browsers and mobile apps. Gibson models this tier faithfully (a banking REST API, web applications, a 3270 terminal that runs in a browser, and an administration dashboard) and it matters enormously for security, because the web tier brings web vulnerabilities to a platform whose data is as valuable as data gets. Default credentials, injection, broken access control, exposed management interfaces: the whole catalog of web weaknesses now sits in front of the mainframe's crown jewels. This chapter tours that tier and the exposures that come with it.

The mainframe as web server

z/OS runs modern application servers (Liberty, WebSphere, and the z/OS Connect layer that turns CICS and Db2 assets into REST APIs) so the mainframe listens on web ports like any server. Gibson's tier spans several:

Port	Service
8080	CBSA banking REST API
9080	FIBS web application
8023	Browser-based 3270 terminal
8443	Administration dashboard (HTTPS)

The web and API tier, modern doors onto a traditional system.

Every one of these wraps the same back end from the earlier chapters. The REST API on 8080 serves the CBSA banking logic; the web app on 9080 is a browser front end to the same core; the terminal on 8023 is a 3270 session rendered in a browser; the dashboard on 8443 administers the system. The essential mental shift is that the mainframe's data now sits behind web code, so every web-application weakness is a potential path to that data. Web security has become mainframe security.

The banking REST API

The REST API is the clearest example. A request for an account is an ordinary HTTP GET, and the response is JSON, the same account data the COBOL and Db2 chapters handled, now served to any web client:

```
curl http://gibson:8080/api/v1/cbsa/vuln/account/00000101
200 OK
{ "account": { "SORT_CODE": "00-00-01", "ACCOUNT_NUMBER": "00000101", "CUSTOMER_ID": "1001",
  "customer": "1001", "type": "CURRENT",
  "ACTUAL_BALANCE": "1500.00", "STATUS": "OPEN"}, "training": "IDOR ownership check
bypassed" }
```

The banking REST API, account data served as JSON to any web client.

This is convenient and it widens the attack surface in three ways worth naming. Authentication: the API must verify who is calling, and a weak or missing check exposes the data directly. Input validation: any parameter built into a back-end query is an injection risk, the same SQL injection from the Db2 chapter, now reachable over HTTP. Authorization: the API must check that the caller may see this particular account, or a user can read others' balances by changing the number in the URL, the classic broken-access-control flaw. The REST tier is the banking core with an HTTP front door, and every one of these must hold.

Web front ends and the browser 3270

Alongside the API sit browser front ends. The FIBS web application on 9080 is a conventional web app over the banking core. More striking is the terminal on 8023: a full 3270 session rendered in a browser, so the green screen is reached without any terminal emulator at all. These modernize access, and each adds a boundary. The web terminal in particular is a bridge (web authentication in front of the terminal's own sign-on) and a bridge is only as strong as its weaker side. If the web layer authenticates poorly, it can become a way to reach the terminal that bypasses controls assumed to sit at the network edge. Every modern front end is a new path to an old system, and each must be secured as carefully as the system behind it.

The admin dashboard: default credentials

The administration dashboard on 8443 is where the web tier's risks become concrete. It authenticates with HTTP Basic auth, and out of the box it ships with a default credential:

```
GET / HTTP/1.1
401 Unauthorized
WWW-Authenticate: Basic area="Gibson Dashboard"
```

(default credentials as shipped)

```
username: admin
password: gibsonadmin!
```

The admin dashboard, HTTP Basic auth protecting it, with a shipped default credential.

This is the single most common web finding in existence, and it is severe here because of what the dashboard can do: it controls the system, including powering it on and off. A default or unchanged credential on an administrative interface is a full compromise waiting to be found (anyone who knows or guesses admin/gibsonadmin! controls the panel. Default credentials are documented, shared, and the very first thing an attacker tries. The finding writes itself: an internet-reachable or even network-reachable admin dashboard on a known default credential is a critical exposure, and the fix) change the credential, require strong authentication, is the first thing any hardening does.

The Tomcat manager: web to code execution

Web application servers ship with a management interface, and Tomcat's manager is the classic example, modeled in Gibson's tier at the well-known manager paths:

```
GET /manager/html          -> Tomcat Web Application Manager
GET /manager/text/list     -> list deployed applications
POST /manager/text/deploy -> deploy a new application (WAR)
```

The Tomcat manager, a web interface that can deploy applications, and thus run code.

The manager's power is deployment: it can install a new web application. That makes it a textbook web-to-code-execution path, and the attack is well known, reach the manager, authenticate with a default or weak credential, deploy a malicious application, and now attacker code runs inside the server, on the mainframe. This is presented here so a defender recognizes and closes it, not to enable it: the manager must never carry default credentials, should be restricted to trusted networks or removed entirely from production, and its access should be tightly controlled and monitored. An exposed manager with weak credentials is among the highest-severity findings on any web tier, mainframe or otherwise, precisely because it turns web access into code execution.

Securing the web tier

The web tier must be held to full web-application-security standards, the mainframe behind it does not lower the bar, it raises the stakes. The exposures and their remediations are the standard ones:

Exposure	Remediation
Default admin credentials	Change them; require strong authentication
Exposed Tomcat manager	Restrict or remove; never default creds
Injection via API parameters	Parameterize queries; validate input
Broken access control	Enforce per-object authorization checks
Cleartext HTTP	Require TLS on every endpoint

Hardening the web tier, the standard web defenses, applied to the mainframe's front door.

None of these is exotic; they are the everyday disciplines of web security, and that is exactly the point. The mainframe's web tier fails in the same ordinary ways any web tier does (a default password, an unvalidated parameter, a missing authorization check) but the data behind it is a bank's core. Change every default credential, require TLS everywhere, parameterize and validate all input, enforce authorization on every object, and keep management interfaces off the open network. The web tier is secure when it meets the same standard a bank's public website would, because, increasingly, it is that website's back end.

The web-tier mistake is assuming the mainframe's reputation for security extends to the web code in front of it. It does not: a default dashboard password, an exposed Tomcat manager, or an API that trusts its input undoes every control beneath. The web tier is ordinary web code guarding extraordinary data, and it must be secured with full web-application rigor, starting with changing every default credential.

Hands-On Labs

These labs explore the web tier and its exposures. The attack paths are studied to recognize and close them.

Lab 37.1

Map the web ports

Goal: identify the web and API services and what each fronts.

Background: the web tier is a set of network-facing services. Mapping them is the first step of assessing it.

1. List the web ports and the service on each.

Check yourself: you can name the REST API on 8080, the web app on 9080, the browser terminal on 8023, and the dashboard on 8443, and explain that each fronts the same back end.

Why it matters: every web port is a door to the mainframe's data. Knowing them is knowing the modern attack surface.

Blue-Team angle: an unexpected web port is an unexpected exposure. Web services on a mainframe deserve the same port review as any web host.

Lab 37.2

Query the REST API

Goal: retrieve account data over the API and identify its three key exposures.

Background: the REST API is the banking core with an HTTP front door. This lab reads it and names the risks.

1. Request an account over the API and read the JSON response.

```
curl http://gibson:8080/api/v1/cbsa/vuln/account/00000101
```

Check yourself: you can read the JSON account data and name the three exposures, authentication, input validation (injection), and authorization (reading other accounts by changing the number).

Why it matters: an API is a programmable door. Its authentication, input handling, and authorization are what stand between a caller and the data.

Blue-Team angle: test the API for broken access control by requesting an account the caller should not see, if it returns, authorization is missing, a top finding.

Lab 37.3

Assess the dashboard credential

Goal: identify the default-credential exposure on the admin dashboard and rate it.

Background: default credentials on an admin panel are the most common severe web finding. This lab assesses it.

1. Examine the dashboard's authentication and determine whether it uses a default or unchanged credential.

Check yourself: you can identify that the dashboard uses HTTP Basic auth with a shipped default credential, and rate it critical because the dashboard controls the system, including power.

Why it matters: a default credential on a powerful admin interface is a full compromise waiting to be found. It is the first thing an attacker tries.

Blue-Team angle: changing default credentials is the single highest-value web hardening step. Confirm no admin interface anywhere carries a shipped password.

Lab 37.4

Understand the Tomcat manager path

Goal: explain how the Tomcat manager turns web access into code execution.

Background: the manager can deploy applications. This lab studies the web-to-RCE path defensively.

1. Identify the manager endpoints and describe how deploy access leads to code execution.

Check yourself: you can explain that the manager can deploy a web application, and that reaching it with a weak credential lets an attacker deploy code that runs on the server, web access becoming code execution.

Why it matters: an exposed manager with weak credentials is among the highest-severity web findings, because it is a direct route to running code.

Blue-Team angle: remove or tightly restrict management interfaces in production, never leave default manager credentials, and alert on access to manager paths.

Lab 37.5

Harden the web tier

Goal: state the controls that bring the web tier to full web-application security.

Background: each web exposure has a standard remediation. This lab plans the hardening.

1. For default credentials, exposed management, injection, broken access control, and cleartext, state the control that closes each.

Check yourself: you can map defaults to strong auth, management to restriction, injection to parameterization, access control to per-object checks, and cleartext to TLS, a complete web-tier hardening plan.

Why it matters: the web tier fails in ordinary ways and is fixed by ordinary web disciplines, applied with the rigor the data behind it demands.

Blue-Team angle: hold the mainframe's web tier to the same standard as a public banking site. The stakes are identical because the data is the same.

Injection, now over HTTP

The SQL injection from the Db2 chapter does not go away when the front end becomes a REST API, it simply arrives over HTTP. A search parameter passed to the API is built into a back-end query exactly as before, so crafted input in a URL changes the query's logic:

```
GET /cbsa/search?customer=1001          -> 2 rows (normal)
```

```
GET /cbsa/search?customer=1001'+OR+'1'='1  ->
```

```
    simulated_sql: ...WHERE CUSTOMER_ID = '1001' OR '1'='1'
```

```
    rows_returned: all accounts
```

SQL injection over the REST API, crafted input in a URL parameter reaches the back-end query.

The lesson from Chapter 30 applies unchanged: the fix is parameterized queries, keeping input as data rather than command. What the web tier adds is reach, the injection is now exploitable by anyone who can send an HTTP request, not only someone at a SPUFI session. That widening is the whole security story of the web tier: the same weaknesses, exposed to a far larger population of potential attackers. Input validation and parameterization at the API boundary are therefore not optional niceties but the primary defense.

Mass assignment: trusting the whole payload

APIs introduce their own characteristic flaw. When an update endpoint binds every field in the incoming JSON straight onto the record, a caller can set fields they were never meant to touch by simply including them in the payload:

```
POST /cbsa/customer/1001
```

```
{ "address1": "1 High Street",      <- legitimate
  "credit_score": "850",             <- should be read-only
  "admin": true }                   <- should not exist
```

```
RESULT: record updated including credit_score and admin
```


Mass assignment, an endpoint that binds the whole payload lets a caller set protected fields.

Here the caller updated their own credit score and set a privileged flag, simply because the endpoint accepted whatever fields arrived. This is mass assignment, and it is one of the most common API-specific vulnerabilities. The fix is to bind only an explicit allow-list of fields the caller may change, and to reject or ignore anything else, never to trust the shape of the incoming payload. It is the same principle as input validation and parameterization: the boundary decides what is accepted, not the caller.

Lab 37.6

Test the API for injection and mass assignment

Goal: demonstrate SQL injection through an API parameter and mass assignment through an update payload, and state each fix.

Background: the two most consequential API flaws are injection and mass assignment. This lab studies both defensively.

1. Send a crafted search parameter and observe the altered query.

```
GET /cbsa/search?customer=1001'+0R+'1'='1
```

2. Send an update payload with extra fields and observe them being applied.

Check yourself: you can show that a URL parameter reaches the back-end query (fix: parameterize) and that extra payload fields are bound onto the record (fix: allow-list the permitted fields).

Why it matters: both flaws come from trusting input at the boundary. The API is where that trust must be withheld, parameterize queries and allow-list fields.

Blue-Team angle: review API endpoints for dynamic queries built from parameters and for updates that bind whole payloads. Both are found by reading the boundary code, and both are closed there.

Checkpoint: can you map the web ports, query the REST API and name its exposures, assess the dashboard's default credential, explain the Tomcat manager as a web-to-code-execution path, and harden the tier with standard web defenses? Those skills secure the modern front door to a traditional system.

The modern mainframe is a web server and API host: the same COBOL, CICS, and Db2 core is served through a REST API, web front ends, a browser-based 3270 terminal, and an administration dashboard. That convenience brings the full catalog of web vulnerabilities to the platform. The REST API exposes the banking core to authentication, injection, and broken-access-control risks over HTTP. The admin dashboard ships with a default credential on an interface that controls the system, the most common severe web finding. The Tomcat manager can deploy applications, making it a textbook web-to-code-execution path. Every one of these is an ordinary web weakness guarding extraordinary data, so the web tier must be held to full web-application security: change default credentials, require TLS, parameterize and validate input, enforce per-object authorization, and keep management interfaces off the open network. Web security is now mainframe security.

What comes next

The chapters so far have driven Gibson through its interfaces. The last chapter of Part VII turns to driving it through code: the languages and interpreters the mainframe runs. Gibson includes working REXX, JCL, COBOL, and HLASM environments, and the next chapter explores programming the mainframe, how these languages work, what each is for, and how understanding them underpins everything from automation to the exploitation and defense that Parts VIII and IX take up.

CHAPTER 38

RESTful APIs, z/OS Connect & API Security

By the end of this chapter you will be able to:

Explain how z/OS Connect exposes CICS and Db2 as REST APIs.

Retrieve mainframe data with a real curl request.

Exploit and fix IDOR/BOLA and SQL injection through an API.

Obtain an OAuth 2.0 token and read a JWT.

Demonstrate and defend against JWT forgery.

Trace the full identity-to-data chain and secure it.

CHAPTER 37 INTRODUCED THE WEB and API tier in outline. This chapter goes deep on the part that matters most for a modern mainframe: the RESTful API layer that z/OS Connect places in front of CICS, COBOL, and Db2, and the security of that layer. This is where the mainframe meets the modern web, and it is where the modern web's hardest problems (broken object authorization, injection, and above all the OAuth 2.0 and JWT identity layer) now sit directly in front of a bank's core data. Every example here is a real, runnable request against the Gibson training system, given as a complete curl command with its actual port, path, and response. We work up from the simplest data return to the full chain in which a forged token authorizes access to CICS- and Db2-backed data. The mainframe's own RACF and Db2 controls remain intact throughout, the lesson of this chapter is that the API identity layer in front of them is the new front line.

z/OS Connect: REST in front of CICS and Db2

z/OS Connect is IBM's technology for turning traditional mainframe assets into RESTful JSON APIs. A CICS transaction that once required a 3270 terminal, or a Db2 query that once required SQL over DDF, is published as an HTTP endpoint that any client can call with JSON. Gibson models this as a z/OS Connect-style provider, and the request path is explicit:

```
Browser / curl
-> FIBS WEB9080 / CBSA API 8080    (z/OS Connect-style API)
    -> CICS transaction (INQACC, INQCUST, TELLER_SEARCH)
        -> COBOL service program
            -> Db2 SQL -> Db2 tables (CBSA.ACCOUNT, CBSA.CUSTOMER)
<- JSON response          (+ SMF / Master Console evidence)
```

The z/OS Connect chain, a REST call becomes a CICS transaction, a COBOL program, and a Db2 query.

Read the chain top to bottom: an HTTP request arrives, the API provider maps it to a CICS transaction, the transaction runs a COBOL program, the program issues Db2 SQL, and the result returns as JSON (with SMF records and console messages emitted along the way).

This is genuinely powerful; it lets modern applications reach decades of proven business logic. It is also the point at which web-application security becomes mainframe security, because everything a REST API can get wrong) authorization, input handling, identity, now sits directly in front of the bank's data. The rest of the chapter attacks and then defends each link, starting with the simplest.

A simple data return, and IDOR

The most basic API call fetches an account. On Gibson's CBSA API this is a single curl to port 8080, and it returns the real Db2 row as JSON:

```
curl http://gibson:8080/api/v1/cbsa/vuln/account/00000101
{
  "account": {
    "SORT_CODE": "00-00-01",
    "ACCOUNT_NUMBER": "00000101",
    "CUSTOMER_ID": "1001",
    "ACCOUNT_TYPE": "CURRENT",
    "ACTUAL_BALANCE": "1500.00",
    "AVAILABLE_BALANCE": "1500.00",
    "OVERDRAFT_LIMIT": "500.00",
    "STATUS": "OPEN"
  },
  "training": "IDOR ownership check bypassed"
}
```

A real API data return, the Db2 account row, and the flaw: no ownership check.

Behind that single response is the whole chain (the API provider ran the CICS INQACC transaction, which ran the COBOL INQACC program, which issued a Db2 SELECT against CBSA.ACCOUNT. The data returned correctly; the flaw is in authorization. This is IDOR) Insecure Direct Object Reference, called BOLA (Broken Object Level Authorization) in the API security world, and the tell is the training note: the ownership check was bypassed. Any caller can request any account number and receive it, because the API never verified that the caller owns that account. Change 00000101 to another number and another customer's balance returns. The fix is to enforce object-level authorization: the API, or the CICS service behind it, must verify that the authenticated caller is entitled to this specific account before Db2 is ever queried. Db2 returning the row is correct; the API asking for it without an ownership check is the defect.

SQL injection through the API

The account search takes a customer parameter, and if that parameter is built into the Db2 query as text, it is injectable, over HTTP. A normal call returns the customer's accounts:

```
curl "http://gibson:8080/api/v1/cbsa/vuln/accounts?customer=1001"
{ "input": "1001",
  "simulated_sql": "SELECT * FROM CBSA.ACCOUNT WHERE CUSTOMER_ID = '1001'",
  "rows_returned": 2, "result": "NORMAL" }
```

A normal API search, the customer parameter built into a Db2 SELECT.

Now inject. Supplying a classic tautology as the customer value changes the query's logic and returns every account, not just the customer's:

```
curl "http://gibson:8080/api/v1/cbsa/vuln/accounts?customer=1001'+OR+'1'='1"
{ "input": "1001' OR '1'='1",
  "simulated_sql":
    "SELECT * FROM CBSA.ACCOUNT WHERE CUSTOMER_ID = '1001' OR '1'='1'",
  "rows_returned": 4, "result": "ROWSET_EXPANDED" }
```

SQL injection over the API, the tautology expands the result set to every account.

The response shows the injected SQL and the expanded row set (the CICS TELLER_SEARCH path built the caller's input straight into the WHERE clause, so the tautology returned all rows. This is the same injection from the Db2 chapter, now reachable by anyone who can send an HTTP request. The fix is unchanged and absolute: parameterized queries, so input is bound as data and can never alter the query's structure. The API layer must validate and parameterize before the request ever reaches CICS and Db2. Gibson records the attempt as a simulated SMF 102 event and raises console message GIBSQLI01W) the detection half, so even a successful injection leaves a trail in the audit data of Part IV.

Excessive data, mass assignment, and verbose errors

Three more data-layer flaws round out the simple set, each a single curl. A debug endpoint over-returns a full customer record; a customer update binds fields it should not; and an error endpoint leaks the backend SQL:

```
curl http://gibson:8080/api/v1/cbsa/vuln/debug/customer/1001
{ "customer": { "CUSTOMER_ID": "1001", "NAME": "ALICE MORGAN",
  "ADDRESS1": "1 HIGH STREET", "DATE_OF_BIRTH": "1980-01-01",
  "CREDIT_SCORE": "740", "STATUS": "ACTIVE" } }
```

Excessive data exposure, a debug endpoint returning the full customer record.

The debug endpoint returns far more than a caller needs (date of birth, credit score, full address) the excessive-data-exposure flaw, fixed by returning only the fields a use case

requires. Mass assignment follows the same shape as an update: a PUT to `/api/v1/cbsa/vuln/customers/1001` that includes extra fields (a role, a raised overdraft limit) binds them onto the record because the endpoint trusts the whole payload, fixed by binding only an explicit allow-list of permitted fields. And the verbose-error endpoint leaks the backend detail an attacker wants:

```
curl "http://gibson:8080/api/v1/cbsa/vuln/sql-error?account=abc"
HTTP/1.1 500
{ "SQLCODE": "-104", "SQLSTATE": "42601",
  "simulated_sql": "SELECT * FROM CBSA.ACCOUNT WHERE ACCOUNT_NUMBER = 'abc'",
  "error": "controlled verbose SQL error" }
```

A verbose error, leaking the Db2 SQLCODE, SQLSTATE, and the query itself.

The verbose error hands the attacker the Db2 SQLCODE, the SQLSTATE, and the query structure, a map of the backend, confirming injection is worth pursuing. The fix is to return a generic error to the client and keep the detail in server-side logs only. Three flaws, three one-line principles: return the minimum, bind an allow-list, and never leak backend detail.

OAuth 2.0 and OIDC on the mainframe API

The data flaws so far were on the deliberately open CBSA API. The FIBS banking API on port 9080 adds what a real mainframe API has: an identity layer. It implements OAuth 2.0 and OpenID Connect, publishing a discovery document and a token endpoint, and issuing JWTs:

```
curl http://gibson:9080/.well-known/openid-configuration
{ "issuer": "http://gibson:9080",
  "authorization_endpoint": "/oauth/authorize",
  "token_endpoint": "/oauth/token",
  "jwks_uri": "/oauth/jwks",
  "id_token_signing_alg_values_supported": ["HS256"] }
```

The OIDC discovery document, the token endpoint, JWKS, and signing algorithm.

The discovery document advertises the endpoints and the signing algorithm (HS256). A client obtains a token from the token endpoint and presents it as a bearer token on every API call; the API validates the token before it reaches CICS and Db2. This is exactly how a modern mainframe API authenticates, the JWT is the identity that the whole z/OS Connect chain trusts. Which means the security of the CICS- and Db2-backed data now depends on the correctness of JWT validation. Get that validation wrong, and a forged identity walks straight through to the mainframe data. That is the subject of the next section, and the most important idea in the chapter.

JWT forgery, the identity layer attacked

A JWT is three base64 parts (header, payload, signature) and its security rests entirely on the signature and on validating the claims. Gibson's JWT lab lets you forge tokens against a series of validation failures. The classic is the alg=none attack: strip the signature and claim the token needs none, with an elevated role:

```
curl -X POST http://gibson:9080/webapi/labs/jwt/forge -d
"lab=alg_none&user=alice&role=admin"
```

VULNERABLE mode:

```
{ "accepted": true, "token": "eyJhbGciOiJIub25lIn0...",
  "claims": { "sub": "alice", "role": "admin" },
  "correlation_id": "..."} }
```

SECURE mode:

```
{ "accepted": false, "error": "algorithm rejected" }
```

The alg=none JWT forgery, an unsigned admin token accepted in VULNERABLE mode, rejected in SECURE.

In VULNERABLE mode the unsigned token is accepted and the forged admin role is trusted; in SECURE mode the alg=none token is rejected. The lab covers the full family of JWT validation failures, each its own forge variant:

Forge variant	Attack	Fix
alg_none	Strip the signature (alg:none)	Reject alg=none; require signature
weak_hmac	Sign with a guessable secret	Strong signing key
expired	Present an expired token	Validate exp (expiry)
wrong_audience	Token minted for another API	Validate aud (audience)
role claim	Elevate the role claim	Do not trust role claims for authz

The JWT forge variants, each a validation failure, each with its fix.

Every variant is the same lesson: if the identity layer trusts something it did not verify (an absent signature, a weak key, an expired token, the wrong audience, a self-asserted role) a forged identity is accepted. And because that identity gates the z/OS Connect chain, acceptance means authorized access to CICS- and Db2-backed data. The fixes are the standard JWT disciplines: verify the signature and reject alg=none, use a strong key, validate issuer, audience, and expiry, and never trust a role claim as authorization on its own. Gibson raises console message GIBAUTH02W for JWT training events, so a forgery attempt is recorded whether it is accepted or rejected.

The full chain: identity to data

Assemble the pieces into the real-world attack. The identity layer is exploited first, and its acceptance authorizes the mainframe data return:

1. POST /oauth/token -> obtain a JWT (or forge one via /jwt/forge)
2. forged token (role=admin) -> JWT validator ACCEPTS (vuln mode)
3. GET /accounts/00000102 -> z/OS Connect -> CICS INQACC -> COBOL

4. `-> Db2 SELECT FROM CBSA.ACCOUNT -> row`
5. JSON returned; no ownership check (IDOR) -> other customer's data
6. evidence: SMF80 GIBAPI401W (authz) + GIBAUTH02W (JWT)

The full API attack chain, a forged JWT authorizes a CICS/Db2 data return it should not.

This is the chapter's worked example end to end, and the teaching point is precise: the mainframe's RACF and Db2 controls never failed. Db2 returned a valid row to a request the API told it was authorized. The compromise happened entirely in the API identity and authorization layer (a forged JWT that was trusted, and a missing ownership check) and that layer, not RACF, is where the defense must sit. The z/OS Connect chain is only as trustworthy as the identity it enforces at the front. Secure the JWT validation and the object authorization, and the same chain that leaked data becomes one where every request is authenticated, authorized, and logged before Db2 is ever touched.

Securing the mainframe API tier

Pulling the flaws together, each has a specific fix and a specific piece of evidence, and a defender works the whole list:

Flaw	Fix	Evidence
IDOR / BOLA	Enforce object-level authorization	SMF80, GIBAPI401W
SQL injection	Parameterize all queries	SMF102, GIBSQLI01W
Excessive data	Return only required fields	API audit
Mass assignment	Bind an explicit field allow-list	GIBAPI403I
Verbose errors	Generic client errors; log server-side	API audit
JWT / OAuth flaws	Strict validation; reject alg=none	GIBAUTH02W

Securing the mainframe API tier, each flaw with its fix and its audit evidence.

None of these fixes is exotic; they are the standard disciplines of API security, applied with the seriousness the mainframe data behind them demands. Enforce authorization on every object, parameterize every query, return and accept only what is needed, and validate identity strictly. And note the evidence column: every flaw, exploited or blocked, produces an SMF record and a console message, so the API tier is not only defensible but observable, the same audit discipline from Part IV, now watching the front door. The mainframe API tier is secure when it meets the standard a bank's public API would, because that is exactly what it is.

The mistake is trusting the mainframe's reputation to cover the API in front of it. RACF and Db2 can be flawless while a forged JWT, a missing ownership check, or an injectable parameter hands over the very data they protect. The API identity and authorization layer is the new front line, secure it with full API-security rigor, verify every token, authorize every object, and parameterize every query, because the z/OS Connect chain will faithfully deliver mainframe data to whatever the API tells it is authorized.

Hands-On Labs

These labs are runnable curl requests against the Gibson training API. Substitute your host for "gibson". The CBSA API on 8080 is intentionally open for the data labs; the FIBS API on 9080 carries the OAuth and JWT identity layer.

Lab 38.1**Return account data and spot the IDOR**

Goal: retrieve a Db2 account row through the API and identify the missing ownership check.

Background: the account endpoint runs the z/OS Connect chain to Db2. This lab reads the data and the flaw.

1. Request an account and read the JSON and the training note.

```
curl http://gibson:8080/api/v1/cbsa/vuln/account/00000101
```

2. Request a different account number and confirm another customer's data returns.

```
curl http://gibson:8080/api/v1/cbsa/vuln/account/00000102
```

Check yourself: you receive the real Db2 row (balance 1500.00) with "IDOR ownership check bypassed," and a different number returns a different customer, proving no object-level authorization.

Why it matters: the data return is correct; the authorization is missing. BOLA is the top API risk precisely because it looks like normal data access.

Blue-Team angle: the fix is object-level authorization before Db2 is queried; the evidence is SMF80 and console GIBAPI401W, confirm both are collected.

Lab 38.2**Inject SQL through the API**

Goal: demonstrate SQL injection through the account-search parameter.

Background: the search parameter reaches a Db2 SELECT through CICS. This lab injects it defensively.

1. Run a normal search and note the two rows and the simulated SQL.

```
curl "http://gibson:8080/api/v1/cbsa/vuln/accounts?customer=1001"
```

2. Inject a tautology and observe the expanded result set.

```
curl "http://gibson:8080/api/v1/cbsa/vuln/accounts?customer=1001'+0R+'1'='1"
```

Check yourself: the normal call returns 2 rows; the injection returns 4 (ROWSET_EXPANDED) and the response shows the tautology in the WHERE clause.

Why it matters: the injection reaches Db2 through the whole z/OS Connect chain.

Parameterization at the API boundary stops it.

Blue-Team angle: parameterize the query; the attempt raises SMF102 and GIBSQLI01W, so a successful injection is still detectable in the audit trail.

Lab 38.3**Excessive data, mass assignment, verbose errors**

Goal: demonstrate three data-layer flaws and their one-line fixes.

Background: over-returning, over-binding, and over-disclosing are common API flaws. This lab runs each.

1. Over-return: read the full customer record from the debug endpoint.

```
curl http://gibson:8080/api/v1/cbsa/vuln/debug/customer/1001
```

2. Verbose error: trigger and read the leaked backend SQL.

```
curl "http://gibson:8080/api/v1/cbsa/vuln/sql-error?account=abc"
```

Check yourself: the debug endpoint returns date of birth and credit score; the error endpoint leaks SQLCODE -104 and the query; and a PUT with extra fields to /customers/1001 binds them, mass assignment.

Why it matters: return the minimum, bind an allow-list, and never leak backend detail, three principles that close three flaws.

Blue-Team angle: mass assignment raises GIBAPI403I when blocked, confirm the allow-list is enforced and the event is logged.

Lab 38.4

Obtain an OAuth token

Goal: read the OIDC discovery and obtain a JWT from the token endpoint.

Background: the FIBS API authenticates with OAuth 2.0 and JWTs. This lab walks the token flow.

1. Read the OIDC discovery document and note the endpoints and algorithm.

```
curl http://gibson:9080/.well-known/openid-configuration
```

2. Read the JWKS to see the published signing key metadata.

```
curl http://gibson:9080/oauth/jwks
```

Check yourself: you can identify the token endpoint, the JWKS URI, and the HS256 signing algorithm, and explain that the API validates the JWT before reaching CICS and Db2.

Why it matters: the JWT is the identity the whole z/OS Connect chain trusts. Understanding the flow is the prerequisite to securing it.

Blue-Team angle: the signing algorithm and key management are the foundation, a weak key or a permitted alg=none undermines every token.

Lab 38.5

Forge a JWT

Goal: forge tokens against the validation failures and see SECURE mode reject them.

Background: JWT security rests on signature and claim validation. This lab attacks each and reads the defense.

1. Forge an alg=none admin token and observe acceptance in VULNERABLE mode.

```
curl -X POST http://gibson:9080/webapi/labs/jwt/forge -d "lab=alg_none&user=alice&role=admin"
```

2. Repeat with the expired and wrong_audience variants.

```
curl -X POST http://gibson:9080/webapi/labs/jwt/forge -d "lab=expired&user=alice"
```

Check yourself: in VULNERABLE mode the forged tokens are accepted (accepted:true); in SECURE mode alg=none is rejected ("algorithm rejected"), and the expired and audience variants are likewise blocked.

Why it matters: a trusted forged token authorizes access to CICS- and Db2-backed data. Strict validation is what stands between a forged claim and the mainframe.

Blue-Team angle: reject alg=none, verify the signature, validate issuer/audience/expiry, and never trust role claims for authorization, GIBAUTH02W records the attempt either way.

Lab 38.6

Run the full identity-to-data chain

Goal: chain a forged token to a data return and identify where the defense belongs.

Background: the real attack combines a forged identity with a missing authorization check. This lab assembles it.

1. Forge an admin token, then use it to request an account that is not the caller's, and trace the chain to Db2.

Check yourself: you can describe the sequence (forged JWT accepted, z/OS Connect to CICS INQACC to COBOL to Db2 SELECT, data returned with no ownership check) and explain that RACF and Db2 never failed; the API identity and authorization layer did.

Why it matters: this is the real-world mainframe API attack. The defense sits in the API tier (validate the token and authorize the object) not in RACF, which was never bypassed.

Blue-Team angle: instrument the whole chain (GIBAUTH02W for the token, GIBAPI401W for the authorization, SMF for both) so the chain is authenticated, authorized, and logged before Db2 is reached.

Checkpoint: can you retrieve mainframe data with a real curl request, exploit and fix IDOR and SQL injection through the API, obtain an OAuth token, forge a JWT against each validation failure, and trace the full identity-to-data chain, explaining that the defense belongs in the API tier, not RACF? Those skills secure the modern front door to CICS, COBOL, and Db2.

z/OS Connect exposes CICS, COBOL, and Db2 as RESTful JSON APIs, and the security of that API tier is now mainframe security. A single curl to the CBSA API returns a real Db2 account row (and reveals IDOR/BOLA when it does so without an ownership check; the account-search parameter is injectable straight into the Db2 SELECT; and debug endpoints, updates, and error pages leak, over-bind, and over-disclose. The FIBS API adds the identity layer that a real mainframe API has: OAuth 2.0 and OIDC issuing JWTs, validated before the z/OS Connect chain reaches CICS and Db2. That makes JWT validation the front line, and Gibson's forge lab attacks each failure) alg=none, weak HMAC, expired, wrong audience, elevated role (all accepted in VULNERABLE mode and rejected in SECURE. The full chain shows a forged token authorizing a data return that RACF and Db2 never refused, because the compromise was in the API identity and authorization layer. The fixes are standard API-security disciplines) object authorization, parameterization, minimal data, strict token validation, each producing SMF and console evidence, so the mainframe API tier is both defensible and observable.

What comes next

With the API tier secured, the connectivity part turns to its final subject: driving the mainframe through code. The next chapter covers programming Gibson (the REXX, JCL, COBOL, and HLASM interpreters) the languages behind the CICS transactions and Db2 services this chapter's APIs called, and the literacy a defender needs to read what the code on a mainframe actually does.

CHAPTER 39

Programming Gibson

By the end of this chapter you will be able to:

Write and run a simple REXX script.

Read JCL as the job-control glue of the mainframe.

Recognize COBOL's structure and compile a program.

Read a High Level Assembler listing.

Explain each language's security relevance.

See why reading code is a core defensive skill.

THE CHAPTERS SO FAR HAVE driven Gibson through its interfaces (terminals, consoles, files, the web. This one drives it through code. The mainframe runs a handful of languages, each with a distinct job: REXX for scripting and automation, JCL for describing batch work, COBOL for business logic, and HLASM) the High Level Assembler (for the low-level, system-level code closest to the metal. Gibson includes working interpreters for all four, and this chapter uses them to introduce each. The purpose is not to make you a mainframe programmer in a single chapter but to make you literate: able to read what these languages do. That literacy underpins everything ahead, because you cannot assess what you cannot read) a malicious script, a rogue job, a COBOL bug, an authorized assembler routine, and Parts VIII and IX depend on being able to read them. This chapter closes Part VII by turning interface users into code readers.

REXX: scripting and automation

REXX is the mainframe's scripting language, interpreted, readable, and everywhere. It is what sysprogs and automation reach for to glue tasks together, write ISPF macros, and drive tools. A first script shows how ordinary it looks:

```
SAY "HELLO FROM REXX" ; X = 2 + 3 ; SAY "X =" X
HELLO FROM REXX
X = 5
```

A REXX script, SAY prints, variables compute, exactly as any scripting language does.

SAY prints, variables need no declaration, and expressions evaluate as you would expect. What makes REXX powerful (and security-relevant) is its reach: a REXX script can issue TSO commands, read and write data sets, and drive ISPF, so it is not a sandboxed toy but a scripting language with the run of the system, executing with its user's authority. That is why REXX is the natural language for automation and, equally, why a malicious REXX script is a real concern: a script that quietly issues privileged commands or exfiltrates data sets is doing so with the reach of the user who runs it. Reading REXX (knowing what a script actually does) is therefore a defensive skill, not just a programming one.

JCL: the job-control language

JCL, the Job Control Language, is how batch work is described. It is not a programming language in the usual sense; it is the glue that says which programs to run and which data sets to give them. Its three core statements do almost everything:

```
Parse a job
//MYJOB JOB (ACCT),CLASS=A      <- the JOB: who and what class
//S1   EXEC PGM=IEFBR14         <- a STEP: run this program
//DD1   DD  DSN=MY.DATA,DISP=SHR <- a DD: give it this data set
```

```
PARSED: JOB(MYJOB) STEP(S1 PGM=IEFBR14) DD(DD1 DSN=MY.DATA)
```

JCL parsed, a JOB, a step that runs a program, and a DD that supplies a data set.

The JOB statement names the job and its class; each EXEC runs a program as a step; each DD supplies a data set the step will use. That is the shape of nearly all batch work (run these programs, in order, against these data sets. Its security relevance is one you have already met: a job runs its programs with the submitter's authority, so JCL is code execution, and anything that can submit a job (a terminal, FTP with FILETYPE=JES, NJE from a trusted node) can run work as its user. Reading JCL tells you exactly what a job will run and touch) the first thing to check about any job whose provenance is uncertain.

COBOL: the business language

COBOL is the language of the mainframe's business logic, the banking programs of Part VI, and a staggering proportion of the world's financial code. A program is organized into divisions, and compiling one produces a listing:

```
Compile a COBOL program
IGYCRCTL Enterprise COBOL for z/OS SIMULATOR
SOURCE LISTING FOLLOWS
000001  IDENTIFICATION DIVISION.
000002  PROGRAM-ID. HELLO.
000003  PROCEDURE DIVISION.
000004      DISPLAY 'HELLO FROM COBOL'.
000005      STOP RUN.
RC=0  COMPILE SUCCESSFUL
```

A COBOL compile, the IGYCRCTL listing with numbered source and a return code.

The IDENTIFICATION division names the program, the DATA division (omitted here) declares its storage, and the PROCEDURE division holds the logic. The compiler lists the source with line numbers and reports a return code (zero for a clean compile. COBOL's security relevance is the one from the banking chapter: its fixed-length field handling is the source of the storage-overflow class, and reading COBOL is how those bugs are found.

A defender who can read a PROCEDURE division can spot an unvalidated MOVE before it becomes an ASRA abend and a corrupted flag) code review is a security control, and it requires reading the language.

HLASM: the assembler

Closest to the metal is HLASM, the High Level Assembler, the language of the operating system itself and of the most performance-critical or privileged code. Assembling a small routine produces a listing that shows the machine at its lowest level:

```
Assemble a routine
ASMA90 High Level Assembler SIMULATOR
SOURCE LISTING
000001 000000 MYPGM      CSECT
000002 000000          LR      R1,R2
000003 000004          BR      R14
000004 000008          END
SYMBOL TABLE  MYPGM  000000
RC=0
```

An HLASM assembly, source, location counter, machine instructions, and a symbol table.

Each line shows a location counter and an instruction: a CSECT begins the code, LR loads one register from another, BR branches to the address in a register (here R14, the return). The symbol table records where each label lives. Assembler is where the mainframe's deepest power (and deepest risk) lives, because authorized assembler programs (the APF concept from Part V) can bypass the normal controls, issue supervisor calls, and touch protected storage. Understanding assembler is therefore essential to the most serious analysis: the authorized-program exploits Part IX examines are written in it, and reading assembler is how a defender understands what such a program really does.

Why programming underpins security

Pulling the four together, each carries a distinct security weight, and reading each is a distinct defensive capability:

Language	Security relevance
REXX	Scripts run with user authority, automation or abuse
JCL	A job is code execution under the submitter's identity
COBOL	Fixed-length handling, the storage-overflow class
HLASM	Authorized/system code, can bypass normal controls

The four languages and what each means for security.

The common thread is that you cannot secure, assess, or investigate code you cannot read. A suspicious REXX script, a job of unknown origin, a COBOL program with an unvalidated move, an authorized assembler routine (each is opaque until someone reads it, and reading it is how the defender decides whether it is benign or hostile. Programming

literacy is security literacy on the mainframe. The goal of this chapter is not fluency in four languages but the ability to look at each and understand what it does) the foundation for the recon, exploitation, and defense that the rest of the book builds.

The mistake is treating programming as a separate discipline from security, something for developers, not defenders. On the mainframe the two are inseparable: a script, a job, a program, and an assembler routine are all code that runs with real authority, and assessing them means reading them. A defender who cannot read REXX, JCL, COBOL, and assembler is blind to what the code on the system actually does.

Hands-On Labs

These labs run each language and connect it to its security relevance.

Lab 39.1

Run a REXX script

Goal: write and run a short REXX script with output and a variable.

Background: REXX is the mainframe's scripting language. Running one is the start of automation, and of reading others' scripts.

1. Write a script that prints a line and computes a value.

```
SAY "HELLO FROM REXX" ; X = 2 + 3 ; SAY "X =" X
```

Check yourself: the script prints its line and the computed value, and you can explain that REXX can also issue TSO commands and reach data sets with the user's authority.

Why it matters: REXX is automation and, in the wrong hands, a tool with system reach. Reading it tells you which.

Blue-Team angle: a REXX script that issues privileged TSO commands or reads sensitive data sets is doing so with its runner's authority, review scripts as you would any code with system access.

Lab 39.2

Read JCL

Goal: read a job's JOB, EXEC, and DD statements and say what it will run.

Background: JCL is the batch glue. Reading it tells you exactly what a job does before it runs.

1. Read a job and identify its program and data set.

Check yourself: you can name the job, the program its step runs, and the data set the DD supplies, and explain that the job runs with the submitter's authority.

Why it matters: a job is code execution. Reading its JCL is how you know what code, against what data, under whose identity.

Blue-Team angle: for a job of uncertain origin, read the EXEC and DD statements first, they tell you what it runs and touches, the fastest triage there is.

Lab 39.3

Compile COBOL

Goal: compile a COBOL program and read its listing and return code.

Background: COBOL is the business logic. Compiling and reading it is how its bugs (including security bugs) are found.

1. Compile a short program and read the listing.

Check yourself: you can identify the IDENTIFICATION and PROCEDURE divisions, read the numbered source, and confirm a clean compile (RC=0).

Why it matters: reading a PROCEDURE division is how you spot an unvalidated move before it becomes an overflow. Code review is a security control.

Blue-Team angle: scan COBOL for unbounded MOVES of untrusted input into fixed fields, the storage-overflow pattern from the banking chapter, found by reading.

Lab 39.4

Read an assembler listing

Goal: assemble a routine and read its instructions and symbol table.

Background: assembler is the metal, and the language of authorized code. Reading it is essential to deep analysis.

1. Assemble a small routine and read the listing.

Check yourself: you can read the CSECT, the register instructions (LR, BR), the location counter, and the symbol table, and explain that authorized assembler can bypass normal controls.

Why it matters: the most serious mainframe exploits are written in assembler. Reading it is how a defender understands what privileged code really does.

Blue-Team angle: authorized (APF) assembler programs deserve the closest review, they can issue supervisor calls and touch protected storage, so what they do must be understood exactly.

Lab 39.5

Read code as a defender

Goal: connect each language to the security question reading it answers.

Background: programming literacy is security literacy here. This lab makes the connection explicit.

1. For a REXX script, a job, a COBOL program, and an assembler routine, state the security question reading each answers.

Check yourself: you can say that reading REXX reveals what a script does with its authority, JCL what a job runs, COBOL whether input is validated, and assembler what privileged code really does.

Why it matters: every code artifact on the system is opaque until read. The ability to read all four is the foundation of assessment and investigation.

Blue-Team angle: build reading fluency in all four languages, it is the single capability that most improves a defender's ability to judge whether code is benign or hostile.

REXX with system reach

REXX's power becomes vivid when a script drives the system itself. A loop prints a few lines, and then ADDRESS TSO hands a command to TSO (here a RACF query) and captures the result back into the script:

```
DO I=1 TO 3; SAY "LINE" I; END; ADDRESS TSO "LISTUSER IBMUSER"
LINE 1
LINE 2
LINE 3
USER=IBMUSER  NAME=IBMUSER  OWNER=#SYSPROG
ATTRIBUTES=SPECIAL
DEFAULT-GROUP=SYS1  PASS-INTERVAL=30
```

REXX driving TSO, a script runs a RACF command and reads the result, all under its user's authority.

The script did more than print: it issued LISTUSER and received IBMUSER's RACF profile, SPECIAL attribute and all. That is the whole point about REXX's reach (a script can query and change the security database, submit jobs, and move data, limited only by what its user is allowed to do. Legitimate automation relies on exactly this. So does a hostile script: the same three lines that print a greeting could enumerate privileged users, and a longer one could act on what it finds. This is why reading a REXX script) following which TSO and ISPF commands it issues, is a security review, not a formality. A script is only as trustworthy as what it does with the authority it runs under.

A complete job

Real JCL is usually more than one step. The classic pattern compiles a program, link-edits it into a load library, and runs it (compile, link, go) each an EXEC step feeding the next through DD statements:

```
//BUILDRUN JOB (ACCT),CLASS=A,MSGCLASS=X
//COMPILE EXEC PGM=IGYCRCTL          <- compile the COBOL
//SYSIN DD DSN=IBMUSER.SOURCE(PGM),DISP=SHR
//LINK EXEC PGM=IEWL                 <- link into a load library
//SYSLMOD DD DSN=IBMUSER.LOADLIB(PGM),DISP=SHR
//GO EXEC PGM=PGM                    <- run the result
```

A three-step compile-link-go job, the classic batch pattern, each step feeding the next.

Read top to bottom and the job's whole intent is visible: compile the source at IBMUSER.SOURCE(PGM), link the result into IBMUSER.LOADLIB(PGM), then execute it. Every step runs with the submitter's authority, and the DD statements name exactly which data sets are read and written, including the load library the job updates. For a

defender that last point matters: a job that writes a load library is placing executable code, so reading the DDs tells you not just what a job computes but what it changes. The provenance and content of a job are both readable in its JCL, which is why reading it is the first step in trusting it.

Lab 39.6

Follow a script's and a job's reach

Goal: read a REXX script that issues a TSO command and a multi-step job that writes a load library, and state what each touches and under whose authority.

Background: reach is what makes scripts and jobs powerful and risky. This lab traces both to their effects.

1. Read a REXX script that issues a RACF command and note what it can learn or change.
2. Read a compile-link-go job and identify the load library it writes.

Check yourself: you can explain that the REXX script runs RACF commands with its user's authority, and that the job writes executable code into a load library, both actions bounded only by the identity they run under.

Why it matters: reading for reach (what a script or job touches and changes) is how you judge trust. The effect, not the syntax, is the security question.

Blue-Team angle: a script that queries the security database or a job that writes a load library are both worth close review, the first can enumerate, the second can place code. Read the commands and the DDs.

Checkpoint: can you run a REXX script, read JCL, compile and read COBOL, read an assembler listing, and explain each language's security relevance? Those reading skills turn you from an interface user into someone who can judge what the code on a mainframe actually does, the foundation for everything in Parts VIII and IX.

The mainframe runs four core languages, and reading them is a defensive skill. REXX is the scripting language for automation, powerful because it runs with its user's authority and can issue TSO commands and reach data sets. JCL is the job-control glue (JOB, EXEC, DD) and a job is code execution under the submitter's identity. COBOL is the business logic, whose fixed-length field handling is the source of the storage-overflow class, found by reading the PROCEDURE division. HLASM is the assembler, the language of the operating system and of authorized code that can bypass normal controls, and reading it is essential to the deepest analysis. The unifying lesson is that you cannot secure or investigate code you cannot read: programming literacy is security literacy on the mainframe, and the ability to read all four languages is the foundation for the recon, exploitation, and defense that follow. This closes Part VII.

What comes next

Parts I through VII have built a complete working knowledge of the mainframe (its foundations, its security, its operation, its subsystems, its connectivity, and its languages.

Part VIII turns that knowledge outward, to reconnaissance: the tools and techniques for discovering and mapping a mainframe from the outside, as both an attacker and a defender must understand them. It opens with an overview of the recon toolkit Gibson models) nmap and its mainframe scripts, EZRecon, web-application scanners, and the specialized mainframe tools, always with the defender's goal of knowing what can be discovered so it can be protected.

Part VIII

Reconnaissance: Mapping the Mainframe

Defence begins with seeing what an attacker sees. Part VIII is reconnaissance from the defender side: the toolkit, scanning, external footprinting, web and OSINT techniques, and the pivots that turn a foothold into a map of the estate.

CHAPTER 40

The Reconnaissance Toolkit

By the end of this chapter you will be able to:

Explain why reconnaissance matters to defenders.

Describe the phases of mainframe reconnaissance.

Survey the recon tools Gibson models and what each is for.

Read the nmap toolkit menu and its bounded, safe options.

Use recon tools to assess your own exposure.

Plan detection of reconnaissance against a mainframe.

PARTS I THROUGH VII BUILT a complete working knowledge of the mainframe. Part VIII turns that knowledge outward, to reconnaissance (the discovery and mapping of a system from the outside. It is essential to be clear about why this is here and how it is framed. Reconnaissance is the first phase of every attack, which makes it the first thing a defender must understand: to protect a mainframe, you have to know what an attacker can learn about it. Every technique in this part is presented for the defender, and paired with the same three questions) what does this reveal, how do we reduce that exposure, and how do we detect the reconnaissance itself. Gibson's toolkit reflects this: its recon tools are explicitly bounded and educational, labelled "safe forensic mode" and "bounded audit," built to teach discovery and defense on a sandbox, not to attack real systems. This chapter is the map of the part: the phases of recon, the tools that perform them, and the defender's use of both.

Why reconnaissance matters to defenders

It is tempting to think of reconnaissance as an attacker's concern, but it is a defender's concern first. An attacker uses recon to build a picture of a target before striking; a defender must build that same picture of their own systems, because you cannot protect an exposure you do not know you have. Running the same discovery a defender's adversary would run (what ports are open, what services answer, what they reveal) is how you find the TN3270 port that should not be internet-facing, the service that leaks version information, the account that can be enumerated. Reconnaissance has a second defensive value: it leaves traces. Scans, enumeration attempts, and credential probes all generate activity, and detecting that activity is early warning, the difference between learning of an attacker during their first phase and learning of them after the last. Understanding recon is thus doubly defensive: it reveals what to protect, and it teaches what to watch for.

The phases of reconnaissance

Reconnaissance follows a recognizable progression, from the broadest external view down to specific weaknesses, and each phase maps to tools Gibson models:

Phase	Goal	Gibson tooling
Footprint	What exists, from outside the network	EZRecon, OSINT
Scan	Which hosts and ports are reachable	nmap
Enumerate	What each service reveals about itself	nmap NSE scripts
Credential	Whether weak or default logins exist	bounded audits

The reconnaissance phases, from external footprint to credential probing.

The progression narrows at each step. Footprinting works from outside the network entirely (DNS records, public information) to learn what an organization exposes. Scanning finds which hosts and ports are reachable. Enumeration asks each reachable service what it is and what it reveals. Credential work tests whether any login is weak or default. A defender walks the same path in reverse when hardening: reduce what the footprint shows, close unnecessary ports, quiet services that leak, and eliminate weak credentials, and instrument each phase for detection.

The Gibson recon toolkit

Gibson models a full recon toolkit, each tool the subject of a later chapter in this part. Taken together they cover the whole progression:

Tool	What it does	Chapter
nmap + NSE	Scan ports and enumerate mainframe services	40
EZRecon	External footprinting, DNS, MX, subdomains	41
Web scanners	Web-application reconnaissance (nikto, etc.)	42
Mainframe tools	cicspwn, Db2 and JES tools, PassTickets	43
OSINT	Open-source intelligence gathering	44
Pivoting	Lateral movement to a second host (LENNOX)	45

The Gibson recon toolkit, each tool covered in its own chapter this part.

The catalog spans the whole model: external footprinting and OSINT for the outside view, nmap for scanning, mainframe-specific tools for enumerating CICS and the other subsystems, and pivoting for moving to a second system once a foothold exists. Each chapter presents its tool the same way (what it discovers, why that matters, how to reduce the exposure, and how to detect the tool in use) so that the part as a whole is a defender's guide to the attacker's toolkit.

The nmap toolkit at a glance

nmap is the workhorse, and Gibson wraps its mainframe capabilities in a guided menu whose options preview the enumeration to come, and whose wording shows the bounded, safe design throughout:

```
nmap -M
```

```
Gibson nmap-sim guided menu
```

- 1 Quick screen grab
- 2 TSO user enumeration
- 3 Safe CICS information gathering
- 4 CICS transaction enumeration
- 5 CICSPWN simulation: safe forensic mode
- 6 Bounded TSO credential audit
- 7 Bounded CICS credential audit
- 8 Full Gibson classroom smoke run

The nmap-sim menu, mainframe enumeration options, all bounded and safe by design.

Read the options and the mainframe recon surface appears in miniature: grab a TN3270 screen, enumerate TSO users, gather CICS information and transactions, and (explicitly bounded and in “safe forensic mode”) audit credentials. Behind the menu sit the mainframe NSE scripts (nmap Scripting Engine): tn3270-screen, tso-enum, cics-enum and cics-info, and the bounded brute options. The next chapter takes these apart one by one. What matters here is the shape: nmap on the mainframe is not generic port scanning, it is mainframe-aware enumeration, and a defender who knows what it can find knows what their own systems reveal to it.

The defender’s use of the toolkit

The single most important idea in Part VIII is that these are assessment tools. A defender runs them against their own systems to see exactly what an attacker would find, then acts on the result. This is not a metaphor; it is the workflow. Point the recon toolkit at your mainframe, read what it discovers, and you have a prioritized list of exposures, an internet-facing TN3270 port, a service that leaks its version, users that enumerate, a default credential that answers. Each finding then gets the same treatment as every finding in this book: reduce the exposure, and add detection. Reduce by closing the port, quieting the service, removing the weak credential; detect by watching for the scan, the enumeration, the failed logins that mark someone else running the same tools. The attacker’s toolkit, turned on your own systems, is one of the most efficient security assessments there is.

What recon reveals, and how to reduce it

Each thing reconnaissance can discover has a matching reduction and a matching detection, and holding the three together is the defender’s frame for the whole part:

Discovery	Reduce	Detect
Open TN3270 port	Restrict to trusted networks	Connection attempts from outside
Version banners	Suppress or minimize banners	Scanning patterns
Enumerable users	Limit enumeration; generic errors	Enumeration attempts
Weak credentials	Enforce strong auth	Failed-login bursts

Recon discoveries paired with their reductions and detections, the defender’s frame.

This table is the template for every chapter that follows. Whatever a given tool discovers, the defender's response is the same shape: make the discovery yield less, and make the attempt to discover it visible. An open port becomes a restricted one and a monitored one; a chatty service becomes a quiet one and a watched one. By the end of Part VIII you will have seen the full attacker toolkit, and for each tool the reduction and the detection that answer it, the defensive value of understanding offense.

The mistake is to treat reconnaissance as purely an attacker's activity and never run it against your own systems. An exposure you have not looked for is one you will not find until someone else does. Run the recon toolkit against your own mainframe, read what it reveals, and act, self-assessment with the adversary's tools is among the most valuable things a defender can do.

Hands-On Labs

These labs orient you in the toolkit and the defender's use of it. Every tool in this part is bounded and runs against the Gibson sandbox.

Lab 40.1

Map the phases

Goal: describe the reconnaissance phases and what each seeks.

Background: recon follows a progression from footprint to credential. Knowing it frames everything ahead.

1. List the phases from footprint to credential and state the goal of each.

Check yourself: you can name footprinting, scanning, enumeration, and credential testing, and explain that each narrows from the external view to specific weaknesses.

Why it matters: the phases are the structure of both attack and assessment. Knowing them tells you what to look for and in what order.

Blue-Team angle: hardening walks the phases in reverse (reduce the footprint, close ports, quiet services, fix credentials) and instruments each for detection.

Lab 40.2

Survey the toolkit

Goal: name the tools in the toolkit and what each discovers.

Background: each tool serves a phase. Surveying them previews the part.

1. For each tool (nmap, EZRecon, web scanners, mainframe tools, OSINT, pivoting) state what it discovers.

Check yourself: you can match each tool to its purpose and to the phase it serves, from external footprinting to lateral movement.

Why it matters: knowing the toolkit is knowing the attacker's capabilities, and therefore what your defenses must answer.

Blue-Team angle: for each tool, ask what it would find on your systems. That question is the start of every hardening effort.

Lab 40.3

Read the nmap menu

Goal: read the nmap-sim menu and identify the enumeration each option performs.

Background: the nmap menu previews mainframe enumeration. Reading it shows the recon surface.

1. Display the nmap-sim guided menu and read its options.

```
nmap -M
```

Check yourself: you can identify the screen grab, TSO and CICS enumeration, and the bounded credential audits, and note the safe, bounded framing of each.

Why it matters: the menu is the mainframe recon surface in miniature. Each option maps to something your systems reveal.

Blue-Team angle: for each menu option, know what it would return against your mainframe, that is the exposure to reduce and the activity to detect.

Lab 40.4

Assess your own exposure

Goal: frame the toolkit as self-assessment and list what to look for.

Background: the highest-value use of recon is against your own systems. This lab frames that use.

1. Describe how you would run the toolkit against your own mainframe and what findings you would expect to act on.

Check yourself: you can explain that self-directed recon yields a prioritized exposure list (open ports, leaky services, enumerable users, weak credentials) each with a reduction and a detection.

Why it matters: an exposure you find yourself is one you can fix before an attacker uses it. Self-assessment is proactive defense.

Blue-Team angle: schedule recurring self-recon so new exposures are caught as systems change, a one-time scan is a snapshot, not a control.

Lab 40.5

Plan recon detection

Goal: for each recon phase, state what activity to detect.

Background: recon leaves traces. Detecting them is early warning. This lab plans that detection.

1. For scanning, enumeration, and credential testing, state the observable activity a defender should monitor.

Check yourself: you can map scanning to connection patterns, enumeration to repeated service queries, and credential testing to failed-login bursts, a detection plan across the phases.

Why it matters: catching recon is catching an attacker in their first phase, the earliest and cheapest place to stop them.

Blue-Team angle: the detections here connect directly to the SMF and monitoring work of Part IV, recon shows up as the very events those tools already collect.

A first look at what a scan sees

Before the detailed chapters, it helps to see the scan phase in one picture. Pointed at a mainframe, nmap reports the open ports and, service-aware, names what answers on each, and the result is a fingerprint that says “mainframe” at a glance:

```
nmap -sV mainframe.example.com
```

PORT	STATE	SERVICE	VERSION
21/tcp	open	ftp	z/OS FTP (FILETYPE JES/SQL capable)
23/tcp	open	tn3270	VTAM TN3270 terminal
175/tcp	open	nje	IBM Network Job Entry (JES2)
2252/tcp	open	nje	NJE over TLS
8443/tcp	open	http	Gibson administration dashboard

A service scan of a mainframe, the open ports and their services form an unmistakable fingerprint.

Every port here is a chapter of this book: FTP with its FILETYPE powers, the TN3270 terminal, NJE clear and TLS, the web dashboard. To an attacker this single view says a mainframe is present and lists its doors; to a defender it is the exposure inventory, read from the outside as an adversary would read it. The very recognizability of the fingerprint is itself worth noting, these ports and services announce the platform, so the defensive questions begin immediately: does each of these need to be reachable from where the scan came, and is each one monitored for exactly this kind of probe?

The mainframe NSE scripts

Scanning finds the doors; the nmap Scripting Engine (NSE) scripts look through them. Gibson models the mainframe-specific scripts, each enumerating one subsystem:

NSE script	What it reveals
tn3270-screen	Captures the initial TN3270 logon screen
tso-enum	Enumerates valid TSO user IDs
cics-info / cics-enum	CICS region details and transactions
tso-brute / cics-user-brute	Bounded credential audits
nje-node-brute	Enumerates NJE node names

The mainframe NSE scripts, each looks through one door a scan has found.

Each script turns an open port into information: the TN3270 screen grab reveals the logon banner and often the system’s identity; tso-enum turns the logon into a list of valid user

IDs; the CICS scripts reveal the transaction environment; the node-brute enumerates the NJE network from Chapter 35. The next chapter runs each of these against Gibson and, for each, gives the reduction and the detection. The pattern to carry forward is that a scan reveals the platform and the NSE scripts reveal its details, and a defender who knows exactly what each returns knows exactly what to quiet and what to watch.

Lab 40.6

Read a mainframe fingerprint

Goal: read a service scan of a mainframe and explain why the open ports form a recognizable fingerprint, then map each to a defensive question.

Background: the scan phase produces a fingerprint. Reading it is the start of self-assessment.

1. Read a service scan and identify the mainframe-indicating ports and services.

```
nmap -sV mainframe.example.com
```

Check yourself: you can name FTP, TN3270, NJE (clear and TLS), and the web dashboard from the scan, and explain that together they identify the platform, and for each you can ask whether it needs to be reachable and whether it is monitored.

Why it matters: the fingerprint is what an attacker sees first. Reading it as a defender turns it into an exposure inventory and a monitoring checklist.

Blue-Team angle: reduce the fingerprint by restricting which ports are reachable from untrusted networks, and detect the scan itself, a sweep across exactly these ports is a recognizable early signal.

Lab 40.7

Turn a finding into a control

Goal: take one recon discovery end to end, name the exposure, the reduction, and the detection.

Background: the whole of Part VIII reduces to this loop. This lab rehearses it once so the pattern is second nature for the chapters ahead.

1. Pick one discovery from the fingerprint (say, an internet-reachable TN3270 port) and write its exposure, its reduction, and its detection.

Check yourself: for the open TN3270 port you can state the exposure (a terminal login reachable from untrusted networks), the reduction (restrict the port to trusted sources), and the detection (alert on connection attempts from outside those sources), a complete finding-to-control loop.

Why it matters: every tool chapter that follows produces discoveries, and each is only useful once it becomes a reduction and a detection. Practicing the loop once makes the rest of the part fall into place.

Blue-Team angle: keep a running table of discovery / reduction / detection as you work through Part VIII, by the end it is a ready-made hardening and monitoring plan for a mainframe.

Checkpoint: can you explain why recon matters to defenders, describe its phases, survey the Gibson toolkit, read the nmap menu, and frame the whole part as self-assessment paired with detection? Those ideas turn the chapters ahead from an attacker's manual into a defender's guide to the attacker's toolkit.

Reconnaissance is the first phase of an attack and therefore the first concern of a defender: to protect a mainframe you must know what it reveals. Recon progresses from external footprinting through scanning and service enumeration to credential testing, and Gibson models a tool for each (EZRecon and OSINT for the outside view, nmap with mainframe NSE scripts for scanning and enumeration, mainframe-specific tools like cicspwn, and pivoting to a second host) all explicitly bounded and educational. The defender's use of this toolkit is self-assessment: run it against your own systems to produce a prioritized exposure list, then answer each finding the same way, by reducing the exposure and adding detection. Recon also leaves traces, so detecting it is early warning. Every chapter in Part VIII follows this frame (what a tool discovers, how to reduce it, and how to detect its use) making the part a defender's guide to the attacker's toolkit.

What comes next

With the map in hand, the next chapter takes up the workhorse in detail: nmap on the mainframe. It walks the mainframe NSE scripts one by one (the TN3270 screen grab, TSO and CICS enumeration, the bounded credential audits, and node discovery) showing exactly what each reveals, and for each, the reduction and detection a defender applies. It is the first deep look at the toolkit this chapter surveyed.

CHAPTER 41

nmap on the Mainframe

By the end of this chapter you will be able to:

Use nmap's mainframe NSE scripts and read their output.

Explain what tn3270-screen and tso-enum reveal.

Enumerate CICS with cics-info and cics-enum.

Understand the bounded credential audits.

See how a secured CICS defeats cicspwn, and correlate it.

Pair each script with a reduction and a detection.

NMAP IS THE WORKHORSE OF reconnaissance, and on the mainframe it is far more than a port scanner. Through the nmap Scripting Engine (NSE) it runs mainframe-aware scripts that look through each open port and enumerate the service behind it. This chapter walks those scripts one by one, using Gibson's bounded, safe implementations, and for every script it answers the defender's three questions: what does this reveal, how do we reduce that exposure, and how do we detect the script in use. The chapter's payoff comes at the end, with cicspwn: run against a properly secured CICS, the tool is blocked at every turn, which is the clearest possible demonstration that the transaction security from Part VI actually works. Reconnaissance, read this way, is a checklist of what to harden and what to watch.

From fingerprint to enumeration

A service scan identifies the mainframe and its open ports; the NSE scripts then interrogate each. The scripts target the mainframe's own protocols (TN3270, TSO, CICS, NJE) and each turns an open port into specific information. The nmap menu from the last chapter is the front end; behind it, each option runs a script against the VTAM front door on the terminal port. The sequence mirrors the recon phases: grab a screen, enumerate users, enumerate the application, then (bounded and safe) test credentials. Reading each in turn shows exactly what a mainframe reveals to an attacker who knows these scripts, which is precisely what a defender needs to know to reduce it.

tn3270-screen: grabbing the logon

The simplest script captures the initial TN3270 screen, what any client sees on connecting:

```
nmap --script tn3270-screen -p 23 <host>
PORT      STATE SERVICE
2023/tcp  open  gibson-vtam
| tn3270-screen:
```

```
| screen:
| GIBSON VTAM FRONT DOOR
```

tn3270-screen, the logon screen captured, revealing the system's front door.

The logon screen is often surprisingly informative: it can name the system, the organization, the VTAM application, sometimes even software levels and (unwisely) hints about how to log on. To an attacker it confirms a mainframe and gives a first orientation. The reduction is to minimize what the banner reveals: a logon screen should not name internal systems, versions, or give usage hints, and a legal-warning banner is preferable to a chatty one. The detection is to watch for connections that grab the screen and disconnect, a client that connects, reads, and leaves without logging on is scraping, not working.

tso-enum: enumerating users

More revealing is user enumeration. The tso-enum script tests user IDs and reads how the system responds, and the response is an oracle:

```
nmap --script tso-enum <host>
| tso-enum:
| IBMUSER      CONFIRMED    PASSWORD_PROMPT
| GRRR         UNAVAILABLE  USER_NOT_FOUND
```

tso-enum, a valid user draws a password prompt; an invalid one is told it does not exist.

The weakness is that the system responds differently to a valid user than an invalid one: IBMUSER, a real ID, is met with a password prompt, while a made-up ID is told the user does not exist. That difference lets an attacker confirm which user IDs are real without knowing a single password (building the target list for a later credential attack. The reduction is the classic one: make the responses identical, so a valid and an invalid user are indistinguishable) a generic message either way. The detection is to watch for a run of logon attempts against many different user IDs, the signature of enumeration.

cics-info and cics-enum

Two scripts enumerate CICS. The first gathers region information; the second lists transactions:

```
nmap --script cics-info,cics-enum <host>
| cics-info:
| region: Region CONFIRMED GIBCICS
| cics-enum:
| CESN    CONFIRMED LOGON_TRANSACTION
| CESL    CONFIRMED LOGON_TRANSACTION
```

cics-info and cics-enum, the region identity and the transactions it exposes.

cics-info confirms a CICS region and names it; cics-enum lists the transactions reachable without signing on. That list is a map of the application's surface, and if it includes the powerful built-in transactions (CEMT, CECI, CEDA) it is a serious finding, because it means they are reachable. The reduction is exactly the transaction security from the CICS chapter: with TCICSTRN active and the dangerous transactions restricted, enumeration reveals far less and the powerful transactions are not reachable to enumerate. The detection is repeated transaction probes against a region, the CICS equivalent of user enumeration.

The bounded credential audit

The credential-testing scripts are where Gibson's bounded, safe design is most visible. The tso-brute audit tests a small, bounded set and reports simulated results, warning plainly that it is a training simulation:

```
| tso-brute:
| Warning: bounded training simulation, sandbox use only
|  IBMUSER   CONFIRMED   SIMULATED_CREDENTIAL_ACCEPTED
|  IBMUSER   UNAVAILABLE SIMULATED_LOGIN_REJECTED
|  GRRR      UNAVAILABLE SIMULATED_LOGIN_REJECTED
```

The bounded TSO credential audit, a small, safe, clearly-labelled simulation.

The purpose here is not to break in but to demonstrate the class of attack so a defender can answer it. Credential testing (guessing or spraying passwords) is defeated by the controls from the RACF chapters: strong password rules, and revocation after a few failures so guessing cannot continue. The detection is the failed-login burst, which lands in SMF exactly as the forensics chapter described, a rash of failures against one or many users is credential testing, and it is one of the most reliable early signals a defender has.

cicspwn: what a secured system does to it

The most instructive script is cicspwn, which attempts to assess and exploit a CICS region through its powerful transactions. Run against Gibson's secured CICS in safe forensic mode, the result is the whole point of the book in one output:

```
| cicspwn: (bounded training simulation)
| Stage: transaction-access
|  CESN   CONFIRMED  LOGON_TRANSACTION
|  CEMT   DENIED     SECURITY_PROTECTED
|  CEDA   DENIED     SECURITY_PROTECTED
|  CECI   DENIED     SECURITY_PROTECTED
| Stage: capability-assessment
|  Administrative path  DENIED  CEMT_PROTECTED
|  Define/install path  DENIED  CEDA_PROTECTED
```



```
|_ Result: BLOCKED, no simulated exploit path completed
```

cicspwn against a secured CICS, the powerful transactions are protected, and the tool is blocked.

Read what happened. cicspwn tried the dangerous transactions (CEMT to control the region, CEDA to define resources, CECI to run commands) and every one was DENIED, SECURITY_PROTECTED. The administrative and define paths were blocked. The final line is the reward for good security: BLOCKED, no exploit path completed. This is transaction security from the CICS chapter working exactly as designed, the tool that would compromise an unsecured region finds every door locked. The lesson could not be clearer: the reductions this book prescribes are what turn a successful attack into a blocked one. And the tool leaves a trail for the defender to follow.

Correlating the attempt

Even blocked, the attempt should be detected, and the safe tooling produces exactly the artifacts a defender needs, a correlation ID and pointers to where the activity was logged:

```
| Stage: forensic-correlation
| Correlation ID   CICSPWN-20260701-141552-88AF
| SDSF             Review ST / O / H output
| OPERLOG          Review CICS transaction activity
| CICS events      Review security / forensic events
```

The forensic-correlation stage, a correlation ID and where to find the attempt in the logs.

The tool hands the defender a correlation ID and tells them where to look: SDSF for the job output, OPERLOG for the console timeline, and the CICS security events. That is the detection half of the frame made concrete, the attempt, blocked or not, is visible in the very logs Part IV taught you to read, and the correlation ID ties the pieces together. A blocked attack that is also detected is the ideal outcome: the reduction stopped it, and the detection caught it. Every NSE script in this chapter fits the same frame:

Script	Reveals	Reduce / Detect
tn3270-screen	Logon banner, system identity	Minimize banner / scrape attempts
tso-enum	Valid user IDs	Generic errors / enum bursts
cics-info,enum	Region and transactions	Transaction security / probes
tso-brute	Weak credentials	Strong auth, revoke / failed-login bursts
cicspwn	Admin-path exposure	Protect CEMT/CEDA/CECI / correlation IDs

The mainframe NSE scripts, what each reveals, and how a defender reduces and detects it.

The mistake is reading this chapter as an attack manual rather than a hardening checklist. Every script here maps to a control already in the book (banner minimization, generic errors, transaction security, strong credentials) and the cicspwn result proves those controls work: against a secured system the tool is blocked at every stage. Run the scripts against your own systems, and treat each thing they reveal as a reduction to make and an activity to detect.

Hands-On Labs

These labs run the mainframe NSE scripts against the Gibson sandbox and pair each with its defense.

Lab 41.1

Grab the logon screen

Goal: run tn3270-screen and assess what the banner reveals.

Background: the logon screen is the first thing recon captures. This lab reads it as a defender.

1. Run the screen-grab option and read the captured screen.

```
nmap -M 1
```

Check yourself: you can read the captured logon screen and identify what it reveals, and explain that a minimal banner reduces that exposure.

Why it matters: the banner is free intelligence for an attacker. Minimizing it costs nothing and reveals less.

Blue-Team angle: a connect-read-disconnect with no logon is screen scraping, detectable as a session that never authenticates.

Lab 41.2

Understand user enumeration

Goal: run tso-enum and explain the valid-versus-invalid oracle.

Background: user enumeration builds the target list. This lab studies the oracle defensively.

1. Run the TSO enumeration option and compare the responses for a real and a fake user.

```
nmap -M 2
```

Check yourself: you can show that a valid user draws a password prompt and an invalid one a not-found message, and explain that identical, generic responses close the oracle.

Why it matters: enumeration is credential attack preparation. Removing the response difference removes the free target list.

Blue-Team angle: detect enumeration as many logon attempts across many user IDs, a pattern SMF records and monitoring can flag.

Lab 41.3

Enumerate CICS

Goal: run cics-info and cics-enum and assess the transaction exposure.

Background: CICS enumeration maps the application surface. This lab reads it against transaction security.

1. Run the CICS information and transaction enumeration options.

```
nmap -M 3
```

```
nmap -M 4
```

Check yourself: you can read the region identity and the enumerated transactions, and explain that transaction security reduces what is reachable to enumerate.

Why it matters: an enumerated CEMT, CECI, or CEDA is a reachable dangerous transaction. Transaction security keeps them off the list.

Blue-Team angle: enumeration probes against a region are detectable, and the best reduction (transaction security) is already the CICS chapter's core control.

Lab 41.4

Read the bounded audit

Goal: run the bounded credential audit and state the controls that defeat credential testing.

Background: the bounded audit demonstrates credential testing safely. This lab reads it and its defenses.

1. Run the bounded TSO credential audit and read its simulated results.

```
nmap -M 6
```

Check yourself: you can read the bounded, simulated results and explain that strong passwords and revocation after failures defeat credential testing, which SMF records as failed-login bursts.

Why it matters: credential testing is common and noisy. Strong auth stops it and the failed-login burst detects it.

Blue-Team angle: the failed-login burst is one of the most reliable detections, connect it to the SMF and monitoring work from Part IV.

Lab 41.5

Watch cicspwn get blocked

Goal: run cicspwn against a secured CICS, observe it blocked, and read the correlation for detection.

Background: this is the payoff, good security defeats the tool, and the attempt is still detectable. This lab shows both.

1. Run the safe cicspwn simulation and read the denied transactions and the blocked result.

```
nmap -M 5
```

Check yourself: you can show that CEMT, CEDA, and CECI are DENIED SECURITY_PROTECTED, that the result is BLOCKED with no exploit path completed, and that the tool provides a correlation ID and log pointers for detection.

Why it matters: this is the reductions of Part VI proven, transaction security turns a compromise into a blocked, detected attempt. The best possible outcome.

Blue-Team angle: use the correlation ID to find the attempt in SDSF, OPERLOG, and the CICS events, a blocked attack you can also see is the ideal, and the logs are the ones Part IV taught you to read.

nje-node-brute: enumerating the network

One more script reaches beyond a single host to the NJE network of Chapter 35. Because the OPEN handshake answers differently for a valid node name than an invalid one, a script can enumerate node names by watching the ACK-versus-NAK response, the same oracle as user enumeration, one level up:

```
nmap --script nje-node-brute -p 175 <host>
| nje-node-brute:
|   GIBSON    VALID    OPEN_ACK
|   HAL       VALID    OPEN_ACK
|   ORAC      VALID    OPEN_ACK
|   ZORK      INVALID   OPEN_NAK
```

nje-node-brute, valid node names draw an ACK, invalid ones a NAK, enumerating the NJE network.

The script confirms which node names the target will talk to (mapping the trust network an attacker would then try to abuse through the cross-node execution and NMR spoofing of Chapter 35. The reduction is the one from that chapter: require TLS so the handshake is not exposed to an anonymous prober, and the ACK/NAK difference cannot be observed without a valid certificate. The detection is repeated OPEN attempts with differing node names from one source) node enumeration has the same burst signature as user enumeration, and the clear NJE port is where to watch for it. The pattern holds across every script in this chapter: a response difference is an oracle, the reduction removes or hides the difference, and the burst of attempts is the detection.

Lab 41.6

Enumerate the NJE network defensively

Goal: run node enumeration, explain the ACK/NAK oracle, and connect it to the NJE reductions from Chapter 35.

Background: node enumeration maps the NJE trust network. This lab reads it as a defender and ties it to TLS.

1. Run the node-brute script and note which node names return an ACK versus a NAK.

```
nmap --script nje-node-brute -p 175 <host>
```

Check yourself: you can explain that valid nodes ACK and invalid ones NAK (enumerating the network) and that requiring TLS hides the handshake so the oracle cannot be probed anonymously.

Why it matters: enumerating the node network is the first step toward abusing its trust. Hiding the handshake behind TLS closes the enumeration.

Blue-Team angle: watch the clear NJE port for repeated OPEN attempts with varying node names, and require TLS, the same control that stops NMR spoofing also stops node enumeration.

Checkpoint: can you run the mainframe NSE scripts, explain what each reveals, and (above all) show how a secured CICS blocks cicspwn while still leaving a detectable trail? Those skills turn nmap from an attacker's tool into a defender's assessment of exactly what to harden and watch.

nmap on the mainframe is mainframe-aware enumeration through NSE scripts. tn3270-screen captures the logon banner, reduced by minimizing what it reveals; tso-enum exploits the valid-versus-invalid response difference to confirm user IDs, reduced by generic errors; cics-info and cics-enum map the region and its transactions, reduced by transaction security; and the bounded tso-brute demonstrates credential testing, defeated by strong authentication and revocation. The centerpiece is cicspwn: run against a secured CICS, it finds CEMT, CEDA, and CECI all SECURITY_PROTECTED and is BLOCKED with no exploit path completed (the transaction security of Part VI proven to work) while still emitting a correlation ID and log pointers so the attempt is detected. Every script maps to a reduction already in the book and a detection in the logs of Part IV, making nmap a defender's checklist of what to harden and what to watch.

What comes next

nmap works against the mainframe once you can reach it. But an attacker's first moves happen before any packet touches the target, in the open information about an organization. The next chapter covers EZRecon, Gibson's external footprinting tool (DNS records, mail servers, name servers, subdomain discovery) the reconnaissance that maps an organization from the outside before a single port is scanned, and what a defender can do to reduce that external footprint.

CHAPTER 42

EZRecon: External Footprinting

By the end of this chapter you will be able to:

Explain footprinting, mapping an organization from public data.

Read DNS records: A, MX, NS, and SOA.

Interpret WHOIS registration data.

Explain why an open zone transfer is a serious leak.

Recognize subdomain discovery and email harvesting.

Use Shodan to find internet-exposed mainframe services.

NMAP WORKS ONCE YOU CAN reach a target. But an attacker's first moves happen before any packet touches the mainframe, in the public information an organization exposes to the world. This is footprinting, and Gibson models it with EZRecon: a tool that maps an organization from the outside using DNS records, registration data, name servers, subdomain discovery, email harvesting, and internet-wide scan databases like Shodan. None of this touches the target directly (it reads what is already public) which is exactly why it matters to a defender. Your external footprint is visible to anyone, and the only way to know what it reveals is to look at it yourself. This chapter walks EZRecon's techniques, and for each asks what it exposes and how to reduce it. Detection is limited here, because the queries hit public data rather than your systems, so reduction (shrinking the footprint) is the defense.

Footprinting from public information

Every organization leaves a public trail: the DNS records that let the world find its services, the registration data behind its domains, the name servers that answer for it. Footprinting gathers that trail into a picture of the organization's systems (what hosts exist, what mail infrastructure, what subdomains, what internet-facing services) without ever probing the target. Because it is passive and public, footprinting is nearly invisible to the target and impossible to prevent outright; you cannot stop the world from reading public DNS. What you can do is control what that public data reveals. EZRecon's value to a defender is showing exactly what an attacker assembles for free, so you can decide what should not be there.

DNS records: A, MX, NS, SOA

DNS is the richest public source. The A records give host addresses, MX the mail servers, NS the name servers, and the SOA record the zone's administrative details:

```
ezrecon soa gibsoncorp.com
```

```
SOA record for gibsoncorp.com:
```

```
Primary Name Server : ns1.gibsoncorp.com
Responsible Person  : hostmaster.gibsoncorp.com
Serial Number       : 2026010152
```

An SOA record, the zone's primary name server, admin contact, and serial number.

The SOA names the authoritative server, an administrative contact (often a real email address), and a serial number that changes when the zone does. The MX records reveal the mail path (useful for phishing and for identifying mail-security gateways) and NS records name the servers to query next. Each is legitimately public, so the reduction is not to hide DNS but to ensure it reveals nothing it should not: no internal host names in public records, no administrative details beyond what is required, and no records for systems that should not be known to exist externally at all.

WHOIS: registration data

WHOIS returns the domain's registration record, who owns it, when it was created, and through which registrar:

```
ezrecon whois gibsoncorp.com
WHOIS record for gibsoncorp.com:
Domain Name: GIBSONCORP.COM
Registrar: Lab Registrar, Inc.
Creation Date: 2014-03-12T00:00:00Z
```

A WHOIS record, domain ownership, registrar, and age, all public.

WHOIS reveals ownership and age, and historically exposed registrant names, addresses, and emails, a gift to an attacker building a social-engineering or phishing campaign. The domain's age and registrar can also hint at how carefully it is managed. The reduction is WHOIS privacy: most registrars offer to replace personal contact details with a privacy service, and using it removes the personal information an attacker would otherwise harvest directly from the registration record.

Zone transfer: the big leak

The most serious DNS misconfiguration is an open zone transfer. A zone transfer (AXFR) is meant to replicate a zone to authorized secondary servers, but if a name server allows it to anyone, it hands over the entire zone, every host name and address the organization has published. EZRecon tests for it, and a well-configured server refuses:

```
ezrecon zone_transfer gibsoncorp.com
AXFR attempt for gibsoncorp.com:
Transfer refused by ns1 (REFUSED) - good hygiene.
Transfer refused by ns2 (REFUSED).
```

A refused zone transfer, good hygiene; an open one would leak the entire zone.

Here both name servers refuse the transfer (the tool itself calls it good hygiene) and that is exactly the desired result. An open AXFR is a catastrophic footprinting win: instead of guessing host names one at a time, an attacker downloads the complete list. The reduction is unambiguous: restrict zone transfers to the specific authorized secondary servers and refuse them from everyone else. This is one of the few footprinting exposures with a clean detection too, an AXFR attempt from an unauthorized source is a clear, loggable event on the name server, and worth alerting on.

Subdomains and emails

Two more techniques enrich the picture. Subdomain discovery tries common names to find hosts that are not obvious, and it often turns up the interesting ones:

```
ezrecon subdomains gibsoncorp.com
Subdomains discovered for gibsoncorp.com:
www.gibsoncorp.com    203.0.113.79
dev.gibsoncorp.com    203.0.113.187
test.gibsoncorp.com   203.0.113.245
```

Subdomain discovery, dev and test hosts surface alongside the public website.

The www host is expected; dev and test are the finds. Development and test systems are often less hardened, less monitored, and more likely to hold real data in an insecure form (an attacker's preferred way in. The reduction is to keep non-production systems off public DNS and behind network controls entirely. Email harvesting is the companion technique, gathering published addresses) info@, security@, careers@, that feed phishing and password-spraying campaigns. There is no removing published business addresses, so the reduction there is awareness and anti-phishing controls rather than concealment.

Shodan: the mainframe on the internet

The most alarming EZRecon result comes from Shodan, a search engine that continuously scans the internet and indexes what answers. Query an organization and it returns the services it found exposed, and for a mainframe, that can include the terminal port itself:

```
ezrecon shodan gibsoncorp.com
Shodan results for gibsoncorp.com:
203.0.113.213:23  IBM z/OS TN3270  (VTAM)
```

Shodan, a mainframe TN3270 terminal found exposed directly to the internet.

This is a critical finding. Shodan has indexed a TN3270 terminal (a mainframe logon) reachable directly from the internet, identified by its VTAM fingerprint. A mainframe terminal facing the open internet is a serious exposure: it is the front door of a system running an organization's most critical workloads, offered to every scanner and attacker on the network. The reduction is categorical, mainframe services should not be internet-facing; the terminal, FTP, NJE, and administrative interfaces belong behind network controls, reachable only from trusted networks. And the detection is one every defender

can run: search Shodan for your own organization and addresses, and treat anything mainframe-related that appears as an incident. The attacker's most powerful footprinting tool is equally the defender's fastest way to find what should never have been exposed.

Technique	Reveals	Reduce
A / MX / NS / SOA	Hosts, mail, name servers, admin contact	No internal names in public DNS
WHOIS	Ownership, age, contacts	WHOIS privacy
Zone transfer	The entire zone, if allowed	Restrict AXFR to secondaries
Subdomains	Hidden dev/test hosts	Keep non-prod off public DNS
Email scraper	Addresses for phishing	Awareness; anti-phishing
Shodan	Internet-exposed services	Remove all mainframe exposure

EZRecon techniques, what each reveals and how a defender reduces the footprint.

The footprinting mistake is assuming that because these queries do not touch your systems, there is nothing to defend. The exposure is real and entirely in what you have made public: an open zone transfer, a dev host in public DNS, a mainframe terminal on the internet. You cannot stop footprinting, but you control what it finds, so run these tools against your own organization and remove what should not be there, starting with anything mainframe-facing that Shodan can see.

Hands-On Labs

These labs footprint the Gibson training organization and reason about reducing the footprint.

Lab 42.1

Read the DNS records

Goal: query A, MX, NS, and SOA records and say what each reveals.

Background: DNS is the richest public source. Reading it is the start of footprinting, and of knowing your own exposure.

1. Query the SOA record and note the name server and admin contact.

```
ezrecon soa gibsoncorp.com
```

Check yourself: you can read the primary name server, the responsible person, and the serial, and explain that MX and NS records reveal mail and name infrastructure.

Why it matters: DNS is public and rich. Knowing what it reveals is knowing part of your external footprint.

Blue-Team angle: audit public DNS for internal host names and records for systems that should not be externally known, remove what does not belong.

Lab 42.2

Interpret WHOIS

Goal: read a WHOIS record and identify the harvestable information.

Background: WHOIS reveals ownership and historically personal contacts. This lab reads it as an attacker would.

1. Query the WHOIS record and note ownership, age, and any contact data.

```
ezrecon whois gibsoncorp.com
```

Check yourself: you can read the domain, registrar, and creation date, and explain that exposed contacts feed social engineering, reduced by WHOIS privacy.

Why it matters: registration data is free intelligence for phishing. Privacy services remove the personal parts.

Blue-Team angle: enable WHOIS privacy so registrant details are not harvested straight from the registration record.

Lab 42.3

Test the zone transfer

Goal: test AXFR and explain why an open transfer is catastrophic.

Background: an open zone transfer leaks the entire zone. This lab tests for it and reads the good-hygiene result.

1. Attempt a zone transfer and read whether it is refused.

```
ezrecon zone_transfer gibsoncorp.com
```

Check yourself: you can explain that a refused AXFR is good hygiene, that an open one would hand over every published host, and that the reduction is restricting transfers to authorized secondaries.

Why it matters: an open AXFR turns host-name guessing into a download. Refusing it is one of the highest-value DNS controls.

Blue-Team angle: an AXFR attempt from an unauthorized source is a clear, loggable event, one of footprinting's few clean detections. Alert on it.

Lab 42.4

Find the hidden hosts

Goal: discover subdomains and harvested emails and assess their risk.

Background: dev and test hosts and published emails are attacker favorites. This lab surfaces them.

1. Run subdomain discovery and email harvesting and note what turns up.

```
ezrecon subdomains gibsoncorp.com
```

Check yourself: you can identify dev and test hosts among the subdomains and explain that they are often weaker targets, and that harvested emails feed phishing and spraying.

Why it matters: non-production hosts and published emails are common first footholds. Keeping non-prod off public DNS removes one; awareness answers the other.

Blue-Team angle: keep development and test systems off public DNS and behind network controls, they should never be an attacker's easy way in.

Lab 42.5

Search Shodan for your mainframe

Goal: use Shodan to find internet-exposed mainframe services and treat any as an incident.

Background: Shodan indexes what is exposed. This lab finds a mainframe terminal on the internet, the exposure that must not exist.

1. Query Shodan for the organization and read the exposed services.

```
ezrecon shodan gibsoncorp.com
```

Check yourself: you can identify a TN3270 mainframe terminal exposed to the internet, explain why that is critical, and state the reduction, no mainframe service should be internet-facing.

Why it matters: a mainframe terminal on the open internet is a front door to the most critical systems. Shodan finds it, for the attacker and for you.

Blue-Team angle: search Shodan for your own addresses regularly and treat any mainframe service that appears as an incident, it is the fastest way to catch accidental exposure.

Reverse lookups and the mail path

Two smaller techniques fill in the map. A reverse (PTR) lookup turns an IP address back into a name, which can reveal hosting providers and naming conventions; and the MX records trace the mail path an attacker would use for phishing and would probe for mail-security gateways:

```
ezrecon reverse 203.0.113.213
```

PTR record for 203.0.113.213:

```
203.0.113.213 -> host-203-0-113-213.in-addr.lab
```

MX records for gibsoncorp.com:

```
10 mail.gibsoncorp.com
```

```
20 mail2.gibsoncorp.com
```

Reverse DNS and MX, the naming convention and the mail path, both public.

The reverse lookup on the mainframe's address ties it into the organization's naming, and the MX records name the mail servers by priority. Neither is secret, and neither can be withheld while the services work, so the reduction is again about hygiene: consistent, uninformative reverse names that do not advertise a host's role, and mail infrastructure that reveals no more than it must. The recurring theme of footprinting is that the individual facts are each innocuous, it is their assembly into a complete picture that creates the exposure.

The assembled footprint

Combining every technique, EZRecon builds a single external picture of the organization, and it is worth seeing assembled, because the whole is far more than its parts:

```
gibsoncorp.com - EXTERNAL FOOTPRINT
```

```

Name servers : ns1, ns2           (AXFR refused - good)
Mail         : mail, mail2
Web          : www 203.0.113.79
Non-prod     : dev, test         (<- weaker targets)
Emails       : info@, security@, careers@
Shodan       : 203.0.113.213:23 TN3270 (<- CRITICAL)

```

The assembled footprint, innocuous facts combine into a complete attack-surface map.

Read as a whole, the footprint is an attack plan: it names the systems, the weaker non-production hosts, the emails for phishing, and (most seriously) the mainframe terminal exposed to the internet. No single query was an attack, yet together they hand an adversary a map of where to aim. That is why a defender assembles the same picture first: only by seeing the whole footprint can you judge what should not be in it. Work down the picture and the priorities are clear, remove the Shodan-visible mainframe exposure immediately, pull the non-production hosts off public DNS, and confirm the good hygiene (the refused zone transfer) holds. The assembled view is the deliverable of footprinting, for attacker and defender alike.

Lab 42.6

Assemble and prioritize the footprint

Goal: combine the EZRecon results into a single external picture and prioritize the reductions.

Background: the exposure is in the assembled whole, not any single query. This lab builds the picture and ranks the fixes.

1. Combine the DNS, subdomain, email, and Shodan results into one footprint, then rank what to reduce first.

Check yourself: you can assemble the full picture and prioritize (the internet-exposed mainframe first, non-production hosts next, then confirm the refused zone transfer) a ranked reduction plan from the footprint.

Why it matters: the footprint's power is in assembly. Building it yourself, and ranking the fixes, is how you shrink your external surface deliberately.

Blue-Team angle: run a full external footprint of your own organization on a schedule and keep the assembled picture current, it is your attacker's-eye view, and the top of it should never be a mainframe.

Checkpoint: can you footprint an organization with EZRecon (DNS, WHOIS, zone transfer, subdomains, emails, Shodan) and for each state what it reveals and how to reduce it, above all removing any mainframe service Shodan finds on the internet? Those skills let you see your organization exactly as an attacker's first reconnaissance does.

Footprinting maps an organization from public data before any packet touches it, and EZRecon models the full technique. DNS records (A, MX, NS, SOA) reveal hosts, mail, name servers, and admin contacts; WHOIS exposes ownership, age, and historically personal contacts; an open zone transfer would hand over the entire zone, so refusing AXFR to all but authorized secondaries is essential; subdomain discovery surfaces weaker dev and test hosts; email harvesting feeds phishing; and Shodan indexes internet-exposed services, potentially including a mainframe terminal facing the open internet (a critical exposure. Footprinting is passive and cannot be prevented, so the defense is reduction: keep internal names and non-production hosts out of public DNS, enable WHOIS privacy, restrict zone transfers, and) above all, ensure no mainframe service is internet-facing, using Shodan yourself to confirm it. Your external footprint is visible to everyone; the only way to control it is to look at it first.

What comes next

Footprinting finds the organization's external surface, including its web presence. The next chapter narrows to that web tier: web-application reconnaissance with tools like nikto, aimed at the banking web applications and the management interfaces from Chapter 37. It is where external footprinting meets the web vulnerabilities already seen, discovering the endpoints, the server software, and the exposed management paths that a web attacker would target, and how a defender reduces and detects that discovery.

CHAPTER 43

Web-Application Reconnaissance

By the end of this chapter you will be able to:

Fingerprint a web server and read its version disclosure.

Run and interpret a nikto-style web vulnerability scan.

Enumerate HTTP methods and check CORS policy.

Discover endpoints and management interfaces.

Recognize the Tomcat manager as the high-value target.

Reduce and detect web-application reconnaissance.

FOOTPRINTING FOUND THE ORGANIZATION’S WEB presence; web-application reconnaissance narrows to the web applications themselves. It fingerprints the server software, enumerates the endpoints and methods, checks the security policies, and hunts for the management interfaces that turn web access into control. This is where the external recon of the last chapters meets the web tier of Chapter 37 (the banking REST APIs, the web front ends, and the administration and management interfaces) and where their weaknesses become discoverable. Tools like nikto automate the scan; the Tomcat manager is the prize. Throughout, the defensive frame holds: every technique here reveals something a defender should already have reduced, and web scanners are noisy enough that their use is one of the more detectable phases of reconnaissance.

Fingerprinting the web server

The first move is to identify the server software and version, because that alone often decides the attack. Web servers announce themselves generously, in headers, page titles, and default pages. The root page of Gibson’s web tier is typical:

```
GET / HTTP/1.1
HTTP/1.1 200 OK
<title>Apache Tomcat/9.0.x</title>
<h1>Apache Tomcat/9.0.x</h1>
<ul><li><a href='/manager/html'>Manager App</a></li>
  <li><a href='/docs'>Documentation</a></li>
  <li><a href='/examples'>Examples</a></li></ul>
```

A web server fingerprint, the exact product and version, and links to the manager.

The page names the exact product and version (Apache Tomcat 9.0.x) and, worse, links straight to the manager application. Version disclosure is a serious help to an attacker: knowing the precise version lets them look up its known vulnerabilities and default behaviors immediately. The reduction is to suppress version disclosure, remove the version from headers and titles, replace default pages, and never link management

interfaces from a public page. A server that reveals only that it is “a web server” gives an attacker far less to work with than one that announces its exact build.

nikto: the web vulnerability scanner

nikto automates web reconnaissance, checking a server against a large database of known issues: version disclosure, dangerous methods, default and backup files, and exposed administrative paths. A scan of the web tier produces a findings list:

```
nikto -h http://webhost:8080
+ Server: Apache Tomcat/9.0.x    (version disclosed)
+ /manager/html: Tomcat Manager interface found
+ Allowed HTTP methods: GET, POST, PUT, DELETE, OPTIONS
+ Access-Control-Allow-Origin: * (permissive CORS)
+ /docs, /examples: default files present
```

A nikto-style scan, version disclosure, exposed manager, dangerous methods, permissive CORS.

Every line is a finding a defender should have pre-empted: the version is disclosed, the manager is reachable, dangerous HTTP methods are enabled, CORS is wide open, and default files remain. None is exotic, and each has a standard fix. The scanner’s value to a defender is exactly this consolidated list (run it against your own web tier and you get the prioritized hardening tasks directly. And nikto is loud: it makes many requests, hits many nonexistent paths, and produces a burst of 404s and probes that is highly detectable in web logs. The detection is that pattern) a rapid sweep of unusual paths from one source is a web scan in progress.

Methods and CORS

Two findings deserve their own look. The allowed HTTP methods and the CORS policy are both security decisions an attacker reads directly. The methods enumerate what the server will do:

```
OPTIONS / HTTP/1.1
Allow: GET, POST, PUT, DELETE, OPTIONS
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
```

HTTP methods and CORS, PUT and DELETE enabled, and any origin allowed.

Two problems sit here. PUT and DELETE, if enabled and not tightly controlled, can let a client write or remove content, methods most applications never need and should not expose. And the wildcard CORS policy, Access-Control-Allow-Origin: *, tells browsers that any web page anywhere may call this API, undermining the browser protections that normally keep one site from reading another’s responses. The reductions are precise: disable the HTTP methods the application does not use, and replace the wildcard CORS

with an explicit list of the origins that legitimately need access. Both are one-line configuration fixes that a scanner would otherwise flag as open doors.

The Tomcat manager: the prize

The highest-value discovery is the management interface. The Tomcat manager (linked, recall, right from the root page) is a web application that can deploy other web applications, and its interface makes that plain:

```
GET /manager/html
Apache Tomcat/9.0.x - Manager App
  Logged in as: admin
  WAR file to deploy: [ choose file ] [ Deploy ]
  /manager/text/list          - list applications
  /manager/text/serverinfo    - server details
  /manager/status             - server status
```

The Tomcat manager, a deploy form and management endpoints, the web-to-code-execution prize.

This is the target from Chapter 37 seen through the scanner's eyes. The manager offers a WAR-file deploy form (upload a web application and it runs) and management endpoints that list applications and reveal server internals. Reaching it with a weak or default credential is the web-to-code-execution path: deploy a malicious application, and attacker code runs on the server. That it is discoverable at all (named on the root page, found by any scanner) is the first problem. The reductions are firm: remove the manager from production or restrict it to trusted networks, never leave default credentials, and certainly never link it from a public page. An exposed, discoverable manager is among the highest-severity web findings, and this chapter's scan found it in the first request.

The bank web apps

Beyond the server and its management interface sit the banking applications themselves, and endpoint discovery maps their surface, the API paths a scanner or a patient attacker enumerates:

```
DISCOVERED ENDPOINTS:
  /cbsa/account/{n}      account inquiry (REST)
  /cbsa/customer/{n}     customer inquiry
  /cbsa/search           account search
  /api/v1/...            versioned API surface
```

Discovered banking endpoints, the API surface a web attacker then probes.

Each discovered endpoint is a place to apply the web vulnerabilities from Chapter 37: the account and customer endpoints for broken access control (changing the number to read another record), the search endpoint for injection, the update endpoints for mass assignment. Endpoint discovery is the map; those flaws are the exploitation. The

reduction is twofold: minimize the discoverable surface (no unnecessary or undocumented endpoints, no verbose error pages that confirm what exists) and ensure every endpoint enforces authentication, input validation, and authorization, so that discovering it yields nothing. A mapped endpoint that is properly secured is a dead end for an attacker.

The defender's web recon

As with every tool in this part, the highest-value use is against your own systems, and the findings sort into a clean reduce-and-detect table:

Finding	Reduce	Detect
Version banner	Suppress version disclosure	Scan bursts of odd paths
Exposed manager	Remove/restrict; no default creds	Access to manager paths
Dangerous methods	Disable unneeded HTTP methods	Unusual method requests
Wildcard CORS	Restrict to known origins	, (config finding)
Discoverable endpoints	Minimize surface; enforce authz	Enumeration 404 bursts

Web-recon findings, reduce the exposure and detect the scan.

Run a web scanner against your own tier and this table writes itself: the version to suppress, the manager to lock down, the methods to disable, the CORS to tighten, the endpoints to secure. Web reconnaissance is also, uniquely among the recon phases, quite detectable, scanners are noisy, generating rapid requests to many nonexistent paths, so a web-application firewall or even attentive log review catches a scan in progress. The web tier is ordinary web software, assessed and defended with ordinary web-security tools, applied with the seriousness the mainframe behind it demands.

The web-recon mistake is leaving the easy wins in place: a version banner, an exposed manager, PUT and DELETE enabled, wildcard CORS. A scanner finds all of them in seconds, and each is a one-line fix. Run the scan against your own web tier first, and close what it finds, starting with any management interface that is discoverable or carries a default credential.

Hands-On Labs

These labs perform web-application reconnaissance against the Gibson web tier and reduce what they find.

Lab 43.1

Fingerprint the server

Goal: identify the web server software and version.

Background: version disclosure decides the attack. This lab reads the fingerprint.

1. Request the root page and read the server version and any linked interfaces.

GET / HTTP/1.1

Check yourself: you can identify Apache Tomcat 9.0.x and note that the manager is linked from the root, and explain that suppressing version disclosure reduces the exposure.

Why it matters: the exact version is a lookup key for known vulnerabilities. Revealing less gives an attacker less.

Blue-Team angle: strip version strings from headers and titles and never link management interfaces from public pages, both are free hardening.

Lab 43.2

Run a web scan

Goal: run a nikto-style scan and read the findings.

Background: a web scanner consolidates known issues into a findings list. This lab reads it as a hardening checklist.

1. Scan the web tier and read the reported findings.

```
nikto -h http://webhost:8080
```

Check yourself: you can list the findings (version disclosure, exposed manager, dangerous methods, wildcard CORS, default files) and treat each as a task to fix.

Why it matters: the scan is your prioritized hardening list, produced in seconds. Running it yourself is the fastest web assessment.

Blue-Team angle: a web scan is noisy, many 404s and odd paths from one source. That burst is a detectable signature in web logs and a WAF.

Lab 43.3

Enumerate methods and CORS

Goal: enumerate allowed HTTP methods and read the CORS policy.

Background: methods and CORS are security decisions an attacker reads directly. This lab checks them.

1. Send an OPTIONS request and read the Allow and CORS headers.

```
OPTIONS / HTTP/1.1
```

Check yourself: you can identify PUT and DELETE as enabled and the wildcard CORS, and state the reductions, disable unused methods and restrict CORS to known origins.

Why it matters: dangerous methods and wide-open CORS are one-line findings a scanner flags immediately. Both are trivial to fix.

Blue-Team angle: allow only the HTTP methods the application uses, and replace wildcard CORS with an explicit origin list, two small configuration changes that close two findings.

Lab 43.4

Find the manager

Goal: locate the Tomcat manager and explain why it is the prize.

Background: the management interface is the highest-value web target. This lab finds and assesses it.

1. Request the manager interface and read its capabilities.

```
GET /manager/html
```

Check yourself: you can identify the WAR deploy form and management endpoints, explain that deploy access is code execution, and state the reductions, remove or restrict the manager and never use default credentials.

Why it matters: a discoverable manager is a direct route to running code. Finding it first is how you close it before an attacker uses it.

Blue-Team angle: management interfaces do not belong on production, and never with default credentials, an exposed manager is a top-severity finding.

Lab 43.5

Map and secure the endpoints

Goal: discover the banking endpoints and connect them to their web vulnerabilities and fixes.

Background: endpoint discovery is the map; the Chapter 37 flaws are the exploitation. This lab connects them defensively.

1. Enumerate the banking API endpoints and, for each, name the vulnerability class and its fix.

Check yourself: you can map account/customer endpoints to broken access control, search to injection, and updates to mass assignment, and state the fixes, authz checks, parameterization, and field allow-lists.

Why it matters: a discovered endpoint is only a risk if it is weakly secured. Mapping the surface tells you exactly what to harden.

Blue-Team angle: minimize the discoverable surface and secure every endpoint, a mapped but properly secured endpoint is a dead end for the scanner and the attacker.

Information leakage from status pages

Even short of the deploy form, a management interface leaks detail that helps an attacker. The server-status and server-info endpoints reveal the exact runtime beneath the application:

```
GET /manager/status
```

```
Server Status
```

```
Server Version: Apache Tomcat/9.0.x
```

```
JVM Version: 17.0.x
```

```
OS: z/OS (USS)
```

```
Deployed apps: /cbsa, /fibs, /manager, /docs, /examples
```

A server-status page, exact versions, the host OS, and every deployed application, all disclosed.

This single page hands over the Tomcat version, the JVM version, the host operating system (z/OS under USS) and a list of every deployed application. To an attacker that is a precise inventory: which versions to look up for known issues, and which applications to target next. The reduction is to lock down the status and info endpoints exactly as the manager itself, restrict them to trusted networks, require strong authentication, and prefer to remove them from production entirely. Information-leakage endpoints are lower-

severity than the deploy form but follow the same rule: a management interface reachable by anyone is an intelligence gift, and the less it discloses to an unauthenticated request, the better.

Lab 43.6

Close the information leaks

Goal: read a server-status page, identify the disclosed detail, and state the reductions.

Background: status and info endpoints leak the runtime inventory. This lab reads one and closes it.

1. Request the server-status page and note the versions, OS, and deployed applications it reveals.

GET /manager/status

Check yourself: you can list what the page discloses (Tomcat and JVM versions, the host OS, the deployed apps) and state the reduction: restrict or remove the status and info endpoints and require authentication.

Why it matters: a status page is a free inventory for an attacker. Restricting it denies the reconnaissance that precedes a targeted attack.

Blue-Team angle: treat status and info endpoints like the manager itself, off the open network, authenticated, and ideally absent from production. Every version they disclose is a lookup an attacker no longer has to perform.

Checkpoint: can you fingerprint a web server, run and read a web scan, enumerate methods and CORS, find the Tomcat manager, and map the banking endpoints, and for each state the reduction and detection? Those skills assess the mainframe's web tier exactly as a web attacker would, so you can harden it first.

Web-application reconnaissance narrows from the organization's footprint to its web tier. Server fingerprinting reads version disclosure (Apache Tomcat 9.0.x, linking straight to its manager) reduced by suppressing version strings and never publicizing management interfaces. A nikto-style scan consolidates the findings: version disclosure, an exposed manager, dangerous HTTP methods, wildcard CORS, and default files, each with a standard fix. The Tomcat manager is the prize, offering a WAR-deploy form that turns web access into code execution, so it must be removed or restricted and never carry default credentials. Endpoint discovery maps the banking API surface, where the Chapter 37 flaws (broken access control, injection, mass assignment) await, reduced by minimizing the surface and securing every endpoint. Web reconnaissance is also notably detectable, since scanners generate noisy bursts of odd requests. The web tier is ordinary web software, assessed and defended with ordinary web-security tools, applied with mainframe seriousness.

What comes next

The recon so far has been platform-agnostic (scanning, footprinting, web tools that work against any target. The next chapter turns to the specialized instruments built specifically

for mainframes: cicspwn for CICS, Db2 connection tools, JES and FTP anonymous-access checks, and PassTicket handling. These are the mainframe-aware attack tools a defender must understand in depth, and) as with cicspwn already, the chapter shows for each how a properly secured system defeats it, and how the attempt is detected.

CHAPTER 44

Mainframe Attack Tools

By the end of this chapter you will be able to:

Explain Gibson's SECURE and VULNERABLE modes.

Describe cicspwn and CICS shell tools and their defeat.

Show how anonymous FTP/JES access is enabled or blocked.

Explain Db2 connection tools and their controls.

Understand PassTicket abuse and how to defend against it.

Map each tool to an ATT&CK technique and a detection.

THE RECON TOOLS OF THE last chapters were platform-agnostic. This chapter covers the instruments built specifically for mainframes: cicspwn and the CICS shell tools, anonymous FTP and JES access checks, Db2 connection tools, and PassTicket handling. Each is presented as a defender must understand it (what it does, how a properly secured system defeats it, and how the attempt is detected. Gibson makes this concrete with a security-mode switch: it runs VULNERABLE by default, so the attacks succeed and you can see them, and it can be switched to SECURE, where the same attacks are blocked. That switch is the whole lesson of the chapter. Every tool here is a demonstration that the controls this book has prescribed actually work) the tool that succeeds against a vulnerable system finds every door closed against a secured one, and every tool maps to a known attack technique with a known detection.

The SECURE and VULNERABLE modes

Gibson runs in one of two security modes, and it is the mechanism that makes defensive learning possible. In VULNERABLE mode (the default for training) the weaknesses are present and the attack tools succeed, so a learner can see exactly how each works. In SECURE mode, the controls are in place and the same tools are blocked. Checking the mode is a single call:

```
is_secure_mode(state)    -> False
config.security_mode     -> 'vuln'
```

```
(SECURE mode: is_secure_mode -> True, attacks blocked)
```

The security mode, VULNERABLE by default for learning, SECURE to block the attacks.

This is the frame for the entire chapter, and it previews the security-model chapter that opens Part IX. Every tool below behaves one way in VULNERABLE mode and the opposite in SECURE mode, and the difference is precisely the controls from Parts IV through VII, transaction security, disabled anonymous access, strong authentication, protected keys.

Reading the two behaviors side by side is the clearest possible proof that hardening is not theoretical: it is the difference between a tool that works and one that does not.

cicspwn and CICS shell tools

The mainframe attack tools most people have heard of target CICS. `cicspwn`, seen already in the `nmap` chapter, assesses and attempts to exploit a CICS region through its powerful transactions; tools in the `tshocker` and `catso` family attempt to obtain an interactive shell the same way. All rely on reaching `CEMT`, `CECI`, or `CEDA`. Against a secured region, all fail identically:

```
cicspwn / tshocker -> target CICS region
CEMT  DENIED  SECURITY_PROTECTED
CECI  DENIED  SECURITY_PROTECTED
CEDA  DENIED  SECURITY_PROTECTED
Result: BLOCKED - no shell, no code execution
```

CICS attack tools against a secured region, the powerful transactions are protected, all blocked.

The single control that defeats this whole family is transaction security from the CICS chapter: with the powerful built-in transactions restricted, the tools have nothing to reach, and every one is blocked. The detection is the correlation and CICS security events the tools generate even when blocked, the attempt is visible in the logs whether or not it succeeds. One control, transaction security, answers `cicspwn`, `tshocker`, `catso`, and the rest at once, which is why it is the highest-priority CICS hardening.

Anonymous FTP and JES access

A simpler but common weakness is anonymous access. In `VULNERABLE` mode, an FTP login as `ANONYMOUS` is accepted (no credential required) and the event is recorded; in `SECURE` mode, the same login is refused:

```
USER anonymous
VULNERABLE: 230 Anonymous access granted (event: FTP ANON LOGON)
SECURE:      530 Login incorrect          (no anonymous access)
```

Anonymous FTP, granted in `VULNERABLE` mode, refused in `SECURE` mode.

Anonymous FTP is exactly the exposure it appears to be: access with no authentication, and (because mainframe FTP reaches data sets and, with `FILETYPE=JES`, submits jobs) potentially anonymous code execution. The reduction is simply to disable anonymous access, which `SECURE` mode does. The detection is built in: even when granted, the anonymous logon is recorded as a security event, so an anonymous FTP login is both preventable and, if it happens, visible. The same reasoning applies to any anonymous or unauthenticated path into JES or the other services, authentication should never be optional on a system with this much reach.

Db2 connection tools

Db2's network door, DDF, is reachable by any DRDA client, and connection tools test it with default or captured credentials (the network path to the data from Chapter 30. In VULNERABLE mode a weak or default DDF credential lets a tool connect and query; in SECURE mode strong authentication refuses it. The exposure is that DDF accepts connections from off the mainframe, so a tool with valid credentials reaches the banking data without any terminal session at all. The reduction is strong DDF authentication) no default or shared credentials, and ideally certificate-based or multi-factor connection, and tight RACF authorization on what a connected user may do once in. The detection is anomalous DDF activity: connections from unexpected sources, or a connected user running queries far outside their normal pattern, are the signals that a connection tool, rather than a legitimate application, is on the other end.

PassTicket abuse

The most powerful mainframe-specific tool targets PassTickets. A PassTicket is a RACF one-time application password, generated from a secret key shared between RACF and an application, that lets a user authenticate to that application without sending a reusable password. It is a good mechanism, until the secret key is exposed. If an attacker obtains the key, they can generate valid PassTickets for any user, including a SPECIAL user, and impersonate them:

```
PassTicket abuse (ATT&CK T1550 - alternate authentication material)
  secret key exposed -> generate valid PassTicket for target user
  authenticate as that user, up to SPECIAL
  DEFENSE: protect the key; require MFA context; monitor SMF 80
```

PassTicket abuse, an exposed key lets an attacker impersonate any user; the key is the control.

This is impersonation toward the highest privilege, which is why the CTI platform of Part IX rates it a top-severity technique. The defense is layered. First and foremost, protect the secret key: it is as sensitive as a master password, and its exposure is the whole attack, so it must be tightly access-controlled and never left in readable configuration or source. Second, require additional context for PassTicket authentication where supported, binding a PassTicket to more than the key alone raises the bar. Third, monitor: RACF logs PassTicket evaluations to SMF 80, so a burst of PassTicket authentications, especially toward privileged users, is a detectable signal. The tool is only as strong as the key it depends on, so the key is where the defense concentrates.

Mapping tools to technique and detection

The defender's organizing view is to map each tool to a known attack technique and its detection, exactly what Gibson's threat-intelligence platform does, tagging activity with ATT&CK technique IDs and the SMF signals that reveal it:

Tool / activity	ATT&CK	Detection
FTP brute force	T1110	Failed-login bursts (SMF)
FTP exfiltration	T1567	Large outbound transfers
PassTicket abuse	T1550	SMF 80 PassTicket evaluations
CICS shell tools	-	CICS security events, correlation IDs
Anonymous FTP	-	FTP ANON LOGON security event

Mainframe attack tools mapped to ATT&CK techniques and their detections.

This mapping is the payoff of the chapter and the bridge to Part IX. Every tool is a known technique with a known signature, and every signature lands in the SMF and monitoring data that Part IV taught you to collect. The defender's job is therefore not to anticipate novel magic but to instrument for the techniques that exist: brute force, exfiltration, alternate-authentication abuse, CICS exploitation, anonymous access. Each has a control that reduces it and a signal that detects it, and a secured system (SECURE mode) has the controls already in place. The attack tools, understood this way, are a checklist of exactly what to harden and what to watch.

The mistake is to treat these tools as unstoppable mainframe magic. Every one is defeated by a control already in this book (transaction security, disabled anonymous access, strong DDF authentication, a protected PassTicket key) and every one leaves a detectable trace. SECURE mode blocks them all; the work is to ensure your real systems run with the same controls SECURE mode represents, and to watch for the traces the tools leave when they try.

Hands-On Labs

These labs run the mainframe attack tools against the Gibson sandbox and show how SECURE mode defeats each.

Lab 44.1

Understand the two modes

Goal: explain the SECURE and VULNERABLE modes and what they represent.

Background: the mode switch is the chapter's frame. This lab establishes it.

1. Check the current security mode and explain what changes between VULNERABLE and SECURE.

Check yourself: you can explain that VULNERABLE mode leaves the weaknesses present so tools succeed, while SECURE mode applies the controls so they are blocked, the difference being the hardening from earlier parts.

Why it matters: the two modes are a controlled before-and-after of hardening. Seeing both proves the controls work.

Blue-Team angle: your real systems should run as SECURE mode represents. The tools that follow are how you confirm they do.

Lab 44.2

Test anonymous FTP in both modes

Goal: attempt anonymous FTP under VULNERABLE and SECURE modes and compare.

Background: anonymous access is a common, serious exposure. This lab shows it granted and then blocked.

1. Attempt an anonymous FTP login and observe the result in each mode.

USER anonymous

Check yourself: you can show that VULNERABLE mode grants anonymous access and records the event, while SECURE mode refuses it, and explain that anonymous FTP can mean anonymous code execution via FILETYPE=JES.

Why it matters: anonymous access with no authentication on a system with FTP's reach is a serious hole. Disabling it closes it; the event detects it.

Blue-Team angle: disable anonymous access everywhere and alert on the FTP ANON LOGON event, an anonymous login should never happen, so any occurrence is an incident.

Lab 44.3

See the CICS tools blocked

Goal: run a CICS attack tool against a secured region and observe it blocked.

Background: transaction security defeats the whole CICS-tool family. This lab confirms it.

1. Run a CICS shell tool against a secured region and read the denied transactions and blocked result.

Check yourself: you can show CEMT, CECI, and CEDA all SECURITY_PROTECTED and the result BLOCKED, and explain that one control (transaction security) defeats cicspwn, tshocker, and catso alike.

Why it matters: a single control answering an entire tool family is the best kind of hardening. Transaction security is that control for CICS.

Blue-Team angle: even blocked, the tools emit CICS security events and correlation IDs, confirm those are collected so the attempt is not just stopped but seen.

Lab 44.4

Understand PassTicket defense

Goal: explain PassTicket abuse and the layered defense against it.

Background: PassTicket abuse is impersonation toward SPECIAL. This lab studies the defense.

1. Explain how an exposed key enables PassTicket abuse and state the three defenses.

Check yourself: you can explain that an exposed secret key lets an attacker generate valid PassTickets for any user, and that the defenses are protecting the key, requiring MFA context, and monitoring SMF 80 PassTicket evaluations.

Why it matters: PassTicket abuse reaches the highest privilege, so its defense (above all the key) deserves top priority.

Blue-Team angle: treat the PassTicket secret key as a crown-jewel secret, and alert on bursts of PassTicket authentications toward privileged users.

Lab 44.5

Map tools to detection

Goal: map each attack tool to its ATT&CK technique and detection signal.

Background: every tool is a known technique with a known signature. This lab builds the mapping.

1. For FTP brute force, exfiltration, PassTicket abuse, CICS tools, and anonymous access, state the technique and detection.

Check yourself: you can map brute force to T1110 and failed-login bursts, exfiltration to T1567 and large transfers, PassTicket abuse to T1550 and SMF 80, and the CICS and anonymous tools to their security events.

Why it matters: mapping tools to techniques and detections turns a list of threats into a monitoring plan grounded in the SMF data you already collect.

Blue-Team angle: this mapping is the bridge to Part IX's detection platform, the same techniques, tagged and detected automatically from the SMF signals.

FTP as brute force and exfiltration

FTP is not only an anonymous-access risk; it is the channel for two more techniques the threat platform tracks by name. Brute force (ATT&CK T1110) hammers a login, and exfiltration (T1567) carries data out, and each has a clear detection signature:

T1110 FTP BRUTE FORCE

USER ibmuser / PASS **** -> 530 (x40 in 60s from one IP)

DETECT: failed-login burst -> SMF; RACF revoke after N fails

T1567 FTP EXFILTRATION

RETR PAYROLL.MASTER (48 MB outbound to external host)

DETECT: unusually large outbound transfer

FTP brute force and exfiltration, each a named technique with a clear detection signal.

The brute-force burst is defeated by the RACF control from Part IV (revoke a user after a few failures, so guessing cannot continue) and detected as a rash of 530 rejections from one source in the SMF logon records. The exfiltration is harder to prevent outright, since legitimate transfers happen, but a large outbound RETR to an external host stands out against a baseline, and TLS plus egress controls limit where data can go. Both techniques live in FTP, both are answered by controls already covered, and both surface in the same SMF data, which is exactly why the threat platform can tag them automatically.

Db2 connection tools in detail

A Db2 connection tool speaks DRDA to the DDF port from Chapter 30, presenting credentials to reach the data over the network. In VULNERABLE mode a weak credential connects; in SECURE mode strong authentication refuses it:

```
db2 connect to GIBSONDB2 user DDFUSER using ****
VULNERABLE:  connected - SELECT * FROM CBSA.ACCOUNT  (rows)
SECURE:      SQL30082N authentication failed (strong auth)
DETECT: DDF connection from unexpected source / off-baseline query
```

A Db2 connection tool, connecting over DDF in VULNERABLE mode, refused under strong auth in SECURE mode.

The exposure is that DDF is a network door straight to the banking data, reachable from anywhere the port is exposed, so a tool with a valid credential needs no terminal and no green screen. The reduction is strong DDF authentication (no default or shared credentials, certificate or multi-factor connection where possible) backed by tight RACF authorization on what a connected identity may do. The detection is behavioral: a DDF connection from a source that never connects, or a connected user issuing queries far outside their normal pattern, is the signature of a tool rather than the application it impersonates. As with FTP, the door is useful and the defense is to authenticate it strongly and watch who comes through.

Lab 44.6

Detect the FTP and Db2 techniques

Goal: identify the detection signature for FTP brute force, FTP exfiltration, and a Db2 connection tool, and the control that reduces each.

Background: these techniques share the SMF and DDF data as their detection surface. This lab connects each to its signal and control.

1. For an FTP brute-force burst, a large outbound RETR, and an off-baseline DDF connection, state the detection signal and the reducing control.

Check yourself: you can map brute force to failed-login bursts (reduced by revoke-after-N), exfiltration to large outbound transfers (reduced by egress control and TLS), and Db2 connection tools to off-baseline DDF activity (reduced by strong authentication).

Why it matters: each technique is detectable in data you already collect. Naming the signal and the control turns the threat into a monitoring rule and a hardening step.

Blue-Team angle: baseline normal FTP and DDF activity so the anomalies (failed-login bursts, oversized transfers, unexpected connections) stand out. The baseline is what makes the detection reliable.

Checkpoint: can you explain the SECURE and VULNERABLE modes, describe the CICS tools, anonymous access, Db2 tools, and PassTicket abuse, and for each state the control that defeats it and the signal that detects it? Those skills turn the mainframe attack toolkit into a hardening and monitoring checklist.

The mainframe-specific attack tools are each defeated by a control already in this book, and Gibson proves it with a security-mode switch: VULNERABLE mode lets the tools succeed for learning, SECURE mode blocks them. cicspwn and the CICS shell tools (tshocker, catso) are all defeated by transaction security and detected by CICS security events. Anonymous FTP is granted in VULNERABLE mode and refused in SECURE mode, detected by the anonymous-logon event, and serious because mainframe FTP can execute code. Db2 connection tools abuse the DDF network door, answered by strong authentication and anomalous-activity detection. PassTicket abuse impersonates users up to SPECIAL by exploiting an exposed secret key, defended by protecting the key, requiring MFA context, and monitoring SMF 80 evaluations. Every tool maps to a known ATT&CK technique (T1110, T1567, T1550) with a detection in the SMF data of Part IV, making the toolkit a checklist of what to harden and what to watch, and the bridge to Part IX's detection platform.

What comes next

The tools so far act against systems. The next chapter covers a quieter kind of reconnaissance that never touches a system at all: OSINT, open-source intelligence. It is the gathering of public information about an organization and its people (documents, social media, breach data, job postings) that an attacker assembles to plan social engineering, guess credentials, and choose targets. Understanding OSINT is understanding what an organization reveals about itself without any system being probed, and what a defender can do to reduce that exposure.

CHAPTER 45

OSINT: Open-Source Intelligence

By the end of this chapter you will be able to:

Explain OSINT and how it differs from active reconnaissance.

Describe the categories of open-source intelligence.

Recognize how people, technology, and documents leak information.

Explain the risk of credentials exposed in breach data.

Assemble an OSINT picture and see the mainframe-specific angle.

Reduce an organization's OSINT footprint.

THE RECONNAISSANCE SO FAR HAS touched systems (scanning ports, probing web applications) or read public technical records like DNS. OSINT, open-source intelligence, touches nothing. It gathers public information about an organization and its people: who works there, what technology they run, what documents they have published, what credentials have leaked in breaches. It is entirely legal, entirely passive, and invisible to the target, which makes it the quietest and often the most productive phase of an attacker's work. For the mainframe it is particularly valuable, because OSINT reveals who administers the system, what versions and products are in use, and (through breach data) sometimes the very credentials to reach it. This chapter covers OSINT as a defender must understand it: what an organization reveals about itself without any system being probed, and how to reduce that exposure. Detection is minimal here (there is nothing to detect when no system is touched) so reduction and awareness are the whole defense.

Intelligence from public sources

OSINT is the human and organizational layer of reconnaissance. Where scanning asks a machine what it runs, OSINT asks the public record what an organization is: its people, its technologies, its documents, its history. The information is scattered (social networks, job sites, published files, breach databases, conference programs, code repositories) and the attacker's skill is assembling the scattered pieces into a coherent picture of whom to target and how. Because none of it involves touching the target, it cannot be blocked or, for the most part, detected; the only defense is to control what is out there. The categories are worth knowing, because each is a different kind of exposure with a different reduction:

Category	Typical source	What it reveals
People	Social and professional networks	Staff, roles, targets
Technology	Job postings, talks, tech blogs	The technology stack
Documents	Published files and their metadata	Username, paths, versions
Credentials	Breach and leak databases	Reusable passwords
Behavior	Case studies, conference talks	Practices and projects

The categories of OSINT, each a different public exposure with a different reduction.

People: the org chart from outside

The most valuable OSINT is about people. Professional networks and social media reveal who works at an organization, their roles, and often their reporting lines (an org chart assembled from the outside. For an attacker targeting a mainframe, the goal is specific: find the people who run it. A public profile listing a role as “z/OS systems programmer” or “RACF administrator” names exactly the individual whose access is most valuable, and whose credentials or good faith an attacker will try to exploit through spear-phishing or social engineering. The reduction is awareness rather than concealment) people will and should have professional presences, but staff can be guided not to publish internal system details, specific privileged roles, or technical specifics that turn a general profile into a precise target. The difference between “works in IT infrastructure” and “administers the RACF security database on our z/OS 2.5 systems” is the difference between a name and a targeting package.

Technology: what they run

Organizations advertise their technology constantly, most reliably in job postings. A posting seeking a mainframe specialist reads, to an attacker, as a specification of the target:

`JOB POSTING (public):`

`Senior z/OS Systems Programmer`

`Required: z/OS 2.5, RACF, CICS TS 5.6, Db2 12, JES2`

`Experience with SMP/E maintenance and USS`

`Familiarity with our IND$FILE and FTP job-submission processes`

A job posting read as reconnaissance, it specifies the exact platform, versions, and products.

This single posting hands over the operating system version, the security product, the transaction monitor and its release, the database, and even operational details about file transfer and job submission. An attacker now knows precisely what they are up against and which known issues to research, the same value as a version banner, volunteered. Conference talks, tech blogs, and vendor case studies leak the same way, often with more detail. The reduction is to sanitize what is published: job postings can describe the role without enumerating exact versions and internal processes, and staff speaking publicly can avoid disclosing specifics of the environment. The role can be filled without publishing the target’s blueprint.

Documents and metadata

Published documents leak in two ways: their content, and their metadata. Every PDF and Office file carries hidden metadata (authors, the software that made it, sometimes internal file paths and server names) and harvesting it from an organization’s public documents assembles a surprising picture:

`METADATA harvested from public documents:`

```

Authors:  j.prince, a.morgan, sysadmin      (-> usernames)
Software:  Enterprise COBOL; ISPF; Word 2019
Paths:    \\GIBFS01\shared\draft\          (-> internal server)

```

Document metadata, usernames, internal software, and server names, extracted from public files.

The authors resolve to usernames (often the same IDs used to log on) the software hints at the environment, and internal paths reveal server names and directory structure. Tools that automate this harvesting turn a set of public documents into a list of usernames and infrastructure. The usernames are especially valuable, because they feed the user-enumeration and credential attacks from earlier chapters with real, valid IDs. The reduction is straightforward and often overlooked: strip metadata from documents before publishing them. A cleaned document reveals its content and nothing else, closing a leak most organizations do not know they have.

Credentials from breaches

The most direct OSINT is breach data. Large collections of credentials from past breaches are public, and an attacker checks an organization's email addresses against them:

```

BREACH CHECK for gibsoncorp.com:
a.morgan@gibsoncorp.com    found in 2 breaches
j.prince@gibsoncorp.com    found in 1 breach
RISK: password reuse -> the same password may work on TSO/FTP

```

A breach check, organization emails found in leaked credential data, a password-reuse risk.

The danger is password reuse. A password exposed in some unrelated breach is often reused, and if an employee used the same password for the mainframe, that leaked credential is a working login, to TSO, to FTP, to the systems this book has spent chapters securing. This is credential stuffing, and it bypasses every technical control by simply logging in as a real user. The reductions are the strongest available: monitor breach data for the organization's email addresses and force a password reset on any that appear; forbid password reuse across systems; and, above all, require multi-factor authentication, so that a leaked password alone is not enough to log on. MFA is the single control that most reliably defeats breach-sourced credentials, which is why it matters so much for a system as valuable as the mainframe.

Assembling the OSINT picture

The power of OSINT, like footprinting, is in assembly. Combine the categories and a targeting plan appears:

```

MAINFRAME OSINT PROFILE - gibsoncorp
WHO:    a.morgan - RACF administrator (profile + doc metadata)
TECH:   z/OS 2.5, RACF, CICS 5.6, Db2 12 (job posting)
ACCESS: a.morgan in 2 breaches - reused password possible

```


PLAN: phish or stuff a.morgan's mainframe login

An assembled OSINT profile, who to target, with what pretext, using what credential.

Read as a whole, the profile is a plan: a named administrator, the exact technology they work with (for a convincing pretext), and a possibly-reused credential from a breach. No system was touched to build it. This is why a defender assembles the same profile first, only by seeing what OSINT reveals about your own organization can you reduce it. The reductions stack: guide staff on what to publish, sanitize job postings, strip document metadata, monitor breach data and enforce MFA. Each closes one input to the profile, and together they turn a precise targeting package back into scattered, low-value fragments.

The OSINT mistake is believing there is nothing to defend because no system is touched. The exposure is entirely in what the organization and its people have made public (named administrators, exact technology in job postings, usernames in document metadata, reused passwords in breaches) and every piece has a reduction. The single most important is multi-factor authentication, which defeats the breach-sourced credential that OSINT most directly enables.

Hands-On Labs

These labs assess an organization's OSINT footprint and reduce it. They use public-information reasoning against the Gibson training organization; no live systems are probed.

Lab 45.1

Map the categories

Goal: describe the categories of OSINT and what each reveals.

Background: OSINT spans people, technology, documents, credentials, and behavior. Knowing the categories frames the assessment.

1. List the OSINT categories and the public source and exposure of each.

Check yourself: you can name people, technology, documents, credentials, and behavior, and explain what each reveals and why it cannot be detected, only reduced.

Why it matters: OSINT is scattered until assembled. Knowing the categories is knowing where the pieces come from.

Blue-Team angle: for each category, ask what your organization exposes, that inventory is the start of reducing the footprint.

Lab 45.2

Assess people exposure

Goal: explain how public profiles reveal mainframe administrators as targets.

Background: the people who run the mainframe are the highest-value targets. This lab assesses their exposure.

1. Explain how a public profile naming a privileged mainframe role turns a person into a target, and what guidance reduces it.

Check yourself: you can explain that a profile naming a RACF or z/OS role names the person to phish, and that guidance to omit internal specifics reduces the exposure without hiding legitimate professional presence.

Why it matters: the administrator's access is the prize. Not advertising who holds it raises the attacker's effort.

Blue-Team angle: awareness training for privileged staff on what to publish is a low-cost, high-value control against targeted phishing.

Lab 45.3

Read a job posting as recon

Goal: extract the technology stack from a job posting and sanitize it.

Background: job postings specify the target. This lab reads one as an attacker and then reduces it.

1. Extract the platform, versions, and products from a posting, then rewrite it to reveal less.

Check yourself: you can pull the OS version, security product, transaction monitor, and database from the posting, and rewrite it to describe the role without enumerating exact versions and internal processes.

Why it matters: a posting can fill a role without publishing the target's blueprint. Sanitizing it is free reduction.

Blue-Team angle: review job postings before publication for exact versions and internal operational detail, the role can be described without the specifics.

Lab 45.4

Understand metadata and breach risk

Goal: explain how document metadata leaks usernames and how breach data enables credential stuffing.

Background: metadata and breach data are the most directly dangerous OSINT. This lab studies both.

1. Explain what document metadata reveals and how a breached, reused password becomes a mainframe login.

Check yourself: you can explain that metadata leaks usernames, software, and paths, and that a reused password from a breach may work on TSO or FTP, defeated by MFA and reset-on-exposure.

Why it matters: these two categories turn OSINT into access. Stripping metadata and requiring MFA close the most direct paths.

Blue-Team angle: strip metadata before publishing documents, monitor breach data for your domains, and require MFA, the control that makes a leaked password insufficient by itself.

Lab 45.5

Reduce the footprint

Goal: assemble an OSINT profile and state the reduction for each input.

Background: the exposure is the assembled whole. This lab builds it and reduces each part.

1. Assemble a targeting profile from the categories, then state the reduction for people, technology, documents, and credentials.

Check yourself: you can assemble a profile (named admin, exact tech, possible reused credential) and map each input to its reduction: awareness, sanitized postings, stripped metadata, and breach monitoring with MFA.

Why it matters: reducing each input turns a precise targeting package back into scattered fragments. That is the whole OSINT defense.

Blue-Team angle: run an OSINT self-assessment of your own organization periodically, it is the only way to see, and then reduce, what the outside world already knows.

Public code repositories

A modern OSINT source deserves special attention: public code. Developers publish code to public repositories, and that code too often carries secrets that were never meant to leave the organization, hardcoded credentials, keys, and internal hostnames committed by accident:

```
PUBLIC REPO search for 'gibsoncorp':
  config.properties:
    DB2_HOST=gibsondb2.internal    DB2_USER=DDFUSER
    DB2_PASS=Summer2025!          <- hardcoded credential
  deploy.jcl: //SYSIN DD DSN=PROD.PAYROLL.SOURCE
  RISK: real credential + internal host + production dataset
```

Secrets in public code, a hardcoded Db2 credential, an internal hostname, and a production data set.

This is among the most damaging OSINT of all, because it is not a hint or a target list (it is a working credential. A hardcoded Db2 password in a public repository, alongside the internal hostname and a production data-set name, is a direct path to the data over DDF, exactly the connection-tool scenario from the last chapter but with the credential handed over for free. Any secret) the PassTicket key, an FTP password, an API token (committed to public code is an immediate exposure. The reduction is layered: scan public repositories for the organization's secrets continuously, rotate anything found immediately, and) the root fix, keep secrets out of code entirely, in a secrets manager, so there is nothing to leak. A secret in public code is a breach that has already happened; the only questions are whether you find it first, and how fast you can rotate it.

The reductions, together

Pulling the chapter together, each OSINT input has a specific reduction, and a defender works the whole list:

OSINT input	Reduction
Named admins on profiles	Awareness; omit internal role specifics
Exact tech in job postings	Sanitize postings; describe, don't enumerate
Usernames in document metadata	Strip metadata before publishing
Reused passwords in breaches	Breach monitoring; force resets; MFA
Secrets in public code	Repo scanning; rotate; secrets manager

The OSINT inputs and their reductions, the defender's checklist for shrinking the footprint.

No single reduction is dramatic, but together they turn a precise targeting package back into scattered fragments. And one control stands above the rest: multi-factor authentication. People will be named, technology will be inferred, and credentials will leak no matter how careful an organization is, but MFA makes a leaked or reused password insufficient on its own, defeating the most direct path OSINT provides into the mainframe. If a defender does only one thing in response to this chapter, it is to require MFA on every path to the system.

Lab 45.6

Hunt for leaked secrets

Goal: explain how secrets leak into public code and state the layered reduction, ending with MFA as the backstop.

Background: a secret in public code is a breach that has already happened. This lab studies finding and closing it.

1. Given a hardcoded credential in a public repository, state the immediate response and the root fix.

Check yourself: you can explain that a public credential must be rotated immediately, that repositories should be scanned continuously, that secrets belong in a secrets manager rather than code, and that MFA limits the damage of any credential that still leaks.

Why it matters: leaked secrets are direct access, not hints. Finding them first, rotating fast, and backing everything with MFA is the difference between a near-miss and a breach.

Blue-Team angle: run continuous secret-scanning on public repositories for your organization's identifiers, and treat any hit as an incident, rotate the secret before anything else.

Checkpoint: can you explain OSINT and its categories, show how people, technology, documents, and breach data leak information, assemble a targeting profile, and state the reduction for each input, above all MFA against breach-sourced credentials? Those skills let you see, and shrink, what your organization reveals without any system being touched.

OSINT gathers public information about an organization and its people without touching any system, making it passive, legal, and nearly undetectable (so the defense is reduction, not detection). Its categories each leak differently: professional profiles reveal the administrators who run the mainframe as targets; job postings and talks specify the exact technology stack; document metadata exposes usernames, software, and internal paths; and breach data supplies credentials that, if reused, log in as real users. Assembled, these produce a targeting profile) who to phish, with what pretext, using what credential (all built from public sources. The reductions stack: guide staff on what to publish, sanitize job postings, strip document metadata, monitor breach data, and) most importantly, require multi-factor authentication, the single control that defeats the breach-sourced credential OSINT most directly enables. A defender must run the same assessment against their own organization to see what the outside world already knows.

What comes next

OSINT closes the survey of reconnaissance from outside. The final chapter of Part VIII turns to what happens after a foothold is gained: pivoting. Gibson models a second host, LENNOX, and the chapter covers lateral movement (using access to one system to reach another) through the connectivity of Part VII, the NJE trust of Chapter 35, and the credentials gathered along the way. It is the bridge from reconnaissance to the kill-chain of Part IX, showing how the pieces combine to move through an environment, and how a defender contains that movement.

CHAPTER 46

LENNOX & Pivoting

By the end of this chapter you will be able to:

Explain why the mainframe is rarely the attacker's first hop.

Describe pivoting, using one system to reach another.

Enumerate a mainframe's location from a compromised host.

Recognize credentials and clues left on a peripheral system.

Understand how a foothold becomes mainframe access.

Contain lateral movement with segmentation, MFA, and monitoring.

EVERY RECON CHAPTER SO FAR has looked at the mainframe from the outside as if it could be reached directly. In real attacks it rarely can. A well-run mainframe is not internet-facing (the Shodan finding of Chapter 42 was a mistake to be fixed, not the norm) so attackers reach it indirectly, by first compromising a weaker system that can reach it, then pivoting. Gibson models this with LENNOX, a small second host that sits near the mainframe and can talk to it. This chapter closes Part VIII with lateral movement: how a foothold on one system becomes access to another, through the network connectivity, the trust relationships, and the credentials an attacker finds along the way. It is the bridge from reconnaissance to the kill-chain of Part IX, and its defensive lesson is that the mainframe's isolation is only as strong as the systems allowed to reach it.

The mainframe is rarely the first hop

Consider the attacker's position after reconnaissance. The mainframe is well defended, not exposed to the internet, and its front doors (TN3270, FTP, the web tier) are reachable only from inside the network. Attacking it directly from outside is usually impossible. What is reachable is everything else: the workstations, the Linux servers, the peripheral systems that make up the ordinary corporate network, defended to ordinary standards. An attacker compromises one of those (through phishing, a web vulnerability, or a reused credential from the OSINT of the last chapter) and now has a foothold inside the network, from which the mainframe is reachable. This is the essential insight: the mainframe's strong perimeter is bypassed not by breaking it but by starting from inside, and the security of the mainframe therefore depends on the security of every system that can reach it. LENNOX is one such system.

LENNOX: the foothold

LENNOX is a small standalone host (an ordinary Linux box) that an administrator uses to reach the mainframe. Assume an attacker has compromised it and has a shell as the user training. The first moves are basic enumeration of where they have landed:

```

training@lennox:~$ whoami
training
training@lennox:~$ hostname
lennox
training@lennox:~$ id
uid=1000(training) gid=1000(training) groups=1000(training)

```

A foothold on LENNOX, an ordinary shell on a peripheral host near the mainframe.

Nothing here is remarkable (an unprivileged user on a small host) and that is exactly the point. The attacker has not touched the mainframe at all; they are on a modest Linux box that happens to sit near it. The value of this foothold is not what it is but what it can reach and what it knows. The next moves are to learn both.

Discovering the mainframe

From the foothold, the attacker looks for the mainframe. The most basic enumeration (reading the host's own network configuration) often finds it immediately:

```

training@lennox:~$ cat /etc/hosts
127.0.0.1    localhost
10.0.0.7    mfhost mainframe    # GIBSON z/OS - TSO/TN3270

```

The mainframe discovered from the foothold, its IP, its names, and its role, in one file.

The hosts file names the mainframe outright: its address (10.0.0.7), its aliases (mfhost, mainframe), and even a comment noting it runs GIBSON z/OS with TSO and TN3270. In one command, the attacker has located the target and learned it is reachable from here. This is the first containment failure, and the reduction is network segmentation: a peripheral Linux box has no business being able to reach the mainframe's administrative ports. Placing the mainframe in a segment reachable only from the specific systems that genuinely need it means a compromised LENNOX finds the address but cannot connect, the pivot is stopped at the network before credentials ever matter.

Finding the keys

Knowing where the mainframe is, the attacker looks for how to log on, and on a system used to reach the mainframe, the clues are often lying in the user's files:

```

training@lennox:~$ cat README.txt
Welcome to LENNOX.
I use this box to reach the mainframe (mfhost / 10.0.0.7).
Check my notes. I left a backup of my creds lying around -
must clean that up.    -- training

```

A README that hands over the attack, the box's purpose, and a credential backup left behind.

The user's own notes complete the picture, and they are a catalog of the failures that make pivoting work:

```
training@lennox:~$ cat notes.txt
```

```
TODD:
```

- rotate the mainframe password (it is in a backup somewhere)
- security keeps nagging about my sudo rights
- mfhost TSO logon is the usual admin id

Notes revealing the admin ID convention and an unrotated, stored mainframe password.

Read the failures. Mainframe credentials are stored on this peripheral host, a backup of them is “lying around.” The TSO logon is “the usual admin id,” a predictable, reused convention rather than a unique identity. And the hygiene is deferred: rotating the password and cleaning up the backup are on a to-do list, not done. Each is a real-world commonplace, and together they hand the attacker a working credential for a privileged mainframe account. The reductions are specific and important: never store mainframe credentials on off-platform systems; use unique administrative identities rather than a shared “usual admin id”; and treat credential rotation and cleanup as immediate, not eventual. The notes.txt on LENNOX is a museum of how footholds become mainframe compromises.

The pivot, and what stops it

With the address, the admin ID convention, and a stored credential, the attacker pivots: from LENNOX they connect to the mainframe's TN3270 or FTP and log on as the administrator. The foothold on a modest Linux box has become privileged access to the mainframe (without ever breaking the mainframe's perimeter, because they started inside it. But notice how many controls, any one of them, would have stopped this. Network segmentation would have kept LENNOX from reaching the mainframe at all. Not storing credentials off-platform would have left the attacker with an address and no way in. Unique administrative identities would have defeated the “usual admin id” guess. And multi-factor authentication) the control the OSINT chapter insisted on, would have made the stored password insufficient on its own, requiring a second factor the attacker does not have. The pivot succeeds only because every one of these was missing; each is a place a defender breaks the chain.

Containing lateral movement

The defenses against pivoting are about containment (limiting what a foothold can reach, learn, and reuse) and monitoring for the movement itself:

Control	What it contains
Network segmentation	Which systems can reach the mainframe at all
No off-platform credentials	Credential discovery on peripheral hosts
Unique admin identities	Reuse of a shared “usual” ID
Multi-factor authentication	A stored or stolen password used alone

Control	What it contains
Source-aware monitoring	Mainframe logons from unusual hosts

Containing lateral movement, limit reach, remove credentials, and watch the source.

Segmentation is the foundation: if a peripheral host cannot reach the mainframe's ports, the pivot dies at the network. Removing credentials from off-platform systems and using unique, MFA-protected administrative identities means that even a host that can reach the mainframe yields no usable way in. And source-aware monitoring adds detection, a mainframe logon for a privileged ID originating from an unexpected system like LENNOX is exactly the anomaly that signals a pivot in progress, visible in the same SMF logon records Part IV taught you to read. Contain the reach, remove the credentials, and watch the source: lateral movement is defeated in depth, not by any single wall.

The bridge to the kill-chain

Pivoting is one link in a longer chain, and this chapter is where reconnaissance hands off to it. The full sequence (reconnaissance, initial access, pivot, mainframe access, privilege escalation, impact) is the kill-chain that Part IX assembles and, essentially, shows how to detect as a whole. LENNOX is the pivot link seen in isolation; the chapters ahead connect it to the others, from the credential attacks that gain initial access to the privilege escalation that turns a foothold into full control, all watched by Gibson's detection platform. Having seen how an attacker finds and reaches the mainframe, Part IX shows what they do once inside, and how a defender sees and stops the entire chain.

The pivoting mistake is defending the mainframe's perimeter while ignoring the systems that can reach it. A hardened mainframe behind a Linux box that stores its credentials, on a flat network, with a shared admin ID and no MFA, is not hardened at all, it is one phishing email away from compromise. Segment the network, keep mainframe credentials off every other system, use unique MFA-protected admin identities, and watch where privileged logons come from.

Hands-On Labs

These labs work the pivot from the LENNOX foothold and then contain it.

Lab 46.1

Understand the indirect path

Goal: explain why the mainframe is rarely the attacker's first hop.

Background: a hardened mainframe is reached indirectly. This lab establishes why pivoting matters.

1. Explain how an attacker reaches a non-internet-facing mainframe through a compromised peripheral system.

Check yourself: you can explain that the mainframe's perimeter is bypassed by starting from inside (a compromised workstation or server) so its security depends on every system that can reach it.

Why it matters: defending only the mainframe's perimeter misses the real path in. The reachable systems are part of its attack surface.

Blue-Team angle: inventory which systems can reach the mainframe's ports, that set is part of the mainframe's security boundary, and it should be as small as possible.

Lab 46.2

Enumerate from the foothold

Goal: from the LENNOX shell, discover the mainframe's location.

Background: a foothold's value is what it can reach and know. This lab finds the mainframe from LENNOX.

1. From the LENNOX shell, read the host's network configuration to locate the mainframe.

```
cat /etc/hosts
```

Check yourself: you can find the mainframe's IP, names, and role in the hosts file, and explain that network segmentation would keep LENNOX from reaching it.

Why it matters: the peripheral host often knows exactly where the mainframe is.

Segmentation stops that knowledge from becoming a connection.

Blue-Team angle: a peripheral host that can both name and reach the mainframe's admin ports is a segmentation failure, the address is unavoidable, the reachability is not.

Lab 46.3

Find the pivot material

Goal: locate the credentials and clues left on LENNOX and name the failures.

Background: a system used to reach the mainframe often holds the keys. This lab finds them and diagnoses why they are there.

1. Read the user's notes and identify the stored credential, the admin-ID convention, and the deferred hygiene.

```
cat README.txt notes.txt
```

Check yourself: you can identify a stored mainframe credential, a shared "usual admin id," and unrotated, uncleaned secrets, and name each as a containment failure with its reduction.

Why it matters: credentials on a peripheral host turn a foothold into mainframe access.

Removing them breaks the pivot.

Blue-Team angle: never store mainframe credentials off-platform, use unique admin identities, and treat rotation and cleanup as immediate, the notes.txt is a checklist of what not to do.

Lab 46.4

Trace the pivot and its breaks

Goal: describe the pivot from LENNOX to the mainframe and the controls that would stop it.

Background: the pivot succeeds only because several controls are missing. This lab traces it and each break point.

1. Describe how the attacker reaches the mainframe from LENNOX, and name a control that would break the chain at each step.

Check yourself: you can trace address → admin ID → stored credential → logon, and name segmentation, off-platform credential removal, unique identities, and MFA as the controls that each break it.

Why it matters: the pivot depends on a chain of failures. Any one control breaks it, so defense in depth is decisive.

Blue-Team angle: MFA is the last-line break, even with a stored password, a second factor the attacker lacks stops the logon. It is the control to prioritize.

Lab 46.5

Contain lateral movement

Goal: state the containment controls and the detection for a pivot.

Background: pivoting is defeated by containment and monitoring. This lab assembles the defense.

1. List the containment controls and the logon anomaly that detects a pivot in progress.

Check yourself: you can list segmentation, off-platform credential removal, unique MFA-protected identities, and source-aware monitoring, and explain that a privileged logon from an unusual host is the pivot's detectable signature.

Why it matters: containment limits the pivot and monitoring catches it. Together they defeat lateral movement in depth.

Blue-Team angle: alert on privileged mainframe logons originating from unexpected sources, the SMF logon records carry the origin, and an admin logon from a peripheral host is exactly the pivot signal to catch.

A second route: NJE trust

Stolen credentials are one pivot path; trust relationships are another. The NJE network of Chapter 35 is itself a pivot surface, a node trusted to send jobs can run work on the mainframe without any password at all. From a foothold on or near a trusted node, cross-node execution reaches the mainframe through trust rather than credentials:

from a foothold near node ORAC (trusted by GIBSON):

```
submit job -> ORAC -> execute at GIBSON
```

```
JOB $HASP122 X FROM ORAC RECEIVED AT GIBSON
```

```
runs as the NODES-mapped identity on GIBSON
```

```
DEFENSE: NODES class (least identity) + TLS + monitoring
```

The NJE pivot, a trusted node runs work on the mainframe through trust, not a stolen password.

This is the cross-node execution of Chapter 35 seen as a pivot: an attacker who reaches a trusted node, or spoofs one on a clear line, moves onto the mainframe through the trust relationship, and the identity the work runs under is decided entirely by the mainframe's NODES class. The defenses are the ones that chapter prescribed, now understood as pivot containment: trim each node's AUTH to the minimum, map inbound work to a least-privilege identity with the NODES class, require TLS so a node cannot be impersonated, and monitor cross-node jobs. The lesson across both routes (stolen credentials and abused trust) is the same: the mainframe's safety depends on the systems and nodes it trusts, and every trust relationship is a pivot path to contain.

Lab 46.6

Contain the trust pivot

Goal: explain how NJE trust is a pivot path and state the containment controls from Chapter 35 as pivot defenses.

Background: trust is a pivot surface as real as a stolen credential. This lab connects the NJE controls to lateral-movement containment.

1. Describe how a trusted node reaches the mainframe, and state the controls that contain the trust pivot.

Check yourself: you can explain that cross-node execution runs work on the mainframe through trust, mapped by the NODES class, and that trimming AUTH, mapping to a least-privilege identity, requiring TLS, and monitoring cross-node jobs contain it.

Why it matters: credentials are not the only pivot. A trusted node is a passwordless route in, contained by the same least-trust discipline applied to identity.

Blue-Team angle: treat every NJE trust relationship as a pivot path (least AUTH, least NODES identity, TLS, and monitoring) and review the node network as part of the mainframe's reachability boundary.

Checkpoint: can you explain why the mainframe is reached indirectly, enumerate it from a foothold, find the credentials a peripheral host leaves lying around, trace the pivot, and contain lateral movement with segmentation, credential hygiene, MFA, and source-aware monitoring? Those skills close the reconnaissance part and open the kill-chain.

The mainframe is rarely an attacker's first hop; being well defended and not internet-facing, it is reached by pivoting from a compromised peripheral system. Gibson models this with LENNOX, an ordinary host near the mainframe. From a foothold there, an attacker enumerates the mainframe's location from the hosts file, and finds (in the user's own notes) a stored mainframe credential, a shared "usual admin id," and deferred rotation and cleanup, a catalog of the failures that make pivoting work. With address, identity convention, and credential in hand, the attacker pivots to privileged mainframe access without ever breaking its perimeter, because they started inside. Yet any of several controls breaks the chain: network segmentation so the foothold cannot reach the mainframe, no credentials stored off-platform, unique administrative identities, multi-factor authentication so a stored password is insufficient, and source-aware monitoring to detect a privileged logon from an unusual host. The mainframe's isolation is only as strong as the systems allowed to reach it, and this chapter bridges reconnaissance to the kill-chain of Part IX. This closes Part VIII.

What comes next

Part VIII has shown how an attacker discovers and reaches the mainframe. Part IX assembles the full kill-chain (what happens once inside. It opens with the security model itself: the SECURE and VULNERABLE modes seen in Chapter 44, made the explicit subject, showing how a system's posture determines whether the attacks that follow succeed or fail. From there the part walks initial access and credential attacks, privilege escalation, lateral movement and persistence, and) the payoff, the CTI platform and HMS detection engine that watch the whole chain, closing with the defender's complete view.

Part IX

The Kill-Chain & Its Defence

Part IX assembles the pieces into a full attack and its defence: the security model, the stages of the kill-chain from initial access to persistence, and the detection platform that watches the whole chain and answers it.

CHAPTER 47

The Security Model

By the end of this chapter you will be able to:

Explain Gibson's SECURE, VULNERABLE, and NORACF modes.

Describe what SECURE mode implements, the CIS z/OS Benchmark.

Connect password, resource, and network controls to CIS.

Explain how posture determines whether an attack succeeds.

Understand the NORACF extreme and what it reveals about RACF.

Read the security-mode banners and block messages.

PART IX ASSEMBLES THE KILL-CHAIN (the full sequence of an attack, from initial access to impact) and it opens here, with the security model, because everything that follows depends on it. Whether any attack in this part succeeds is decided not by the attacker's skill but by the system's posture: a hardened system blocks what a weak one permits. Gibson makes that posture explicit and switchable through its security modes, and the essential fact is that its SECURE mode is not an arbitrary invention. It implements the CIS z/OS Security Benchmark, the recognized industry standard for mainframe hardening. That means the defensive advice spread across this whole book (password rules, RACLISTed classes, protected libraries, PROTECTALL(FAIL), TLS on the network) is not opinion; it is a named, auditable standard, and SECURE mode is that standard turned on. This chapter makes the model the subject: the modes, what SECURE actually does, and how posture determines the outcome of every attack ahead.

The three modes

Gibson runs in one of three security modes, and each represents a different posture. The banner at startup announces which is active:

```
GIBSON VULNERABLE TRAINING MODE ACTIVE
```

```
GIBSON SECURE MODE ACTIVE
```

```
GIBSON NORACF MODE ACTIVE - RACF AUTHORIZATION CHECKING DISABLED
```

The three security-mode banners, each a different posture, announced at startup.

Mode	Posture	Attacks
VULNERABLE	Weaknesses present (training default)	Succeed, for learning
SECURE	CIS Benchmark controls applied	Blocked
NORACF	RACF authorization checking disabled	Trivially succeed

The three modes, VULNERABLE to learn, SECURE to defend, NORACF to show RACF's absence.

VULNERABLE mode is the training default: the weaknesses are present so a learner can see each attack work. SECURE mode applies the CIS controls, and the attacks are blocked. NORACF mode is a teaching extreme (RACF authorization checking is switched off

entirely, so everything is permitted) included to show what RACF does by demonstrating its absence. Real systems live somewhere on the road from VULNERABLE to SECURE; the goal of hardening is to reach SECURE, and NORACF is the cautionary opposite no real system should ever resemble.

What SECURE mode is: the CIS Benchmark

SECURE mode implements the CIS z/OS Security Benchmark, a published set of specific, auditable hardening controls maintained by the Center for Internet Security. Gibson models a representative set of them, each with a control number that maps to the real benchmark:

CIS	Control	Area
1.1.2	PASSWORD(HISTORY) >= 4	Password
1.1.5	PASSWORD(REVOKE) after failures	Password
1.3.1	SPECIAL attribute justified	Identity
2.2.5	PARMLIB protected	Libraries
2.3.3	PROTECTALL(FAIL)	Data sets
6.2.4	AT-TLS for FTP	Network
6.6.4	AT-TLS for TN3270	Network
9.23	USS Telnet server not active	Network

Representative CIS z/OS Benchmark controls, what SECURE mode applies.

This table is the quiet climax of the book's defensive argument. Every control here has appeared already, chapters apart (password history and revocation from the RACF chapters, protected PARMLIB from the configuration chapter, PROTECTALL(FAIL) from the data-set chapters, TLS on FTP and the terminal from the connectivity chapters) and now they resolve into a single named standard with control numbers an auditor recognizes. SECURE mode is that standard applied in one switch. The remaining sections group the controls the way the benchmark does, so their cumulative effect is clear.

Password and identity controls (CIS 1.x)

The benchmark's first section governs identity. It requires password rules (a maximum change interval, a history that prevents reuse, and revocation after repeated failures) and it requires that the SPECIAL attribute, RACF's highest privilege, be justified rather than sprawling. These are the SETROPTS and user-attribute controls from Part IV, now with names: PASSWORD(INTERVAL), PASSWORD(HISTORY), PASSWORD(REVOKE), and SPECIAL justification. Their combined effect is to defeat the credential attacks of the recon chapters: revocation stops brute force, history defeats reuse, and justified SPECIAL limits how much a single compromised account can do. In VULNERABLE mode these are relaxed and the credential attacks work; in SECURE mode they are enforced and the attacks fail. The identity controls are where hardening begins, because identity is where most attacks begin.

Resource protection (CIS 2.x)

The second section protects resources, data sets and the libraries that hold executable code. Its keystone is PROTECTALL(FAIL), which makes access to any unprofiled data set fail rather than fall through to a permissive default:

```
SETOPTS PROTECTALL(FAIL)    (CIS 2.3.3)
  access to an unprofiled data set -> DENIED (fail-closed)
  PARMLIB, PROCLIB, LINKLIB      -> RACF-protected
  RACF security data sets        -> tightly controlled
```

Resource controls, fail-closed access and protected libraries, the CIS 2.x controls.

PROTECTALL(FAIL) changes the system's default answer from yes to no: nothing is accessible unless a profile explicitly permits it, which closes the whole class of exposures where a forgotten resource is readable because no one protected it. Alongside it, the benchmark requires the critical libraries (PARMLIB, PROCLIB, LINKLIB) and the RACF security data sets themselves to be tightly protected, since control of those is control of the system. These are the configuration and data-set protections from Parts III and IV, and in SECURE mode they are the reason an attacker who reaches the system still cannot read or alter what matters.

Network controls (CIS 6.x and 9.x)

The network sections require encryption and forbid plaintext services. AT-TLS (Application Transparent TLS) encrypts FTP and TN3270, and the plaintext USS Telnet server is not active:

```
CIS 6.2.4  AT-TLS for FTP      -> file transfer encrypted
CIS 6.6.4  AT-TLS for TN3270   -> terminal encrypted (secure listener)
CIS 9.23   USS Telnet inactive -> no plaintext shell listener
```

Network controls, TLS on FTP and the terminal, and no plaintext services, the CIS 6.x/9.x controls.

These are the TLS protections from the connectivity chapters, named and required. Encrypting the terminal and file transfer defeats the eavesdropping and credential-capture that plaintext invites, and disabling plaintext services removes the unencrypted doors entirely. In VULNERABLE mode the plaintext listeners are present and traffic is readable; in SECURE mode the encrypted listeners replace them and the plaintext ones are gone. Together with the identity and resource controls, the network controls complete a posture where the attacks of the recon chapters find encrypted channels, protected resources, and enforced identity at every turn.

How posture determines outcome

The point of the whole model is this: in SECURE mode, an attack that would succeed is stopped, and the system says so plainly:

```
GIBSON SECURE MODE ACTIVE
```

REQUEST BLOCKED BY ACTIVE SECURITY PROFILE

reason: control enforced (CIS-mapped)

A SECURE-mode block, the request denied by the active security profile.

Every attack in the chapters ahead behaves this way: it succeeds in VULNERABLE mode, where you can study it, and is blocked in SECURE mode by a specific CIS control. The kill-chain of Part IX is, read the other way, a tour of the CIS Benchmark, each attack a demonstration of what a particular control prevents. That is why the security model comes first. It is the frame for everything that follows: initial access blocked by identity controls, escalation blocked by resource controls, movement blocked by segmentation and encryption. Posture, not the attacker, determines the outcome, and SECURE mode is the posture to reach.

The mistake is treating hardening as a matter of judgment when a named standard exists. SECURE mode is the CIS z/OS Benchmark, a published, auditable baseline, not one person's opinion of what is safe. The work is to measure a real system against that benchmark and close the gaps, moving from VULNERABLE toward SECURE. NORACF, meanwhile, is the reminder of what RACF does: turn it off and everything is permitted, so its presence and correct configuration are everything.

Hands-On Labs

These labs explore the security model and the CIS controls behind SECURE mode.

Lab 47.1**Read the modes**

Goal: identify the three security modes and what each represents.

Background: the mode is the system's posture. This lab reads the banners and their meaning.

1. Read the three mode banners and state the posture each represents.

Check yourself: you can explain that VULNERABLE leaves weaknesses for learning, SECURE applies the CIS controls, and NORACF disables RACF authorization entirely.

Why it matters: the mode decides whether attacks succeed. Reading it is reading the system's security posture at a glance.

Blue-Team angle: a real system's posture is measured against SECURE (the CIS Benchmark) and the job is to close the gap between where it is and there.

Lab 47.2**Map SECURE mode to CIS**

Goal: connect the controls in SECURE mode to the CIS Benchmark.

Background: SECURE mode is the CIS z/OS Benchmark. This lab makes the mapping explicit.

1. For several SECURE-mode controls, give the CIS control number and area.

Check yourself: you can map password history to 1.1.2, revocation to 1.1.5, PARMLIB protection to 2.2.5, PROTECTALL(FAIL) to 2.3.3, and TLS to 6.2.4 and 6.6.4.

Why it matters: naming the standard turns hardening from opinion into an auditable baseline an assessor recognizes.

Blue-Team angle: use the CIS z/OS Benchmark as your hardening checklist, it is the same standard SECURE mode implements, control by numbered control.

Lab 47.3

Trace PROTECTALL(FAIL)

Goal: explain how PROTECTALL(FAIL) changes the system's default answer.

Background: PROTECTALL(FAIL) is the keystone resource control. This lab traces its effect.

1. Explain what happens to access of an unprofiled data set with and without PROTECTALL(FAIL).

Check yourself: you can explain that without it, unprofiled access may fall through to a permissive default, and with it, such access fails closed, nothing is readable unless explicitly permitted.

Why it matters: fail-closed is the difference between a forgotten resource being exposed and being denied. It closes a whole class of exposure.

Blue-Team angle: PROTECTALL(FAIL) is one of the highest-value single controls, it makes the system deny by default rather than permit by omission.

Lab 47.4

Understand the network controls

Goal: explain the CIS network controls and what they defeat.

Background: the network controls require encryption and forbid plaintext. This lab explains their effect.

1. State what AT-TLS for FTP and TN3270 and the disabled USS Telnet each protect against.

Check yourself: you can explain that TLS on FTP and the terminal defeats eavesdropping and credential capture, and that disabling plaintext services removes unencrypted doors entirely.

Why it matters: plaintext services leak credentials and data. Encrypting them and removing the plaintext ones closes those leaks.

Blue-Team angle: require TLS on every mainframe network service and disable plaintext listeners, the CIS 6.x and 9.x controls are the connectivity hardening in standard form.

Lab 47.5

Posture and the NORACF extreme

Goal: explain how posture determines outcome and what NORACF reveals.

Background: the mode decides whether attacks succeed, and NORACF is the cautionary extreme. This lab connects both.

1. Explain why the same attack is blocked in SECURE mode and trivially succeeds in NORACF mode.

Check yourself: you can explain that SECURE mode blocks the attack with a CIS control while NORACF disables RACF authorization so everything is permitted, showing what RACF does by its absence.

Why it matters: the outcome is set by posture, not the attacker. SECURE is the posture to reach; NORACF is the one no system should resemble.

Blue-Team angle: NORACF is a teaching extreme, never a configuration, its lesson is that RACF's presence and correct setup underpin every other control.

The complete control set

Grouped as the benchmark groups them, the controls SECURE mode applies fall into four families, and one of those families is easy to overlook, auditing:

Family	Controls (representative)
Identity (1.x)	INTERVAL<=90, HISTORY>=4, REVOKE, SPECIAL justified, OPERCMDS/FACILITY RACLISTed
Resource (2.x)	RACF datasets, PARMLIB, PROCLIB, LINKLIB protected; PROTECTALL(FAIL)
Audit (3.x)	CMDVIOL logging, SPECIAL activity logging, AUDIT classes
Network (6.x/9.x)	AT-TLS for FTP and TN3270, TN3270 SMF recording, USS Telnet inactive

The CIS control families in SECURE mode, identity, resource, audit, and network.

The audit family (CIS 3.x) is the bridge to the rest of Part IX. It requires command-violation logging, logging of all SPECIAL-user activity, and active AUDIT classes (the very SMF records the forensics chapters taught you to read, now mandated by the benchmark. This is the standard's recognition that prevention and detection are inseparable: SECURE mode does not only block attacks, it ensures the attempts are recorded. The identity controls stop the credential attacks, the resource controls stop escalation, the network controls stop eavesdropping, and the audit controls make certain that whatever is tried leaves a trace) which the detection platform of Chapter 51 then reads. A hardened system is a logged system; the two arrive together in the benchmark.

The same attack, three postures

The clearest way to see the model is to hold one attack against all three modes. Take anonymous FTP from Chapter 44, its outcome is decided entirely by posture:

USER anonymous -> outcome by mode:

```
VULNERABLE: 230 Anonymous access granted (weakness present)
SECURE:      530 blocked by security profile (CIS control)
NORACF:      230 granted - no authorization checking at all
```

One attack, three postures, the outcome set by the mode, not the attacker.

The attacker's action is identical in all three; only the system's posture changes, and with it the result. VULNERABLE grants access because the weakness is present; SECURE blocks it with a CIS-mapped control; NORACF grants it because authorization checking is switched off entirely. This is the whole thesis of Part IX in one figure: the outcome of every attack ahead is determined by which of these postures the target holds. The chapters that follow walk the kill-chain in VULNERABLE mode to show each attack, then show SECURE mode blocking it, and NORACF stands as the reminder of how completely a system fails when its authorization layer is absent. Reach SECURE, and the attacker's skill stops mattering; the controls decide.

Lab 47.6

Hold one attack against three postures

Goal: trace a single attack through VULNERABLE, SECURE, and NORACF modes and explain each outcome.

Background: posture determines outcome. This lab proves it with one attack across all three modes.

1. Take anonymous FTP (or another attack) and state its result and the reason in each mode.

Check yourself: you can explain that the same action is granted in VULNERABLE (weakness present), blocked in SECURE (CIS control), and granted in NORACF (no authorization checking), and that only posture changed.

Why it matters: the outcome is set by the system, not the attacker. Reaching SECURE posture makes the attacker's skill irrelevant to the result.

Blue-Team angle: this exercise is the mental model for the whole kill-chain, for each attack ahead, ask which control in SECURE mode blocks it, and confirm your real system has that control.

Checkpoint: can you explain the three security modes, describe SECURE mode as the CIS z/OS Benchmark, connect password, resource, and network controls to their CIS numbers, and explain how posture determines whether an attack succeeds? Those ideas are the frame for the entire kill-chain, where each attack is blocked (or not) by a specific control.

The security model is the frame for the whole kill-chain, because posture, not the attacker, determines whether an attack succeeds. Gibson runs in one of three modes: VULNERABLE, the training default where weaknesses are present and attacks succeed for study; SECURE, which applies the controls and blocks them; and NORACF, a cautionary extreme that disables RACF authorization to show what RACF does by its absence. The essential point is that SECURE mode implements the CIS z/OS Security Benchmark (a published, auditable standard) so the book's defensive advice resolves into named controls: password history and revocation and justified SPECIAL (CIS 1.x), PROTECTALL(FAIL) and protected PARMLIB, PROCLIB, and LINKLIB (CIS 2.x), and AT-TLS on FTP and the terminal with no plaintext services (CIS 6.x and 9.x). Every attack in the chapters ahead succeeds in VULNERABLE mode and is blocked in SECURE mode by a specific CIS control, making the kill-chain a tour of the benchmark and SECURE mode the posture to reach.

What comes next

With the model established, the kill-chain begins at its beginning: initial access. The next chapter covers how an attacker first gets in (the credential attacks that turn recon into a logon: brute force, password spraying, default and reused credentials, and the enumeration-then-guess pattern from the recon chapters. And, following the model, it shows for each how the CIS identity controls of SECURE mode) revocation, history, strong authentication, MFA, turn a successful login into a blocked one.

CHAPTER 48

Initial Access & Credential Attacks

By the end of this chapter you will be able to:

- Log on with a default credential and reach SPECIAL.
- Confirm valid user IDs through the logon enumeration oracle.
- Explain brute force and password spraying against TSO.
- Read the failed-logon tracking a foothold generates.
- Enumerate the security database from a foothold with RACF.
- Name the RACF identity controls that stop each attack.

THE KILL-CHAIN BEGINS WITH INITIAL access (getting the first valid logon. On the mainframe that means credentials for TSO, and the recon chapters showed the path toward them: enumerate the users, then obtain a password. This chapter shows what actually happens when you do that on Gibson as it ships) in its default vulnerable mode (using real TSO and RACF, because that is the system a learner runs and the outcomes they will see. We start with the simplest and most common real-world compromise of all: a default credential that was never changed. From there we confirm targets by enumeration, guess passwords by brute force and spraying, and read the security database from the foothold. The RACF identity controls that stop each of these) the CIS 1.x controls of the security model, are gathered at the end; the body of the chapter is the attacks and their real outcomes.

The default credential

The easiest way onto a mainframe is a credential that ships with it and was never changed. Gibson, like a real z/OS system, seeds the default administrator IBMUSER, and in the training system its password is the classic default SYS1. Logging on is immediate:

```
LOGON IBMUSER
PASSWORD ==> SYS1
```

```
IKJ56455I IBMUSER LOGON IN PROGRESS
READY
```

A default-credential logon, IBMUSER with the shipped password SYS1 reaches READY.

That is initial access in a single step, and it is worth pausing on how serious it is, because IBMUSER is not an ordinary account. Asking RACF who this user is shows the prize:

```
LISTUSER IBMUSER
USER=IBMUSER  NAME=IBMUSER  OWNER=#SYSPROG
DEFAULT-GROUP=SYS1  PASS-INTERVAL=30
ATTRIBUTES=SPECIAL
```

```
REVOKE DATE=NONE    RESUME DATE=NONE
```

The default account is SPECIAL, a default-credential logon lands full RACF authority.

ATTRIBUTES=SPECIAL means this account has the highest RACF privilege, it can read and change any security profile on the system. So a single unchanged default credential did not just get an attacker in; it got them in as the most powerful identity on the mainframe, with no escalation required. This is not a contrived scenario. Default and unchanged administrator credentials are among the most common real-world mainframe findings, precisely because IBMUSER exists on every system and its default password is documented everywhere. In vulnerable mode the credential works and the attacker is immediately SPECIAL; the entire rest of the kill-chain becomes unnecessary.

Confirming the target: user enumeration

When the default credential has been changed, the attacker must find valid user IDs to attack. The TSO logon is an enumeration oracle, because it answers differently for a real user than an invalid one, the same behavior the nmap tso-enum script automated:

```
LOGON IBMUSER    ->  PASSWORD ====>          (valid: prompts for password)
LOGON GRRR       ->  IKJ56420I USERID (GRRR) NOT AUTHORIZED FOR TSO
```

```
tso-enum:  IBMUSER  CONFIRMED  PASSWORD_PROMPT
           GRRR     UNAVAILABLE USER_NOT_FOUND
```

The logon enumeration oracle, a valid user draws a password prompt, an invalid one is rejected.

A valid user ID is met with a password prompt; an invalid one is told it does not exist. That difference lets an attacker confirm which IDs are real without knowing any password, turning a guess into a target list. Combined with the OSINT usernames of Part VIII (the IDs harvested from document metadata) enumeration confirms which of them are live accounts to attack. In vulnerable mode the oracle is wide open: every valid ID is confirmable. The confirmed list is the input to the next step.

Guessing the password: brute force and spraying

With a confirmed user, the attacker obtains the password by guessing. Two shapes dominate. Brute force tries many passwords against one user; password spraying tries one common password against many users, staying quiet by never hammering a single account. Gibson tracks the attempts:

```
BRUTE FORCE (one user, many passwords):
LOGON IBMUSER / PASS ****  ->  IKJ56421I PASSWORD NOT AUTHORIZED  (x N)
LOGON IBMUSER / PASS SYS1  ->  LOGON IN PROGRESS
```

```
SPRAYING (one password, many users):
IBMUSER/Summer2025  CICSUSER/Summer2025  ...  -> first reuse hits
```


Brute force and spraying, guessing a password against one user, or one password against many.

In vulnerable mode the guessing is not stopped: each failed attempt is recorded in the failed-logon tracking, but the account is not revoked, so an attacker can keep trying until a weak or default password is found. Spraying is the more dangerous of the two in practice, because a single reused password (the kind OSINT surfaces from a breach) tried across the confirmed user list will often hit at least one account without ever tripping a per-account lockout. The real outcome in the default configuration is that credential guessing eventually succeeds, which is exactly why the controls at the end of the chapter matter so much.

What a valid logon gives

Whatever the route (default credential, brute force, spraying) the result is a session at READY as a real user. If that user is IBMUSER, the attacker already has SPECIAL and the kill-chain is effectively over. If it is an ordinary user, the logon is a foothold: a valid identity from which to enumerate the system and look for a way up. Either way, the foothold's first use is reconnaissance from inside, and RACF is the richest source. The commands from the RACF chapters now serve the attacker:

```
LISTUSER           -> my own attributes and groups
LISTGRP SYS1       -> members of a powerful group
SEARCH CLASS(USER) -> enumerate user IDs (if permitted)
LISTUSER IBMUSER   -> confirm who holds SPECIAL
```

Enumerating the security database from a foothold, the RACF commands now serve the attacker.

From a foothold, LISTUSER reveals the attacker's own privileges, LISTGRP reveals the membership of powerful groups, and (where permissions allow) the security database can be searched for high-value accounts. The attacker is building the map for privilege escalation: who is SPECIAL, which groups grant power, what the foothold can already reach. In vulnerable mode a foothold user can often read far more of the security database than they should, because the resource controls that would restrict it are relaxed, which is the subject of the next chapter. Here, the point is that initial access is not the end but the beginning: a valid logon turns the attacker from an outsider into an insider with a view of the system.

The RACF controls that stop this

Each attack in this chapter is closed by a specific RACF identity control, the CIS 1.x controls that SECURE mode applies. In brief:

Attack	Real vuln-mode outcome	Control that stops it
Default credential	IBUSER/SYS1 → immediate SPECIAL	Change defaults; no shipped passwords
User enumeration	Valid IDs confirmed via oracle	Generic logon errors
Brute force	Guessing continues, no lockout	PASSWORD(REVOKE) after N (CIS 1.1.5)
Password spraying	A reused password hits	MFA; unique strong passwords

The credential attacks and the RACF identity controls that stop each in SECURE mode.

The single most important is the first: change the default credentials, so IBMUSER/SYS1 and its kind simply do not work. After that, PASSWORD(REVOKE) stops brute force by revoking an account after a few failures; generic logon errors close the enumeration oracle by answering the same way for valid and invalid users; and multi-factor authentication defeats spraying and any guessed or stolen password by requiring a second factor the attacker does not have. In SECURE mode these are all enforced and initial access by credential attack fails; in the default vulnerable mode they are relaxed and the attacks succeed as shown. The gap between the two is exactly the CIS 1.x identity controls.

The initial-access mistake is leaving IBMUSER (or any default administrator account) with its shipped password. It is the first credential an attacker tries, it exists on every z/OS system, and it is SPECIAL, so a single unchanged default is a complete compromise with no further work required. Change every default credential before anything else, then enforce revocation, generic errors, and MFA to close the guessing and enumeration that follow.

Hands-On Labs

These labs work initial access against the Gibson sandbox in its default mode, then name the control that closes each.

Lab 48.1

Log on with the default credential

Goal: gain initial access with a default credential and confirm the privilege it grants.

Background: an unchanged default is the simplest initial access. This lab uses it and reads the result.

1. Log on as IBMUSER with the default password and reach READY.

```
LOGON IBMUSER (password SYS1)
```

2. Confirm the account's privilege with LISTUSER.

```
LISTUSER IBMUSER
```

Check yourself: the logon succeeds to READY, and LISTUSER shows ATTRIBUTES=SPECIAL, a default credential landed the highest RACF privilege with no escalation.

Why it matters: a default administrator credential is a complete compromise. It is the first thing an attacker tries and the first thing a defender must change.

Blue-Team angle: changing default credentials is the single highest-value initial-access control, IBMUSER/SYS1 must never work on a real system.

Lab 48.2

Enumerate valid users

Goal: use the logon oracle to confirm which user IDs are real.

Background: the logon answers differently for valid and invalid users. This lab reads the oracle.

1. Attempt a logon for a real ID and an invalid ID and compare the responses.

LOGON IBMUSER vs LOGON GRRR

Check yourself: the valid ID prompts for a password; the invalid ID is rejected as not found, confirming which IDs are real without any password.

Why it matters: enumeration turns a guess into a target list, and OSINT-harvested usernames into confirmed accounts to attack.

Blue-Team angle: generic logon errors close the oracle, valid and invalid users should draw identical responses; enumeration bursts are detectable in SMF.

Lab 48.3

Brute force and spraying

Goal: guess a password against a confirmed user and observe the failed-logon tracking.

Background: with a confirmed user, guessing obtains the password. This lab runs it in the default mode.

1. Attempt several passwords against a confirmed user and observe that guessing is tracked but not stopped.

Check yourself: failed attempts are recorded but the account is not revoked in the default mode, so guessing continues, and a weak or default password eventually succeeds; spraying a reused password across users hits at least one.

Why it matters: without revocation, guessing works. Spraying a reused password is the quiet, effective variant.

Blue-Team angle: PASSWORD(REVOKE) stops brute force; MFA stops spraying, and the failed-logon burst is a reliable SMF detection either way.

Lab 48.4

Enumerate the database from a foothold

Goal: from a valid logon, use RACF commands to map privileges.

Background: a foothold's first use is reconnaissance. This lab enumerates the security database with RACF.

1. From a foothold, read your own attributes and a powerful group's membership.

LISTUSER ; LISTGRP SYS1

Check yourself: you can read your own privileges and the members of a powerful group, and identify who holds SPECIAL, the map for privilege escalation.

Why it matters: initial access is the beginning, not the end. From a foothold the attacker maps the path up.

Blue-Team angle: restrict how much of the security database a foothold user can read, the resource controls of the next chapter limit this enumeration.

Lab 48.5

Name the controls

Goal: map each credential attack to the RACF identity control that stops it.

Background: each attack has a specific control. This lab assembles the defense.

1. For default credentials, enumeration, brute force, and spraying, state the control that closes each.

Check yourself: you can map default credentials to changing them, enumeration to generic errors, brute force to PASSWORD(REVOKE), and spraying to MFA, the CIS 1.x identity controls.

Why it matters: initial access is closed by identity controls. Naming them turns the attacks into a hardening checklist.

Blue-Team angle: the identity controls are where hardening begins, because identity is where most attacks begin, and changing defaults is first among them.

Not every account is a target

Enumeration turns up accounts, but not all of them are attackable, and recognizing the difference is part of reading the system. Alongside IBMUSER, Gibson seeds CICSUSER, and it is a very different kind of account:

```
LISTUSER CICSUSER
USER=CICSUSER  NAME=CICS  DEFAULT USER  OWNER=#SYSPROG
ATTRIBUTES=PROTECTED
PASSWORD ==> *NOPASSWORD*
```

```
LOGON CICSUSER -> rejected: PROTECTED id cannot log on
```

A PROTECTED account, CICSUSER has no password and cannot be logged on to or brute-forced.

CICSUSER is PROTECTED, with the special value *NOPASSWORD*, which means it cannot be used for an interactive logon and cannot be revoked by failed attempts (it exists only for a started task or surrogate context to assume, never for a person to log on with. To an attacker this is a dead end: there is no password to guess, so brute force and spraying are pointless against it. The contrast with IBMUSER is the lesson. IBMUSER ships with a usable, default password and full SPECIAL authority) the ideal target, while CICSUSER is PROTECTED and unusable for logon. The PROTECTED attribute is itself a defensive measure, and applying it to every service and surrogate identity removes those accounts from the credential-attack surface entirely. Even in the default vulnerable mode, a PROTECTED account gives the attacker nothing.

The foothold's enumeration

From an ordinary foothold (say a guessed non-privileged user) the attacker's first work is to map the security database and find the accounts that matter. In the default mode a foothold can often read more than it should:

```
LISTUSER -> ATTRIBUTES= (none)  DEFAULT-GROUP=SYS1
```

```

SEARCH CLASS(USER)      -> IBMUSER  CICSUSER
LISTUSER IBMUSER        -> ATTRIBUTES=SPECIAL  <- the target
LISTGRP SYS1            -> SUPERIOR-GROUP / members

```

Enumeration from an ordinary foothold, finding who holds SPECIAL and which groups grant power.

The attacker reads their own (empty) attributes, enumerates the user IDs, confirms that IBMUSER holds SPECIAL, and inspects the powerful SYS1 group. In a few commands they have identified the target of privilege escalation: the SPECIAL account, or a group whose membership would grant power. This is why initial access matters even when it lands only an ordinary user, the foothold is a vantage point, and in the default configuration it sees far into the security database. Restricting what a foothold can enumerate is a resource-control problem, and the next chapter's controls limit exactly this. Here the real outcome is plain: an ordinary logon, in the default mode, is enough to map the road to SPECIAL.

Lab 48.6

Tell a target from a dead end

Goal: distinguish an attackable account from a PROTECTED one, and enumerate the escalation target from a foothold.

Background: not every account is a credential-attack target, and a foothold's job is to find the one that is. This lab does both.

1. Compare IBMUSER and CICSUSER with LISTUSER and identify which is attackable.

```
LISTUSER IBMUSER ; LISTUSER CICSUSER
```

2. From a foothold, enumerate the users and confirm who holds SPECIAL.

Check yourself: you can explain that IBMUSER (usable password, SPECIAL) is the target while CICSUSER (PROTECTED, *NOPASSWORD*) is a dead end, and that a foothold can enumerate the database to find the SPECIAL account.

Why it matters: recognizing PROTECTED accounts saves wasted effort and, defensively, shows how to remove accounts from the attack surface. Enumeration finds the real target.

Blue-Team angle: make every service and surrogate identity PROTECTED so it cannot be logged on to or brute-forced, and restrict how much of the database a foothold can enumerate.

Checkpoint: can you gain initial access with a default credential and reach SPECIAL, confirm users through the logon oracle, guess passwords by brute force and spraying, enumerate the security database from a foothold, and name the RACF identity control that stops each? Those skills are the first link of the kill-chain, and the first place a defender breaks it.

Initial access on the mainframe means a valid TSO logon, and in Gibson's default vulnerable mode the outcomes are stark. The simplest and most common is a default credential: IBMUSER with the shipped password SYS1 logs straight in, and LISTUSER shows the account is SPECIAL (the highest RACF privilege, reached with no escalation. Where defaults have been changed, the TSO logon acts as an enumeration oracle, confirming valid user IDs by responding differently to real and invalid ones; brute force then guesses many passwords against one user, and spraying tries one reused password across many, succeeding in the default mode because failed attempts are tracked but the account is not revoked. Any valid logon becomes a foothold from which RACF commands enumerate the security database and map the path to higher privilege. Each attack is closed by a CIS 1.x identity control) changing defaults, generic logon errors, PASSWORD(REVOKE), and multi-factor authentication, with changing the default administrator credential the single most important, because an unchanged default is a complete compromise on its own.

What comes next

Initial access as an ordinary user is a foothold, not full control. The next chapter covers privilege escalation (turning that foothold into SPECIAL) through the real mechanisms an attacker abuses: APF-authorized libraries, SURROGAT and PassTicket delegation, UID(0) and BPX.SUPERUSER in OMVS, and group-SPECIAL. It shows each through real RACF and OMVS output in the default mode, and the resource controls that turn every one of them into a denied request.

CHAPTER 49

Privilege Escalation

By the end of this chapter you will be able to:

- Distinguish RACF SPECIAL from OMVS superuser (UID 0).
- Explain how an APF-authorized library enables escalation.
- Read a SURROGAT profile and explain identity delegation.
- Describe PassTicket abuse toward SPECIAL.
- Show OMVS escalation via UID(0) and BPX.SUPERUSER.
- Name the resource controls that stop each path.

INITIAL ACCESS AS AN ORDINARY user is a foothold, not control. Privilege escalation is the step that turns the foothold into the highest privilege (RACF SPECIAL, or OMVS superuser) and it is the heart of a mainframe attack. This chapter walks the real escalation mechanisms as Gibson models them, through real RACF and OMVS output in the default mode: APF-authorized libraries, SURROGAT delegation, PassTicket abuse, UID(0) and BPX.SUPERUSER in OMVS, and group-SPECIAL. Each is governed by a RACF profile, and the escalation happens when that profile is more permissive than it should be, which, in the default vulnerable mode, it generally is. The resource controls that tighten every one of them, the CIS 2.x controls, are gathered at the end; the body of the chapter is the mechanisms and what they actually do.

Two privilege domains

The mainframe has two separate top privileges, and it is essential not to confuse them. RACF SPECIAL is control of the security database (the power to read and change any profile. OMVS superuser, UID 0, is control of the z/OS UNIX side) the equivalent of Unix root. They are distinct: a SPECIAL user is not automatically a superuser, and a superuser is not automatically SPECIAL. A foothold shows only ordinary privilege in each:

```
TS0:  LISTUSER          -> ATTRIBUTES= (none)      <- not SPECIAL
OMVS:  id                -> uid=10535(user) gid=335  <- not UID 0
```

escalation targets one or both: SPECIAL and/or UID 0

The two privilege domains, RACF SPECIAL and OMVS superuser (UID 0), separate and both targets.

An ordinary foothold is neither SPECIAL nor UID 0, and escalation aims at one or both. Because they are separate, they have separate mechanisms and separate controls, and a real attack often chains them, use one domain to reach the other. The sections that follow take the mechanisms in turn, each governed by a RACF profile that the attacker inspects and abuses, and the defender tightens.

APF: authorized code

The most powerful escalation runs through APF. As Chapter 25 showed, a program loaded from an APF-authorized library can run in an authorized state that bypasses the normal controls, it can touch protected storage, issue privileged instructions, and rewrite its own authority. The escalation is simple to state: if a foothold can write to an APF-authorized library, it can place a program there that grants itself SPECIAL, and the system will run it with full authority. Gibson's health check flags exactly this exposure:

```
GIBAPF01   Broad APF exposure           MEDIUM
-> an APF library is writable more broadly than it should be
-> authorized code placed there runs bypassing RACF
```

The APF exposure, a writable authorized library is a direct path to bypassing RACF.

In the default mode the APF libraries are more broadly writable than they should be, which is why the check raises GIBAPF01. The reason this is so dangerous is that authorized code is above RACF (it does not ask permission, it takes it) so write access to an APF library is effectively the power to become SPECIAL. The control is to protect the APF libraries as tightly as the LINKLIB control of the security model requires (CIS 2.2.15), so that no foothold can write to them. APF is the reason library protection is not housekeeping but the core of escalation defense.

SURROGAT: acting as another user

A subtler escalation uses SURROGAT, the RACF class that lets one user act under another's identity. A SURROGAT profile names a user whose identity may be borrowed; permission to it lets the holder submit work as that user. Reading the profile shows the delegation:

```
RLIST SURROGAT * ALL
CLASS      NAME
SURROGAT   IBMUSER.SUBMIT
UNIVERSAL ACCESS  YOUR ACCESS  WARNING
NONE          ALTER          NO
```

A SURROGAT profile, IBMUSER.SUBMIT with ALTER access delegates IBMUSER's identity.

The profile IBMUSER.SUBMIT with ALTER access means the holder can submit jobs that run as IBMUSER, and IBMUSER is SPECIAL. So a foothold with this SURROGAT permission does not need to guess IBMUSER's password or find another way in; it simply submits a job that runs as IBMUSER and inherits SPECIAL. That is escalation by delegation, and Gibson's health check rates it high:

```
GIBSUR01   SURROGAT delegation          HIGH
-> a foothold can run work as a more privileged user
-> escalation without a password
```

The SURROGAT exposure, delegation to a privileged identity is escalation without a password.

The high rating is right: SURROGAT to a SPECIAL user is a direct, quiet path to SPECIAL that leaves no failed logons and needs no guessing. The control is minimal SURROGAT permission, delegation to a privileged identity should be rare, tightly scoped, and justified, never broadly granted. Where it exists, it should be watched closely, because a job running as a more privileged user is exactly what escalation looks like.

PassTicket: forged one-time credentials

Related to SURROGAT is PassTicket abuse, met in the attack-tools chapter. A PassTicket is a one-time application credential generated from a secret key; if the key is exposed, an attacker generates valid PassTickets for any user (including a SPECIAL user) and authenticates as them. Gibson's threat model maps it precisely:

```
TTP surrogat -> ATT&CK T1550 (alternate authentication material)
"Surrogate / PassTicket impersonation toward SPECIAL"
detection: SMF 80 PassTicket evaluation (event 81)
```

PassTicket abuse mapped, impersonation toward SPECIAL via forged one-time credentials.

The escalation is impersonation toward the highest privilege, achieved by exploiting an exposed key rather than a password, which is why it maps to ATT&CK T1550. In the default mode the PassTicket controls are relaxed and the abuse succeeds; the health check GIBPTK01 flags the exposure. The control concentrates on the key: protect the PassTicket secret as a crown-jewel secret, require additional authentication context where supported, and monitor SMF 80 PassTicket evaluations, a burst toward privileged users is the detection.

OMVS: UID(0) and BPX.SUPERUSER

The second privilege domain, OMVS superuser, has its own escalation paths. UID 0 is root; an account assigned UID 0 is a superuser outright. Short of that, the FACILITY profile BPX.SUPERUSER grants the ability to become superuser with su. A foothold checks and, if permitted, uses it:

```
RLIST FACILITY BPX.SUPERUSER ALL -> who may become superuser
PERMIT BPX.SUPERUSER CLASS(FACILITY) ID(user) ACCESS(READ)
ICH06011I PERMIT SUCCESSFUL FOR BPX.SUPERUSER
id -> uid=10535(user) su -> id -> uid=0 (superuser)
```

OMVS escalation, BPX.SUPERUSER access, or an assigned UID 0, grants superuser.

Anyone with READ to BPX.SUPERUSER can su to UID 0 and become root on the Unix side, and anyone assigned UID 0 is superuser from the start. Note that this is separate from RACF SPECIAL (a SPECIAL user who wants Unix root still needs one of these, and a superuser who wants the security database still needs SPECIAL) but a real attacker chains them, using SPECIAL to grant BPX.SUPERUSER, or superuser to reach RACF-controlled files. The controls are to restrict BPX.SUPERUSER to the few identities that genuinely need

it, never share UID 0 across accounts, and treat both as the privileged assignments they are. Broad BPX.SUPERUSER access is an OMVS escalation waiting to be used.

group-SPECIAL: scoped power that isn't

A last mechanism is group-SPECIAL. RACF allows SPECIAL to be granted within a group's scope rather than system-wide (a reasonable delegation for a team's own resources. The escalation is that if the group's scope is broad, group-SPECIAL is effectively system SPECIAL, and adding a foothold to such a group escalates it. A foothold enumerates groups (LISTGRP), finds one with group-SPECIAL and a wide scope, and) if it can get itself added, perhaps via another mechanism, inherits sweeping power. The control is to keep group scopes narrow, so that group-SPECIAL grants only the limited authority it appears to, and to review which groups carry it. Scoped power is only scoped if the scope is actually small.

The controls that stop escalation

Every mechanism here is closed by a resource control (the CIS 2.x controls of SECURE mode) and Gibson's health checks flag the exposures. In brief:

Mechanism	Real profile / check	Control that stops it
APF authorized code	GIBAPF01 (broad APF)	Protect APF/LINKLIB tightly
SURROGAT delegation	SURROGAT IBMUSER.SUBMIT; GIBSUR01	Minimal, scoped SURROGAT
PassTicket abuse	GIBPTK01; T1550	Protect the key; MFA context
BPX.SUPERUSER / UID 0	FACILITY BPX.SUPERUSER	Restrict; never share UID 0
group-SPECIAL	wide-scope group profile	Narrow group scopes

Escalation mechanisms and the CIS 2.x resource controls that stop each in SECURE mode.

The unifying control is least privilege on resources: protect the authorized libraries, grant SURROGAT and BPX.SUPERUSER to only the few who need them, guard the PassTicket key, and keep group scopes narrow (all backstopped by PROTECTALL(FAIL), which denies access to anything not explicitly permitted. In the default vulnerable mode these are relaxed and escalation succeeds by the paths shown; in SECURE mode they are tight and each path is a denied request. And the health checks) GIBAPF01, GIBSUR01, GIBPTK01, are the defender's early warning, naming the over-permissive profiles before an attacker finds them.

The escalation mistake is protecting logon while leaving the escalation profiles broad, a writable APF library, SURROGAT to a SPECIAL user, wide BPX.SUPERUSER, a broad group-SPECIAL. Any one of these turns an ordinary foothold into full control without a password. Escalation defense is resource least-privilege: tighten every profile that can grant power, and let the health checks flag the ones that are too broad.

Hands-On Labs

These labs work the escalation mechanisms against the Gibson sandbox in its default mode, then name the resource control that closes each.

Lab 49.1

Tell the two domains apart

Goal: distinguish RACF SPECIAL from OMVS superuser and read a foothold's privilege in each.

Background: the two top privileges are separate. This lab reads both from a foothold.

1. Read your RACF attributes and your OMVS identity.

```
LISTUSER ; id
```

Check yourself: you can show a foothold is neither SPECIAL nor UID 0, and explain that SPECIAL controls the security database while UID 0 is Unix root, separate targets.

Why it matters: confusing the two domains misjudges both the attack and the defense.

Escalation aims at one or both, by different mechanisms.

Blue-Team angle: audit SPECIAL holders and UID 0 assignments separately, they are different privileges with different controls.

Lab 49.2

Read a SURROGAT delegation

Goal: read a SURROGAT profile and explain escalation by delegation.

Background: SURROGAT delegates an identity. This lab reads one to a SPECIAL user.

1. List the SURROGAT profiles and identify delegation to a privileged user.

```
RLIST SURROGAT * ALL
```

Check yourself: you can read IBMUSER.SUBMIT with ALTER access and explain that it lets the holder submit jobs as IBMUSER, inheriting SPECIAL without a password.

Why it matters: SURROGAT to a SPECIAL user is a quiet, passwordless path to SPECIAL. It leaves no failed logons.

Blue-Team angle: SURROGAT to a privileged identity should be rare and justified; GIBSUR01 flags it, and a job running as a more privileged user is the detection.

Lab 49.3

The APF exposure

Goal: explain how a writable APF library enables escalation.

Background: authorized code bypasses RACF. This lab studies the APF path defensively.

1. Read the GIBAPF01 health check and explain the escalation it warns of.

Check yourself: you can explain that a writable APF library lets a foothold place authorized code that bypasses RACF and grants SPECIAL, and that GIBAPF01 flags the broad exposure.

Why it matters: authorized code is above RACF. Write access to an APF library is effectively the power to become SPECIAL.

Blue-Team angle: protect APF libraries as tightly as LINKLIB (CIS 2.2.15), no foothold should be able to write to one.

Lab 49.4

OMVS superuser

Goal: read the BPX.SUPERUSER profile and explain UID 0 escalation.

Background: OMVS superuser is the Unix-side top privilege. This lab reads its controls.

1. List who may become superuser and explain the su-to-UID-0 path.

```
RLIST FACILITY BPX.SUPERUSER ALL
```

Check yourself: you can explain that READ to BPX.SUPERUSER allows su to UID 0, that an assigned UID 0 is superuser outright, and that both are separate from RACF SPECIAL.

Why it matters: OMVS superuser is Unix root on the mainframe. Broad BPX.SUPERUSER access is an escalation path waiting to be used.

Blue-Team angle: restrict BPX.SUPERUSER to the few identities that need it and never share UID 0, audit both regularly.

Lab 49.5

Name the controls

Goal: map each escalation mechanism to the resource control that stops it.

Background: each mechanism has a specific control and a health check. This lab assembles the defense.

1. For APF, SURROGAT, PassTicket, BPX.SUPERUSER, and group-SPECIAL, state the control and the check that flags the exposure.

Check yourself: you can map APF to library protection (GIBAPF01), SURROGAT to minimal delegation (GIBSUR01), PassTicket to key protection (GIBPTK01), BPX.SUPERUSER to restriction, and group-SPECIAL to narrow scope.

Why it matters: escalation defense is resource least-privilege. Naming the controls and checks turns the mechanisms into a hardening and monitoring plan.

Blue-Team angle: run the health checks regularly, GIBAPF01, GIBSUR01, GIBPTK01 name the over-permissive profiles before an attacker finds them.

The payoff: granting SPECIAL

Each mechanism above delivers the same result (the ability to run a privileged command) and the escalation is complete the moment the attacker uses it to grant SPECIAL to an account they control. Whether they reached this point by submitting a job as IBMUSER through SURROGAT, or by running authorized code from a writable APF library, the final step is a single RACF command:

```
(running as IBMUSER via SURROGAT, or as authorized APF code)
```

```
ALTUSER HACKER SPECIAL
```

```
    ICH01032I ATTRIBUTE SPECIAL ASSIGNED TO HACKER
```

```
LISTUSER HACKER
```

```
    ATTRIBUTES=SPECIAL
```

The escalation payoff, one ALTUSER command grants SPECIAL to an attacker-controlled account.

The system confirms it plainly: ICH01032I ATTRIBUTE SPECIAL ASSIGNED, and LISTUSER now shows the attacker's account is SPECIAL. From an ordinary foothold, through one over-permissive profile, the attacker now holds the highest RACF privilege on a durable account of their own (which is also the beginning of persistence, the subject of the next chapter. This is why escalation defense concentrates on the profiles that grant power rather than on logon alone: the whole chain collapses to a single ALTUSER once any one mechanism is reachable. Two defensive facts follow. First, the command itself is logged) an ALTUSER that assigns SPECIAL is a high-value SMF event and one of the most important things to alert on. Second, the real fix is upstream: if no mechanism let the attacker run privileged commands in the first place, the ALTUSER never happens. Detection catches the payoff; the resource controls prevent it.

Lab 49.6

Watch the escalation payoff

Goal: observe the ALTUSER that grants SPECIAL and identify both the upstream prevention and the detection.

Background: every escalation mechanism ends in a privileged command. This lab reads the payoff and its two defenses.

1. From a privileged context, grant SPECIAL to an account and confirm it.

```
ALTUSER HACKER SPECIAL ; LISTUSER HACKER
```

Check yourself: you see ICH01032I ATTRIBUTE SPECIAL ASSIGNED and LISTUSER showing ATTRIBUTES=SPECIAL, and can explain that the resource controls prevent reaching this command while SMF detects it when it runs.

Why it matters: the whole escalation chain collapses to one ALTUSER. Preventing the mechanisms upstream is the fix; alerting on the command is the safety net.

Blue-Team angle: alert on any ALTUSER that assigns SPECIAL, OPERATIONS, or group-SPECIAL, it is a high-value SMF event and a strong indicator of escalation in progress.

Checkpoint: can you distinguish SPECIAL from superuser, explain escalation via APF, SURROGAT, PassTicket, and BPX.SUPERUSER, read the governing RACF profiles, and name the resource control and health check for each? Those skills are the middle link of the kill-chain, turning a foothold into full control, and the profiles a defender must tighten to prevent it.

Privilege escalation turns a foothold into the mainframe's highest privilege (RACF SPECIAL or OMVS superuser, two separate domains) and Gibson models the real mechanisms through RACF and OMVS profiles. An APF-authorized library, if writable by a foothold, runs authorized code that bypasses RACF and grants SPECIAL (flagged by GIBAPF01). A SURROGAT profile such as IBMUSER.SUBMIT with ALTER access lets a foothold submit jobs as IBMUSER and inherit SPECIAL without a password (GIBSUR01, rated high). PassTicket abuse forges one-time credentials toward SPECIAL by exploiting an exposed key (GIBPTK01, ATT&CK T1550). On the OMVS side, READ to BPX.SUPERUSER allows su to UID 0, and an assigned UID 0 is superuser outright. And a broad group-SPECIAL is effectively system SPECIAL. In the default mode these profiles are over-permissive and escalation succeeds; the CIS 2.x resource controls (tight library protection, minimal SURROGAT and BPX.SUPERUSER, a guarded PassTicket key, narrow group scopes, all backstopped by PROTECTALL(FAIL)) turn every path into a denied request, with the health checks flagging the over-broad profiles first.

What comes next

With full privilege obtained, an attacker's next goals are to spread and to stay. The next chapter covers lateral movement and persistence, using the foothold and its privilege to reach other systems, through the NJE trust and pivoting of Part VIII, and to establish durable access that survives a reboot or a password change: backdoor users, altered started tasks, and modified authorized libraries. It shows each through real RACF output in the default mode, and the controls and detections that find and remove them.

CHAPTER 50

Lateral Movement & Persistence

By the end of this chapter you will be able to:

Explain lateral movement through NJE trust and pivots.

Create a disguised backdoor user and see why it persists.

Read a STARTED profile and explain started-task persistence.

Describe authorized-library persistence.

Explain why persistence is hard to remove.

Name the audits and detections that find each foothold.

WITH FULL PRIVILEGE OBTAINED, AN attacker's goals become two: to spread, and to stay. Lateral movement reaches other systems; persistence establishes access that survives a reboot, a password change, or a first round of remediation. This chapter covers both through real RACF output in the default mode. Lateral movement reuses the trust and pivots of Part VIII (the NJE network and the LENNOX foothold) rather than finding new exploits. Persistence plants durable footholds: backdoor users, altered started tasks, and modified authorized libraries, each designed to outlast the obvious responses. The audits and detections that find and remove them are gathered at the end; the body of the chapter is how an attacker spreads and stays.

Lateral movement: reusing trust

Lateral movement rarely needs a new exploit; it reuses trust the attacker now controls. Two paths from earlier chapters carry them. The NJE network of Chapter 35 lets a trusted node run work on another, so an attacker on one node executes jobs on the mainframe through the trust relationship:

```
cross-node execution (NJE):
  submit job -> trusted node -> execute at GIBSON
  JOB $HASP122 X FROM ORAC RECEIVED AT GIBSON
  runs as the NODES-mapped identity
```

Lateral movement via NJE, a trusted node runs work on the mainframe through trust, not a password.

The other path is the LENNOX pivot of Chapter 46: a peripheral host that stores mainframe credentials, or that the mainframe can reach, becomes a bridge in either direction. Both are the same idea (movement along established trust rather than fresh compromise) and both are contained the same way: least NJE trust with TLS so a node cannot be impersonated, network segmentation so hosts cannot reach what they should not, and unique credentials so a password stolen on one system does not work on another. In the default mode the trust is broad and the movement is easy; the controls are the least-trust discipline applied across systems, not just within one.

Persistence: the goal of staying

Persistence is durable access, the ability to return without re-exploiting. An attacker who must repeat the whole kill-chain each time is fragile: change one password and they are locked out. Persistence makes them resilient, surviving the reboots, password resets, and remediation that would otherwise evict them. The mainframe offers several durable footholds, and a thorough attacker plants more than one, so that removing the obvious one leaves another behind. The sections that follow show the main three, each through the real RACF profile that governs it.

Backdoor users

The classic persistence is a new privileged account of the attacker's own, disguised to blend into the user list. Having reached SPECIAL, the attacker creates one that looks like a system account:

```
ADDUSER SYSDIAG DFLTGRP(SYS1) PASSWORD(***** ) NAME('SYSTEM DIAGNOSTIC')
ALTUSER SYSDIAG SPECIAL OPERATIONS
  ICH01032I ATTRIBUTE SPECIAL ASSIGNED TO SYSDIAG
  ICH01032I ATTRIBUTE OPERATIONS ASSIGNED TO SYSDIAG
```

A backdoor user, disguised as a system diagnostic account, granted SPECIAL and OPERATIONS.

The account is named to look official (SYSDIAG, "SYSTEM DIAGNOSTIC") so it does not stand out in a list of hundreds of users, and it is granted SPECIAL and OPERATIONS, the two most powerful attributes. It is the attacker's own credential, independent of every existing account, so resetting the password they originally compromised does nothing to it. This is why a backdoor user is such effective persistence: it survives the most common response (the password reset) entirely, and it hides in plain sight. The detection is to audit user creation, especially the assignment of SPECIAL or OPERATIONS: an ADDUSER followed by an ALTUSER granting privilege is a high-value SMF event and, for an account no one recognizes, a strong indicator of a planted backdoor.

Started-task persistence

A more durable foothold uses started tasks, the long-running system services that start at every IPL. The RACF STARTED class maps each started task to the identity it runs under, and the profiles are readable:

```
RLIST STARTED * ALL
CLASS      NAME
STARTED    FTPD1.*
  UNIVERSAL ACCESS  YOUR ACCESS  WARNING
  NONE              ALTER        NO
```

A STARTED profile, mapping a started task to the identity it runs under.

The escalation to persistence is this: if an attacker can alter a STARTED profile, they change the identity a system service runs as, or point a service at code they control (and because started tasks launch at every IPL, the attacker's code runs with privilege on every boot. That survives a reboot, which is exactly what defeats many remediation efforts: an administrator reboots to clean the system, and the persistence reloads with it. The ALTER access shown here is the exposure) the ability to change the profile, and the control is to protect STARTED profiles tightly and alert on any change to them, since a modified started-task identity is rarely legitimate and always serious.

Authorized-library persistence

The most powerful persistence combines with the APF escalation of the last chapter. If an attacker can modify an APF-authorized library, they can plant code there that runs in an authorized state every time it is loaded (at IPL, or whenever a program calls it. This foothold is durable and dangerous for the same reasons APF is the top escalation path: the code runs above RACF, so it can re-establish any other persistence the defender removes, and it reloads on every boot, so a reboot does not clear it. It is also hard to spot, because a modified authorized library looks like a normal one unless its integrity is checked. The controls are the CIS library controls again) protect APF libraries tightly so they cannot be modified, backed by integrity monitoring that detects any change to an authorized library. Authorized-library persistence is the reason library protection and integrity checking are not optional: a single modified library can undo an entire cleanup.

Why persistence is hard to remove

Persistence is designed to survive the obvious responses, and a thorough attacker layers it so that no single action evicts them. Reset the compromised password, and the backdoor user remains. Delete the backdoor user, and the started task recreates it at the next IPL. Reboot to clean the system, and the started task and the authorized-library code reload. Remove one foothold, and another re-establishes it. This is why removing persistence requires finding all of it (a full audit of the users, the started tasks, and the authorized libraries, comparing each against a known-good baseline) rather than reacting to the first thing found. And it is why detection at the moment of creation matters so much: catching the ADDUSER, the STARTED change, or the library modification as it happens is far easier than untangling layered persistence after the fact. The best time to find persistence is when it is planted.

The audits and detections

Each foothold is found by a specific audit, and each is best caught at creation. In brief:

Mechanism	Survives	Detection
Backdoor user	Password resets	Audit ADDUSER / ALTUSER SPECIAL
Started task	Reboot (IPL)	Alert on STARTED profile changes
Authorized library	Reboot and resets	Library integrity monitoring

Mechanism	Survives	Detection
Layered persistence	Single-action cleanup	Baseline comparison of all three

Persistence mechanisms and the audits that find each, best applied at creation.

The through-line is that persistence is planted with privileged commands that all leave SMF records (the ADDUSER, the ALTUSER, the STARTED change, the library write) so the detection already exists in the audit data of Part IV; the work is to watch for it. Least privilege limits what a foothold can create in the first place, and regular baseline comparison of users, started tasks, and authorized libraries finds anything planted despite that. In the default mode the persistence is easy to plant and easy to miss; the defense is to alert on the privileged commands that create it and to compare the system against its baseline, so that a planted foothold is caught before it is needed.

The persistence mistake is remediating the obvious foothold and stopping. A password reset leaves the backdoor user; deleting the user leaves the started task; a reboot reloads the authorized-library code. Persistence is layered to survive single actions, so removal requires finding all of it against a baseline, and the far easier path is to alert on the privileged commands that plant it, catching each foothold as it is created.

Hands-On Labs

These labs work lateral movement and persistence against the Gibson sandbox in its default mode, then name the audit that finds each.

Lab 50.1

Move laterally through trust

Goal: describe lateral movement via NJE trust and the LENNOX pivot, and the controls that contain each.

Background: lateral movement reuses trust. This lab traces the two paths defensively.

1. Describe cross-node execution over NJE and the LENNOX pivot, and name the containment for each.

Check yourself: you can explain that NJE runs work through node trust (contained by least trust and TLS) and that LENNOX pivots through stored credentials or reachability (contained by segmentation and unique credentials).

Why it matters: movement reuses trust the attacker controls. Least trust across systems contains it.

Blue-Team angle: treat every trust relationship (NJE nodes, peripheral hosts) as a movement path, and apply least trust and monitoring to each.

Lab 50.2

Plant and spot a backdoor user

Goal: create a disguised backdoor user and identify how it persists and how it is detected.

Background: a backdoor user survives password resets. This lab plants one and reads its detection.

1. Create a disguised account and grant it privilege.

```
ADDUSER SYSDIAG NAME( 'SYSTEM DIAGNOSTIC' ) ; ALTUSER SYSDIAG SPECIAL OPERATIONS
```

Check yourself: the account is granted SPECIAL and OPERATIONS and named to blend in, and you can explain that it survives resetting other accounts' passwords, detected by auditing ADDUSER and privilege assignment.

Why it matters: a backdoor user is an independent credential that survives the most common response. It hides in plain sight.

Blue-Team angle: alert on ADDUSER and any ALTUSER granting SPECIAL or OPERATIONS, and review the user list against a baseline for accounts no one recognizes.

Lab 50.3

Started-task persistence

Goal: read a STARTED profile and explain persistence that survives a reboot.

Background: started tasks launch at every IPL. This lab reads their identity mapping.

1. List the STARTED profiles and explain what altering one would achieve.

```
RLIST STARTED * ALL
```

Check yourself: you can explain that STARTED maps a task to an identity, and that altering it makes a system service run as an attacker-controlled identity at every IPL, persistence that survives reboot.

Why it matters: started-task persistence reloads on every boot, defeating the reboot that many cleanups rely on.

Blue-Team angle: protect STARTED profiles tightly and alert on any change, a modified started-task identity is rarely legitimate.

Lab 50.4

Authorized-library persistence

Goal: explain authorized-library persistence and its integrity control.

Background: a modified APF library runs attacker code authorized on every load. This lab studies it defensively.

1. Explain how a modified authorized library persists and why it is hard to spot.

Check yourself: you can explain that authorized code runs above RACF and reloads at IPL, so it can re-establish other footholds and survive a reboot, and that integrity monitoring detects the change.

Why it matters: a single modified authorized library can undo an entire cleanup by re-establishing persistence. Library integrity is essential.

Blue-Team angle: protect APF libraries so they cannot be modified, and monitor their integrity, any change to an authorized library is a serious event.

Lab 50.5

Find all of it

Goal: explain why layered persistence requires a full baseline audit to remove.

Background: persistence is layered to survive single actions. This lab plans complete removal.

1. Describe how the footholds re-establish one another, and the audit that finds all of them.

Check yourself: you can explain that a reset leaves the backdoor user, deletion leaves the started task, and a reboot reloads the library code, so removal requires auditing users, started tasks, and libraries against a baseline.

Why it matters: reacting to the first foothold found leaves the others. Finding all of it (and catching creation) is the real defense.

Blue-Team angle: maintain baselines of users, started tasks, and authorized libraries, compare regularly, and alert on the privileged commands that plant persistence, catching it at creation is far easier than untangling it later.

The lateral move in full

Seen end to end, cross-node execution is how an attacker turns node trust into code running on the mainframe. A job submitted from a trusted node is received and run on GIBSON, with no logon and no password:

```
xeq_execute rhost=GIBSON ohost=ORAC asuser=GIBSON
$HASP100 JOB ON INTRDR
JOB $HASP122 X FROM ORAC RECEIVED AT GIBSON
IEF403I X - STARTED
IEF404I X - ENDED
-> attacker's job ran on the mainframe, no logon
```

A full NJE lateral move, a job from another node runs on the mainframe through trust alone.

The job is received from ORAC and started and ended on GIBSON (IEF403I and IEF404I are the mainframe running the attacker's work) and it ran as the identity the NODES class mapped for inbound work from that node, with no authentication step anywhere. This is why the NJE trust relationships are part of the mainframe's attack surface: a compromised or spoofed trusted node is a passwordless way to execute code. The containment is the least-trust discipline of Chapter 35 (minimal AUTH per node, a least-privilege NODES identity, TLS so a node cannot be impersonated, and monitoring of cross-node jobs) now understood as lateral-movement containment. Trust that is not tightly scoped is movement waiting to happen.

The baseline audit in practice

Because persistence is layered, the reliable way to find all of it is to compare the system against a known-good baseline. A single audit surfaces the three footholds together:

BASELINE AUDIT (compare against known-good):

```

USERS:      + SYSDIAG (SPECIAL,OPERATIONS)    <- UNEXPECTED
STARTED:    FTPD1.* identity CHANGED          <- UNEXPECTED
APF LIB:    SYS1.LINKLIB member hash MISMATCH <- UNEXPECTED
-> three planted footholds, found by comparison

```

A baseline audit, comparing users, started tasks, and libraries finds all three planted footholds.

The comparison catches what reacting to a single alert would miss: an unexpected privileged user, a changed started-task identity, and a modified authorized library, all at once. This is the difference between chasing symptoms and removing the infection, the baseline shows everything that differs from known-good, so the whole layered persistence is visible in one view and can be removed together rather than one piece at a time while the others re-establish it. Maintaining baselines of the users, the STARTED profiles, and the authorized libraries, and comparing regularly, is therefore one of the highest-value defensive practices against a determined attacker. Combined with alerting on the privileged commands that plant persistence, it closes the gap between planting and discovery: caught at creation by the alerts, or caught soon after by the baseline, the footholds never become the durable access the attacker intended.

Lab 50.6

Run a baseline audit

Goal: use a baseline comparison to find layered persistence across users, started tasks, and libraries.

Background: layered persistence is found by comparison, not by chasing single alerts. This lab runs the audit.

1. Compare the current users, STARTED profiles, and authorized libraries against a known-good baseline and list every difference.

Check yourself: the comparison surfaces the unexpected SPECIAL user, the changed started-task identity, and the modified authorized library together, the full layered persistence in one view.

Why it matters: a baseline finds all of the persistence at once, so it can be removed together before the pieces re-establish one another.

Blue-Team angle: baseline users, STARTED profiles, and authorized libraries and compare on a schedule, the comparison is what turns scattered alerts into a complete picture of what was planted.

Checkpoint: can you explain lateral movement through NJE trust and pivots, plant and detect a backdoor user, explain started-task and authorized-library persistence, and describe why layered persistence requires a full baseline audit to remove? Those skills are the final attacker link of the kill-chain (spreading and staying) and the audits a defender uses to find and evict them.

After privilege, an attacker spreads and stays. Lateral movement reuses trust the attacker controls (NJE cross-node execution runs work on the mainframe through node trust, and the LENNOX pivot bridges through stored credentials or reachability) contained by least NJE trust with TLS, segmentation, and unique credentials. Persistence plants durable footholds: a backdoor user disguised as a system account (SYSDIAG, granted SPECIAL and OPERATIONS) that survives password resets; a STARTED profile altered so a system service runs as an attacker-controlled identity at every IPL, surviving reboots; and a modified APF-authorized library whose code runs above RACF on every load, able to re-establish anything removed. Because these are layered, no single action evicts them (a reset leaves the backdoor user, deletion leaves the started task, a reboot reloads the library) so removal requires auditing users, started tasks, and libraries against a baseline. Each is planted with a privileged command that leaves an SMF record, so the strongest defense is to alert on those commands and catch persistence at creation, when it is far easier to remove.

What comes next

The kill-chain is now complete (reconnaissance, initial access, escalation, movement, and persistence) and every stage left traces in the SMF and console data. The next chapter is the payoff: Gibson's CTI platform and HMS detection engine, which watch the whole chain. It covers the /cti console, the kill-chain detection that correlates the stages into a single picture, and the mapping of each technique to MITRE ATT&CK, the defender's view of the entire attack, assembled from the traces this part has been noting all along.

CHAPTER 51

The CTI Platform & HMS Detection

By the end of this chapter you will be able to:

- Access the CTI console and explain the HMS engine.
- Map mainframe attack techniques to MITRE ATT&CK.
- Read a sighting and what HMS records for each technique.
- Follow the kill-chain HMS correlates across phases.
- Explain the alarm threshold and mid-chain detection.
- Turn a detection into a contained response.

EVERY STAGE OF THE KILL-CHAIN in this part left traces (SMF records, console messages, correlation IDs) and this chapter is where they pay off. Gibson's CTI platform (Cyber Threat Intelligence) and its HMS detection engine watch the whole chain. HMS observes the attacks from Parts VIII and IX as they happen, maps each to a MITRE ATT&CK technique, correlates the individual observations into a progressing attack, and raises an alarm when a chain forms. This is the defender's assembled view: not scattered events, but a single, ATT&CK-mapped picture of an attack in progress, built from the very traces every earlier chapter has been noting. If the recon and kill-chain chapters were the offense a defender must understand, this chapter is the defense that understanding was for.

The CTI platform and HMS

CTI is the threat-intelligence console; HMS is the detection engine behind it, an intrusion-detection and kill-chain correlation system built for the mainframe. A defender reaches it through the CTI web console:

```
CTI console:  https://gibson:8443/cti
login:  ctiadmin / gibson
```

```
HMS engine:  observes security events -> sightings -> kill-chain
```

The CTI console and HMS engine, the defender's window on the whole attack chain.

HMS consumes the security events the mainframe already generates (the SMF records and console messages of Part IV, the correlation IDs of the recon chapters) and turns each into a sighting: a recognized attack technique, tagged and timestamped. It does for the mainframe what a modern detection platform does for any estate: watch continuously, recognize known techniques, and assemble them into a coherent story. The difference is that HMS speaks the mainframe's own telemetry natively, so the traces this book has pointed to all along become its input.

Mapping techniques to ATT&CK

HMS's common language is MITRE ATT&CK, the industry framework of adversary techniques. Every attack from the recon and kill-chain chapters maps to an ATT&CK technique, and HMS tags each observation with its technique ID:

Technique	ATT&CK	Where seen
Port scan (nmap)	T1046	Ch40 scanning
TSO user enumeration	T1087	Ch40, Ch47
FTP brute force	T1110	Ch43, Ch47
APF privilege escalation	T1068	Ch48
Surrogat / PassTicket abuse	T1550	Ch43, Ch48
NJE cross-node execution	T1021	Ch35, Ch49
FTP exfiltration	T1567	Ch43

The HMS technique catalog, each mainframe attack mapped to a MITRE ATT&CK technique.

This mapping is what makes the whole part cohere. Each attack a reader studied (the nmap scan, the TSO enumeration, the FTP brute force, the APF escalation, the SURROGAT abuse, the NJE execution) is a named ATT&CK technique that HMS recognizes, and speaking ATT&CK lets a mainframe defender share a language with the rest of security. The techniques are not exotic mainframe curiosities; they are the same adversary behaviors ATT&CK catalogs everywhere, expressed in mainframe terms. HMS is the translation layer that puts the mainframe on the same map as every other platform a defender protects.

A sighting

When HMS observes a technique, it records a sighting, the atomic unit of detection. Each carries the timestamp, the technique, its ATT&CK code, and the source:

```
HmsSighting(
    ts='2026-07-01 23:18:06',
    ttp_id='elv_apf', attack='T1068',
    name='APF privilege escalation',
    src_ip=... )
```

A sighting, HMS recording an observed technique with its ATT&CK code and source.

Every attack the earlier chapters ran produces a sighting like this: the scan, the enumeration, the escalation each land as a timestamped, ATT&CK-tagged record. A single sighting on its own is informative but not alarming, a lone port scan is background noise on any network. The power of HMS is not in the individual sighting but in what it does with a series of them, which is to recognize when they form a chain.

The kill-chain correlated

HMS carries a model of the full attack kill-chain (the ATT&CK phases in order) and correlates sightings against it, so isolated events become a recognized progression:

Phase	ATT&CK	Mainframe example
Reconnaissance	T1592	OSINT on job posts, public pages
Initial access	T1566	Spear-phishing link
Credential access	T1557	Adversary-in-the-middle on TN3270
Execution	T1106	CICS spool write to JES
Privilege escalation	T1068	Writable APF library
Collection	T1005	Extract RACF / Db2 artifacts
Exfiltration	T1567	Data leaves over FTP
Defense evasion	T1070	Purge JES SYSOUT
Impact (tabletop)	T1486	Would encrypt, described only

The HMS kill-chain, the ATT&CK phases HMS correlates sightings into.

Read down the chain and it is the whole of Parts VIII and IX in nine phases, from the OSINT of the reconnaissance chapters to the impact that a real attack would end in. HMS places each sighting into its phase and watches the picture build: a scan is reconnaissance, a brute force is credential access, an APF write is privilege escalation, an FTP transfer out is exfiltration. What was a series of unrelated alerts becomes a single attack seen progressing through its stages. Note the final phase deliberately: impact is a tabletop (described, not executed) because Gibson is a training system and the destructive end of the chain is studied, not performed. The value is that HMS shows the attack as a story, and a defender who sees the story forming can act before it reaches the end.

The alarm

Correlation becomes actionable through the alarm. HMS raises an alarm once enough sightings accumulate to indicate a chain rather than noise, a low threshold, because a forming chain is worth interrupting early:

```
sightings: nmap(T1046) -> tso_enum(T1087) -> ftp_brute(T1110)
count reaches ALARM_THRESHOLD (3)
*** HMS ALARM: kill-chain in progress ***
phase: credential access -> escalation likely next
```

The HMS alarm, a forming chain crosses the threshold and the defender is alerted mid-attack.

Three related sightings (a scan, an enumeration, a brute force) are no longer three alerts to triage separately but one recognized kill-chain, and HMS says so. This is the detection payoff the whole part has built toward: the defender is alerted while the attack is mid-chain, in credential access, before escalation and exfiltration, the earliest practical place to intervene. It echoes the recon chapters' lesson that catching an attacker early is catching them cheaply. The alarm turns the correlated story into a call to action at the moment it still matters most.

From detection to response

Detection exists to drive response, and HMS hands the defender everything needed to act. The correlated chain names the phase, so the response fits the stage: block the source of a scan, revoke the account under brute force, remove the persistence of the last

chapter, contain the movement. The correlation IDs from the recon chapters tie the sightings to the underlying SMF and console evidence, so an analyst can pivot from the alarm to the raw records in SDSF and OPERLOG and confirm exactly what happened. And the controls of Parts IV through IX are the response toolkit: PASSWORD(REVOKE) and MFA against the credentials, the resource controls against the escalation, the baseline audit against the persistence, segmentation against the movement. HMS is the trigger; the hardening this book has taught is the response. Together they close the loop, the attack is seen forming, understood in ATT&CK terms, traced to its evidence, and answered with the specific control for its stage.

The detection mistake is collecting the traces and never correlating them, treating each scan, failed logon, and privileged command as an isolated alert. An attack is a chain, and its danger is the progression, not any single step. HMS correlates the sightings into the kill-chain and alarms mid-attack; a defender's job is to feed it the mainframe's telemetry and act on the correlated story, not to drown in unconnected alerts.

Hands-On Labs

These labs use the CTI console and HMS engine against the Gibson sandbox to detect the attacks of the earlier chapters.

Lab 51.1

Access the CTI console

Goal: reach the CTI console and describe the HMS engine's role.

Background: CTI is the defender's window on the attack chain. This lab opens it.

1. Log on to the CTI console and locate the HMS detection view.

`https://gibson:8443/cti` (ctiadmin / gibson)

Check yourself: you can reach the console and explain that HMS observes security events, records sightings, and correlates them into the kill-chain.

Why it matters: the CTI console is where the traces this book has noted become a picture. It is the defender's vantage point.

Blue-Team angle: change the default CTI credential, a detection platform on a default password is itself an exposure, the very finding from the web-tier chapter.

Lab 51.2

Map techniques to ATT&CK

Goal: match the earlier attacks to their MITRE ATT&CK techniques.

Background: HMS speaks ATT&CK. This lab reads the technique catalog.

1. For the nmap scan, TSO enumeration, FTP brute force, APF escalation, and SURROGAT abuse, give the ATT&CK technique ID.

Check yourself: you can map the scan to T1046, enumeration to T1087, brute force to T1110, APF escalation to T1068, and SURROGAT to T1550.

Why it matters: ATT&CK is the shared language of detection. Mapping mainframe attacks to it puts the platform on the same map as every other.

Blue-Team angle: use ATT&CK technique IDs in your own mainframe detection rules, they let mainframe alerts integrate with enterprise security operations.

Lab 51.3

Read a sighting

Goal: trigger a technique and read the sighting HMS records.

Background: a sighting is the atomic unit of detection. This lab generates and reads one.

1. Cause a detectable technique and read the resulting sighting.

Check yourself: the sighting carries a timestamp, the technique ID, its ATT&CK code, the name, and the source, a complete, tagged record of the observation.

Why it matters: a sighting turns a raw event into a recognized technique. It is the building block of the correlated chain.

Blue-Team angle: a single sighting is often noise, its value is as input to correlation. Do not triage sightings in isolation.

Lab 51.4

Watch the chain correlate and alarm

Goal: generate a sequence of techniques and watch HMS correlate and alarm.

Background: HMS correlates sightings into the kill-chain and alarms at threshold. This lab runs a short chain.

1. Trigger a scan, an enumeration, and a brute force in sequence, and observe the alarm.

Check yourself: the three sightings are placed into their kill-chain phases, the count reaches the alarm threshold, and HMS raises a kill-chain alarm naming the current phase.

Why it matters: the alarm fires mid-chain, in credential access, before escalation and exfiltration, the earliest practical place to intervene.

Blue-Team angle: tune the alarm threshold to your environment, low enough to catch a forming chain early, high enough to see past isolated background noise.

Lab 51.5

Detection to response

Goal: turn an HMS alarm into a staged, contained response.

Background: detection drives response. This lab connects the alarm to the controls of earlier parts.

1. For an alarm naming a phase, state the containing control and the evidence to pivot to.

Check yourself: you can match credential access to revocation and MFA, escalation to the resource controls, persistence to the baseline audit, and movement to segmentation, and use the correlation IDs to pivot to the SMF and console evidence.

Why it matters: HMS is the trigger; the hardening of Parts IV through IX is the response. Together they close the loop from detection to containment.

Blue-Team angle: pre-map each kill-chain phase to its containing control so response is immediate, the alarm should trigger a known action, not a scramble.

A full scenario detected

HMS is best seen running against a whole attack rather than a single technique. Driving a scenario (a source address working through the chain) lights up sighting after sighting across the phases:

```
run_scenario(src_ip='10.4.22.17', userid='HACKER')
T1046  Port scan (nmap)          -> reconnaissance
T1087  TSO userid enumeration    -> recon
T1110  FTP brute force          -> credential access
T1068  APF privilege escalation -> privilege escalation
4 sightings across the chain -> ALARM
```

A full scenario detected, HMS records sightings across reconnaissance, credential access, and escalation.

In a few steps the source at 10.4.22.17, acting as HACKER, produces four sightings (a scan, an enumeration, a brute force, an APF escalation) spanning three kill-chain phases, and the accumulation crosses the alarm threshold. This is exactly the attack the kill-chain chapters walked, now seen from the defender's side as a coherent, progressing detection rather than the isolated actions it was when performed. The scenario is the proof that the traces were worth noting: every step the attacker took surfaced here as a tagged, correlated sighting, and the whole chain became visible from a single console.

Threat-actor attribution

HMS goes beyond detection to intelligence, the CTI in the platform's name. It matches the observed techniques and tools against known adversary profiles and attributes the activity to a named actor, then notifies the defenders:

```
*** HMS ALARM ***
actor:   HEAVY METAL SPIDER
message: Potential Heavy Metal Spider detected
ttps_seen:  nmap, tso_enum, ftp_brute
associated_tools: hydra, nikto, surrogat
notified: KEVIN, IBMUSER (delivery queued)
```

Threat-actor attribution, HMS matches the TTPs and tools to a known actor and notifies defenders.

The combination of techniques (nmap, TSO enumeration, FTP brute force) and associated tools (hydra, nikto, surrogat) matches a known adversary profile (here the training actor HEAVY METAL SPIDER) and HMS raises a named alarm and queues notifications to the responders. This is the intelligence layer that distinguishes a CTI platform from a bare intrusion-detection system: it does not only say that an attack is happening, it says who it looks like, based on the pattern of techniques and tools. For a defender that context shapes the response (a known actor's known objectives suggest what they will do next) and it connects the mainframe's detections to the wider threat-intelligence picture the rest of the security world uses. Detection recognizes the behavior; attribution recognizes the adversary.

Lab 51.6

Detect and attribute a full scenario

Goal: run a full attack scenario through HMS and read both the correlated chain and the actor attribution.

Background: HMS detects and attributes. This lab runs a whole scenario and reads both outputs.

1. Drive a scenario from one source through several techniques and read the accumulated sightings.

```
run_scenario(src_ip=..., userid=HACKER)
```

2. Read the alarm's actor attribution and the notified responders.

Check yourself: you see sightings across reconnaissance, credential access, and escalation crossing the threshold, and an alarm attributing the activity to a named actor from the matched techniques and tools, with responders notified.

Why it matters: detection says an attack is happening; attribution says who it resembles and hints what comes next. Together they shape a faster, better-targeted response.

Blue-Team angle: feed HMS your real mainframe telemetry and connect its attribution to enterprise threat intelligence, a mainframe detection tagged with ATT&CK and an actor integrates directly into security operations.

Checkpoint: can you access the CTI console, map mainframe attacks to MITRE ATT&CK, read a sighting, follow HMS correlating the kill-chain and raising an alarm mid-attack, and turn that alarm into a staged response? Those skills are the detection payoff of the whole part, the defender's assembled view of the entire attack, built from the traces every stage left.

Gibson's CTI platform and HMS detection engine are the payoff of the kill-chain part. Reached through the CTI console, HMS consumes the mainframe's own telemetry (the SMF records, console messages, and correlation IDs the earlier chapters noted) and turns each observed attack into a sighting tagged with a MITRE ATT&CK technique: the nmap scan is T1046, TSO enumeration T1087, FTP brute force T1110, APF escalation T1068, SURROGAT abuse T1550, and so on. HMS carries a model of the full ATT&CK kill-chain (reconnaissance, initial access, credential access, execution, privilege escalation, collection, exfiltration, defense evasion, and a tabletop impact phase) and correlates sightings into it, so isolated alerts become a recognized progression. When enough related sightings accumulate to cross the alarm threshold, HMS raises a kill-chain alarm mid-attack, alerting the defender while there is still time to intervene. That detection drives response: the phase names the containing control from Parts IV through IX, and the correlation IDs pivot to the underlying evidence. HMS speaks ATT&CK, putting the mainframe on the same map as every other platform, and closes the loop from trace to detection to contained response.

What comes next

HMS assembles the attack into a picture; the final chapter steps back to the defender who reads it. The Defender's View closes Part IX and the narrative of the book, drawing together the security model, the kill-chain, and the detection platform into a single defensive posture, and returning to where the book began: that understanding the offense exists entirely to serve the defense. It is the synthesis toward which every chapter has been building.

CHAPTER 52

The Defender's View

By the end of this chapter you will be able to:

Frame mainframe security as prevention, detection, and response.

Measure your posture with the health checks.

Map every kill-chain phase to a control that breaks it.

Assemble the whole attack-and-defense picture on one page.

Explain why understanding offense serves the defense.

Run the defender's continuous cycle.

THE BOOK BEGAN WITH A promise: that understanding the offense exists entirely to serve the defense. This final chapter of Part IX makes good on it. Stepping back from the individual attacks and detections, it draws the security model, the kill-chain, and the HMS platform into a single defensive posture (the defender's view of the whole. That posture has three layers: prevent what you can, detect what you cannot prevent, and respond to what you detect. Every part of this book maps onto those three, and this chapter assembles them, returning to where it started) with the mainframe as a system worth defending, now understood well enough to defend it. It is the synthesis toward which every chapter has been building.

Defense in depth: three layers

No single control holds a system; defense is layered. The defender's posture is three layers working together, and the whole book sorts into them:

Layer	What it does	Where in the book
Prevention	Block attacks before they succeed	SECURE mode, CIS controls (Ch46)
Detection	Catch what prevention misses	SMF, HMS (Part IV, Ch50)
Response	Contain what detection finds	Controls of Parts IV-IX

Defense in depth, prevention, detection, and response, and where each lives in the book.

Prevention is the SECURE posture (the CIS controls that block attacks outright. Detection is the SMF telemetry and the HMS engine that catch what prevention misses. Response is the containment that limits what detection finds. The layers cover for one another: prevention stops most attacks, detection catches the ones that slip through, and response contains those. A defender who relies on any one layer alone) prevention without detection, or detection without a ready response, has a gap an attacker will find. The strength is in the combination, and the rest of the chapter walks each layer in turn.

Prevention: the SECURE posture

Prevention is the CIS Benchmark applied (SECURE mode from Chapter 47. Every attack in the kill-chain was blocked in SECURE mode by a specific control: default-credential logon by changed credentials, brute force by PASSWORD(REVOKE), escalation by the resource controls, movement by segmentation. The defender's prevention job is to reach and hold the SECURE posture) to close the gap between where a real system sits and the benchmark. This is not a one-time project but a standing state: systems drift, new exposures appear, and the posture must be maintained. Prevention is the first and largest layer, because an attack blocked outright never needs detecting or responding to.

Knowing your posture: the health checks

A defender does not guess at their posture; they measure it. Gibson's health checks assess the security posture continuously, naming each exposure and its severity:

GIBSUR01	SURROGAT delegation	HIGH
GIBAPF01	Broad APF exposure	MEDIUM
GIBPTK01	PassTicket profile review	MEDIUM
GIBAUD01	Console security audit echo	LOW
GIBNET01	TN3270/TELNET parameters	LOW

The health-check posture, each exposure named and rated, a prioritized hardening list.

This is prevention made measurable. Each check corresponds to an attack from the kill-chain (GIBSUR01 to the SURROGAT escalation, GIBAPF01 to the APF path, GIBPTK01 to PassTicket abuse, GIBNET01 to the network controls) and the severity ranks the work: SURROGAT delegation is HIGH and comes first, the APF and PassTicket exposures are MEDIUM, the audit and network parameters LOW. The list is produced automatically and continuously, so the defender always knows how close the system is to the SECURE posture and what to fix next. Running the health checks regularly is proactive posture management, finding the over-permissive profiles before an attacker does, which is exactly the self-assessment the recon chapters urged, now built into the platform.

Detection: HMS and the traces

What prevention misses, detection catches. The traces every attack leaves (the SMF records, console messages, and correlation IDs) feed the HMS engine of the last chapter, which maps each to an ATT&CK technique, correlates them into the kill-chain, and alarms when a chain forms. The defender's detection job is to feed HMS the mainframe's telemetry and act on the correlated alarms rather than drowning in isolated alerts. Detection is the safety net under prevention: no posture is perfect, some attack will slip through or come from a trusted place, and detection is how the defender learns of it, ideally early, mid-chain, while there is still time to break it. Prevention and detection are partners: the audit controls that SECURE mode mandates (CIS 3.x) are what make detection possible, so a hardened system is also a watched one.

Response: breaking the chain

Detection drives response, and the kill-chain's great gift to the defender is that it can be broken at any link. Each phase has a control that stops the attack there:

Phase	Break it with	Detection signal
Reconnaissance	Reduce footprint; restrict access	Scan / enum sightings
Initial access	Change defaults; MFA; revoke	Failed-logon burst
Privilege escalation	Resource controls; least privilege	ALTUSER SPECIAL alert
Lateral movement	Segmentation; least trust	Cross-node / pivot logon
Persistence	Baseline audit; alert on creation	ADDUSER / STARTED change

Breaking the kill-chain, a control for every phase, and the signal that detects it.

The earlier the break, the cheaper: stopping reconnaissance costs nothing, while responding to persistence means a full baseline audit. So the defender's response is staged, the alarm names the phase, and the phase names the control: block the source, revoke the account, remove the privilege, contain the movement, evict the persistence. The response is not improvised; it is prepared, a known action mapped to each phase in advance, so that an HMS alarm triggers a practiced containment rather than a scramble. Breaking the chain at any link defeats the attack, and breaking it early defeats it cheaply.

The whole picture

Assembled, the three layers give the defender's complete view, for every attack, the prevention that blocks it, the detection that catches it, and the response that contains it. This is the entire book on one page:

Attack	Prevent	Detect / Respond
Recon & footprinting	Reduce exposure; SECURE	Sightings / block source
Credential attacks	Defaults, revoke, MFA	Failed logons / revoke
Escalation (APF, SURROGAT)	Resource controls; checks	SPECIAL alert / remove
Movement (NJE, pivot)	Least trust; segmentation	Trust logon / contain
Persistence	Least privilege	Baseline / evict all

The defender's complete view, every attack met by prevention, detection, and response.

Read across any row and it is a complete defense: the attack, the control that prevents it, and the signal and action that catch and contain it if prevention fails. Read down the columns and it is the three layers, a SECURE posture across the whole chain, HMS detection across the whole chain, prepared response across the whole chain. This table is the defender's view in one frame, and it is the destination the book has been travelling toward: not a list of attacks to fear, but a matrix of attacks answered, each with a prevention, a detection, and a response the reader now understands.

Why understanding offense serves defense

Return, finally, to the thesis. You cannot defend what you do not understand, and every attack in Parts VIII and IX was studied so the defender could recognize it, prevent it,

detect it, and respond to it. That is why the offense was taught (not to enable it, but because a defender blind to how attacks work cannot see them coming or know what to close. Gibson's whole design reflects this: the bounded, safe attack tools, the SECURE mode that proves the controls work, the tabletop impact phase that describes rather than performs, the correlation IDs that turn every attack into evidence. It is a training system built to teach defense through understanding, and the defender's view is its purpose. The reader who reaches here can look at a mainframe and see both the attacks it faces and the posture that answers them) which is what it means to defend it.

The defender's mistake is to treat security as a single layer (to harden and stop, or to watch without a ready response. Attacks are chains met by depth: prevention that blocks most, detection that catches the rest, and response that contains what remains. Measure your posture with the health checks, feed the traces to HMS, and prepare a response for every phase) the three layers together are the defense, and no one of them is enough alone.

Hands-On Labs

These labs assemble the defender's view against the Gibson sandbox, measuring posture, mapping the layers, and preparing the response.

Lab 52.1

Measure your posture

Goal: run the health checks and read the prioritized exposure list.

Background: a defender measures posture rather than guessing. This lab reads the checks.

1. Run the health checks and rank the exposures by severity.

Check yourself: you can read GIBSUR01 (HIGH), GIBAPF01 and GIBPTK01 (MEDIUM), and GIBAUD01 and GIBNET01 (LOW), and explain that the ranking is your prioritized hardening list.

Why it matters: measured posture is actionable. The checks name what to fix and in what order, produced continuously.

Blue-Team angle: run the health checks on a schedule, posture drifts, and continuous measurement finds new exposures before an attacker does.

Lab 52.2

Map the three layers

Goal: sort the book's controls into prevention, detection, and response.

Background: defense is layered. This lab organizes the whole book into the three layers.

1. For prevention, detection, and response, list the controls and tools the book provides.

Check yourself: you can place SECURE-mode CIS controls in prevention, SMF and HMS in detection, and the containment controls in response, and explain how the layers cover for one another.

Why it matters: seeing the layers is seeing the whole defense. Each covers the others' gaps.

Blue-Team angle: audit your own program for all three layers, a gap in any one is where an attacker succeeds.

Lab 52.3

Break the chain at every phase

Goal: map each kill-chain phase to the control that breaks it and the signal that detects it.

Background: the chain breaks at any link. This lab prepares the break for each.

1. For each phase, state the breaking control and the detection signal.

Check yourself: you can map reconnaissance, initial access, escalation, movement, and persistence each to a control and a signal, a staged, prepared response across the chain.

Why it matters: breaking the chain early is breaking it cheaply. A prepared response per phase turns an alarm into immediate action.

Blue-Team angle: pre-map each phase to its response so an HMS alarm triggers a practiced containment, not a scramble.

Lab 52.4

Assemble the whole picture

Goal: build the one-page matrix of attack, prevention, detection, and response.

Background: the defender's view fits on one page. This lab assembles it.

1. For each attack class, fill in the prevention, the detection, and the response.

Check yourself: you can complete a row for recon, credentials, escalation, movement, and persistence, each with its prevention, detection, and response, the whole defense in one frame.

Why it matters: the matrix is the defender's view. It turns a list of attacks into a list of attacks answered.

Blue-Team angle: keep this matrix as your reference, it is the book's defensive content on one page, and a checklist for any mainframe.

Lab 52.5

Run the continuous cycle

Goal: describe the defender's ongoing cycle of measure, harden, watch, respond.

Background: defense is continuous, not a project. This lab frames the cycle.

1. Describe the ongoing cycle: measure posture, harden toward SECURE, watch with HMS, respond to alarms, and repeat.

Check yourself: you can describe a continuous loop (health checks to measure, CIS controls to harden, HMS to detect, prepared containment to respond) that repeats as the system and threats change.

Why it matters: posture drifts and threats evolve, so defense is a cycle, not a milestone. The loop keeps the defense current.

Blue-Team angle: institutionalize the cycle (scheduled health checks, ongoing hardening, continuous HMS monitoring, and rehearsed response) so defense is a standing practice, not a one-time effort.

The kill-chain, end to end

One figure holds the whole of Parts VIII and IX from the defender's side, the chain the attacker climbs, and the control that breaks it at each rung:

THE KILL-CHAIN, AND WHERE THE DEFENDER BREAKS IT

Recon	-> reduce footprint	(Shodan self-check)
Initial acc.	-> change defaults, MFA	(revoke on failure)
Escalation	-> resource controls	(GIBSUR01/GIBAPF01)
Movement	-> segmentation, TLS	(least NJE trust)
Persistence	-> baseline audit	(alert on creation)
[Impact] -> tabletop only, never reached if broken above		

The kill-chain end to end, a break point at every phase, and impact reached only if none holds.

Read top to bottom, the attacker's progress is a ladder, and every rung has a control that removes it. Reduce the footprint and reconnaissance finds less; change defaults and require MFA and initial access fails; tighten the resource profiles the health checks flag and escalation is denied; segment the network and require TLS and movement is contained; audit against a baseline and persistence is evicted. The final rung, impact, is reached only if every control above it failed, and in Gibson it is a tabletop in any case, described rather than performed. The defender's task is not to hold a single wall but to remove as many rungs as possible, because an attacker who cannot climb past credential access never reaches the data. Every rung removed is an attack that ends earlier, and an attack that ends earlier is an attack that costs the defender less.

The continuous cycle

Defense is not a project with an end but a cycle that repeats, because a system's posture drifts and the threats against it evolve. The defender's loop ties the whole book together as an ongoing practice:

THE DEFENDER'S CONTINUOUS CYCLE

1. MEASURE -> run health checks (GIBSUR01, GIBAPF01, ...)
2. HARDEN -> close gaps toward SECURE (CIS controls)
3. WATCH -> feed telemetry to HMS; read sightings
4. RESPOND -> alarm names phase -> prepared containment
5. REPEAT -> posture drifts, threats evolve -> loop

The defender's continuous cycle, measure, harden, watch, respond, and repeat.

Measure the posture with the health checks; harden the gaps toward the SECURE benchmark; watch with HMS, feeding it the telemetry and reading the correlated sightings; respond to alarms with the containment prepared for each phase; and then repeat, because tomorrow the posture will have drifted and a new exposure may have appeared. This loop is the defender's view in motion, not a state reached once and held, but a discipline run continuously. The health checks make the measuring automatic, SECURE mode makes the hardening concrete, HMS makes the watching correlated, and the phase-mapped controls make the responding fast. Institutionalizing the cycle is what turns the knowledge in this book into a standing defense, one that keeps a mainframe secure not on the day it is hardened but every day after.

Lab 52.6

Run one full turn of the cycle

Goal: run a complete turn of the defender's cycle (measure, harden, watch, respond) against the sandbox.

Background: the cycle is the defense in motion. This lab runs one turn end to end.

1. Measure posture with the health checks and pick the highest-severity exposure.
2. State the hardening for it, the HMS signal that would detect its abuse, and the prepared response.

Check yourself: you can run health checks (finding GIBSUR01 HIGH first), state its hardening (minimal SURROGAT), name its detection (a job running as a privileged user), and its response (remove the delegation), one full turn of measure, harden, watch, respond.

Why it matters: a single turn rehearses the whole defense. Repeated, it is what keeps a mainframe secure over time, not just once.

Blue-Team angle: schedule the cycle (regular health checks, ongoing hardening, continuous HMS monitoring, rehearsed response) so the defense is a standing practice that tracks a drifting posture and an evolving threat.

Checkpoint: can you frame mainframe security as prevention, detection, and response, measure your posture with the health checks, break the kill-chain at every phase, assemble the whole picture on one page, and explain why understanding offense serves defense? Those are the defender's view, the synthesis of everything the book has taught, and the posture that answers everything it has shown.

The defender's view is the synthesis the whole book built toward: mainframe security as three layers (prevention, detection, and response) working in depth. Prevention is the SECURE posture, the CIS Benchmark applied, blocking each kill-chain attack with a specific control; a defender measures how close they are with the health checks, which name and rank the exposures (SURROGAT delegation HIGH, APF and Passticket MEDIUM, audit and network LOW) as a prioritized hardening list. Detection is the SMF telemetry and the HMS engine that catch what prevention misses, correlating traces into the kill-chain and alarming mid-attack. Response breaks the chain at any link, staged so the alarm's phase names the control (the earlier the break, the cheaper. Assembled, the three layers give a one-page matrix in which every attack is met by a prevention, a detection, and a response the reader now understands. And that is the point of having studied the offense at all: a defender cannot protect what they do not understand, so the attacks of Parts VIII and IX were taught to be recognized, prevented, detected, and answered. Gibson's bounded, safe, SECURE-provable design exists to teach exactly this) the defender's view is its purpose, and reaching it is what it means to defend the mainframe. This closes Part IX.

What comes next

The narrative of the book is complete (from first logon to the defender's view. What remains is reference. Part X is the Lab Catalog: the book's hands-on labs gathered and grouped so they can be found and run as a set, a practical companion to the chapters. Part XI is the Reference) the quick-reference appendices a working defender keeps at hand: RACF and console commands, ports, the API routes and JWT details, the CIS control map, the ATT&CK mapping, and the health checks. Together they turn the book from a course into a manual to return to.

The Lab Catalog

This catalog gathers every hands-on lab in the book (more than two hundred and seventy of them) into one place, grouped by part and chapter, so they can be found and run as a set rather than hunted for across the chapters. Each lab follows the same contract used throughout: a Goal, Background, the steps to Do it, a Check-yourself, Why it matters, and a Blue-Team angle. Every lab runs against the Gibson training sandbox, and every one that touches an API is a runnable curl command with a real port and path. The listing below is the index; the full lab text lives in the chapter named for each group. After the index, a set of suggested lab tracks sequences the labs toward particular goals, learning RACF, securing the API tier, or walking the kill-chain end to end.

How to use the catalog: find the part and chapter that matches what you want to practice, run its labs in order, and use the Check-yourself in each to confirm you succeeded before moving on. The labs within a chapter build on one another, and the chapters build across a part, so running a part's labs in sequence is a self-contained course in that topic. Labs that carry a Blue-Team angle (all of them) pair every action with the control or detection that answers it, so the catalog is as much a hardening checklist as a set of exercises.

Part I, Foundations

Part II, Getting Started

Chapter 5, Installing Gibson

- Lab 5.1, Confirm Gibson is installed and listening
- Lab 5.2, Check the ports are free before you start
- Lab 5.3, Find and read your configuration
- Lab 5.4, Verify every service is reachable

Chapter 6, Launching & IPL

- Lab 6.1, Cold-start Gibson and reach a ready system
- Lab 6.2, Confirm the listener port
- Lab 6.3, Read the startup console
- Lab 6.4, Stop and restart a service

Chapter 7, The Network Map

- Lab 7.1, Scan the estate
- Lab 7.2, Inventory the services
- Lab 7.3, Map the Gibson host
- Lab 7.4, Grab a service banner

Chapter 8, Connecting with c3270 & x3270

- Lab 8.1, Install the emulator and connect

Lab 8.2, Recover a locked keyboard

Lab 8.3, Open a 3270 session

Lab 8.4, Send a screen with an AID key

Lab 8.5, Diagnose a refused connection

Chapter 9, Your First Logon

Lab 9.1, Log on to TSO

Lab 9.2, Log off cleanly

Lab 9.3, Log on and read the panel

Lab 9.4, Log off cleanly and log back on

Lab 9.5, Read a refused logon

Lab 9.6, Read your last-access time

Part III, Core z/OS

Chapter 10, The TSO READY Prompt & Essential Commands

Lab 10.1, Inspect a library and its members

Lab 10.2, Submit a job and capture its number

Lab 10.3, Check your session profile

Lab 10.4, Explore a RACF group

Chapter 11, ISPF: Panels, Navigation, and the Management Menu

Lab 11.1, Navigate by option and equals-jump

Lab 11.2, Reach the Management menu tools

Lab 11.3, Jump straight to the Data Set List

Lab 11.4, Open the Settings panel

Chapter 12, Datasets & the Catalog

Lab 12.1, Find your data sets through the catalog

Lab 12.2, Allocate a new data set

Lab 12.3, Read a data set's attributes with LISTDS

Lab 12.4, Inventory your data sets with LISTCAT

Lab 12.5, Browse a member of a library

Chapter 13, VSAM & IDCAMS

Lab 13.1, Define a VSAM cluster

Lab 13.2, Delete a cluster cleanly

Lab 13.3, Read a cluster's anatomy

Lab 13.4, Delete a cluster and confirm all three parts

Chapter 14, The ISPF Editor

Lab 14.1, Find text in the editor

Lab 14.2, Change every occurrence

- Lab 14.3, Copy a block of lines
- Lab 14.4, Find and change text
- Lab 14.5, Exclude lines to focus on what matters
- Lab 14.6, Insert and repeat lines

Chapter 15, SDSF: Watching the System Work

- Lab 15.1, Submit a job and read its result
- Lab 15.2, Filter the panels to your own work
- Lab 15.3, Read the system log with FIND
- Lab 15.4, Purge finished output

Chapter 16, JES2 & JCL

- Lab 16.1, Write and submit a minimal job
- Lab 16.2, Run a TSO command in batch
- Lab 16.3, See where a job's identity comes from
- Lab 16.4, Write a multi-step job that creates a data set

Chapter 17, Abends & Debugging

- Lab 17.1, Read an S0C7 data-exception dump
- Lab 17.2, Diagnose a module-not-found S806
- Lab 17.3, Recognize a security abend (S913)
- Lab 17.4, Read a time-limit abend (S322)

Chapter 18, OMVS: The z/OS UNIX Side

- Lab 18.1, Enter OMVS and find your bearings
- Lab 18.2, Explore the file system
- Lab 18.3, Read a file's permissions
- Lab 18.4, See who is root
- Lab 18.5, Tighten a file's permissions

Part IV, Security & Identity

Chapter 19, RACF Fundamentals

- Lab 19.1, Read a user profile
- Lab 19.2, Read a group and its authorities
- Lab 19.3, Find the powerful users
- Lab 19.4, Check the system-wide posture
- Lab 19.5, Create, connect, and revoke an account

Chapter 20, RACF Resource Protection

- Lab 20.1, Protect a data set
- Lab 20.2, Grant an exception with PERMIT
- Lab 20.3, Find an over-exposed profile

Lab 20.4, Protect a general resource

Lab 20.5, Read your effective access

Lab 20.6, Read an access denial

Chapter 21, The RACF Database

Lab 21.1, Read the database status and hash inventory

Lab 21.2, Verify the database

Lab 21.3, Build the remediation list

Lab 21.4, Confirm the database is protected

Lab 21.5, Confirm the backup is current and protected

Lab 21.6, Reason about the unload

Lab 21.7, Trace the exposure to the policy

Chapter 22, zSecure: Monitoring the Posture

Lab 22.1, Review events and isolate the risky ones

Lab 22.2, Read a structured finding

Lab 22.3, Capture a baseline

Lab 22.4, Detect drift

Lab 22.5, Read the summary and drill down

Lab 22.6, Turn the filter into alerts

Lab 22.7, Prove a change shows up as drift

Chapter 23, SMF & Forensics

Lab 23.1, Read a security record

Lab 23.2, Identify the record types

Lab 23.3, Sketch a timeline

Lab 23.4, Protect the evidence

Lab 23.5, Review PassTicket activity

Lab 23.6, Go straight to the right record

Lab 23.7, Locate the complete record

Lab 23.8, Read the attack in the sequence

Part V, Operating the System

Chapter 24, The Master Console

Lab 24.1, Display the active work

Lab 24.2, Identify the system

Lab 24.3, Survey the network and the APF list

Lab 24.4, Stop and start a service

Lab 24.5, Read the operations log as a timeline

Lab 24.6, Go to the display that answers the question

Chapter 25, PARMLIB, PROCLIB & APF

- Lab 25.1, Explore PARMLIB
- Lab 25.2, List the started-task procedures
- Lab 25.3, Audit the APF list
- Lab 25.4, Reason about dynamic APF
- Lab 25.5, Confirm the sensitive members are protected
- Lab 25.6, Read the APF list at its source

Chapter 26, WLM, RMF, GRS & Health Checks

- Lab 26.1, Read the workload priorities
- Lab 26.2, Check for contention
- Lab 26.3, Run the Health Checker
- Lab 26.4, Act on the highest finding
- Lab 26.5, Read RMF as a security lens
- Lab 26.6, Read a finding in full and plan the fix
- Lab 26.7, Spot the anomalous job in RMF

Chapter 27, Sysprog Tooling

- Lab 27.1, Explore SMP/E
- Lab 27.2, Read the SYSMOD status
- Lab 27.3, Monitor with SYSVIEW
- Lab 27.4, Explore Endeavor's change path
- Lab 27.5, Reason about tool privilege
- Lab 27.6, Read the zones and a subsystem monitor

Part VI, The Subsystems**Chapter 28, CICS: The Transaction Monitor**

- Lab 28.1, Inquire into the region
- Lab 28.2, Read the running tasks
- Lab 28.3, Explore CECI
- Lab 28.4, Explore CEDA
- Lab 28.5, Audit transaction security
- Lab 28.6, Inventory the region's programs

Chapter 29, The Banking Stack

- Lab 29.1, Map the stack
- Lab 29.2, Read the data model
- Lab 29.3, Trace a transfer
- Lab 29.4, Understand the overflow
- Lab 29.5, Apply the secure fix

Lab 29.6, Read the bug and write the fix

Chapter 30, Db2: The Database

Lab 30.1, Query with SPUFI

Lab 30.2, Read the catalog for authorities

Lab 30.3, Inspect DDF

Lab 30.4, Understand injection

Lab 30.5, Apply parameterized queries

Lab 30.6, Audit the grants

Chapter 31, IMS: Hierarchical Data

Lab 31.1, Read the hierarchy

Lab 31.2, Navigate with DL/I

Lab 31.3, Read the program view

Lab 31.4, Display active work

Lab 31.5, Audit IMS access

Lab 31.6, Prove the read-only view holds

Chapter 32, z/VM: The Hypervisor

Lab 32.1, Work in CMS

Lab 32.2, Read the directory

Lab 32.3, Understand the classes

Lab 32.4, Run the class-creep audit

Lab 32.5, Apply Just Enough Authority

Lab 32.6, See the exposure, then close it

Lab 32.7, Confirm a guest's reach matches its role

Chapter 33, z/TPF: Extreme Throughput

Lab 33.1, Read system status

Lab 33.2, Understand the ECB

Lab 33.3, Use functional messages

Lab 33.4, Examine loadsets

Lab 33.5, Reason about z/TPF security

Lab 33.6, Trace a transaction and a code change

Part VII, Connectivity & Programming

Chapter 34, FTP & TCP/IP

Lab 34.1, Map the listening ports

Lab 34.2, Transfer with the right type

Lab 34.3, Submit a job through FTP

Lab 34.4, Assess the FTP exposure

Lab 34.5, Harden FTP

Lab 34.6, Trace the round trip and the SQL mode

Chapter 35, NJE: Network Job Entry

Lab 35.1, Map the NJE network

Lab 35.2, Read the trust levels

Lab 35.3, Observe cross-node execution

Lab 35.4, Understand NMR spoofing

Lab 35.5, Secure the trust boundary

Lab 35.6, Inspect the sockets and handshake

Lab 35.7, Constrain inbound work with NODES

Chapter 36, IND\$FILE: Terminal File Transfer

Lab 36.1, Understand the mechanism

Lab 36.2, Transfer a file both ways

Lab 36.3, Choose the right transfer type

Lab 36.4, Read a transfer in progress

Lab 36.5, Assess the IND\$FILE exposure

Lab 36.6, Recognize a corrupted transfer

Lab 36.7, Handle record formats

Lab 36.8, Target a PDS member safely

Chapter 37, The Web & API Tier

Lab 37.1, Map the web ports

Lab 37.2, Query the REST API

Lab 37.3, Assess the dashboard credential

Lab 37.4, Understand the Tomcat manager path

Lab 37.5, Harden the web tier

Lab 37.6, Test the API for injection and mass assignment

Chapter 38, RESTful APIs, z/OS Connect & API Security

Lab 38.1, Return account data and spot the IDOR

Lab 38.2, Inject SQL through the API

Lab 38.3, Excessive data, mass assignment, verbose errors

Lab 38.4, Obtain an OAuth token

Lab 38.5, Forge a JWT

Lab 38.6, Run the full identity-to-data chain

Chapter 39, Programming Gibson

Lab 39.1, Run a REXX script

Lab 39.2, Read JCL

Lab 39.3, Compile COBOL

- Lab 39.4, Read an assembler listing
- Lab 39.5, Read code as a defender
- Lab 39.6, Follow a script's and a job's reach

Part VIII, Reconnaissance & Attack Tools

Chapter 40, The Reconnaissance Toolkit

- Lab 40.1, Map the phases
- Lab 40.2, Survey the toolkit
- Lab 40.3, Read the nmap menu
- Lab 40.4, Assess your own exposure
- Lab 40.5, Plan recon detection
- Lab 40.6, Read a mainframe fingerprint
- Lab 40.7, Turn a finding into a control

Chapter 41, nmap on the Mainframe

- Lab 41.1, Grab the logon screen
- Lab 41.2, Understand user enumeration
- Lab 41.3, Enumerate CICS
- Lab 41.4, Read the bounded audit
- Lab 41.5, Watch cicspwn get blocked
- Lab 41.6, Enumerate the NJE network defensively

Chapter 42, EZRecon: External Footprinting

- Lab 42.1, Read the DNS records
- Lab 42.2, Interpret WHOIS
- Lab 42.3, Test the zone transfer
- Lab 42.4, Find the hidden hosts
- Lab 42.5, Search Shodan for your mainframe
- Lab 42.6, Assemble and prioritize the footprint

Chapter 43, Web-Application Reconnaissance

- Lab 43.1, Fingerprint the server
- Lab 43.2, Run a web scan
- Lab 43.3, Enumerate methods and CORS
- Lab 43.4, Find the manager
- Lab 43.5, Map and secure the endpoints
- Lab 43.6, Close the information leaks

Chapter 44, Mainframe Attack Tools

- Lab 44.1, Understand the two modes
- Lab 44.2, Test anonymous FTP in both modes

- Lab 44.3, See the CICS tools blocked
- Lab 44.4, Understand PassTicket defense
- Lab 44.5, Map tools to detection
- Lab 44.6, Detect the FTP and Db2 techniques

Chapter 45, OSINT: Open-Source Intelligence

- Lab 45.1, Map the categories
- Lab 45.2, Assess people exposure
- Lab 45.3, Read a job posting as recon
- Lab 45.4, Understand metadata and breach risk
- Lab 45.5, Reduce the footprint
- Lab 45.6, Hunt for leaked secrets

Chapter 46, LENNOX & Pivoting

- Lab 46.1, Understand the indirect path
- Lab 46.2, Enumerate from the foothold
- Lab 46.3, Find the pivot material
- Lab 46.4, Trace the pivot and its breaks
- Lab 46.5, Contain lateral movement
- Lab 46.6, Contain the trust pivot

Part IX, Kill-Chain & Detection

Chapter 47, The Security Model

- Lab 47.1, Read the modes
- Lab 47.2, Map SECURE mode to CIS
- Lab 47.3, Trace PROTECTALL(FAIL)
- Lab 47.4, Understand the network controls
- Lab 47.5, Posture and the NORACF extreme
- Lab 47.6, Hold one attack against three postures

Chapter 48, Initial Access & Credential Attacks

- Lab 48.1, Log on with the default credential
- Lab 48.2, Enumerate valid users
- Lab 48.3, Brute force and spraying
- Lab 48.4, Enumerate the database from a foothold
- Lab 48.5, Name the controls
- Lab 48.6, Tell a target from a dead end

Chapter 49, Privilege Escalation

- Lab 49.1, Tell the two domains apart
- Lab 49.2, Read a SURROGAT delegation

- Lab 49.3, The APF exposure
- Lab 49.4, OMVS superuser
- Lab 49.5, Name the controls
- Lab 49.6, Watch the escalation payoff

Chapter 50, Lateral Movement & Persistence

- Lab 50.1, Move laterally through trust
- Lab 50.2, Plant and spot a backdoor user
- Lab 50.3, Started-task persistence
- Lab 50.4, Authorized-library persistence
- Lab 50.5, Find all of it
- Lab 50.6, Run a baseline audit

Chapter 51, The CTI Platform & HMS Detection

- Lab 51.1, Access the CTI console
- Lab 51.2, Map techniques to ATT&CK
- Lab 51.3, Read a sighting
- Lab 51.4, Watch the chain correlate and alarm
- Lab 51.5, Detection to response
- Lab 51.6, Detect and attribute a full scenario

Chapter 52, The Defender's View

- Lab 52.1, Measure your posture
- Lab 52.2, Map the three layers
- Lab 52.3, Break the chain at every phase
- Lab 52.4, Assemble the whole picture
- Lab 52.5, Run the continuous cycle
- Lab 52.6, Run one full turn of the cycle

Suggested Lab Tracks

The labs can also be run as themed tracks that cut across parts, each sequencing the labs toward a particular capability. Four tracks are suggested below; each names the chapters to work through in order for that goal.

Track 1, The RACF & Identity Track

For mastering identity and access control: work through RACF Fundamentals, RACF Resource Protection, and The RACF Database (Part IV), then zSecure and SMF & Forensics for posture and audit. Follow with the identity attacks that test it (Initial Access & Credential Attacks and Privilege Escalation (Part IX)) so the controls are seen both configured and defended. This track turns the RACF chapters and their attacks into a single course on mainframe identity.

Track 2, The API Security Track

For securing the modern mainframe API tier: start with The Web & API Tier, then RESTful APIs, z/OS Connect & API Security for the full curl-based lab set, the account IDOR, SQL injection, mass assignment, verbose errors, the OAuth token flow, and the JWT forgery variants. Follow with Web-Application Reconnaissance (Part VIII) to see the tier assessed from outside. This track is a complete, runnable API-security course grounded in the z/OS Connect → CICS → COBOL → Db2 chain.

Track 3, The Kill-Chain Track

For walking an attack end to end and defending it: begin with The Security Model, then Initial Access, Privilege Escalation, and Lateral Movement & Persistence (Part IX), watching each attack succeed in the default mode. Close with The CTI Platform & HMS Detection and The Defender's View to see the whole chain detected, attributed, and answered. Pair with the Part VIII reconnaissance chapters for the phases that precede initial access. This track is the complete kill-chain, offense and defense together.

Track 4, The Connectivity Track

For the ways on and off the platform: work through FTP & TCP/IP, NJE: Network Job Entry, IND\$FILE, The Web & API Tier, and the API-security chapter (Part VII). Follow with The Reconnaissance Toolkit and nmap on the Mainframe (Part VIII) to see those same doors discovered and enumerated from outside. This track covers every network path into the mainframe and how each is secured and monitored.

Together the catalog indexes 276 labs across the book's chapters. Whether run part by part as a structured course, or track by track toward a specific capability, they turn the reading into practice, and, because every lab carries its Blue-Team angle, into a working defender's checklist for the mainframe.

PART XI

Reference

This part collects the quick-reference material a working defender keeps at hand: the ports and services, the RACF and console commands, the API routes and their authentication, the CIS z/OS Benchmark control map, the MITRE ATT&CK technique mapping, and the health checks. Every table is drawn from the Gibson training system as used throughout the book, so the reference matches exactly what the chapters and labs demonstrate. Where a real z/OS system differs in detail, the shape and the security meaning are the same.

Appendix A, Ports & Services

The network services Gibson exposes and the ports they listen on. Every port is a door to assess: does it need to be reachable from where it is, and is it monitored?

Port	Service	Notes
21	FTP	SITE FILETYPE SEQ/JES/SQL; anonymous risk
23	Telnet	plaintext; prefer TLS
175	NJE	clear; SECURE=OPTIONAL exposure
2022	z/OS UNIX (SSH)	OMVS shell access
2252	NJE over TLS	labelled/encrypted node link
2380	LENNOX	peripheral host (pivot)
3023	z/VM	hypervisor console
3270	TN3270	terminal; AT-TLS in SECURE
8023	Web 3270 terminal	browser terminal
8080	CBSA REST API	z/OS Connect-style; vuln API
8443	Dashboard / CTI	HTTPS; default cred risk
9080	FIBS web	OAuth/OIDC + JWT identity
50000	Db2 DDF	DRDA network data access
50001	Db2 web services	REST/HTTP data access

Appendix A, Gibson ports and services.

Appendix B, RACF Command Reference

The RACF commands used throughout the book, for identity, resource protection, and posture. Read commands (LISTUSER, RLIST, SEARCH) enumerate; change commands (ADDUSER, ALTUSER, PERMIT, SETROPTS) configure, and are the high-value events to audit.

Command	Purpose
LISTUSER userid	Show a user's attributes, groups, and privileges
LISTGRP group	Show a group's members and superior group
RLIST class profile ALL	Show a general-resource profile and access list
LISTDSD DA(dsn) ALL	Show a data-set profile and access list
SEARCH CLASS(USER)	Enumerate profiles in a class

Command	Purpose
ADDUSER userid ...	Create a user (audit: possible backdoor)
ALTUSER userid SPECIAL	Grant an attribute (audit: escalation)
PERMIT profile CLASS(..) ID(..) ACCESS(..)	Grant access to a resource
RDEFINE class profile	Define a resource profile
CONNECT userid GROUP(grp)	Add a user to a group
SETROPTS RACLIST(class) REFRESH	Activate/refresh in-storage profiles
SETROPTS PROTECTALL(FAIL)	Fail-closed on unprofiled data sets (CIS 2.3.3)

Appendix B, RACF commands for enumeration, configuration, and posture.

Appendix C, Console & TSO Quick Reference

Common operator-console and TSO commands seen in the operating and subsystem chapters. Console commands display and control the running system; TSO commands work with data sets and the session.

Command	Purpose
D A,L	Display active jobs and started tasks
D T	Display the system time
D GRS,C	Display global resource serialization contention
D ASM	Display page/auxiliary storage
\$D JOBQ	(JES2) display the job queue
\$D SPOOL	(JES2) display spool usage
HZSPRINT	Print health-check output (HZS0004I accepted)
LISTDS 'dsn'	(TSO) list a data set's attributes
ALLOCATE / FREE	(TSO) allocate or free a data set to the session
SUBMIT 'dsn(member)'	(TSO) submit JCL as a batch job

Appendix C, console and TSO commands.

Appendix D, API Routes & JWT/OAuth Reference

The API routes that return data or handle identity, as runnable curl targets. The CBSA API on port 8080 is intentionally open (the vuln labs); the FIBS API on 9080 carries the OAuth/OIDC and JWT identity layer. Substitute your host for "gibson".

Method & path (host:port)	Returns	Auth
GET 8080 /api/v1/cbsa/vuln/account/{n}	Account row (Db2)	none (IDOR)
GET 8080 /api/v1/cbsa/vuln/accounts?customer={q}	Account rows	none (SQLi)
GET 8080 /api/v1/cbsa/vuln/debug/customer/{id}	Full customer	none
PUT 8080 /api/v1/cbsa/vuln/customers/{id}	Updated record	none (mass-assign)
POST 8080 /api/v1/cbsa/vuln/transfer	Transfer result	none
GET 8080 /api/v1/cbsa/vuln/sql-error?account={x}	SQLCODE -104	none (verbose)
POST 8080 /api/v1/cbsa/vuln/login	Unsigned token	weak
GET 9080 /.well-known/openid-configuration	OIDC discovery	none
GET 9080 /oauth/jwks	JWKS keys	none
POST 9080 /oauth/token	JWT access token	client creds
GET 9080 /webapi/accounts	Accounts JSON	JWT/session
GET 9080 /webapi/teller/search	Teller data	staff (403 else)

Method & path (host:port)	Returns	Auth
POST 9080 /webapi/labs/jwt/forge	Forge lab result	none (lab)

Appendix D, API routes, what they return, and their authentication.

JWT forge variants at /webapi/labs/jwt/forge: alg_none (strip signature), weak_hmac (guessable key), expired (past exp), wrong_audience (aud mismatch), and role (elevated claim). All are accepted in VULNERABLE mode and rejected in SECURE. The token issued by /oauth/token is HS256, with issuer, audience, and a kid; validation must check the signature, reject alg=none, and validate issuer, audience, and expiry.

Appendix E, CIS z/OS Benchmark Control Map

The CIS controls SECURE mode implements, grouped as the benchmark groups them. This is the hardening baseline: measure a system against it and close the gaps.

CIS	Control	Area
1.1.1	PASSWORD(INTERVAL) <= 90	Identity
1.1.2	PASSWORD(HISTORY) >= 4	Identity
1.1.5	PASSWORD(REVOKE) after failures	Identity
1.2.6	OPERCMD5 active / RACLSTed	Identity
1.2.8	FACILITY active / RACLSTed	Identity
1.3.1	SPECIAL attribute justified	Identity
2.1.8	RACF security data sets protected	Resource
2.2.5	PARMLIB protected	Resource
2.2.13	PROCLIB protected	Resource
2.2.15	LINKLIB protected	Resource
2.3.3	PROTECTALL(FAIL)	Resource
3.1	CMDVIOL logging	Audit
3.2	SPECIAL activity logging	Audit
3.3	AUDIT classes active	Audit
6.2.4	AT-TLS for FTP	Network
6.6.4	AT-TLS for TN3270	Network
6.6.5	TN3270 SMF recording	Network
9.23	USS Telnet server not active	Network

Appendix E, the CIS z/OS Benchmark controls SECURE mode applies.

Appendix F, MITRE ATT&CK Technique Mapping

The mainframe attacks in the book mapped to MITRE ATT&CK techniques, as HMS tags them. This is the shared language that puts mainframe detections on the same map as the rest of security.

Technique	ATT&CK	Detection signal
Port scan (nmap)	T1046	Scan sightings
VTAM / NJE enumeration	T1590	Enumeration bursts
TSO user enumeration	T1087	Logon oracle probes
FTP brute force	T1110	Failed-login bursts
JCL/REXX upload	T1105	Ingress transfer
NJE cross-node execution	T1021	Cross-node job

Technique	ATT&CK	Detection signal
NJE NMR command injection	T1059	Operator-command events
APF privilege escalation	T1068	Authorized-code events
Surrogat / PassTicket abuse	T1550	SMF 80 PassTicket eval
FTP exfiltration	T1567	Large outbound transfer
Nikto web scan	T1595	Web-scan 404 bursts

Appendix F, mainframe attacks mapped to MITRE ATT&CK.

Appendix G, Health Checks

The Gibson health checks measure security posture, naming each exposure and its severity, a prioritized hardening list, produced continuously. Run them regularly; posture drifts.

Check	Exposure	Severity	Related control
GIBSUR01	SURROGAT delegation	HIGH	Minimal, scoped SURROGAT
GIBAPF01	Broad APF exposure	MEDIUM	Protect APF/LINKLIB
GIBPTK01	PassTicket profile review	MEDIUM	Protect PassTicket key
GIBAUD01	Console security audit echo	LOW	Audit logging (CIS 3.x)
GIBNET01	TN3270/TELNET parameters	LOW	AT-TLS (CIS 6.x)

Appendix G, the health checks and the control each points to.

Appendix H, The Kill-Chain Reference

The kill-chain phases with the control that breaks each and the signal that detects it, the defender's one-page response map. Break the chain at any link; the earlier, the cheaper.

Phase (ATT&CK)	Break it with	Detect
Reconnaissance (T1592)	Reduce footprint; restrict	Scan/enum sightings
Initial access (T1566)	Change defaults; MFA; revoke	Failed-logon burst
Credential access (T1557)	MFA; AT-TLS	AiTM / logon anomalies
Execution (T1106)	Transaction/command security	CICS/console events
Privilege escalation (T1068)	Resource controls; checks	ALTUSER SPECIAL alert
Collection (T1005)	Least privilege; encryption	Bulk-access anomalies
Lateral movement (T1021)	Segmentation; least trust	Cross-node/pivot logon
Exfiltration (T1567)	Egress control; AT-TLS	Large outbound transfer
Persistence	Least privilege; baseline	ADDUSER/STARTED change
Impact (T1486, tabletop)	All of the above	Reached only if all fail

Appendix H, the kill-chain, its break points, and its detections.

These appendices condense the book's technical content into the tables a defender returns to: the doors to watch (A), the commands to run and audit (B, C), the API surface to secure (D), the baseline to harden toward (E), the techniques to detect (F), the posture to measure (G), and the chain to break (H). Together with the acronym table published alongside the book, they turn the manual into a working reference. This closes the book.

Index

Page references to the principal topics, commands, and technologies covered in the book. Bold ranges in the text and the Reference appendices (Part XI) give the fullest treatment of each.

Abend.....	25, 70, 112, 113, 122, 125, 126, 127
APF.....	12, 13, 15, 26, 72, 132, 167, 168
AT-TLS.....	375, 376, 378, 379, 381, 433, 435, 436
ATT&CK.....	25, 349, 351, 352, 353, 354, 356, 392
BOLA.....	298, 299, 303, 304, 307
Brute force.....	351, 352, 354, 355, 375, 381, 382, 383
Catalog.....	17, 19, 20, 22, 25, 68, 70, 71
CICS.....	9, 13, 14, 15, 16, 17, 24, 25
CIS Benchmark.....	374, 375, 377, 414, 421
COBOL.....	8, 25, 26, 90, 91, 102, 224, 225
Console.....	9, 13, 24, 25, 26, 27, 37, 38
CTI.....	7, 8, 9, 10, 11, 12, 13, 14
Dataset.....	15, 18, 19, 20, 22, 23, 24, 26
Db2.....	9, 13, 14, 16, 17, 24, 25, 26
DDF.....	16, 232, 233, 234, 235, 237, 238, 239
EZRecon.....	25, 26, 81, 83, 315, 318, 320, 324
Footprinting.....	318, 320, 324, 332, 333, 334, 335, 336
FTP.....	16, 25, 26, 31, 32, 34, 41, 42
GRS.....	26, 189, 190, 192, 198, 200, 201, 202
Health check.....	41, 163, 200, 201, 202, 203, 204, 205
HLASM.....	26, 297, 307, 308, 310, 314
HMS.....	26, 372, 405, 406, 407, 408, 409, 410
IBMUUSER.....	21, 22, 26, 61, 62, 63, 64, 65
IDCAMS.....	93, 94, 95, 96, 97, 98, 99, 100
IDOR.....	231, 291, 298, 299, 303, 304, 306, 307
IMS.....	8, 13, 16, 24, 26, 34, 35, 54
IND\$FILE.....	55, 168, 172, 281, 282, 283, 284, 285
IPL.....	9, 10, 13, 14, 20, 27, 29, 35
ISPF.....	9, 13, 15, 17, 19, 23, 24, 26
JCL.....	15, 18, 20, 21, 24, 26, 46, 51
JES2.....	13, 17, 19, 21, 24, 26, 32, 39
JWT.....	16, 298, 301, 302, 303, 305, 306, 307
Kill-chain.....	50, 98, 182, 364, 365, 368, 371, 372
Lateral movement.....	275, 318, 320, 364, 365, 367, 368, 370
LENNOX.....	25, 26, 48, 318, 364, 365, 366, 367
LINKLIB.....	19, 87, 185, 194, 197, 376, 379, 381

Mass assignment.....	295, 296, 300, 301, 303, 304, 305, 343
MFA.....	15, 61, 62, 63, 68, 351, 353, 356
nikto.....	25, 26, 318, 340, 341, 342, 345, 347
NJE.....	15, 25, 26, 32, 34, 43, 48, 49
nmap.....	25, 26, 45, 46, 47, 48, 49, 51
OAuth.....	153, 298, 301, 302, 303, 305, 306, 307
OIDC.....	62, 301, 305, 307, 433, 434
OMVS.....	13, 15, 24, 25, 26, 39, 69, 71
OSINT.....	16, 25, 26, 318, 320, 324, 356, 357
PARMLIB.....	13, 19, 20, 26, 38, 39, 71, 74
PassTicket.....	25, 71, 168, 170, 172, 173, 175, 176
Persistence.....	192, 194, 195, 196, 221, 372, 396, 397
Pivoting.....	318, 320, 324, 364, 365, 366, 367, 368
Privilege escalation.....	113, 138, 171, 174, 198, 253, 368, 372
PROCLIB.....	19, 26, 87, 88, 121, 185, 191, 192
PROTECTALL.....	26, 374, 375, 376, 378, 379, 381, 393
RACF.....	9, 10, 13, 14, 15, 16, 17, 18
REXX.....	26, 91, 102, 297, 307, 308, 310, 311
RMF.....	26, 193, 198, 200, 201, 202, 204, 205
SDSF.....	15, 24, 26, 54, 70, 71, 73, 74
Shodan.....	333, 335, 336, 337, 338, 339, 340, 365
SMF.....	10, 13, 14, 15, 18, 19, 22, 23
SPECIAL.....	7, 15, 16, 22, 75, 101, 105, 120
SQL injection.....	233, 236, 240, 291, 295, 296, 298, 300
SURROGAT.....	15, 75, 106, 115, 118, 123, 124, 153
TN3270.....	16, 23, 25, 26, 32, 40, 41, 43
TSO.....	9, 13, 14, 15, 17, 19, 23, 24
VSAM.....	10, 16, 17, 18, 19, 23, 24, 70
VTAM.....	9, 10, 13, 14, 16, 17, 18, 19
WLM.....	26, 189, 198, 200, 201, 202, 203, 206
z/OS Connect.....	290, 298, 301, 302, 303, 304, 305, 306
z/TPF.....	8, 24, 26, 54, 216, 256, 257, 258
z/VM.....	8, 11, 14, 24, 25, 26, 48, 54
zSecure.....	17, 24, 80, 81, 83, 160, 166, 167